Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mbuf functions

Synopsis

The functions prefixed with Mbuf make up the Buffer module. The Buffer module allows you to allocate and control data buffers (storage areas) and containers (Aurora Imaging Library objects that hold buffers and other containers) that are typically operated on by functions of several Aurora Imaging Library modules. Examples of buffers include image buffers and lookup table (LUT) buffers. The module allows you to isolate regions of a buffer using child buffers or regions of interest, copy regions or bit-planes of a buffer to another buffer, and convert images grabbed from a camera with a Bayer filter into 3-band color images. The module can archive and retrieve buffer data in common storage formats (such as TIFF, JPEG, and AVI). The module also allows you to compress and decompress images and sequences, lossily or losslessly.

Functions

MbufAlloc1d
MbufAlloc2d
MbufAllocColor
MbufAllocComponent
MbufAllocContainer
MbufAllocDefault
MbufBayer
MbufChild1d
MbufChild2d
MbufChildColor
MbufChildColor2d
MbufChildColor2dClip
MbufChildContainer
MbufChildMove
MbufClear
MbufClearCond
MbufClone
MbufControl
MbufControlArea
MbufControlContainer
MbufControlFeature
MbufConvert3d
MbufCopy
MbufCopyClip
MbufCopyColor
MbufCopyColor2d
MbufCopyComponent
MbufCopyCond
MbufCopyMask
MbufCreate2d

MbufCreateColor
MbufCreateComponent
MbufDiskInquire
MbufExport
MbufExportSequence
MbufFree
MbufFreeComponent
MbufGet
MbufGet1d
MbufGet2d
MbufGetArc
MbufGetColor
MbufGetColor2d
MbufGetHookInfo
MbufGetLine
MbufGetList
MbufHookFunction
MbufImport
MbufImportSequence
MbufInquire
MbufInquireContainer
MbufInquireFeature
MbufLink
MbufLoad
MbufPut
MbufPut1d
MbufPut2d
MbufPutColor
MbufPutColor2d
MbufPutLine
MbufPutList
MbufRestore
MbufSave
MbufSetRegion
MbufStream
MbufTransfer

MbufAlloc1d

Synopsis

Allocate a 1D data buffer.

Syntax

```
AIL_ID MbufAlloc1d(
    AIL_ID SystemId,     //in
    AIL_INT SizeX,       //in
    AIL_INT Type,        //in
    AIL_INT64 Attribute, //in
    AIL_ID *BufIdPtr     //out
)
```

Description

This function allocates a one-dimensional 1-band data buffer on the specified system.

After allocating the one-dimensional 1-band data buffer, you should check if the operation was successful, using MappGetError() or by verifying that the one-dimensional 1-band data buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the one-dimensional 1-band data buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the one-dimensional 1-band data buffer's lifetime and you must not manually free it.

Parameters

SystemId INQ

Specifies the Aurora Imaging Library system on which to allocate the buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

SizeX INQ

Specifies the buffer width. The units are determined by the selected buffer attribute. For example, if the buffer has a LUT buffer attribute, specify the number of LUT entries to allocate.

Type INQ

Specifies a combination of two values: the depth and type of the data. Specify the depth of the buffer as one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the data depth of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 +	Specifies a 1-bit (packed binary) buffer. Note that you cannot allocate a 1-bit LUT, kernel, or structuring element buffer.	c		e	f	g	a		i		j	k	l	m	n	b
8 +	Specifies an 8-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
16 +	Specifies a 16-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
32 +	Specifies a 32-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the Type parameter.

You can add one of the following values to the above-mentioned values to set the data type.

Supported data types depend on the specified depth.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type of the buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLOAT	Specifies that the type of data is floating-point. The valid data depth is 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the type of data is signed. The valid data depths are 8, 16, or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the type of data is unsigned. The valid data depths are 1, 8, 16, or 32 bits. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute INQ

Specifies the buffer usage. Aurora Imaging Library uses this information to determine where to allocate the buffer in physical memory.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer usage																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ARRAY +	Specifies a buffer to store array type data. Note that some functions take an M_ARRAY buffer rather than a user-defined array.	c		e	f	g	a		i		j	k	l	m	n	b

M_IMAGE +	Specifies a buffer to store image data. You must specify a combination value from the following table: • To set the intended purpose of this buffer	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a kernel buffer to store a custom filter for convolution functions. The depth must be 8, 16, or 32-bit integer or floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT +	Specifies a buffer to store lookup table data. The valid data depths are 8, 16, or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a buffer to store structuring element data for morphology functions. The data depth must be 32-bit. If signed, the range is -32767 to +32767. If unsigned, the range is 0 to +32767. Note that the data can be set to **M_DONT_CARE** to ignore specific structuring elements.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You must add one or more of the following values to the above-mentioned value to set the intended purpose of this buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESS +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an image buffer that can hold compressed data. Note that a buffer with this attribute must have the M_UNSIGNED data type. When allocating buffers for operations that require a source and destination buffer, and one of the buffers has an M_COMPRESS attribute, the data will be compressed or decompressed depending on the attributes of the destination buffer. If both the source and destination buffers have M_COMPRESS specifies but different compression types, the data will be re-compressed according to the settings of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISP	Specifies an image buffer that can be displayed.	c		e	f	g	a		i		j	k	l	m	n	b

M_GRAB	Specifies an image buffer in which to grab data. This type of buffer is allocated in Aurora Imaging Library-reserved, physically contiguous, non-paged memory unless otherwise specified. For Host buffers (M_HOST_MEMORY), the maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (M_NON_PAGED). For on-board buffers (M_ON_BOARD), the maximum (total) number of grab (M_GRAB) buffers that can be allocated is restricted by the total amount of on-board memory.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific														
	Allocating a buffer with M_GRAB will force the buffer in pageable memory (M_PAGED) by default.				f	g								n	b
	Pageable memory grab buffers are supported. Allocating a buffer with M_GRAB will not force the buffer in Aurora Imaging Library-reserved, non-pageable memory (M_NON_PAGED).			e		g	a							n	
	Single-band (monochrome) buffers must have a depth of 8 or 16 bits to have an M_GRAB attribute.			e	f	g				j	k	l	m	n	b
	Single-band (monochrome) buffers must have a depth of 8 bits to have an M_GRAB attribute.							i							
M_PROC +	Specifies an image buffer that can be processed. If you intend to use the buffer as the source or destination buffer of a processing or analysis operation, you must allocate the buffer with an M_PROC attribute.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PROC.

You can add the following value to the above-mentioned value to set the allocation of an overscan region.

This overrides the MsysControl() M_ALLOCATION_OVERSCAN system control setting and forces the allocation of an overscan region. When this attribute is set, the setting of the M_ALLOCATION_OVERSCAN_SIZE system control determines the size of the overscan region and the M_ALLOCATION_OVERSCAN setting is ignored.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For allocating an overscan region																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOCATION_OVERSCAN	Specifies that the buffer is allocated with an overscan region. Specify the size of the region using MsysControl() with M_ALLOCATION_OVERSCAN_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type INQ																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data format is 1-band, 8-bit data. If no attribute is specified, M_JPEG_LOSSY is assumed. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data format is 1-band, 8-bit data.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format and a location specifier.

Aurora Imaging Library automatically selects the most appropriate storage format according to the specified intended usage attribute. For general processing, Aurora Imaging Library will convert the data when the

function requires a different format. If the default storage format is not appropriate and you want to avoid conversion during a time critical operation, you can add a storage format and a location specifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format and a location specifier																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIB	[This is only applicable to Windows] Forces the buffer to be a DIB buffer. This ensures your buffer is stored with a DIB header. Use MbufInquire() with M_BITMAPINFO to return a pointer (LPBITMAPINFO) to the header of the Aurora Imaging Library buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GDI	Forces the buffer to be compatible with GDI. When using a device context (using MbufControl() with M_DC_ALLOC) for drawing, the buffer should be internally stored in GDI format, and cannot be a child buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINUX_MXIMAGE	[This is only applicable to Linux] Forces the buffer to be an X11 Ximage.	c		e	f	g	a				j	k	l	m	n	b

Combination value for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add the following value to the above-mentioned values to set a location for a buffer that is FPGA accessible.

Note that the following value cannot be used with M_COMPRESS or values in the For specifying a storage format and a location specifier table.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the buffer is FPGA accessible																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FPGA_ACCESSIBLE +	Forces the buffer to be allocated in a bank of memory that is accessible from the Processing FPGA. The exact bank of on-board memory is determined by the other attributes used with this value. If combined with the M_HOST_MEMORY attribute, the memory allocated will be in non-paged Host memory.										j	k	l	m		

Combination values for any of the possible values of the Attribute parameter.

You can add one of the following values to the above-mentioned values to set a location for a buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location for a buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_HOST_MEMORY +	Forces the buffer in Host memory. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF_BOARD	Ensures that the buffer is not in on-board memory.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	To ensure that an M_OFF_BOARD buffer can be used by the DMA transfer section of the board, the buffer must be allocated in non-paged memory (M_OFF_BOARD + M_NON_PAGED or M_HOST_MEMORY + M_NON_PAGED).								i		j	k	l	m		
M_ON_BOARD +	Forces the buffer in on-board memory.	c							i		j	k	l	m		
	System specific															
	By default, all on-board buffers are allocated in memory that is not mapped on the PCI bus. This unshared memory cannot be accessed by the Host or any system other than the one on which it was allocated. See combination values below (M_SHARED) to change this default behavior.										j	k	l	m		
	This is only available for grab buffers (M_GRAB).								i							
	This is only available for image buffers (M_IMAGE).	c											l	m		
	This is only available for image (M_IMAGE) and LUT buffers (M_LUT).										j	k				

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following values: M_HOST_MEMORY; M_ON_BOARD.

You can add the following value to the above-mentioned values to specify a memory bank to be used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a memory bank to be used																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEMORY_BANK_*n*	Forces the buffer to be allocated in the specified memory bank.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															

		[This is only applicable to Windows] Valid values for *n* are between 0 and 6, inclusive. However, not all memory banks are necessarily present on a given system. Typically, if there is more than 1 memory bank, it is NUMA-enabled. Note that, to ensure that you are accessing a NUMA-enabled memory bank, you must specify the M_HOST_MEMORY combination value. To determine the actual number of memory banks available on the Aurora Imaging Library system, use MappInquireMp() with M_MEMORY_BANK_NUM and MappInquireMp() with M_MEMORY_BANK_AFFINITY_MASK. For more information, refer to the NUMA Support subsection of the Transparent multi-core use section of Chapter 66: Multi-processing, multi-core, and multi-threading. Note that this value cannot be used with M_COMPRESS or M_NON_PAGED. Also note that the combination of M_HOST_MEMORY with this value, in paged memory, is only useful when doing multi-core processing on a Host system.																
		[This is only applicable to Linux] Note that this value is not supported with the M_HOST_MEMORY combination value.																
		System specific																
		Valid values for *n* are between 0 and 1, inclusive. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value. Memory bank number: 0 Type of memory: Acquisition (SDRAM) Description: This is the default memory bank for an on-board buffer created with the M_GRAB attribute.									j	k				
		In this case, *n* must be set to 0. This is the default memory bank for an on-board buffer created with the M_GRAB attribute. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.											l	m		

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following value: M_ON_BOARD.

You can add the following value to the above-mentioned values to set a location in a specific type of memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location in a specific type of memory																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SHARED	Specifies that the buffer is in shared processing memory. This memory is mapped on the PCI bus. Note that this combination value can only be added to M_ON_BOARD.										j	k	l	m		

Combination values for any of the possible values of the Attribute parameter.

You can add one of the following values to the above-mentioned values to set the attribute of the specified physical memory type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the attribute of the specified physical memory type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NON_PAGED	Forces the buffer in Aurora Imaging Library reserved, non-pageable memory, if possible. The maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (specified at installation time or using the Aurora Imaging Configurator utility).	c		e	f	g	a		i		j	k	l	m	n	b
M_PAGED	Forces the buffer in pageable memory.	c		e	f	g	a		i		j	k	l	m	n	b

BufIdPtr

Specifies the address of the variable in which to write the buffer identifier or specifies the data type that the function should use to return the buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the one-dimensional 1-band data buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remarks

- If you are creating a DLL that includes a call to **MbufAlloc1d**(), ensure that the call is not made from the `DllMain()` function, because **MbufAlloc1d**() might load a required DLL and you cannot load a DLL from `DllMain()`. If necessary, call **MbufAlloc1d**() from an initialization function in your DLL instead.
- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.
- *[Aurora Imaging Library Lite]*
  You can allocate an image buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute with Aurora Imaging Library Lite. However, these attributes are not required to work under Aurora Imaging Library Lite.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufAlloc2d

Synopsis

Allocate a 2D data buffer.

Syntax

```
AIL_ID MbufAlloc2d(
    AIL_ID SystemId,     //in
    AIL_INT SizeX,       //in
    AIL_INT SizeY,       //in
    AIL_INT Type,        //in
    AIL_INT64 Attribute, //in
    AIL_ID *BufIdPtr     //out
)
```

Description

This function allocates a two-dimensional 1-band data buffer on the specified system.

After allocating the two-dimensional 1-band data buffer, you should check if the operation was successful, using MappGetError() or by verifying that the two-dimensional 1-band data buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the two-dimensional 1-band data buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the two-dimensional 1-band data buffer's lifetime and you must not manually free it.

Parameters

SystemId INQ

Specifies the Aurora Imaging Library system on which to allocate the buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

SizeX INQ

Specifies the buffer width. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, specify the width in pixels.

SizeY INQ

Specifies the buffer height. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, specify the height in pixels.

Type INQ

Specifies a combination of two values: the depth and type of the data. Specify the depth of the buffer as one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the data depth of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 +	Specifies a 1-bit (packed binary) buffer. Note that you cannot allocate a 1-bit LUT, kernel, or structuring element buffer.	c		e	f	g	a		i		j	k	l	m	n	b
8 +	Specifies an 8-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
16 +	Specifies a 16-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
32 +	Specifies a 32-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the Type parameter.

You can add one of the following values to the above-mentioned values to set the data type.

Supported data types depend on the specified depth.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type of the buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLOAT	Specifies that the type of data is floating-point. The valid data depth is 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the type of data is signed. The valid data depths are 8, 16, or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the type of data is unsigned. The valid data depths are 1, 8, 16, or 32 bits. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute INQ

Specifies the buffer usage. Aurora Imaging Library uses this information to determine where to allocate the buffer in physical memory.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer usage

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ARRAY +	Specifies a buffer to store array type data. Note that some functions take an M_ARRAY buffer rather than a user-defined array.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data. You must specify a combination value from the following table: • To set the intended purpose of the image buffer	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a kernel buffer to store a custom filter for convolution functions. The depth must be 8, 16, or 32-bit integer or floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT +	Specifies a buffer to store lookup table data. The valid data depths are 8, 16, or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a buffer to store structuring element data for morphology functions. The data depth must be 32-bit. If signed, the range is -32767 to +32767. If unsigned, the range is 0 to +32767. Note that the data can be set to **M_DONT_CARE** to ignore specific structuring elements.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You must add one or more of the following values to the above-mentioned value to set the intended purpose of the image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COMPRESS +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an image buffer that can hold compressed data. Note that a buffer with this attribute must have the M_UNSIGNED data type. When allocating buffers for operations that require a source and destination buffer, and one of the buffers has an M_COMPRESS attribute, the data will be compressed or decompressed depending on the attributes of the destination buffer. If both the source and destination buffers have M_COMPRESS specifiers but different compression types, the data will be re-compressed according to the settings of the destination buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISP	Specifies an image buffer that can be displayed.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific														
	Single-band display buffers with a depth greater than 16 bits cannot be allocated. 3-band display buffers can only be allocated with 8 bits per band.								i						
M_GRAB	Specifies an image buffer in which to grab data. This type of buffer is allocated in Aurora Imaging Library-reserved, physically contiguous, non-paged memory unless otherwise specified. For Host buffers (M_HOST_MEMORY), the maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (M_NON_PAGED). For on-board buffers (M_ON_BOARD), maximum (total) number of grab (M_GRAB) buffers that can be allocated is restricted by the total amount of on-board memory.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific														
	Allocating a buffer with M_GRAB will force the buffer in pageable memory (M_PAGED) by default.				f	g								n	b
	Pageable memory grab buffers are supported. Allocating a buffer with M_GRAB will not force the buffer in Aurora Imaging Library-reserved, non-pageable memory (M_NON_PAGED).			e		g	a							n	
	Single-band (monochrome) buffers must have a depth of 8 or 16 bits to have an M_GRAB attribute.			e	f	g					j	k	l	m	n	b
	Single-band (monochrome) buffers must have a depth of 8 bits to have an M_GRAB attribute.								i						

M_PROC +	Specifies an image buffer that can be processed. If you intend to use the buffer as the source or destination buffer of a processing or analysis operation, you must allocate the buffer with an M_PROC attribute.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PROC.

You can add the following value to the above-mentioned value to set the allocation of an overscan region.

This overrides the MsysControl() M_ALLOCATION_OVERSCAN system control setting and forces the allocation of an overscan region. When this attribute is set, the setting of the M_ALLOCATION_OVERSCAN_SIZE system control determines the size of the overscan region and the M_ALLOCATION_OVERSCAN setting is ignored.

Note that this attribute is only useful when added to a buffer on which you want to perform neighborhood processing operations.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For allocating an overscan region																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOCATION_OVERSCAN	Specifies that the buffer is allocated with an overscan region. Specify the size of the region using MsysControl() with M_ALLOCATION_OVERSCAN_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type INQ																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data format is 1-band, 8-bit data. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data format is 1-band, 8-bit data.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format and a location specifier.

Aurora Imaging Library automatically selects the most appropriate storage format according to the specified intended usage attribute. For general processing, Aurora Imaging Library will convert the data when the function requires a different format. If the default storage format is not appropriate and you want to avoid conversion during a time critical operation, you can add a storage format and a location specifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format and a location specifier																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIB	[This is only applicable to Windows] Forces the buffer to be a DIB buffer. This ensures your buffer is stored with a DIB header. Use MbufInquire() with M_BITMAPINFO to return a pointer (LPBITMAPINFO) to the header of the Aurora Imaging Library buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GDI	Forces the buffer to be compatible with GDI. When using a device context (using MbufControl() with M_DC_ALLOC) for drawing, the buffer should be internally stored in GDI format, and cannot be a child buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINUX_MXIMAGE	[This is only applicable to Linux] Forces the buffer to be an X11 Ximage.	c		e	f	g	a				j	k	l	m	n	b

Combination value for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add the following value to the above-mentioned values to specify that the buffer is FPGA accessible.

Note that the following value cannot be used with M_COMPRESS or values in the For specifying a storage

format and a location specifier table.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the buffer is FPGA accessible																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FPGA_ACCESSIBLE +	Forces the buffer to be allocated in a bank of memory that is accessible from the Processing FPGA. The exact bank of on-board memory is determined by the other attributes used with this value. If combined with the M_HOST_MEMORY attribute, the memory allocated will be in non-paged Host memory.										j	k	l	m		

Combination values for any of the possible values of the Attribute parameter.

You can add one of the following values to the above-mentioned values to set a location for a buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location for a buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HOST_MEMORY +	Forces the buffer in Host memory. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF_BOARD	Ensures that the buffer is not in on-board memory.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	To ensure that an M_OFF_BOARD buffer can be used by the DMA transfer section of the board, the buffer must be allocated in non-paged memory (M_OFF_BOARD + M_NON_PAGED or M_HOST_MEMORY + M_NON_PAGED).								i		j	k	l	m		
M_ON_BOARD +	Forces the buffer in on-board memory.	c							i		j	k	l	m		
	System specific															
	By default, all on-board buffers are allocated in memory that is not mapped on the PCI bus. This unshared memory cannot be accessed by the Host or any system other than the one on which it was allocated. See combination values below (M_SHARED) to change this default behavior.										j	k	l	m		
	This is only available for grab buffers (M_GRAB).								i							
	This is only available for image buffers (M_IMAGE).	c											l	m		

		This is only available for image (M_IMAGE) and LUT buffers (M_LUT).										j	k				

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following values: M_HOST_MEMORY; M_ON_BOARD.

You can add the following value to the above-mentioned values to specify a memory bank to be used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a memory bank to be used																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEMORY_BANK_*n*	Forces the buffer to be allocated in the specified memory bank.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															
	[This is only applicable to Windows] Valid values for *n* are between 0 and 6, inclusive. However, not all memory banks are necessarily present on a given system. Typically, if there is more than 1 memory bank, it is NUMA-enabled. Note that, to ensure that you are accessing a NUMA-enabled memory bank, you must specify the M_HOST_MEMORY combination value. To determine the actual number of memory banks available on the Aurora Imaging Library system, use MappInquireMp() with M_MEMORY_BANK_NUM and MappInquireMp() with M_MEMORY_BANK_AFFINITY_MASK. For more information, refer to the NUMA Support subsection of the Transparent multi-core use section of Chapter 66: Multi-processing, multi-core, and multi-threading. Note that this value cannot be used with M_COMPRESS or M_NON_PAGED. Also note that the combination of M_HOST_MEMORY with this value, in paged memory, is only useful when doing multi-core processing on a Host system.															
	[This is only applicable to Linux] Note that this value is not available with the M_HOST_MEMORY combination value.															
	System specific															

		Valid values for *n* are between 0 and 1, inclusive. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value. Memory bank number: Type of memory: Description 0: Acquisition (SDRAM): This is the default memory bank for an on-board buffer created with the M_GRAB attribute.										j	k				
		In this case, *n* must be set to 0. This is the default memory bank for an on-board buffer created with the M_GRAB attribute. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.												l	m		

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following value: M_ON_BOARD.

You can add the following value to the above-mentioned values to set a location in a specific type of memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location in a specific type of memory																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SHARED	Specifies that the buffer is in shared processing memory. This memory is mapped on the PCI bus. Note that this combination value can only be added to M_ON_BOARD.										j	k	l	m		

Combination values for any of the possible values of the Attribute parameter.

You can add one of the following values to the above-mentioned values to set the attribute of the specified physical memory type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the attribute of the specified physical memory type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_NON_PAGED	Forces the buffer in Aurora Imaging Library reserved, non-pageable memory, if possible. The maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (specified at installation time or using the Aurora Imaging Configurator utility).	c		e	f	g	a		i		j	k	l	m	n	b
M_PAGED	Forces the buffer in pageable memory.	c		e	f	g	a		i		j	k	l	m	n	b

BufIdPtr

Specifies the address of the variable in which to write the buffer identifier or specifies the data type that the function should use to return the buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, an Aurora Imaging Library smart identifier of type *M_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

Return value

The returned value is the two-dimensional 1-band data buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remarks

- If you are creating a DLL that includes a call to **MbufAlloc2d**(), ensure that the call is not made from the `DllMain()` function, because **MbufAlloc2d**() might load a required DLL and you cannot load a DLL from `DllMain()`. If necessary, call **MbufAlloc2d**() from an initialization function in your DLL instead.
- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.
- *[Aurora Imaging Library Lite]*
  You can allocate an image buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute with Aurora Imaging Library Lite. However, these attributes are not required to work under Aurora Imaging Library Lite.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufAllocColor

Synopsis

Allocate a multi-band data buffer.

Syntax

```
AIL_ID MbufAllocColor(
    AIL_ID SystemId,     //in
    AIL_INT SizeBand,    //in
    AIL_INT SizeX,       //in
    AIL_INT SizeY,       //in
    AIL_INT Type,        //in
    AIL_INT64 Attribute, //in
    AIL_ID *BufIdPtr     //out
)
```

Description

This function allocates a data buffer with multiple bands on the specified system. This type of buffer allows for the representation of color images (for example, RGB).

Note that you can also allocate non-color multi-band image buffers. Image buffers allocated with 2 bands or more than 3 bands are always non-color.

This function allocates buffers that have a two-dimensional surface for each specified band. You can use MbufAlloc1d() and MbufAlloc2d() to create single band one- or two-dimensional data buffers, respectively.

For more information about color buffer formats, refer to the RGB buffers section of Chapter 23: Data buffers and the YUV buffers section of Chapter 23: Data buffers.

After allocating the data buffer, you should check if the operation was successful, using MappGetError() or by verifying that the data buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the data buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the data buffer's lifetime and you must not manually free it.

Parameters

SystemId INQ

Specifies the Aurora Imaging Library system on which to allocate the buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b

System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

SizeBand INQ

Specifies the number of (x,y) surfaces (also called bands) to allocate to the buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of surfaces to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 <= *Value* <= 1024	Specifies the number of bands. Specify 1 band for each color band, or other type of data, that the buffer needs to store (for example, monochrome images require 1 band; RGB color images require 3 color bands). This value can be any non-zero integer value up to 1024 for an image buffer, or 1 or 3 for an array or LUT buffer.	c		e	f	g	a		i		j	k	l	m	n	b

SizeX INQ

Specifies the buffer width. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, width is specified in pixels.

SizeY INQ

Specifies the buffer height. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, height is specified in pixels.

Type INQ

Specifies a combination of two values: data type and data depth per band. Specify the depth of the buffer per band as one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the data depth of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 +	Specifies a 1-bit (packed binary) buffer. Note that you cannot allocate a 1-bit LUT buffer.	c		e	f	g	a		i		j	k	l	m	n	b
8 +	Specifies an 8-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
16 +	Specifies a 16-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
32 +	Specifies a 32-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the Type parameter.

You can add one of the following values to the above-mentioned values to set the data type.

Supported data types depend on the specified depth.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type of the buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLOAT	Specifies that the type of data is floating-point. The valid data depth is 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the type of data is signed. The valid data depths are 8, 16, or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the type of data is unsigned. The valid data depths are 1, 8, 16, or 32 bits. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute INQ

Specifies the buffer usage. Aurora Imaging Library uses this information to determine where to allocate the buffer in physical memory.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer usage																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ARRAY +	Specifies a buffer to store array type data. Note that some functions take an M_ARRAY buffer rather than a user-defined array. Note that this attribute is only supported for 1-band and 3-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data. You must specify a combination value from the following table: • To set the intended purpose of the image buffer	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT +	Specifies a buffer to store lookup table data. The valid data depths are 8, 16, or 32 bits and only the planar internal storage format is valid. Note that this attribute is only supported for 1-band and 3-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You must add one or more of the following values to the above-mentioned value to set the intended purpose of the image buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESS +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an image buffer that can hold compressed data. Note that a buffer with this attribute must have the M_UNSIGNED data type. When allocating buffers for operations that require a source and destination buffer, and one of the buffers has an M_COMPRESS attribute, the data will be compressed or decompressed depending on the attributes of the destination buffer. If both the source and destination buffers have M_COMPRESS specifiers but different compression types, the data will be re-compressed according to the settings of the destination buffer. Only 1-band and 3-band buffers can have an M_COMPRESS attribute.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISP	Specifies an image buffer that can be displayed.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies an image buffer in which to grab data. This type of buffer is allocated in Aurora Imaging Library-reserved, physically contiguous, non-paged memory unless otherwise specified. For Host buffers (M_HOST_MEMORY), the maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (M_NON_PAGED). For on-board buffers (M_ON_BOARD), the maximum (total) number of grab (M_GRAB) buffers that can be allocated is restricted by the total amount of on-board memory. Only 1-band and 3-band buffers can have an M_GRAB attribute.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Allocating a buffer with M_GRAB will force the buffer in pageable memory (M_PAGED) by default.				f	g									n	b
	Pageable memory grab buffers are supported. Allocating a buffer with M_GRAB will not force the buffer in Aurora Imaging Library-reserved, non-pageable memory (M_NON_PAGED).			e		g	a								n	
	Single-band (monochrome) buffers must have a depth of 8 bits or 16 bits to have an M_GRAB attribute. Only some three-band (color) buffers can have an M_GRAB attribute. See the constants for color buffer formats to determine which are supported with an M_GRAB attribute.			e	f	g					j	k	l	m	n	b

		Single-band (monochrome) buffers must have a depth of 8 bits or 16 bits to have an M_GRAB attribute. Only some three-band (color) buffers can have an M_GRAB attribute. See the constants for color buffer formats to determine which are supported with an M_GRAB attribute. Note that color buffers with an M_GRAB attribute are only available for color versions of Zebra Iris GTX (such as 2000C).							i							
M_PROC +		Specifies an image buffer that can be processed. If you intend to use the buffer as the source or destination buffer of a processing or analysis operation, you must allocate the buffer with an M_PROC attribute.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PROC.

You can add the following value to the above-mentioned value to set the allocation of an overscan region.

This overrides the MsysControl() M_ALLOCATION_OVERSCAN system control setting and forces the allocation of an overscan region. When this attribute is set, the setting of the M_ALLOCATION_OVERSCAN_SIZE system control determines the size of the overscan region and the M_ALLOCATION_OVERSCAN setting is ignored.

Note that this attribute is only useful when added to a buffer on which you want to perform neighborhood processing operations.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For allocating an overscan region																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOCATION_OVERSCAN	Specifies that the buffer is allocated with an overscan region. Specify the size of the region using MsysControl() with M_ALLOCATION_OVERSCAN_SIZE.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type INQ																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_H264	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold H.264 data. This compression type is used when specifying a source of an input or a destination of an output for a sequence operation as an array of buffers (MseqDefine() with M_BUFFER_LIST).	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24 + M_PLANAR, M_RGB48 + M_PLANAR, or M_YUV24 + M_PLANAR format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24, M_RGB48, M_YUV9, M_YUV12, M_YUV16 + M_PLANAR, or M_YUV24 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band data in M_RGB24 or M_RGB48 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_RGB24, M_YUV24, M_YUV12, M_YUV9, M_YUV16 + M_PLANAR, or M_YUV16 + M_PACKED format. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_YUV16 + M_PACKED format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format and a location specifier.

Aurora Imaging Library automatically selects the most appropriate storage format according to the specified intended usage attribute. For general processing, Aurora Imaging Library will convert the data when the

function requires a different format. If the default storage format is not appropriate and you want to avoid conversion during a time critical operation, you can add a storage format and a location specifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format and a location specifier																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIB	[This is only applicable to Windows] Forces the buffer to be a DIB buffer. This ensures your buffer is stored with a DIB header. Use MbufInquire() with M_BITMAPINFO to return a pointer (LPBITMAPINFO) to the header of the Aurora Imaging Library buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_GDI	Forces the buffer to be compatible with GDI. When using a device context (using MbufControl() with M_DC_ALLOC) for drawing, the buffer should be internally stored in GDI format, and cannot be a child buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_LINUX_MXIMAGE	[This is only applicable to Linux] Forces the buffer to be an X11 Ximage.	c		e	f	g	a				j	k	l	m	n	b

Combination value for M_ARRAY; M_IMAGE; M_LUT.

You can add the following value to the above-mentioned values to set a location for a buffer that is FPGA accessible.

Note that the following value cannot be used with M_COMPRESS or values in the For specifying a storage format and a location specifier table.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the buffer is FPGA accessible																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FPGA_ACCESSIBLE +	Forces the buffer to be allocated in a bank of memory that is accessible from the Processing FPGA. The exact bank of on-board memory is determined by the other attributes used with this value. If combined with the M_HOST_MEMORY attribute, the memory allocated will be in non-paged Host memory.										j	k	l	m		

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format for image buffers.

Note that the buffer must be a 3-band buffer. Also note that it might be slower to use buffers that have been forced with one of these attributes. Although there is no right or wrong storage format to use, certain operations are optimized for some formats.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format for image buffers INQ																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). Typically, packed buffers are used for display (M_DISP). Note that it might be slower to process buffers with the M_PACKED attribute. Also, note that you cannot allocate a packed LUT buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. Typically, planar buffers are used for processing (M_PROC).	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_PLANAR.

You can add one of the following values to the above-mentioned value to set a planar color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB3	Specifies 3-bit color depth (RGB 1:1:1) planar pixels.	c					a		i		j	k	l	m		
M_YUV9	Specifies YUV9 planar pixels.	c			f		a		i		j	k	l	m		
M_YUV12	Specifies YUV12 planar pixels.	c			f		a		i		j	k	l	m		
M_YUV24	Specifies YUV24 planar pixels.	c			f		a		i		j	k	l	m		

Combination values for M_PACKED.

You can add one of the following values to the above-mentioned value to set a packed color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth packed pixels (BGRBGR).	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															

	This value can be used with grab (M_GRAB) buffers.	c		e	f	g	a				j	k	l	m	n	b
M_BGR32	Specifies 32-bit color depth packed pixels (BGRXBGRX). The most-significant byte is a "don't care" byte.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB15	Specifies 16-bit color depth packed pixels (XRGB 1:5:5:5). Note that when accessing an M_RGB15 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB16	Specifies 16-bit color depth packed pixels (RGB 5:6:5). Note that when accessing an M_RGB16 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. For more information, see the YUV buffers section of Chapter 23: Data buffers. This value can be used with grab (M_GRAB) buffers.			e	f	g									n	b
M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. For more information, see the YUV buffers section of Chapter 23: Data buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c					a				j	k	l	m		
	This value can be used with grab (M_GRAB) buffers if those buffers are also packed (M_PACKED).								i							

Combination values for the values listed in For specifying a storage format for image buffers.

You can add one of the following values to the above-mentioned values to set a packed or planar color buffer format. Alternatively, you can add a value from For specifying a packed color buffer format or from For specifying a planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed or planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_RGB24	Specifies 24-bit color depth (RGB 8:8:8) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers.								i		j	k	l	m		
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g									n	b
M_RGB48	Specifies 48-bit color depth (RGB 16:16:16). packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g					j	k	l	m	n	b
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers if the buffer is allocated on the Host (M_HOST_MEMORY) and not on-board.										j	k	l	m		
M_RGB96	Specifies 96-bit color depth (RGB 32:32:32) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16	Specifies YUV16 (4:2:2) pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for any of the possible values of the Attribute parameter.

You can add one of the following values to the above-mentioned values to set a location for a buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location for a buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HOST_MEMORY +	Forces the buffer in Host memory. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF_BOARD	Ensures that the buffer is not in on-board memory.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															

		To ensure that an M_OFF_BOARD buffer can be used by the DMA transfer section of the board, the buffer must be allocated in non-paged memory (M_OFF_BOARD + M_NON_PAGED or M_HOST_MEMORY + M_NON_PAGED).							i		j	k	l	m		
M_ON_BOARD +		Forces the buffer in on-board memory.	c							i		j	k	l	m		
		System specific														
		By default, all on-board buffers are allocated in memory that is not mapped on the PCI bus. This unshared memory cannot be accessed by the Host or any system other than the one on which it was allocated. See combination values below (M_SHARED) to change this default behavior.									j	k	l	m		
		This is only available for grab buffers (M_GRAB).								i						
		This is only available for image buffers (M_IMAGE).	c										l	m		
		This is only available for image (M_IMAGE) and LUT buffers (M_LUT).										j	k			

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following values: M_HOST_MEMORY; M_ON_BOARD.

You can add the following value to the above-mentioned values to specify a memory bank to be used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a memory bank to be used																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEMORY_BANK_*n*	Forces the buffer to be allocated in the specified memory bank.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															

		[This is only applicable to Windows] Valid values for *n* are between 0 and 6, inclusive. However, not all memory banks are necessarily present on a given system. Typically, if there is more than 1 memory bank, it is NUMA-enabled. Note that, to ensure that you are accessing a NUMA-enabled memory bank, you must specify the M_HOST_MEMORY combination value. To determine the actual number of memory banks available on the Aurora Imaging Library system, use MappInquireMp() with M_MEMORY_BANK_NUM and MappInquireMp() with M_MEMORY_BANK_AFFINITY_MASK. For more information, refer to the NUMA Support subsection of the Transparent multi-core use section of Chapter 66: Multi-processing, multi-core, and multi-threading. Note that this value cannot be used with M_COMPRESS or M_NON_PAGED. Also note that the combination of M_HOST_MEMORY with this value, in paged memory, is only useful when doing multi-core processing on a Host system.																
		[This is only applicable to Linux] Note that this value is not available with the M_HOST_MEMORY combination value.																
		System specific																
		Valid values for *n* are between 0 and 1, inclusive. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value. Memory bank number: 0 Type of memory: Acquisition (SDRAM) Description: This is the default memory bank for an on-board buffer created with the M_GRAB attribute.										j	k				
		In this case, *n* must be set to 0. This is the default memory bank for an on-board buffer created with the M_GRAB attribute. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.												l	m		

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following value: M_ON_BOARD.

You can add the following value to the above-mentioned values to set a location in a specific type of memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location in a specific type of memory																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SHARED	Specifies that the buffer is in shared processing memory. This memory is mapped on the PCI bus. Note that this combination value can only be added to M_ON_BOARD.										j	k	l	m		

Combination values for any of the possible values of the Attribute parameter.

You can add one of the following values to the above-mentioned values to set the attribute of the specified physical memory type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the attribute of the specified physical memory type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NON_PAGED	Forces the buffer in Aurora Imaging Library reserved, non-pageable memory, if possible. The maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (specified at installation time or using the Aurora Imaging Configurator utility).	c		e	f	g	a		i		j	k	l	m	n	b
M_PAGED	Forces the buffer in pageable memory.	c		e	f	g	a		i		j	k	l	m	n	b

BufIdPtr

Specifies the address of the variable in which to write the buffer identifier or specifies the data type that the function should use to return the buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the image buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated image buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the LUT buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated LUT buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the data buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remarks

- If you are creating a DLL that includes a call to **MbufAllocColor**(), ensure that the call is not made from the `DllMain()` function, because **MbufAllocColor**() might load a required DLL and you cannot load a DLL from `DllMain()`. If necessary, call **MbufAllocColor**() from an initialization function in your DLL instead.
- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the

compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufAllocComponent

Synopsis

Allocate a buffer and add it as a component of a container.

Syntax

```
AIL_ID MbufAllocComponent(
    AIL_ID ContainerBufId,   //in
    AIL_INT SizeBand,        //in
    AIL_INT SizeX,           //in
    AIL_INT SizeY,           //in
    AIL_INT Type,            //in
    AIL_INT64 Attribute,     //in
    AIL_INT64 ComponentType, //in
    AIL_ID *ComponentIdPtr   //out
)
```

Description

This function allocates a buffer and adds it as a component of a container. The buffer is allocated on the same system as the container.

> Note that typically you do not need to manually allocate components. Aurora Imaging Library typically allocates/reallocates components as required when the container is used as a destination for a grabbing or processing function.

After allocating the component, you should check that the operation was successful (either using MappGetError(), or by verifying that the buffer identifier returned is not Null).

When the component is no longer required, free it using MbufFreeComponent() or free its container using MbufFree().

> Note that in some cases, a component allocated using **MbufAllocComponent**() might be freed and reallocated automatically if its container is the destination for an Aurora Imaging Library function (such as a grabbing or 3D processing function). If the buffer is reallocated, its Aurora Imaging Library identifier will change. If you need to access the component directly using its Aurora Imaging Library buffer identifier, you should inquire its Aurora Imaging Library identifier (using MbufInquireContainer() with M_COMPONENT_ID) any time its container is passed as the destination of an Aurora Imaging Library function.

Parameters

ContainerBufId

Specifies the Aurora Imaging Library identifier of the container to which to add the component.

SizeBand INQ

Specifies the number of (x,y) surfaces (also called bands) to allocate to the buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of surfaces to allocate

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 <= *Value* <= 1024	Specifies the number of bands. When allocating a buffer to store color, specify 1 band for each color band, or other type of data, that the buffer needs to store (for example, monochrome images require 1 color band; RGB color images require 3 color bands). This value can be any non-zero integer value up to 1024 for an image buffer, 1 or 3 for an array or LUT buffer, or 1 for a kernel or structuring element buffer.	c		e	f	g	a		i		j	k	l	m	n	b

SizeX INQ

Specifies the buffer width. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, width is specified in pixels.

SizeY INQ

Specifies the buffer height. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, height is specified in pixels.

Type INQ

Specifies a combination of two values: data type and data depth per band. Specify the depth of the buffer per band as one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the data depth of the component																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
1 +	Specifies a 1-bit (packed binary) buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
8 +	Specifies an 8-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
16 +	Specifies a 16-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
32 +	Specifies a 32-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for 1; 8; 16; 32.

You can add one of the following values to the above-mentioned values to set the data type.

Supported data types depend on the specified depth.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type of the buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_FLOAT	Specifies that the type of data is floating-point. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the type of data is signed. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the type of data is unsigned. INFO This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute INQ

Specifies the buffer usage. Aurora Imaging Library uses this information to determine where to allocate the buffer in physical memory.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer usage																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ARRAY +	Specifies a buffer to store array type data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data.	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL +	Specifies a kernel buffer to store a custom filter for convolution functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT +	Specifies a buffer to store lookup table data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT +	Specifies a buffer to store structuring element data for morphology functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You can add one or more of the following values to the above-mentioned value to set the intended purpose of the image buffer.

> Note that the component is always automatically assigned the intended usage attributes of its container (such as M_PROC or M_DISP). You can therefore optionally omit the intended purpose setting when allocating a component. You can still specify valid combination values for these settings if you omit them.

> Attempting to allocate a component with one of the following attributes in a container that does not have the attribute will generate an error (aside from M_COMPRESS).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COMPRESS +	Specifies an image buffer that can hold compressed data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISP	Specifies an image buffer that can be displayed.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies an image buffer in which to grab data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC +	Specifies an image buffer that can be processed. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PROC.

You can add the following value to the above-mentioned value to set the allocation of an overscan region.

This overrides the MsysControl() M_ALLOCATION_OVERSCAN system control setting and forces the allocation of an overscan region. When this attribute is set, the setting of the M_ALLOCATION_OVERSCAN_SIZE system control determines the size of the overscan region and the M_ALLOCATION_OVERSCAN setting is ignored.

Note that this attribute is only useful when added to a buffer on which you want to perform neighborhood processing operations.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For allocating an overscan region																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOCATION_OVERSCAN	Specifies that the buffer is allocated with an overscan region. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_H264	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold H.264 data. This compression type is used when specifying a source of an input or a destination of an output for a sequence operation as an array of buffers (MseqDefine() with M_BUFFER_LIST).	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24 + M_PLANAR, M_RGB48 + M_PLANAR, or M_YUV24 + M_PLANAR format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24, M_RGB48, M_YUV9, M_YUV12, M_YUV16 + M_PLANAR, or M_YUV24 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band data in M_RGB24 or M_RGB48 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_RGB24, M_YUV24, M_YUV12, M_YUV9, M_YUV16 + M_PLANAR, or M_YUV16 + M_PACKED format. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_YUV16 + M_PACKED format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format and a location specifier.

Aurora Imaging Library automatically selects the most appropriate storage format according to the specified intended usage attribute. For general processing, Aurora Imaging Library will convert the data when the

function requires a different format. If the default storage format is not appropriate and you want to avoid conversion during a time critical operation, you can add a storage format and a location specifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format and a location specifier																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIB	Forces the buffer to be a DIB buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GDI	Forces the buffer to be compatible with GDI. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LINUX_MXIMAGE	[This is only applicable to Linux] Forces the buffer to be an X11 Ximage.	c		e	f	g	a				j	k	l	m	n	b

Combination value for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add the following value to the above-mentioned values to specify that the buffer is FPGA accessible.

Note that the following value cannot be used with M_COMPRESS or values in the For specifying a storage format and a location specifier table.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the buffer is FPGA accessible																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FPGA_ACCESSIBLE +	Forces the buffer to be allocated in a bank of memory that is accessible from the Processing FPGA. INFO										j	k				

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format for image buffers.

Note that the buffer must be a 3-band buffer. Also note that it might be slower to use buffers that have been forced with one of these attributes. Although there is no right or wrong storage format to use, certain operations are optimized for some formats.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format for image buffers INQ																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_PLANAR.

You can add one of the following values to the above-mentioned value to set a planar color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB3	Specifies 3-bit color depth (RGB 1:1:1) planar pixels.	c					a		i		j	k	l	m		
M_YUV9	Specifies YUV9 planar pixels.	c			f		a		i		j	k	l	m		
M_YUV12	Specifies YUV12 planar pixels.	c			f		a		i		j	k	l	m		
M_YUV24	Specifies YUV24 planar pixels.	c			f		a		i		j	k	l	m		

Combination values for M_PACKED.

You can add one of the following values to the above-mentioned value to set a packed color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth packed pixels (BGRBGR).	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c		e	f	g	a				j	k	l	m	n	b
M_BGR32	Specifies 32-bit color depth packed pixels (BGRXBGRX). The most-significant byte is a "don't care" byte.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB15	Specifies 16-bit color depth packed pixels (XRGB 1:5:5:5). Note that when accessing an M_RGB15 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b

M_RGB16	Specifies 16-bit color depth packed pixels (RGB 5:6:5). Note that when accessing an M_RGB16 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. For more information, see the YUV buffers section of Chapter 23: Data buffers. This value can be used with grab (M_GRAB) buffers.			e	f	g									n	b
M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. For more information, see the YUV buffers section of Chapter 23: Data buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c					a				j	k	l	m		
	This value can be used with grab (M_GRAB) buffers if those buffers are also packed (M_PACKED).								i							

Combination values for the values listed in For specifying a storage format for image buffers.

You can add one of the following values to the above-mentioned values to set a packed or planar color buffer format. Alternatively, you can add a value from For specifying a packed color buffer format or from For specifying a planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed or planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB24	Specifies 24-bit color depth (RGB 8:8:8) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers.								i		j	k	l	m		
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g									n	b
M_RGB48	Specifies 48-bit color depth (RGB 16:16:16). packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															

	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g					j	k	l	m	n	b
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers if the buffer is allocated on the Host (M_HOST_MEMORY) and not on-board.										j	k	l	m		
M_RGB96	Specifies 96-bit color depth (RGB 32:32:32) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16	Specifies YUV16 (4:2:2) pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add one of the following values to the above-mentioned values to set a location for a buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location for a buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HOST_MEMORY +	Forces the buffer in Host memory. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF_BOARD	Ensures that the buffer is not in on-board memory.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	To ensure that an M_OFF_BOARD buffer can be used by the DMA transfer section of the board, the buffer must be allocated in non-paged memory (M_OFF_BOARD + M_NON_PAGED or M_HOST_MEMORY + M_NON_PAGED).								i		j	k	l	m		
M_ON_BOARD +	Forces the buffer in on-board memory.	c							i		j	k	l	m		
	System specific															
	By default, all on-board buffers are allocated in memory that is not mapped on the PCI bus. This unshared memory cannot be accessed by the Host or any system other than the one on which it was allocated. See combination values below (M_SHARED) to change this default behavior.										j	k	l	m		

		This is only available for grab buffers (M_GRAB).								i						
		This is only available for image buffers (M_IMAGE).	c											l	m		
		This is only available for image (M_IMAGE) and LUT buffers (M_LUT).										j	k				

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following values: M_HOST_MEMORY; M_ON_BOARD.

You can add the following value to the above-mentioned values to specify a memory bank to be used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a memory bank to be used																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEMORY_BANK_*n*	Forces the buffer to be allocated in the specified memory bank.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															
	[This is only applicable to Windows] Valid values for *n* are between 0 and 6, inclusive. However, not all memory banks are necessarily present on a given system. Typically, if there is more than 1 memory bank, it is NUMA-enabled. Note that, to ensure that you are accessing a NUMA-enabled memory bank, you must specify the M_HOST_MEMORY combination value. To determine the actual number of memory banks available on the Aurora Imaging Library system, use MappInquireMp() with M_MEMORY_BANK_NUM and MappInquireMp() with M_MEMORY_BANK_AFFINITY_MASK. For more information, refer to the NUMA Support subsection of the Transparent multi-core use section of Chapter 66: Multi-processing, multi-core, and multi-threading. Note that this value cannot be used with M_COMPRESS or M_NON_PAGED. Also note that the combination of M_HOST_MEMORY with this value, in paged memory, is only useful when doing multi-core processing on a Host system.															
	[This is only applicable to Linux] Note that this value is not available with the M_HOST_MEMORY combination value.															
	System specific															

		Valid values for *n* are between 0 and 1, inclusive. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value. Memory bank number / Type of memory / Description 0 / Acquisition (SDRAM) / This is the default memory bank for an on-board buffer created with the M_GRAB attribute.										j	k				
		In this case, *n* must be set to 0. This is the default memory bank for an on-board buffer created with the M_GRAB attribute. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.												l	m		

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following value: M_ON_BOARD.

You can add the following value to the above-mentioned values to set a location in a specific type of memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location in a specific type of memory																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SHARED	Specifies that the buffer is in shared processing memory. This memory is mapped on the PCI bus. Note that this combination value can only be added to M_ON_BOARD.										j	k	l	m		

Combination values for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add one of the following values to the above-mentioned values to set the attribute of the specified physical memory type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the attribute of the specified physical memory type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_NON_PAGED	Forces the buffer in Aurora Imaging Library reserved, non-pageable memory, if possible. The maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (specified at installation time or using the Aurora Imaging Configurator utility).	c		e	f	g	a		i		j	k	l	m	n	b
M_PAGED	Forces the buffer in pageable memory.	c		e	f	g	a		i		j	k	l	m	n	b

ComponentType

Specifies the component type, which identifies what kind of information the component stores.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of component																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default component type. The default component type for image buffers is M_COMPONENT_INTENSITY. The default component type for all other buffers is M_COMPONENT_UNDEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. Coordinates associated with the confidence value 0 are considered invalid data and will not be used by 3D image processing functions. A confidence component is associated with a range or disparity component in the same container when there are no other range, disparity, or confidence components in the container. If there is more than one range, disparity, and/or confidence component in a container (for example, because a single grab into the container transmitted multiple range and confidence components), you will need to either create a child container which contains only the required components using MbufChildContainer(), or free the extra components using MbufFreeComponent().	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the column index of each corresponding pixel and will result in a better representation of the camera's lens model when generating X-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the row index of each corresponding pixel and will result in a better representation of the camera's lens model when generating Y-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 254. You can use custom component types to identify components which store information that does not fit one of the standard component types. In addition, some cameras transmit components with custom component types. Refer to your camera manual to determine what type of information is stored in transmitted components.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. Each pixel of a disparity map indicates the apparent distance (typically measured in pixel units) between where an object appears in the left and right images captured by a stereoscopic camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. Only depth map containers can have a matrix component. A matrix component is associated with a range component in the same container when there is only one range component and no other matrix components in that container.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. A mesh component is associated with a range component in the same container when there are no other range or mesh components in that container. A point cloud container which has a mesh component is referred to as a meshed point cloud container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_METADATA	Specifies that the component stores metadata information. Metadata components are used by Aurora Imaging Library internally. Typically, you should ignore metadata components in your application.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. Unlike an intensity component, a multispectral component might include information about non-visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies that the component stores normals information for each point in the M_COMPONENT_RANGE component of the container. A normals component is associated with a range component in the same container when there are no other range or normals components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. This can be either a 1-band buffer that stores a depth map, or a 3-band buffer that stores coordinates of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. Each pixel of a reflectance map indicates how much of the light hitting an object at that location is reflected back. Typically, this is an intensity image of the light spectrum used by a 3D sensor to detect 3D distance/position information. Typically, if the map was generated by a laser profiler, each row indicates the detected intensity of the laser for a single scan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. 3D image processing functions will only operate on points inside the ROI. Only depth map containers can have a region component. A region component is associated with a range component in the same container when there is only one range component and no other region components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. Each pixel of a scatter map indicates how much of the light hitting an object at that location is detected scattering beneath the object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_UNDEFINED	Specifies that the component stores information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

ComponentIdPtr

Specifies the address of the variable in which to write the buffer identifier. Since the **MbufAllocComponent**() function also returns the buffer identifier, you can set this parameter to **M_NULL**. If allocation fails, **M_NULL** is written as the identifier.

Return value

The returned value is the buffer identifier. If allocation fails, **M_NULL** is returned.

Remarks

- If you are creating a DLL that includes a call to **MbufAllocComponent**(), ensure that the call is not made

from the `DllMain()` function, because **MbufAllocComponent**() might load a required DLL and you cannot load a DLL from `DllMain()`. If necessary, call **MbufAllocComponent**() from an initialization function in your DLL instead.

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.
- *[Aurora Imaging Library Lite]*
  You can allocate a buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute with Aurora Imaging Library Lite. However, these attributes are not required to work under Aurora Imaging Library Lite.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufAllocContainer

Synopsis

Allocate a container.

Syntax

```
AIL_ID MbufAllocContainer(
    AIL_ID SystemId,          //in
    AIL_INT64 Attribute,      //in
    AIL_INT64 ControlFlag,    //in
    AIL_ID *ContainerBufIdPtr //out
)
```

Description

This function allocates a container on the specified system. A container is a data object that stores related buffers, referred to as the container's buffer components. By default, a container is empty. Components are automatically added to a container, when you grab into it or use it as a destination of a supported function. You can also add components to a container using MbufAllocComponent().

After allocating the container, you should check if the operation was successful, using MappGetError() or by verifying that the container identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the container is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the container's lifetime and you must not manually free it.

Parameters

SystemId INQ

Specifies the Aurora Imaging Library system on which to allocate the container. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

Attribute INQ

Specifies the container usage.

This attribute is one of the factors used to determine whether a container can be used for a specific purpose. This attribute will also be automatically applied to all image buffer components added to the container.

The values in the tables below can be added together to specify multiple attributes. For example, to allocate a container for both grabbing and display, specify M_GRAB + M_DISP.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the container usage																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DISP	Specifies that image buffer components of the container can be displayed, and that the container can become 3D-displayable. You can inquire whether a container is currently 3D-displayable using MbufInquireContainer() with M_3D_DISPLAYABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies that the image buffer components of the container can be used to grab data, and that the container itself can be used to grab data.				f	g	a						l	m	n	b
M_PROC	Specifies that image buffer components of the container can be processed, and that the container can become 3D-processable. You can inquire whether a container is currently 3D-processable using MbufInquireContainer() with M_3D_PROCESSABLE for a point cloud container or with M_3D_PROCESSABLE_DEPTH_MAP for a depth map container.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContainerBufIdPtr

Specifies the address of the variable in which to write the container identifier or specifies the data type that the function should use to return the container identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated container ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated container ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the container (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated container. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the container identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufAllocDefault

Synopsis

Allocate an image buffer or a container (automatically determined by Aurora Imaging Library) with settings appropriate for grabbing using the specified digitizer.

Syntax

```
AIL_ID MbufAllocDefault(
    AIL_ID SystemId,               //in
    AIL_ID ReferenceDigId,         //in
    AIL_INT64 Attribute,           //in
    AIL_INT64 ControlFlag,         //in
    AIL_INT64 ControlValue,        //in
    AIL_ID *VarContainerOrBufIdPtr //out
)
```

Description

This function allocates an image buffer or a container (automatically determined by Aurora Imaging Library) on the specified system with settings appropriate for grabbing using the specified digitizer. For example, when the digitizer is associated with a 2D camera, typically an image buffer is allocated. If the digitizer is associated with a 3D sensor, typically a container is allocated. When an image buffer is allocated, any required attributes (such as buffer size) are set automatically.

After allocating the image buffer or container, you should check if the operation was successful, using MappGetError() or by verifying that the image buffer or container identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the image buffer or container is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the image buffer or container's lifetime and you must not manually free it.

Parameters

SystemId INQ

Specifies the Aurora Imaging Library system on which to allocate the image buffer or the container. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library system																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ReferenceDigId

Specifies the digitizer to use as a reference for determining the settings of the image buffer or container to

allocate.

Attribute INQ

Specifies the usage of the image buffer or container.

This attribute determines whether the buffer or container can be used for a specific purpose.

The values in the table below can be added together to specify multiple attributes. For example, to allocate an image buffer or container for both grabbing and display, specify M_GRAB + M_DISP.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer or container usage																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DISP	Specifies that the object is allocated with the display attribute. If an image buffer is allocated, this attribute specifies that the image buffer can be displayed. If a container is allocated, this attribute specifies that the container can become 3D-displayable. To learn how to make a container 3D-displayable, see the Preparing a container for display or processing section of Chapter 41: 3D Containers.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies that the object is allocated with the grab attribute. This attribute specifies that the image buffer or container can be used to grab data (for example, using MdigGrab().	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC	Specifies that the object is allocated with the processable attribute. If an image buffer is allocated, this attribute specifies that the image buffer can be processed. If you intend to use the image buffer as the source or destination buffer of a processing or analysis operation, you must allocate the image buffer with an M_PROC attribute. If a container is allocated, this attribute specifies that the container can become 3D-processable. To learn how to make a container 3D-processable, see the Preparing a container for display or processing section of Chapter 41: 3D Containers.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ControlValue

Reserved for future expansion and must be set to **M_DEFAULT**.

VarContainerOrBufIdPtr

Specifies the address of the variable in which to write the image buffer or container identifier or specifies the data type that the function should use to return the image buffer or container identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated image buffer or container ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated image buffer or container ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the image buffer or container (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the container identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated container. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the image buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated image buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the image buffer or container identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remark

- If you are creating a DLL that includes a call to **MbufAllocDefault**(), ensure that the call is not made from the `DllMain()` function, because **MbufAllocDefault**() might load a required DLL and you cannot load a DLL from `DllMain()`. If necessary, call **MbufAllocDefault**() from an initialization function in your DLL instead.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufBayer

Synopsis

Decode the color information of a single-band, Bayer color-encoded image.

Syntax

```
void MbufBayer(
    AIL_ID SrcImageBufId,  //in
    AIL_ID DestImageBufId, //out
    AIL_ID CoefOrExpId,    //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function converts a single-band, Bayer color-encoded image into a 1- or 3-band image. When using a single-band buffer as the destination, the destination will be filled with the Y-band obtained from the RGB to Y conversion when the Bayer image is decoded.

Optionally, you can specify to perform gamma correction and/or white balancing on the image if the appropriate exponents and coefficients are provided, respectively. First, gamma correction is applied to the raw RGB values of the Bayer color-encoded image. Second, white balancing is performed on the gamma corrected RGB values. Third, the demosaicing process converts the single-band, Bayer color-encoded image to a 1- or 3-band image. Finally, color correction is performed if M_COLOR_CORRECTION is specified. Color correction can only be used with the M_ADAPTIVE algorithm.

Note that you can use **MbufBayer**() to automatically calculate the appropriate white balancing coefficients. To do so, specify a Bayer color-encoded source image that in reality should be a uniform shade of gray that is not too saturated, and add M_WHITE_BALANCE_CALCULATE to the ControlFlag parameter. After converting the image and calculating the appropriate coefficients, the coefficients are written to the specified Aurora Imaging Library array buffer and the converted image is then corrected using these automatically calculated coefficients. Alternatively, you could fill an array with custom values, and then load these values (in order) into the Aurora Imaging Library array using MbufPut1d() or MbufPut2d(). For more information, see White balancing your Bayer images. Ensure that the image was grabbed under the same lighting conditions as subsequent source images.

> Note that you cannot calculate the white balance coefficients and specify an adaptive demosaicing in the same call to **MbufBayer**().

For more information on Bayer color-encoded images, the conversion, and the actual equations used to perform white balancing and gamma correction, see the Using images acquired with a Bayer color filter section of Chapter 23: Data buffers.

Note that if the scale of the image is changed (using MdigControl() with M_GRAB_SCALE...) to a value other than 1 prior to grabbing a Bayer image, the Bayer image will not be converted properly; some of the Bayer pattern is lost during the scaling process, rendering color recovery impossible.

[Zebra Iris GTX]

When using a digitizer that supports on-board Bayer conversion, the conversion of a single-band, Bayer color-encoded image into a 1- or 3-band image can be performed using either on-board buffers, or off-board (Host) buffers. When the operation is performed using both source and destination on-board image buffers, your digitizer must be able to perform the Bayer conversion on-board; otherwise, the operation will be performed on

the Host.

In addition, the Bayer operation can only be performed on-board if there is sufficient memory for the operation to complete (for example, to hold internal buffers); otherwise, the operation will be performed on the Host, and the results copied to your on-board destination buffer. This will result in a speed-decrease of the Bayer operation.

To force the Bayer operation to be performed on-board, use MdigControl(). For more information, refer to the Hardware-specific Notes and the Installation and Hardware Reference for your product. Note that, when using MdigControl(), gamma correction is not available.

Parameters

SrcImageBufId

Specifies the identifier of the source image buffer. The image buffer must be a 1- or 3-band buffer. To achieve maximum conversion performance, the source image buffer should be an unsigned 1-band image buffer. If you specify a 3-band buffer, only the first band will be used.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

The source buffer must be an 8- or 16-bit buffer.

DestImageBufId

Specifies the identifier of the destination image buffer. The results of the white balancing, gamma correction, and color correction are clipped, if necessary, to the maximum value of the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the destination buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies the destination buffer will not be used. Note that this can only be used when calculating white balance coefficients (using M_WHITE_BALANCE_CALCULATE). In this case, the white balance coefficients are calculated but the source image is not converted.	c		e	f	g	a		i		j	k	l	m	n	b

Destination image buffer identifier	Specifies the identifier of the destination image buffer. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. When you convert the image using the adaptive algorithm (specified using M_ADAPTIVE), it is recommended that you use a planar RGB buffer (any RGB color buffer format + M_PLANAR). The other destination formats mentioned below can be used, but require an extra conversion operation that increases processing time. The RGB result is saturated according to the source buffer's maximum value, set using MbufControl() with M_MAX. With YUV or 1-band destination buffers, the saturation is applied to the intermediate RGB result, before doing the color conversion. The destination buffer should be either an 8- or 16-bit monochrome, or a 3-band color buffer in one of the following formats: • M_BGR32 + M_PACKED. • M_YUV16_YUYV (equivalent to M_YUV16 + M_PACKED).	c		e	f	g	a		i		j	k	l	m	n	b
	System specific																
	On specific boards, the destination image can also be any RGB color buffer format + M_PLANAR.	c					a		i								

CoefOrExpId

Specifies whether white balancing and gamma correction are performed, and the coefficients and exponents used to perform these corrections, respectively.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For white balancing and gamma correction																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Performs no white balancing or gamma correction on the source image; only the conversion is performed.	c		e	f	g	a		i		j	k	l	m	n	b

Array buffer identifier	Specifies the Aurora Imaging Library array buffer which contains the coefficients and exponents to perform the corrections. The buffer must be allocated with MbufAlloc1d() or MbufAlloc2d(), as a single-band 32-bit floating-point buffer with an M_ARRAY attribute. The first 3 elements in the Aurora Imaging Library array buffer specify the coefficients to perform the white balancing on the gamma corrected RGB values (if gamma correction is performed at all). If you add M_WHITE_BALANCE_CALCULATE to the ControlFlag parameter, these three elements are overwritten with the coefficients that are automatically calculated. Note that white balancing coefficients must be positive. The next 3 elements, if any, specify the exponents to perform the gamma correction on the raw RGB values, respectively. Note that the gamma correction exponents must be positive, are typically in the range of 0.0 to 1.0, and are most commonly specified as 0.45.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific														
	The buffer must have either a 3x1 or 6x1 dimension. If a 3x1 Aurora Imaging Library array buffer is specified, no gamma correction is performed.			e	f	g	a		i						n	b

ControlFlag

Specifies the Bayer pattern to use in the conversion, and whether the white balancing coefficients will be calculated. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the Bayer pattern																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BAYER_BG +	Specifies the Bayer pattern where the top-left pixel is band and the next pixel is green. That is, pixel [0,0] is blue, pixel [1,0] is green and pixel [1,1] is red in the source image. B G G R	c		e	f	g	a		i		j	k	l	m	n	b
M_BAYER_GB +	Specifies the Bayer pattern where the top-left pixel is green and the next pixel is blue. That is, pixel [0,0] is green, pixel [1,0] is blue and pixel [0,1] is red in the source image. G B R G	c		e	f	g	a		i		j	k	l	m	n	b

M_BAYER_GR +	Specifies the Bayer pattern where the top-left pixel is green and the next pixel is red. That is, pixel [0,0] is green, pixel [1,0] is red and pixel [0,1] is blue in the source image. G R B G	c		e	f	g	a		i		j	k	l	m	n	b
M_BAYER_RG +	Specifies the Bayer pattern where the top-left pixel is red and the next pixel is green. That is, pixel [0,0] is red, pixel [1,0] is green and pixel [1,1] is blue in the source image. R G G B	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For the Bayer pattern.

You can add the following value to the above-mentioned values to set whether to automatically calculate the appropriate white balance coefficients.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For white balancing the source image																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_WHITE_BALANCE_CALCULATE	Determines the 3 coefficients required to white balance the source image, and writes these coefficients into the CoefOrExpId array. The image is then white balanced using the 3 calculated coefficients. If gamma exponents are specified, the image is first gamma corrected before the white balance coefficients are calculated. Note that to use M_WHITE_BALANCE_CALCULATE, the source image must be a uniform shade of gray that is not too saturated.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value is not supported on the FPGA, although precalculated coefficients can still be applied.				f						j	k	l	m		

Combination value for the values listed in For the Bayer pattern.

You can add the following value to the above-mentioned values to set whether to bit-shift the result.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For bit shifting the result																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_BAYER_BIT_SHIFT(AIL_INT NbBit)	Indicates to shift the result by the specified number of bits. Note that this operation is applied to the RGB values resulting from the demosaicing process (after the color conversion); with YUV or 1-band destination buffers, the shift is applied to the intermediate RGB result, before doing the color conversion. Signed buffers are considered as unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	NbBit This parameter specifies the number of bits by which to shift the result. You can set this parameter to the following:														
	0 <= *Value* < *Depth of source buffer* — Specifies the number of bits by which to shift the result.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For the Bayer pattern.

You can add one of the following values to the above-mentioned values to set whether to override the use of the bilinear interpolation demosaicing algorithm.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For overriding the bilinear interpolation demosaicing algorithm																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ADAPTIVE +	Specifies to use the adaptive demosaicing algorithm for the conversion process. Using this algorithm minimizes false colors and the zipper effect. > Note you cannot use this constant in combination with M_WHITE_BALANCE_CALCULATE.	c		e		g	a		i						n	b
M_ADAPTIVE_FAST	Specifies to use a fast version of the adaptive demosaicing algorithm for the conversion process; using this algorithm gives you a good compromise between speed of the default bilinear algorithm and M_ADAPTIVE quality. > Note you cannot use this constant in combination with M_WHITE_BALANCE_CALCULATE.	c		e		g	a		i						n	b

M_AVERAGE_2X2	Specifies to use a 2x2 neighborhood kernel to perform the demosaicing. By using this algorithm, the zipper effect is minimized, but the resulting image is shifted upwards and to the left by a small amount.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ADAPTIVE.

You can add the following value to the above-mentioned value to set whether to perform a color correction on the image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_ADAPTIVE																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COLOR_CORRECTION	Specifies that a color correction is performed to further eliminate false colors.	c		e		g	a		i						n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufChild1d

Synopsis

Allocate a 1D child data buffer within a specific region of the first row of a parent data buffer.

Syntax

```
AIL_ID MbufChild1d(
    AIL_ID ParentBufId, //in
    AIL_INT OffX,       //in
    AIL_INT SizeX,      //in
    AIL_ID *BufIdPtr    //out
)
```

Description

This function allocates a one-dimensional child data buffer within a region of the first row of the specified, previously allocated, parent data buffer. If the parent buffer is multi-band, this function allocates a multi-band child buffer; the child is allocated within the specified one-dimensional region of the first row in each band. To allocate a child in one specific band, use MbufChildColor2d() instead of **MbufChild1d**().

The child buffer is not allocated in its own memory space; it remains part of the parent buffer. Therefore, any modification to the child buffer affects the parent and vice versa. Note, a parent buffer can have several child buffers.

A child buffer is considered a data buffer in its own right, and can be used in the same circumstances as its parent buffer. A child buffer inherits its type and attributes from the parent buffer.

If a child buffer is allocated using a parent buffer with an ROI, set using MbufSetRegion(), the child buffer inherits the parent buffer's ROI. You can also set an ROI in a child buffer directly. Note that the ROI of the child buffer is only accessible using the child buffer's Aurora Imaging Library identifier.

After allocating the child buffer, you should check if the operation was successful, using MappGetError() or by verifying that the child buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the child buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the child buffer's lifetime and you must not manually free it.

Parameters

ParentBufId

Specifies the identifier of the parent buffer.

OffX

Specifies the offset of the child buffer relative to the parent buffer's top-left pixel. The offset must be within the width of the parent buffer.

SizeX

Specifies the width of the child buffer.

BufIdPtr

Specifies the address of the variable in which to write the child buffer identifier or specifies the data type that the function should use to return the child buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the child buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated child buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the child buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufChild2d

Synopsis

Allocate a child buffer within a specific region of a parent buffer.

Syntax

```
AIL_ID MbufChild2d(
    AIL_ID ParentBufId, //in
    AIL_INT OffX,       //in
    AIL_INT OffY,       //in
    AIL_INT SizeX,      //in
    AIL_INT SizeY,      //in
    AIL_ID *BufIdPtr    //out
)
```

Description

This function allocates a two-dimensional child buffer within a region of the specified, previously allocated data buffer. If the parent buffer is multi-band, this function allocates a multi-band child buffer; the child is allocated within the specified region in each band. To allocate a child region in one specific band, or specifically in all bands, use MbufChildColor2d() instead of **MbufChild2d**().

The child buffer is not allocated in its own memory space; it remains part of the parent buffer. Therefore, any modification to the child buffer affects the parent and vice versa. Note, a parent buffer can have several child buffers.

A child buffer is considered a data buffer in its own right, and can be used in the same circumstances as its parent buffer. A child buffer inherits its type and attributes from the parent buffer.

If a child buffer is allocated using a parent buffer with an ROI, set using MbufSetRegion(), the child buffer inherits the parent buffer's ROI. You can also set an ROI in a child buffer directly. Note that the ROI of the child buffer is only accessible using the child buffer's Aurora Imaging Library identifier.

After allocating the child buffer, you should check if the operation was successful, using MappGetError() or by verifying that the child buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the child buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the child buffer's lifetime and you must not manually free it.

Parameters

ParentBufId

Specifies the identifier of the parent buffer.

OffX

Specifies the horizontal pixel offset of the child buffer's top-left pixel, relative to the parent buffer's top-left pixel. The given offset must be within the width of the parent buffer.

OffY

Specifies the vertical pixel offset of the child buffer's top-left pixel, relative to the parent buffer's top-left pixel. The given offset must be within the height of the parent buffer.

SizeX

Specifies the width of the child buffer.

SizeY

Specifies the height of the child buffer.

BufIdPtr

Specifies the address of the variable in which to write the child buffer identifier or specifies the data type that the function should use to return the child buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the child buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated child buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the child buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufChildColor2d

Synopsis

Allocate a child data buffer within a parent buffer.

Syntax

```
AIL_ID MbufChildColor2d(
    AIL_ID ParentBufId, //in
    AIL_INT Band,       //in
    AIL_INT OffX,       //in
    AIL_INT OffY,       //in
    AIL_INT SizeX,      //in
    AIL_INT SizeY,      //in
    AIL_ID *BufIdPtr    //out
)
```

Description

This function allocates a child data buffer within the specified, previously allocated, parent data buffer. It selects a two-dimensional region in one or all of the bands of the parent data buffer and allocates the region as a child of that buffer.

The child buffer is not allocated in its own memory space; it remains part of the parent buffer. Therefore, any modification to the child buffer affects the parent and vice versa. Note, a parent buffer can have several child buffers.

A child buffer is considered a data buffer in its own right. It can be used in the same circumstances as its parent buffer. A child buffer inherits its type and attributes from the parent buffer.

If a child buffer is allocated using a parent buffer with an ROI, set using MbufSetRegion(), the child buffer inherits the parent buffer's ROI. You can also set an ROI in a child buffer directly. Note that the ROI of the child buffer is only accessible using the child buffer's Aurora Imaging Library identifier.

After allocating the child buffer, you should check if the operation was successful, using MappGetError() or by verifying that the child buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the child buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the child buffer's lifetime and you must not manually free it.

Parameters

ParentBufId

Specifies the identifier of the parent buffer.

The parent buffer cannot have an M_COMPRESS attribute unless the Band parameter is set to M_ALL_BANDS.

Band

Specifies the band of the parent data buffer from which to allocate the child data buffer. The specified band should be valid in the parent buffer.

The Band parameter can be set to one of the following values:

For specifying the band

Value	Description
M_ALL_BANDS	Specifies all bands (for multi-band parent buffers). That is, M_ALL_BANDS allocates a child buffer with the same number of bands as the parent buffer.
M_BLUE	Specifies the blue color band (for RGB parent buffers).
M_GREEN	Specifies the green color band (for RGB parent buffers).
M_HUE	Specifies the hue band (for HSL and HSV parent buffers).
M_LUMINANCE	Specifies the luminance band (for HSL parent buffers).
M_RED	Specifies the red color band (for RGB parent buffers).
M_SATURATION	Specifies the saturation band (for HSL and HSV parent buffers).
M_U	Specifies the U band (for YUV parent buffers). Note that the dimensions of the child buffer must not exceed the dimensions of the U band of the parent.
M_V	Specifies the V band (for YUV and HSV parent buffers). Note that the dimensions of the child buffer must not exceed the dimensions of the V band of the parent.
M_Y	Specifies the Y band (for YUV parent buffers).
0 <= *Value* < *#bands*	Specifies the index of the band to use. The relationship between index value and band for RGB, HSL, HSV, and YUV buffers is the following: 0 — Corresponds to the red band (for RGB parent buffers), the hue band (for HSL and HSV parent buffers), and the Y band (for YUV parent buffers). 1 — Corresponds to the green band (for RGB parent buffers), the saturation band (for HSL and HSV parent buffers), and the U band (for YUV parent buffers). 2 — Corresponds to the blue band (for RGB parent buffers), the luminance band (for HSL parent buffers), and the V band (for YUV and HSV parent buffers).

OffX

Specifies the horizontal offset of the child buffer, relative to the parent buffer's top-left pixel. The offset must be within the width of the parent buffer.

OffY

Specifies the vertical offset of the child buffer, relative to the parent buffer's top-left pixel. The offset must be within the height of the parent buffer.

SizeX

Specifies the width of the child buffer.

SizeY

Specifies the height of the child buffer.

BufIdPtr

Specifies the address of the variable in which to write the child buffer identifier or specifies the data type that the function should use to return the child buffer identifier.

For retrieving the identifier or specifying how to return it	
Value	Description
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the child buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated child buffer. If allocation fails, **M_NULL** is written as the identifier.

Return value

The returned value is the child buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufChildColor2dClip

Synopsis

Allocate a child data buffer within a parent buffer and, if necessary, perform a clipping operation.

Syntax

```
AIL_ID MbufChildColor2dClip(
    AIL_ID ParentBufId, //in
    AIL_INT Band,       //in
    AIL_INT OffX,       //in
    AIL_INT OffY,       //in
    AIL_INT SizeX,      //in
    AIL_INT SizeY,      //in
    AIL_INT *StatusPtr, //out
    AIL_ID *BufIdPtr    //out
)
```

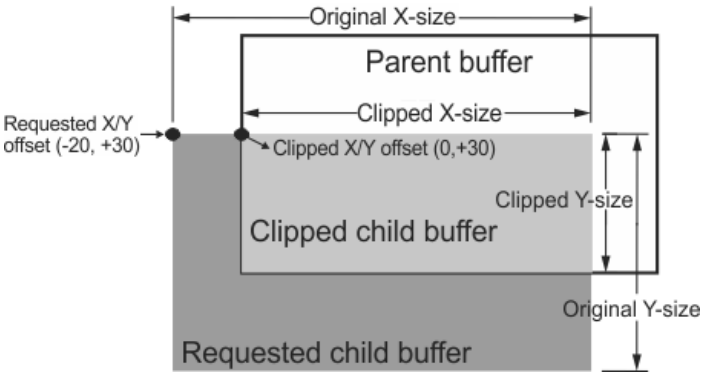
Description

This function allocates a child data buffer within the specified, previously allocated, parent data buffer. It selects a rectangular area in one or all of the bands of the parent data buffer and allocates this area as a child of that buffer. If necessary, the specified rectangular area will be adjusted (clipped) to ensure it fits within the limits of the parent buffer. If the specified rectangular area is completely outside the limits of the parent buffer, the allocation fails and an error is reported.

The child buffer is not allocated in its own memory space; it remains part of the parent buffer. Therefore, any modification to the child buffer affects the parent and vice versa. Note, a parent buffer can have several child buffers.

A child buffer is considered a data buffer in its own right. It can be used in the same circumstances as its parent buffer. A child buffer inherits its type and attributes from the parent buffer.

If the child buffer is larger or positioned outside the parent buffer, the child buffer is clipped and the clipping status is returned using the StatusPtr parameter. In the following example, both the X-offset and the X/Y-size are clipped, so M_CLIPPED_OFFSET_X + M_CLIPPED_SIZE_X + M_CLIPPED_SIZE_Y is returned as the clipping status. If no clipping was necessary, **M_NULL** is returned as the clipping status.



If a child buffer is allocated using a parent buffer with an ROI, set using MbufSetRegion(), the child buffer inherits the parent buffer's ROI. You can also set an ROI in a child buffer directly. Note that the ROI of the child buffer is only accessible using the child buffer's Aurora Imaging Library identifier.

After allocating the child buffer, you should check if the operation was successful, using MappGetError() or by verifying that the child buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the child buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the child buffer's lifetime and you must not manually free it.

Parameters

ParentBufId

Specifies the identifier of the parent buffer.

The parent buffer cannot have an M_COMPRESS attribute unless the Band parameter is set to M_ALL_BANDS.

Band

Specifies the band of the parent data buffer from which to allocate the child data buffer. The specified band should be valid in the parent buffer.

The Band parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the band																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL_BANDS	Specifies all bands (for multi-band parent buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_BLUE	Specifies the blue color band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_GREEN	Specifies the green color band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_HUE	Specifies the hue band (for HSL and HSV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_LUMINANCE	Specifies the luminance band (for HSL buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_RED	Specifies the red color band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Specifies the saturation band (for HSL and HSV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_U	Specifies the U band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_V	Specifies the V band (for YUV and HSV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_Y	Specifies the Y band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b

0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between index value and band for RGB, HSL, HSV, and YUV buffers is the following: 0 – Corresponds to the red band for RGB buffers, the hue band for HSL and HSV buffers, or the Y band for YUV buffers. 1 – Corresponds to the green band for RGB buffers, or the saturation band for HSL and HSV buffers, or the U band for YUV buffers. 2 – Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for YUV and HSV buffers.	c		e	f	g	a		i		j	k	l	m	n	b

OffX

Specifies the horizontal offset of the child buffer, relative to the parent buffer's top-left pixel. If necessary, the given offset will be adjusted (clipped) to make sure that the top-left pixel of the child buffer is within the limits of the parent buffer.

OffY

Specifies the vertical offset of the child buffer, relative to the parent buffer's top-left pixel. If necessary, the given offset will be adjusted (clipped) to make sure that the top-left pixel of the child buffer is within the limits of the parent buffer.

SizeX

Specifies the width of the child buffer. If necessary, the given dimension will be adjusted (clipped) to make sure that the bottom-right pixel of the child buffer is within the limits of the parent buffer.

SizeY

Specifies the height of the child buffer. If necessary, the given dimension will be adjusted (clipped) to make sure that the bottom-right pixel of the child buffer is within the limits of the parent buffer.

StatusPtr

Specifies the address of the variable in which the clipping status is written. If no status is required, set this parameter to M_NULL.

One or more of the following values can be returned together to denote that the child buffer was clipped. For example, when both the X-offset and the X-size are clipped, then M_CLIPPED_OFFSET_X + M_CLIPPED_SIZE_X is returned as the clipping status.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving clipping results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that no clipping was necessary.	c		e	f	g	a		i		j	k	l	m	n	b

M_CLIPPED_OFFSET_X	Specifies that the horizontal offset of the child buffer was clipped to fit within the width of the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLIPPED_OFFSET_Y	Specifies that the vertical offset of the child buffer was clipped to fit within the height of the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLIPPED_SIZE_X	Specifies that the width of the child buffer was clipped to fit within the width of the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLIPPED_SIZE_Y	Specifies that the height of the child buffer was clipped to fit within the height of the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b

BufIdPtr

Specifies the address of the variable in which to write the child buffer identifier or specifies the data type that the function should use to return the child buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the child buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated child buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

Return value

The returned value is the child buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufChildColor

Synopsis

Allocate a band child data buffer within a parent buffer.

Syntax

```
AIL_ID MbufChildColor(
    AIL_ID ParentBufId, //in
    AIL_INT Band,       //in
    AIL_ID *BufIdPtr    //out
)
```

Description

This function allocates a band or multi-band child data buffer within the specified, previously allocated, parent data buffer. It selects one or all of the bands of the parent data buffer and allocates the band or bands as a child of that buffer. The parent buffer can be a single band or multi-band buffer, and the child buffer can encompass the entire parent buffer.

The child buffer is not allocated in its own memory space; it remains part of the parent buffer. Therefore, any modification to the child buffer affects the parent and vice versa. Note, a parent buffer can have several child buffers.

The child buffer is considered a data buffer in its own right. It can be any band or all bands of its parent buffer, and can be used in the same circumstances as its parent buffer. A child buffer inherits its type and attributes from the parent buffer.

If a child buffer is allocated within a parent buffer which contains an ROI, set using MbufSetRegion(), the child buffer inherits a copy of the parent buffer's ROI. You can also set an ROI in a child buffer directly. Note that the ROI of the child buffer is only accessible using the child buffer's Aurora Imaging Library identifier.

After allocating the band child buffer, you should check if the operation was successful, using MappGetError() or by verifying that the band child buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the band child buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the band child buffer's lifetime and you must not manually free it.

Parameters

ParentBufId

Specifies the identifier of the parent buffer.

The parent buffer cannot have an M_COMPRESS attribute unless the Band parameter is set to M_ALL_BANDS.

Band

Specifies the band or bands of the parent data buffer from which to allocate the child data buffer. The specified band or bands should be valid in the parent buffer.

The Band parameter can be set to one of the values below.

For specifying the band

Value	Description
M_ALL_BANDS	Specifies all the bands of the parent buffer. It is useful to create a new full size buffer (a buffer with the same size as the parent, even if the parent is another child buffer) when you have more than one ROI or fixture and you want to add or disable them to the source buffer. > Note that you should typically select this value if the parent buffer is a monochrome buffer.
M_BLUE	Specifies the blue color band (for RGB parent buffers).
M_GREEN	Specifies the green color band (for RGB parent buffers).
M_HUE	Specifies the hue band (for HSL and HSV parent buffers).
M_LUMINANCE	Specifies the luminance band (for HSL parent buffers).
M_RED	Specifies the red color band (for RGB parent buffers).
M_SATURATION	Specifies the saturation band (for HSL and HSV parent buffers).
M_U	Specifies the U band (for YUV parent buffers).
M_V	Specifies the V band (for YUV and HSV parent buffers).
M_Y	Specifies the Y band (for YUV parent buffers).
0 <= *Value* < *#bands*	Specifies the index of the band to use. The relationship between index value and band for RGB, HSL, HSV, and YUV buffers is the following: 0 — Corresponds to the red band (for RGB parent buffers), the hue band (for HSL and HSV parent buffers), and the Y band (for YUV parent buffers). 1 — Corresponds to the green band (for RGB parent buffers), the saturation band (for HSL and HSV parent buffers), and the U band (for YUV parent buffers). 2 — Corresponds to the blue band (for RGB parent buffers), the luminance band (for HSL parent buffers), and the V band (for YUV and HSV parent buffers). > Note that this value must be set to zero if the parent buffer is a monochrome buffer.

BufIdPtr

Specifies the address of the variable in which to write the band child buffer identifier or specifies the data type that the function should use to return the band child buffer identifier.

For retrieving the identifier or specifying how to return it	
Value	Description
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated child buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the child buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.
Address for the image buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated child image buffer. If allocation fails, **M_NULL** is written as the identifier.
Address for the LUT buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated child LUT buffer. If allocation fails, **M_NULL** is written as the identifier.

Return value

The returned value is the band child buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufChildContainer

Synopsis

Allocate a child container.

Syntax

```
AIL_ID MbufChildContainer(
    AIL_ID ParentContainerBufId,                //in
    AIL_INT ComponentCriteriaSize,              //in
    const AIL_INT64 *ComponentCriteriaArrayPtr, //in
    AIL_INT64 ControlFlag,                      //in
    AIL_ID *ContainerBufIdPtr                   //out
)
```

Description

This function allocates a child container that behaves as though it contains components of the specified, previously allocated container. Which components are in the child container is determined dynamically by filtering the contents of the parent container using the specified criteria. Whenever the parent container is modified, Aurora Imaging Library determines the current contents of the child container.

For example, child containers are useful for working with 3D sensors that transmit multiple sets of components. If your 3D sensor transmits 3 range and confidence components with different group IDs in a single grab, you can create a child container of the grab container with the criteria M_COMPONENT_BY_GROUP_ID(); this allocates a child container that only has the range and confidence components of a single group. Each time you grab into the parent container, the child container will be updated with the components of that group (even if components were reallocated). You can pass this child container as a source to MbufConvert3d().

> You can create multiple child containers of a single parent container. You can also create a child container of a child container.

To allocate a child container, you must pass an array of criteria to ComponentCriteriaArrayPtr. If you set ControlFlag to M_ANY_CRITERIA, the child container will contain the components of the parent container which meet at least one of the criteria. If you set ControlFlag to M_ALL_CRITERIA, the child container will only contain the components of the parent container that meet all of the specified criteria. Once you have allocated the child container, you cannot change these criteria. To remove a component from the child container, you can either free the component from the parent container (using MbufFreeComponent()) or allocate a child container of the child container that has additional criteria. To add a component to the child container, you must allocate a new component for the parent container (using MbufAllocComponent()) that meets the existing criteria.

A component of a child container is not allocated in its own memory space; it remains a component of the parent container. Therefore, any modification to a component in the child container affects that component in the parent and vice versa.

A child container is considered a container in its own right, and can be used in some of the same circumstances as its parent container. You cannot directly allocate or free components of a child container. A child container therefore cannot be passed as a destination for a grab, copy, or processing function; it can, however, be passed as a source to relevant functions.

A child container inherits its attributes (such as M_DISP) from the parent container.

After allocating the child container, you should check if the operation was successful, using MappGetError() or by verifying that the child container identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was

specified).

When the child container is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the child container's lifetime and you must not manually free it.

Parameters

ParentContainerBufId INQ

Specifies the identifier of the parent container.

ComponentCriteriaSize

Specifies the number of elements in the array of criteria that will be applied to the parent container.

ComponentCriteriaArrayPtr
Accepts the address of one of the following:

- array of type AIL_INT64 [optionally, in C++: a reference to a constant std::vector<AIL_INT64>] (and a maximum array rank for C# of 1)
  Required array size:(ComponentCriteriaSize)

Specifies the address of an array of criteria that will be used to determine which components from the parent container to make part of the child container.

These settings can be used to include a single component that can be uniquely identified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which component to include in the child container using a uniquely identifying criterion																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_BY_ID(AIL_ID ID)	Specifies the component with the specified Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ID Specifies the Aurora Imaging Library identifier of the component.															
M_COMPONENT_BY_INDEX(AIL_INT ComponentIndex)	Specifies the component with the specified index in the container.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ComponentIndex Specifies the index of the component.															

These settings can be used to include one or more components that can be identified by a generalized criterion.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which components to include in the child container using a generalized criterion

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_BY_GROUP_ID(AIL_INT GroupIndex)	Specifies that the component(s) have the specified group ID. The group ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_GROUP_ID.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	GroupIndex Specifies the group ID.															
M_COMPONENT_BY_REGION_ID(AIL_INT RegionID)	Specifies that the component(s) have the specified region ID. The region ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_REGION_ID.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	RegionID Specifies the region ID.															
M_COMPONENT_BY_SOURCE_ID(AIL_INT SourceID)	Specifies that the component(s) have the specified source ID. The source ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_SOURCE_ID.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	SourceID Specifies the source ID.															

These settings can be used to include one or more components with the specified component type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the component type selection criteria																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_CONFIDENCE	Specifies the components that store confidence information for an M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies the components that store the A coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies the components that store the B coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_CUSTOM + *n*	Specifies the components that have a custom component type , identified by *n*, where n can be a value between 0 and 254. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_DISPARITY	Specifies the components that store a disparity map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INFRARED	Specifies the components that store an intensity image of infrared light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INTENSITY	Specifies the components that store an intensity image of visible light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MATRIX_AIL	Specifies the components that store a 4x4 matrix that transforms depth map coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MESH_AIL	Specifies the components that store mesh information for an M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_METADATA	Specifies the components that store metadata information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MULTISPECTRAL	Specifies the components that store an intensity image where each band represents the intensity of a specific wavelength of light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies the components that store normals information for each point in the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies the components that store 3D distance/position information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REFLECTANCE	Specifies the components that store a reflectance map. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_REGION_AIL	Specifies the components that store a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SCATTER	Specifies the components that store a scatter map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ULTRAVIOLET	Specifies the components that store an intensity image of ultraviolet light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_UNDEFINED	Specifies the components that store information of an unknown type. INFO	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies whether to have the child container contain the components that meet at least one of the criteria, or only the components that meet all of the criteria.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether the child container should contain components that meet any or all of the criteria																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL_CRITERIA	Specifies that the child container will only contain the components of the parent container which meet all of the specified criteria.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY_CRITERIA	Specifies that the child container will contain the components of the parent container which meet at least one of the specified criteria.	c		e	f	g	a		i		j	k	l	m	n	b

ContainerBufIdPtr

Specifies the address of the variable in which to write the child container identifier or specifies the data type that the function should use to return the child container identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated child container ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated child container ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the child container (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated child container. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the child container identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufChildMove

Synopsis

Move and resize a child buffer within the parent buffer.

Syntax

```
void MbufChildMove(
    AIL_ID BufId,         //out
    AIL_INT OffsetX,      //in
    AIL_INT OffsetY,      //in
    AIL_INT SizeX,        //in
    AIL_INT SizeY,        //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function moves and resizes a child buffer within the parent buffer. If the specified new offset or size causes the child buffer to be partially outside of the parent buffer, the child buffer can be clipped to the parent buffer (using M_CLIP).

Parameters

BufId

Specifies the identifier of the child buffer to move and/or resize. This buffer should be allocated using MbufChild...().

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

OffsetX

Specifies the new X-offset of the child buffer relative to the parent buffer's top-left pixel.

For the X-offset	
Value	Description
M_DEFAULT	Specifies the default value. The default is the current X-offset of the child buffer.
Value	Specifies the X-offset, in pixels.

OffsetY

Specifies the new Y-offset of the child buffer relative to the parent's top-left pixel.

For the Y-offset	
Value	Description
M_DEFAULT	Specifies the default value. The default is the current Y-offset of the child buffer.

Value	Specifies the Y-offset, in pixels.

SizeX

Specifies the new width of the child buffer.

For the width	
Value	Description
M_DEFAULT	Specifies the default value. The default is the current width of the child buffer.
Value > 0	Specifies the width, in pixels.

SizeY

Specifies the new height of the child buffer.

For the height	
Value	Description
M_DEFAULT	Specifies the default value. The default is the current height of the child buffer.
Value > 0	Specifies the height, in pixels.

ControlFlag

Specifies the function's control flag. This parameter must be set to one of the following:

For specifying the function's control flag	
Value	Description
M_DEFAULT	Specifies to use the values passed to the offset and size parameters as is, when moving the child buffer. Note that, if you try to move the child buffer outside the limits of the parent buffer, an error is reported.
M_CLIP	Specifies that the values passed to the offset and size parameters are automatically adjusted (if necessary) to ensure that the child buffer stays within the limits of the parent buffer. Note that, if you try to move the child buffer completely outside the limits of the parent buffer, an error is reported.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufClear

Synopsis

Clears a buffer, or the components of a container, to a specified color.

Syntax

```
void MbufClear(
    AIL_ID DstContainerOrBufId, //out
    AIL_DOUBLE Color            //in
)
```

Description

This function clears the specified buffer, or the components of the specified container, to the specified color.

To clear a 16-bit or 32-bit color image buffer to a color value, use MgraClear().

For an image buffer, the clear can be limited to a region of interest (ROI) set using MbufSetRegion().

Using this function with a container is equivalent to using it individually with each component of the container. An error will be generated if the container has a component that cannot be cleared (for example, because it has an ROI defined only in a vector format).

Using this function with a buffer always results in an uncalibrated image. Any previously associated camera calibration context is removed. You can use MgraClear() to clear a buffer while preserving the camera calibration context associated with the empty image.

Parameters

DstContainerOrBufId

Specifies the identifier of the buffer or container to clear. If you specify an image buffer, it can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

Color

Specifies the color value with which to clear the buffer.

For a 3-band image buffer, this parameter can be set to one of the following color values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a color value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies an RGB value used to clear an 8-bit 3-band buffer to an RGB color. Each value (red, green or blue) must not exceed 8 bits.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

		Red Specifies the red band, as a value between 0 and 255.														
		Green Specifies the green band, as a value between 0 and 255.														
		Blue Specifies the blue band, as a value between 0 and 255.														
M_COLOR_BLACK		Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE		Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY		Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN		Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE		Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN		Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN		Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA		Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED		Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW		Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY		Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN		Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE		Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY		Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN		Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE		Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_MAGENTA	Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b

For a 1-band or multi-band buffer, you can set this parameter to the following grayscale value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a grayscale value to be used with all bands																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value	Specifies a grayscale value used to clear the 1-band or multi-band buffer. This value is cast to the buffer type and depth. If the destination buffer is multi-band, this value will be replicated in each band.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufClearCond

Synopsis

Conditionally clears a buffer to a specified color.

Syntax

```
void MbufClearCond(
    AIL_ID DstBufId,         //out
    AIL_DOUBLE RedOrMonoVal, //in
    AIL_DOUBLE GreenVal,     //in
    AIL_DOUBLE BlueVal,      //in
    AIL_ID CondBufId,        //in
    AIL_INT64 Condition,     //in
    AIL_DOUBLE CondValue     //in
)
```

Description

This function clears a buffer to the specified color (the clear color), modifying only those entries of the buffer which have a corresponding entry in the condition buffer that satisfies the specified condition. Other entries are unchanged.

The clear color is specified by RedOrMonoVal, GreenVal, and BlueVal. If the buffer is one-band, only RedOrMonoVal is used. The clear color is treated as if it had the same type and depth as the buffer.

> Note that if a 1-band condition buffer is used with a 3-band buffer, the one band of the condition buffer will be used for each band in the buffer to clear.

For an image buffer, the clear can be limited to a region of interest (ROI) set using MbufSetRegion().

Parameters

DstBufId

Specifies the identifier of the data buffer to conditionally clear. The data buffer must be a 1- or 3-band image, array, or LUT buffer, or 1-band kernel or structuring element buffer. If you specify an image buffer, it can have an ROI set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).

RedOrMonoVal

Specifies the red band of the clear color, or, if DstBufId is a single band, this value is used as the clear color.

For specifying the red or mono band	
Value	Description
Value	Specifies the red band of the clear color, or the entire clear color.

GreenVal

Specifies the green band of the clear color.

For specifying the green band

Value	Description
M_NULL	Specifies that DstBufId is a single band, and that this parameter is ignored.
Value	Specifies the green band of the clear color.

BlueVal

Specifies the blue band of the clear color.

For specifying the blue band	
Value	Description
M_NULL	Specifies that DstBufId is a single band, and that this parameter is ignored.
Value	Specifies the blue band of the clear color.

CondBufId

Specifies the identifier of the condition buffer. The condition buffer must be a 1- or 3-band image, array, or LUT buffer, or 1-band kernel or structuring element buffer. If you specify an image buffer, it can have a region of interest (ROI) associated with it, but it must be a raster region. Using an image buffer with an other type of ROI will cause an error.

Condition

Specifies the condition for which the condition buffer is tested. This parameter can be set to one of the following:

For specifying the condition for which the buffer is tested	
Value	Description
M_DEFAULT	Modify destination buffer entries corresponding to condition buffer entries that are non-zero.
M_EQUAL	Modify destination buffer entries corresponding to condition buffer entries that are equal to CondValue.
M_NOT_EQUAL	Modify destination buffer entries corresponding to condition buffer entries that are not equal to CondValue.

CondValue

Specifies the color value for the specified condition. Even though this value is of type *AIL_DOUBLE*, it is treated as if it had the same type and depth as the condition buffer. If the Condition parameter is set to M_DEFAULT, CondValue is ignored.

For a 3-band condition buffer, this parameter can be set to one of the following color values:

For specifying a color value	
Value	Description

M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies an RGB value. This is only available when CondBufId is a 3-band buffer, and both DstBufId and CondBufId are 8-bit buffers. This allows you to compare each band of the condition buffer against a different value.	
		Parameters
		Red Specifies the red band, as a value between 0 and 255.
		Green Specifies the green band, as a value between 0 and 255.
		Blue Specifies the blue band, as a value between 0 and 255.
M_COLOR_BLACK	Specifies the color black.	
M_COLOR_BLUE	Specifies the color blue.	
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	
M_COLOR_CYAN	Specifies the color cyan.	
M_COLOR_DARK_BLUE	Specifies the color dark blue.	
M_COLOR_DARK_CYAN	Specifies the color dark cyan.	
M_COLOR_DARK_GREEN	Specifies the color dark green.	
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.	
M_COLOR_DARK_RED	Specifies the color dark red.	
M_COLOR_DARK_YELLOW	Specifies the color dark yellow.	
M_COLOR_GRAY	Specifies the color gray.	
M_COLOR_GREEN	Specifies the color green.	
M_COLOR_LIGHT_BLUE	Specifies the color light blue.	
M_COLOR_LIGHT_GRAY	Specifies the color light gray.	

M_COLOR_LIGHT_GREEN	Specifies the color light green.
M_COLOR_LIGHT_WHITE	Specifies the color light white.
M_COLOR_MAGENTA	Specifies the color magenta.
M_COLOR_RED	Specifies the color red.
M_COLOR_WHITE	Specifies the color white.
M_COLOR_YELLOW	Specifies the color yellow.

For a 1-band or 3-band condition buffer, you can set this parameter to the following grayscale value:

For specifying a grayscale value to be used with all bands	
Value	Description
Value	Specifies an entry value. If the condition buffer is binary, this value must be set to 0 or 1. If the condition buffer is 3-band, this value will be used for each band individually.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufClone

Synopsis

Clones a data buffer or container.

Syntax

```
AIL_ID MbufClone(
    AIL_ID SrcContainerOrBufId,    //in
    AIL_ID SysId,                  //in
    AIL_INT SizeX,                 //in
    AIL_INT SizeY,                 //in
    AIL_INT Type,                  //in
    AIL_INT64 Attribute,           //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *VarContainerOrBufIdPtr //out
)
```

Description

This function allocates a buffer or container that is similar to the source buffer or container. If the source is a buffer, the clone is allocated with the same size, number of bands, type, and attributes as the source, unless you specify that certain characteristics of the source buffer be different for the clone (for example, its size). If the source is a container, the clone is allocated with the same attributes as the source and components that are clones of the components in the source.

If you clone a child buffer, the clone will be similar to the source child, but it will not be a child itself. A clone of a band child of a packed buffer will be a one band planar equivalent.

If requested, you can also copy the source buffer or container's data to the clone (using M_COPY_SOURCE_DATA).

After cloning the buffer, you should check if the operation was successful, using MappGetError() or by verifying that the buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the cloned buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the cloned buffer's lifetime and you must not manually free it.

Parameters

SrcContainerOrBufId

Specifies the identifier of the source buffer or container to clone.

If the source is an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

SysId

Specifies the system on which to allocate the clone.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the clone is allocated on the same system as the source buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

SizeX

Specifies the width of the clone. If the source is a container, this parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the width of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the width of the clone is the same as the source buffer, or that the destination is a container.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the width of the clone buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SizeY

Specifies the height of the clone. If the source is a container, this parameter must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the height of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the height of the clone is the same as the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 1	Specifies the height of the clone buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Type

Reserved for future expansion and must be set to **M_DEFAULT**.

Attribute

Specifies the attributes of the clone. For both buffers and containers, you can specify that the clone will retain the same attributes as the source by setting this parameter to M_DEFAULT.

For containers, you can specify that the clone will have different attributes than the source by setting this parameter to M_DISP, M_GRAB, and/or M_PROC.

For buffers, you can set this parameter to the following values. The intended usage attribute (such as M_IMAGE) of the clone must match the intended usage attribute of the source. The other attributes (such as M_GRAB) can be set to any combination supported for the data type of the source buffer and the system specified by SysId.

> Note that some settings are only available for buffers with a specific usage attribute, data type, and/or number of bands. An error is generated if you specify attributes that are incompatible with the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the clone attributes																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to allocate the clone with the same attributes as the source.	c		e	f	g	a		i		j	k	l	m	n	b
M_ARRAY +	Specifies a buffer to store array type data. Note that some functions take an M_ARRAY buffer rather than a user-defined array.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data. You must specify a combination value from the following table: • To set the intended purpose of the image buffer or container	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a kernel buffer to store a custom filter for convolution functions. The depth must be 8, 16, or 32-bit integer or floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT +	Specifies a buffer to store lookup table data. The valid data depths are 8, 16, or 32 bits and only the planar internal storage format is valid.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a buffer to store structuring element data for morphology functions. The data depth must be 32-bit. If signed, the range is -32767 to +32767. If unsigned, the range is 0 to +32767. Note that the data can be set to **M_DONT_CARE** to ignore specific structuring elements.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values that can be used alone or as combination values for M_IMAGE.

You must use one or more of the following values in combination with each other or with the above-mentioned value, to set the intended purpose of the image buffer or container.

Note that you can only use these values alone when the source is a container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer or container																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DISP	Specifies an image buffer or container intended for display.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies an image buffer or container in which to grab data. Only 1-band and 3-band buffers can have an M_GRAB attribute.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This setting is not supported for containers.	c			f				i		j	k	l	m		
	Single-band (monochrome) buffers must have a depth of 8 bits or 16 bits to have an M_GRAB attribute. Only some three-band (color) buffers can have an M_GRAB attribute. See the constants for color buffer formats to determine which are supported with an M_GRAB attribute.			e	f	g					j	k	l	m	n	b
	Single-band (monochrome) buffers must have a depth of 8 bits or 16 bits to have an M_GRAB attribute. Only some three-band (color) buffers can have an M_GRAB attribute. See the constants for color buffer formats to determine which are supported with an M_GRAB attribute. Note that color buffers with an M_GRAB attribute are only available for color versions of Zebra Iris GTX (such as 2000C).								i							
M_PROC +	Specifies an image buffer or container intended for processing.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_IMAGE.

You can add the following value to the above-mentioned value to set that the image buffer is compressed.

Note that only 1-band and 3-band buffers can have an M_COMPRESS attribute.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the image buffer is compressed																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESS +	Specifies an image buffer that can hold compressed data. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PROC.

You can add the following value to the above-mentioned value to set the allocation of an overscan region.

This overrides the MsysControl() M_ALLOCATION_OVERSCAN system control setting and forces the allocation of an overscan region. When this attribute is set, the setting of the M_ALLOCATION_OVERSCAN_SIZE system control determines the size of the overscan region and the M_ALLOCATION_OVERSCAN setting is ignored.

Note that this attribute is only useful when added to a buffer on which you want to perform neighborhood

processing operations.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For allocating an overscan region																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOCATION_OVERSCAN	Specifies that the buffer is allocated with an overscan region. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_H264	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold H.264 data. This compression type is used when specifying a source of an input or a destination of an output for a sequence operation as an array of buffers (MseqDefine() with M_BUFFER_LIST).	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24 + M_PLANAR, M_RGB48 + M_PLANAR, or M_YUV24 + M_PLANAR format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24, M_RGB48, M_YUV9, M_YUV12, M_YUV16 + M_PLANAR, or M_YUV24 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band data in M_RGB24 or M_RGB48 format.	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_RGB24, M_YUV24, M_YUV12, M_YUV9, M_YUV16 + M_PLANAR, or M_YUV16 + M_PACKED format. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_YUV16 + M_PACKED format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format and a location specifier.

Aurora Imaging Library automatically selects the most appropriate storage format according to the specified intended usage attribute. For general processing, Aurora Imaging Library will convert the data when the function requires a different format. If the default storage format is not appropriate and you want to avoid conversion during a time critical operation, you can add a storage format and a location specifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format and a location specifier																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIB	Forces the buffer to be a DIB buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GDI	Forces the buffer to be compatible with GDI. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LINUX_MXIMAGE	[This is only applicable to Linux] Forces the buffer to be an X11 Ximage.	c		e	f	g	a				j	k	l	m	n	b

Combination value for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add the following value to the above-mentioned values to specify that the buffer is FPGA accessible.

Note that the following value cannot be used with M_COMPRESS or values in the For specifying a storage format and a location specifier table.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the buffer is FPGA accessible																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FPGA_ACCESSIBLE +	Forces the buffer to be allocated in a bank of memory that is accessible from the Processing FPGA. The exact bank of on-board memory is determined by the other attributes used with this value. If combined with the M_HOST_MEMORY attribute, the memory allocated will be in non-paged Host memory.										j	k	l	m		

Combination values for M_IMAGE.

You can add one of the following values to the above-mentioned value to set a storage format for image buffers.

Note that the buffer must be a 3-band buffer. Also note that it might be slower to use buffers that have been forced with one of these attributes. Although there is no right or wrong storage format to use, certain operations are optimized for some formats.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format for image buffers																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_PLANAR.

You can add one of the following values to the above-mentioned value to set a planar color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB3	Specifies 3-bit color depth (RGB 1:1:1) planar pixels.	c					a		i		j	k	l	m		
M_YUV9	Specifies YUV9 planar pixels.	c			f		a		i		j	k	l	m		
M_YUV12	Specifies YUV12 planar pixels.	c			f		a		i		j	k	l	m		

M_YUV24	Specifies YUV24 planar pixels.	c			f		a		i		j	k	l	m		

Combination values for M_PACKED.

You can add one of the following values to the above-mentioned value to set a packed color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth packed pixels (BGRBGR).	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c		e	f	g	a				j	k	l	m	n	b
M_BGR32	Specifies 32-bit color depth packed pixels (BGRXBGRX). The most-significant byte is a "don't care" byte.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB15	Specifies 16-bit color depth packed pixels (XRGB 1:5:5:5). Note that when accessing an M_RGB15 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB16	Specifies 16-bit color depth packed pixels (RGB 5:6:5). Note that when accessing an M_RGB16 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. For more information, see the YUV buffers section of Chapter 23: Data buffers. This value can be used with grab (M_GRAB) buffers.			e	f	g									n	b
M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. For more information, see the YUV buffers section of Chapter 23: Data buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.	c					a				j	k	l	m		

		This value can be used with grab (M_GRAB) buffers if those buffers are also packed (M_PACKED).								i						

Combination values for the values listed in For specifying a storage format for image buffers.

You can add one of the following values to the above-mentioned values to set a packed or planar color buffer format. Alternatively, you can add a value from For specifying a packed color buffer format or from For specifying a planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed or planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB24	Specifies 24-bit color depth (RGB 8:8:8) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers.								i		j	k	l	m		
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g									n	b
M_RGB48	Specifies 48-bit color depth (RGB 16:16:16). packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g					j	k	l	m	n	b
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers if the buffer is allocated on the Host (M_HOST_MEMORY) and not on-board.										j	k	l	m		
M_RGB96	Specifies 96-bit color depth (RGB 32:32:32) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16	Specifies YUV16 (4:2:2) pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add one of the following values to the above-mentioned values to set a location for a buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location for a buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HOST_MEMORY +	Forces the buffer in Host memory. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF_BOARD	Ensures that the buffer is not in on-board memory.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	To ensure that an M_OFF_BOARD buffer can be used by the DMA transfer section of the board, the buffer must be allocated in non-paged memory (M_OFF_BOARD + M_NON_PAGED or M_HOST_MEMORY + M_NON_PAGED).								i		j	k	l	m		
M_ON_BOARD +	Forces the buffer in on-board memory.	c							i		j	k	l	m		
	System specific															
	By default, all on-board buffers are allocated in memory that is not mapped on the PCI bus. This unshared memory cannot be accessed by the Host or any system other than the one on which it was allocated. See combination values below (M_SHARED) to change this default behavior.										j	k	l	m		
	This is only available for grab buffers (M_GRAB).								i							
	This is only available for image buffers (M_IMAGE).	c											l	m		
	This is only available for image (M_IMAGE) and LUT buffers (M_LUT).										j	k				

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following values: M_HOST_MEMORY; M_ON_BOARD.

You can add the following value to the above-mentioned values to specify a memory bank to be used.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a memory bank to be used																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEMORY_BANK_*n*	Forces the buffer to be allocated in the specified memory bank.	c		e	f	g	a		i		j	k	l	m	n	b

Operating system specific

[This is only applicable to Windows]

Valid values for *n* are between 0 and 6, inclusive. However, not all memory banks are necessarily present on a given system. Typically, if there is more than 1 memory bank, it is NUMA-enabled. Note that, to ensure that you are accessing a NUMA-enabled memory bank, you must specify the M_HOST_MEMORY combination value.

To determine the actual number of memory banks available on the Aurora Imaging Library system, use MappInquireMp() with M_MEMORY_BANK_NUM and MappInquireMp() with M_MEMORY_BANK_AFFINITY_MASK. For more information, refer to the NUMA Support subsection of the Transparent multi-core use section of Chapter 66: Multi-processing, multi-core, and multi-threading.

Note that this value cannot be used with M_COMPRESS or M_NON_PAGED.

Also note that the combination of M_HOST_MEMORY with this value, in paged memory, is only useful when doing multi-core processing on a Host system.

[This is only applicable to Linux]

Note that this value is not available with the M_HOST_MEMORY combination value.

System specific

j k

Valid values for *n* are between 0 and 1, inclusive. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.

Memory bank number	Type of memory	Description
0	Acquisition (SDRAM)	This is the default memory bank for an on-board buffer created with the M_GRAB attribute.

l m

In this case, *n* must be set to 0. This is the default memory bank for an on-board buffer created with the M_GRAB attribute. Refer to your Hardware-specific Notes for more information about on-board memory. Note that, to assure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.

Combination value for the values listed in For specifying that the buffer is FPGA accessible; and for the following value: M_ON_BOARD.

You can add the following value to the above-mentioned values to set a location in a specific type of memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location in a specific type of memory																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SHARED	Specifies that the buffer is in shared processing memory. This memory is mapped on the PCI bus. Note that this combination value can only be added to M_ON_BOARD.										j	k	l	m		

Combination values for M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT.

You can add one of the following values to the above-mentioned values to set the attribute of the specified physical memory type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the attribute of the specified physical memory type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NON_PAGED	Forces the buffer in Aurora Imaging Library reserved, non-pageable memory, if possible. The maximum (total) number of grab (M_GRAB) buffers that can be allocated might be restricted by the total amount of Aurora Imaging Library-reserved, non-paged memory (specified at installation time or using the Aurora Imaging Configurator utility).	c		e	f	g	a		i		j	k	l	m	n	b
M_PAGED	Forces the buffer in pageable memory.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies the control settings for the cloning operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which information to copy																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to copy only the number of bands, type, and attributes of the source buffer (or components of the source container). The data, information dependent on the content of the buffer (such as M_MIN and M_MAX), and the camera calibration information associated with the source buffer are not copied.	c		e	f	g	a		i		j	k	l	m	n	b

M_COPY_SOURCE_DATA	Specifies to copy the number of bands, type, attributes, and data of the source buffer (or components of the source container), including the camera calibration information and information dependent on the content of the buffer (such as M_MIN and M_MAX).	c		e	f	g	a		i		j	k	l	m	n	b

VarContainerOrBufIdPtr

Specifies the address of the variable in which to write the buffer or container identifier or specifies the data type that the function should use to return the buffer or container identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer or container ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer or container ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer or container (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated buffer. If cloning fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the container identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated container. If cloning fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the buffer or container identifier (of the new, cloned buffer or container) either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remarks

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.
- *[Aurora Imaging Library Lite]*
  You can allocate a buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute with Aurora Imaging Library Lite. However, these attributes are not required to work under Aurora Imaging Library Lite.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufControl

Synopsis

Control a specified data buffer setting.

Syntax

```
void MbufControl(
    AIL_ID BufId,           //out
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function allows you to control a specified data buffer setting.

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MbufControl()** internally calls the *AIL_INT64* version of this function (MbufControlInt64()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MbufControlDouble()).

Parameters

BufId

Specifies the identifier of the buffer to control.

ControlType

Specifies the buffer setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the value needed for the setting.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For specifying general buffer settings
- For specifying image buffer settings
- For M_IMAGE + M_COMPRESS image buffers
- For controlling general neighborhood operation settings of an M_KERNEL or an M_STRUCT_ELEMENT data buffer
- For the neighborhood operation settings of M_KERNEL data buffers
- For the overscan operation settings of M_STRUCT_ELEMENT data buffers
- For specifying settings useful with buffers that are components
- For specifying settings useful with image buffers that store 3D data and are components

The following ControlType and corresponding ControlValue parameter settings are used to control general buffer settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying general buffer settings

ControlType	ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DC_ALLOC		[This is only applicable to Windows] Allocates a device context (DC) for drawing. Determine the DC handle (HDC) using MbufInquire() with the M_DC_HANDLE inquire type. When using this control type, the buffer should be internally stored in M_GDI format, and cannot be a child buffer. No Aurora Imaging Library operation can be done between a device context allocation and free, therefore you should use the device context for a short period of time.															
	M_DEFAULT	Implements the default behavior.			e	f	g	a								n	b
M_DC_FREE		[This is only applicable to Windows] Frees a device context (DC).			e	f	g	a								n	b
	M_DEFAULT	Implements the default behavior.			e	f	g	a								n	b
M_GC_FEATURE_BROWSER		[This is only applicable to Windows] Sets whether to open or close a dialog box that allows you to view and edit the GenTL buffer configuration information interactively. This window is referred to as the Feature Browser.			e												b
	M_DEFAULT	Same as M_OPEN.			e												b
	M_CLOSE	Closes Feature Browser.			e												b
	M_OPEN +	Opens Feature Browser. You must specify a combination value from the following table: • To set whether the Feature Browser should be synchronous or asynchronous			e												b
M_LOCK +		Locks the specified buffer to allow a memory pointer to obtain a valid buffer address. The address is valid while the buffer is locked, but becomes invalid after it is unlocked. The calling thread will be blocked if the buffer is already locked by a different thread. To unlock the specified buffer, use M_UNLOCK.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

M_MAX	Sets the expected maximum pixel value of the buffer. This information is used to optimize certain processing operations on the buffer and the display of the buffer. Note that this information is not validated against the content of the buffer and is not updated automatically. M_MIN and M_MAX should specify the expected range of pixel values in the buffer. Pixel values between M_MIN and M_MAX are remapped linearly to values between the minimum and maximum possible display values. Note that M_MIN and M_MAX cannot be used with child buffers; this information is obtained directly from the parent buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the expected maximum pixel value within the range of the buffer type. For example, for an 8-bit unsigned buffer, set this value to a value between 0 and 255, inclusive.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Sets the expected minimum pixel value of the buffer. This information is used to optimize certain processing operations on the buffer and the display of the buffer. Note that this information is not validated against the content of the buffer and is not updated automatically. M_MIN and M_MAX should specify the expected range of pixel values in the buffer. Pixel values between M_MIN and M_MAX are remapped linearly to values between the minimum and maximum possible display values. Note that M_MIN and M_MAX cannot be used with child buffers; this information is obtained directly from the parent buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the expected minimum pixel value within the range of the buffer type. For example, for an 8-bit unsigned buffer, set this value to a value between 0 and 255, inclusive.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_HOOK	Sets whether to run the user-defined functions hooked to the buffer's modification event upon the event. These user-defined functions are initially hooked to the buffer modification event using MbufHookFunction(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the user-defined functions should not be called. Note that the modification counter of the image buffer will still be incremented. To inquire the count, use MbufInquire() with M_MODIFICATION_COUNT.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ENABLE	Specifies that the user-defined functions should be called. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFIED		Signals Aurora Imaging Library that the buffer content was modified without using Aurora Imaging Library. This control must be used to ensure that Aurora Imaging Library updates its internal information on the buffer. For example, if a display buffer was modified outside Aurora Imaging Library, the display will not be updated until you use this control. Note, if only a certain region of the buffer was modified, it is more efficient to specify an appropriate child buffer as BufId. If a specific area of the buffer was modified without using Aurora Imaging Library, it is faster to use MbufControlArea() with the control type to signal Aurora Imaging Library to update the content of the specified area of the buffer. M_MODIFIED increments the buffer's modification counter. You can inquire about the counter's current value using MbufInquire() with M_MODIFICATION_COUNT.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_ALLOC		[This is only applicable to Linux] Allocates a Cairo surface for drawing. Determine the Cairo surface handle using MbufInquire() with the M_SURFACE_HANDLE inquire type. When using this control type, the buffer should be internally stored in M_GDI format, and cannot be a child buffer. No Aurora Imaging Library operation can be done between a Cairo surface allocation and free; therefore, you should use the device context for a short period of time.			e	f	g	a								n	b
	M_DEFAULT	Implements the default behavior.			e	f	g	a								n	b
M_SURFACE_FREE		[This is only applicable to Linux] Frees a Cairo surface.			e	f	g	a								n	b
	M_DEFAULT	Implements the default behavior.			e	f	g	a								n	b
M_UNLOCK		Unlocks the specified buffer. The buffer must have been previously locked with M_LOCK.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when ControlType=M_GC_FEATURE_BROWSER: M_OPEN].

You must add one of the following values to the above-mentioned value to set whether the Feature Browser should be synchronous or asynchronous.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether Feature Browser should be synchronous or asynchronous																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ASYNCHRONOUS	Specifies that this function returns immediately once Feature Browser window opens.			e												b
M_SYNCHRONOUS	Specifies that this function is blocked until Feature Browser window closes. This is the default value.			e												b

Combination values for M_LOCK.

You can add one or more of the following values to the above-mentioned value to set the buffer's read and write permission.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_LOCK (Read and write permission)																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_READ	Locks the buffer to be read only.	c		e	f	g	a		i		j	k	l	m	n	b
M_READ+M_WRITE	Locks the buffer to be read and write. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_WRITE	Locks the buffer to be write only.	c		e	f	g	a		i		j	k	l	m	n	b

For buffers with an M_IMAGE attribute, ControlType and ControlValue can also be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying image buffer settings																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_ASSOCIATED_LUT		Sets whether to associate or disassociate a LUT buffer with the specified image buffer. The image buffer must be a 1-band 8-bit or 16-bit buffer. INQ															
	M_DEFAULT	Specifies to remove the association between the LUT buffer and the image buffer.	c		e	f	g	a		i		j	k	l	m	n	b

LUT buffer identifier	Specifies the Aurora Imaging Library identifier of the LUT buffer to associate with the image buffer. If and when the image buffer is selected to a display, the required changes occur to produce the display effect of the LUT, unless the display is also associated with a LUT (MdispLut()). If associated with a LUT buffer, the image buffer cannot be selected to a display whose view mode is set to M_AUTO_SCALE or M_MULTI_BYTES.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISPLAY_BANDS	Sets the bands to use for display when the buffer is not a 1-band or 3-band buffer. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISPLAY_MONO_BAND(AIL_INT M)	Specifies to use 1 band for display.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	M Specifies the band to use, as a value between 0 and the number of bands in the buffer - 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISPLAY_RGB_BAND(AIL_INT R, AIL_INT G, AIL_INT B)	Specifies to use 3 bands for display.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters														
	R Specifies the red band to use, as a value between 0 and the number of bands in the buffer - 1.	c		e	f	g	a		i		j	k	l	m	n	b
	G Specifies the green band to use, as a value between 0 and the number of bands in the buffer - 1.	c		e	f	g	a		i		j	k	l	m	n	b
	B Specifies the blue band to use, as a value between 0 and the number of bands in the buffer - 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFORMATION_TYPE	Sets the GenDC part type of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_1D	Specifies to set the GenDC part type to 1D.	c		e	f	g	a		i		j	k	l	m	n	b

M_INFO_2D	Specifies to set the GenDC part type to 2D.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_2D_H264	Specifies to set the GenDC part type to H.264 data.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_2D_JPEG	Specifies to set the GenDC part type to JPEG data.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_2D_JPEG2000	Specifies to set the GenDC part type to JPEG 2000 data.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_GENICAM_CHUNK	Specifies to set the GenDC part type to GenICam chunk data.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_GENICAM_XML	Specifies to set the GenDC part type to GenICam XML data.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_USE	Sets whether the ROI associated with the image buffer is used with supported image processing operations. This setting is ignored if the buffer is not associated with an ROI.	c		e	f	g	a		i		j	k	l	m	n	b
M_IGNORE	Specifies that image processing functions will ignore the ROI associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE	Specifies that image processing functions will use the ROI associated with the buffer. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESOLUTION_X	Sets the X-resolution of the image buffer in pixels per inch (PPI). Typically, you would specify the X-resolution if you are exporting the image, using MbufExport(), for third-party software that makes use of the specified pixels per inch. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the X resolution in PPI.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESOLUTION_Y	Sets the Y-resolution of the image buffer in pixels per inch (PPI). Typically, you would specify the Y-resolution if you are exporting the image, using MbufExport(), for third-party software that makes use of the specified pixels per inch. INQ	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	Specifies the Y resolution in PPI.	c		e	f	g	a		i		j	k	l	m	n	b
M_YCBCR_RANGE	Sets whether the YUV buffer's pixel values are YCbCr encoded; this limits the range of the buffer's pixel values. The Y-band's pixel values are limited between 16 and 235. The Cb- and Cr-bands are limited to pixel values between 16 and 240. This control type is not supported for an RGB buffer. The buffer must be of an uncompressed packed YUV16 type; otherwise, an error is generated. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	When grabbing into the specified buffer from a source that transmits in a YCbCr format (for example, an SDI source), the data is not converted to match the encoding that you specify. Instead, this control type is automatically changed to match the encoding of the grabbed data.	c														
M_DISABLE	Specifies not to encode the YUV buffer's pixel values in YCbCr. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_YCBCR_HD	Specifies to encode the YUV buffer's pixel values using the high-definition YCbCr standard.	c		e	f	g	a		i		j	k	l	m	n	b
M_YCBCR_SD	Specifies to encode the YUV buffer's pixel values using the standard-definition YCbCr standard.	c		e	f	g	a		i		j	k	l	m	n	b
M_YCBCR_UHD	Specifies to encode the YUV buffer's pixel values using the ultra-high-definition YCbCr standard.	c		e	f	g	a		i		j	k	l	m	n	b

For buffers with an M_IMAGE + M_COMPRESS attribute, ControlType and ControlValue can also be set to one of the values below.

> Note that, if the buffer contains any data, setting one of these control types automatically deletes the data. This is because, for Aurora Imaging Library to decompress the buffer's data, it must know the control values that were used in the compression. If you change one of these controls, Aurora Imaging Library will be unable to decompress the data and the data is therefore irrelevant. Note that the control type M_TARGET_SIZE is the only control type that can be changed without affecting the data. For more information, see Chapter 30: JPEG and JPEG2000 compression.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS image buffers																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_DECOMPOSITION_LEVEL +		Sets the number of iterations (decomposition levels) the discrete wavelet transform is applied to the image. This control type is only supported with JPEG2000 buffers (both lossy and lossless). The number of decomposition levels is applied to all bands by default. INQ															
	Value >= 0	Specifies the number of iterations (decomposition levels). Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_AC		Associates an AC Huffman table to the buffer. This control type is only supported with JPEG lossy buffers. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer identifier	Specifies the identifier of the array buffer containing the table. If the buffer is 3-band, the same table is applied to all bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_AC_CHROMINANCE		Associates an AC Huffman table to the U and V bands (chrominance) of the buffer. This control type is only supported with JPEG lossy buffers in YUV format. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_AC_LUMINANCE		Associates an AC Huffman table to the Y band (luminance) of the buffer. This control type is only supported with JPEG lossy buffers in YUV format. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_DC		Associates a DC Huffman table to the buffer. This control type is only supported with JPEG buffers (both lossy and lossless). INQ	c		e	f	g	a		i		j	k	l	m	n	b

	Array buffer identifier	Specifies the identifier of the array buffer containing the table. If the buffer is 3-band, the same table is applied to all bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_DC_CHROMINANCE		Associates a DC Huffman table to the U and V bands (chrominance) of the buffer. This control type is only supported with JPEG lossy buffers in YUV format. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_DC_LUMINANCE		Associates a DC Huffman table to the Y band (luminance) of the buffer. This control type is only supported with JPEG lossy buffers in YUV format. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICTOR		Sets the type of predictor. This control type is only supported with JPEG lossless buffers. If the buffer is 3-band, the same predictor is applied to all bands. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	0	Specifies predictor #0 (no prediction).	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies predictor #1 (the "pixel-to-the-left" predictor). This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	2	Specifies predictor #2 (the "pixel-above" predictor).	c		e	f	g	a		i		j	k	l	m	n	b
M_Q_FACTOR +		Sets the quantization factor. This control type is only supported with lossy buffers. For JPEG lossy buffers, the Q factor is always applied to all bands. For JPEG2000 lossy buffers, the Q factor is applied to all bands by default. To change this default behavior, see the combination values below. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 99	Specifies the quantization factor. Only integer values are accepted. The higher the factor, the more the compression, but the lower the image quality. The default value is 50.	c		e	f	g	a		i		j	k	l	m	n	b

M_Q_FACTOR_CHROMINANCE	Sets the chrominance quantization factor. This control type is only supported with JPEG or JPEG2000 lossy buffers in YUV format. INQ	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 99	Specifies the factor. Only integer values are accepted. The higher the factor, the more the compression, but the lower the image quality. The factor is applied to the U and V bands (chrominance). The default value is 50.	c		e	f	g	a		i		j	k	l	m	n	b
M_Q_FACTOR_LUMINANCE	Sets the luminance quantization factor. This control type is only supported with JPEG or JPEG2000 lossy buffers in YUV format. INQ	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 99	Specifies the factor. Only integer values are accepted. The higher the factor, the more the compression, but the lower the image quality. The factor is applied only to the Y band (luminance). The default value is 50.	c		e	f	g	a		i		j	k	l	m	n	b
M_QUANTIZATION +	Associates a quantization table to the buffer. This control type is only supported with lossy buffers. For JPEG lossy buffers, this table is associated with all bands. For JPEG2000 lossy buffers, this table is associated with all bands by default. Note that setting this control type will reset the M_Q_... control types to their default value (50). If you set the M_Q_FACTOR control type after specifying a custom table with the M_QUANTIZATION control type, the custom table will be scaled accordingly. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b

M_QUANTIZATION_CHROMINANCE	Associates a quantization table to the U and V bands (chrominance) of the buffer. This control type is only supported with JPEG or JPEG2000 lossy buffers in YUV format. Note that setting this control type will reset the M_Q_FACTOR_CHROMINANCE control type to its default value (50). If you set an M_Q_... control type after specifying a custom table with the M_QUANTIZATION_CHROMINANCE control type, the custom table will be scaled accordingly. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_QUANTIZATION_LUMINANCE	Associates a quantization table to the Y band (luminance) of the buffer. This control type is only supported with JPEG or JPEG2000 lossy buffers in YUV format. Note that setting this control type will reset the M_Q_FACTOR_LUMINANCE control type to its default value (50). If you set an M_Q_... control type after specifying a custom table with the M_QUANTIZATION_LUMINANCE control type, the custom table will be scaled accordingly. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTART_INTERVAL	Sets the number of rows or 8x8 blocks between restart markers in a compressed image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies after how many rows (for JPEG lossless buffers) or 8x8 blocks (for JPEG lossy buffers) of data to place restart markers. Only integer values are accepted. The default value is 8 for JPEG lossy buffers and 2 for JPEG lossless buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_SIZE	Sets the size of the buffer into which the source image is compressed. This control type is only supported with JPEG2000 lossy buffers. To ensure that the compressed data fits into the specified buffer size, less-significant data is discarded. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0	Specifies the size, in bytes. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_DECOMPOSITION_LEVEL; M_Q_FACTOR; M_QUANTIZATION.

You can add one of the following values to the above-mentioned values to specify the color bands to affect.

When dealing with M_Q_FACTOR and M_QUANTIZATION, the following values are only supported with JPEG2000 lossy buffers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the color bands to affect																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL_BANDS	[For essential Aurora Imaging Library Lite information, see remarks] Applies the specified control value to all bands of the buffer. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_BLUE	[For essential Aurora Imaging Library Lite information, see remarks] Applies the specified control value to the blue band only (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_GREEN	[For essential Aurora Imaging Library Lite information, see remarks] Applies the specified control value to the green band only (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_RED	[For essential Aurora Imaging Library Lite information, see remarks] Applies the specified control value to the red band only (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_U	[For essential Aurora Imaging Library Lite information, see remarks] Applies the specified control value to the U band only (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_V	[For essential Aurora Imaging Library Lite information, see remarks] Applies the specified control value to the V band only (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_Y	[For essential Aurora Imaging Library Lite information, see remarks] Applies the specified control value to the Y band only (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b

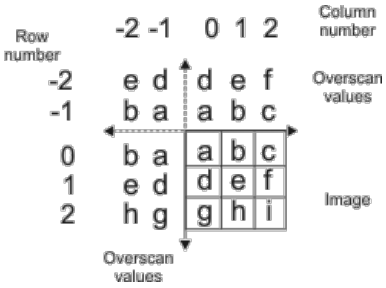
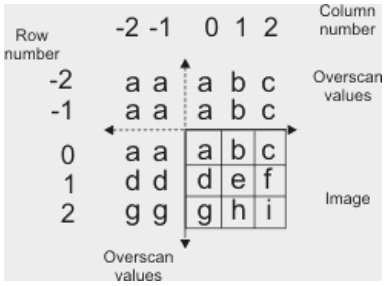
The following ControlType and corresponding ControlValue parameter settings can be specified for an

M_KERNEL or an M_STRUCT_ELEMENT data buffer.

The following ControlType values change the setting of a neighborhood operation for the specified kernel buffer or structuring element buffer. The ControlType values establish how to perform a neighborhood operation when using the specified kernel buffer or structuring element buffer. Neighborhood operations that do not use a kernel buffer or structuring element buffer typically use the default values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling general neighborhood operation settings of an M_KERNEL or an M_STRUCT_ELEMENT data buffer																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_DEFAULT		Sets all neighborhood operation control types to their default value.															
	M_NULL	Implements the default behavior. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_CENTER_X		Sets the X-coordinate of the center pixel of the kernel or structuring element. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the X-coordinate of the top-left pixel of the central elements.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < *SizeX*	Specifies the value of the X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_CENTER_Y		Sets the Y-coordinate of the center of the kernel or structuring element. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the Y-coordinate of the top-left pixel of the central elements.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < *SizeY*	Specifies the value of the Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN		Sets the type of overscan used to handle the border pixels of a source image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies that Aurora Imaging Library automatically selects the type of overscan to optimize speed and logic according to the specified operation and the target system.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that no overscan will be used, unless processing the border pixels is faster than ignoring them; in the latter case, Aurora Imaging Library automatically selects the overscan to optimize speed according to the specified operation and the target system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAST	Specifies that Aurora Imaging Library automatically selects the overscan to optimize speed according to the specified neighborhood operation and the target system. The overscan could be hardware-specific thereby having a different behavior than the other supported overscan modes. Note that when using M_FAST, the destination pixels in the overscan area are undefined. The pixels can therefore contain different values from one function call to the next, even if the function's parameter values are the same.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIRROR	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values that mirror the source buffer pixel values. That is, the overscan pixel values will be a mirror copy of the source buffer's borders. For example: 	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLACE	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values set to the overscan replacement value (M_OVERSCAN_REPLACE_VALUE).	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLICATE	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values that replicate the border pixels. For example: 	c		e	f	g	a		i		j	k	l	m	n	b

	M_TRANSPARENT	Specifies a type of overscan that processes the border pixels of a source image using transparent overscan pixel values. The overscan pixel values will be those of the ancestor buffer, and a mirror is done on the ancestor for any missing pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_REPLACE_VALUE		Sets a replacement value for the overscan pixel values. Note that to use this control type, M_OVERSCAN must be set to M_REPLACE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLACE_MAX	Specifies that the overscan neighborhood pixel values will be set to the maximum value of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLACE_MIN	Specifies that the overscan neighborhood pixel values will be set to the minimum value of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the value of the overscan neighborhood pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION		Sets whether to saturate the results. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to saturate results, except when Aurora Imaging Library can take advantage of optimization routines to accelerate the processing. In the latter case, results will be saturated and processing will be done in hardware. When it is not possible to run hardware processing optimization routines, results that overflow the buffer are undefined.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to saturate results. A result that overflows or underflows will be set to the maximum or minimum value (respectively) that can be represented in the destination buffer. If the saturation, normalization, and absolute value settings are specified for a kernel buffer, the saturation is performed after the normalization factor and the absolute value operations have been applied.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified for an M_KERNEL data buffer. These settings establish how to perform a neighborhood operation when using the specified kernel buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the neighborhood operation settings of M_KERNEL data buffers																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_ABSOLUTE_VALUE		Sets whether to take the absolute value of the results. INQ															
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to take the absolute value of the result.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to take the absolute value of the result. Note that for a structuring element buffer, you cannot take the absolute value of a result.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_FACTOR		Sets the normalization factor to apply to the result. Note that for a structuring element buffer, you cannot specify a normalization factor. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the normalization factor. > If the factor produces an overflow in the destination, saturation might occur, depending on the M_SATURATION control type.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings can be specified for an M_STRUCT_ELEMENT data buffer. These settings are useful for overscan operations with iterations when using the specified structuring element.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the overscan operation settings of M_STRUCT_ELEMENT data buffers																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_ITER_OVERSCAN		Sets the how to handle overscan operations with multiple iterations. INQ															

	M_DEFAULT	Same as M_PER_ITER.	c		e	f	g	a		i		j	k	l	m	n	b
	M_GLOBAL	Specifies that for overscan operations with multiple iterations, to use a size equivalent to all iterations.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PER_ITER	Specifies that for overscan operations with multiple iterations, each iteration will be according to the structuring element's size and center pixel.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control buffer settings that are primarily useful when working with a buffer that is a component of a container, or a buffer that will be copied to a component of a container using MbufCopyComponent().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying settings useful with buffers that are components																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_COMPONENT_TYPE		Sets the component type of the buffer, used when the buffer is a component of a container. The component type specifies the type of information stored in the component. INQ															

	M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. Coordinates associated with the confidence value 0 are considered invalid data and will not be used by 3D image processing functions. A confidence component is associated with a range or disparity component in the same container when there are no other range, disparity, or confidence components in the container. If there is more than one range, disparity, and/or confidence component in a container (for example, because a single grab into the container transmitted multiple range and confidence components), you will need to either create a child container which contains only the required components using MbufChildContainer(), or free the extra components using MbufFreeComponent().	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the column index of each corresponding pixel and will result in a better representation of the camera's lens model when generating X-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the row index of each corresponding pixel and will result in a better representation of the camera's lens model when generating Y-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 255. You can use custom component types to identify components which store information that does not fit one of the standard component types. In addition, some cameras transmit components with custom component types. Refer to your camera manual to determine what type of information is stored in these components.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. Each pixel of a disparity map indicates the apparent distance (typically measured in pixel units) between where an object appears in the left and right images captured by a stereoscopic camera.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. Only depth map containers can have a matrix component. A matrix component is associated with a range component in the same container when there is only one range component and no other matrix components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. A mesh component is associated with a range component in the same container when there are no other range or mesh components in that container. A point cloud container which has a mesh component is referred to as a meshed point cloud container.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_METADATA	Specifies that the component stores metadata information. Metadata components are used by Aurora Imaging Library internally. Typically, you should ignore metadata components in your application.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. Unlike an intensity component, a multispectral component might include information about non-visible light.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_NORMALS_AIL	Specifies that the buffer stores normals information for each point in the M_COMPONENT_RANGE component of the container. A normals component is associated with a range component in the same container when there are no other range or normals components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. The component can be either a 1-band buffer that stores a depth map, or a 3-band buffer that stores coordinates of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. Each pixel of a reflectance map indicates how much of the light hitting an object at that location is reflected back. Typically, this is an intensity image of the light wavelength used by a 3D sensor to detect 3D distance/position information. Typically, if the map was generated by a laser profiler, each line indicates the detected intensity of the laser for a single scan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. 3D image processing functions will only operate on points inside the ROI. Only depth map containers can have a region component. A region component is associated with a range component in the same container when there is only one range component and no other region components in that container.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. Each pixel of a scatter map indicates how much of the light hitting an object at that location is detected scattering beneath the object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_UNDEFINED	Specifies that the component contains information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

For buffers with an M_IMAGE attribute and with M_COMPONENT_TYPE set to M_COMPONENT_RANGE or M_COMPONENT_DISPARITY, ControlType and ControlValue can also be set to one of the values below. These settings are used only when the buffer is the range or disparity component of a container passed as a source to MbufConvert3d().

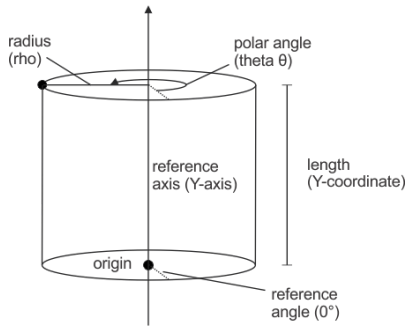
A container cannot be 3D-processable or 3D-displayable unless it has a single range component with all of these settings at their default values (except for M_3D_DISTANCE_UNIT which can be any value and M_3D_REPRESENTATION, which must be set to M_CALIBRATED_XYZ or M_CALIBRATED_XYZ_UNORGANIZED).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying settings useful with image buffers that store 3D data and are components																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_3D_ASPECT_RATIO		Sets by how much the focal length in the Y-direction stored in the buffer will be scaled in the destination point cloud to correct for non-square pixels in 3D reconstruction modes that rely on projective geometry. This is useful when the focal length parameters in the camera intrinsic matrix are not equal (in the X- and Y- directions). INQ															
	Value > 0.0	Specifies by how much the focal length the Y-direction will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_COORDINATE_SYSTEM_TYPE	Sets which type of coordinate system to use to interpret the coordinates stored in the bands of the buffer if it is a range component. Note that Aurora Imaging Library does not support any functionality with coordinates stored using a coordinate system type other than M_CARTESIAN. If a buffer stores information defined using another type of coordinate system, you will need to manually convert that information to cartesian coordinates before the buffer can be used with any Aurora Imaging Library processing function. This conversion cannot be done using MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_CARTESIAN	Specifies that the buffer stores right-handed cartesian coordinates. If the buffer is a 1-band buffer, its values will be interpreted as Z-coordinates (equivalent to the pixels of a depth map). If the buffer is a 3-band buffer, the bands store the following: Band: Information Band 0: Stores the X-coordinates. Band 1: Stores the Y-coordinates. Band 2: Stores the Z-coordinates. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

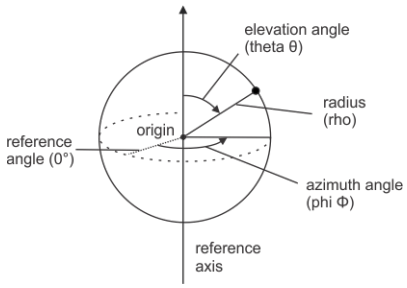
			c		e	f	g	a		i		j	k	l	m	n	b
	M_CYLINDRICAL	Specifies that the buffer stores cylindrical coordinates (theta-Y-rho). In a cylindrical coordinate system, points are defined relative to a reference axis and reference angle perpendicular to that axis. The Y-coordinate is the distance from the origin along the reference axis, theta (θ) is the polar angle of the point relative to a reference angle, and rho is the distance from that axis along the line described by theta and perpendicular to the reference axis (the radius).															



If the buffer is a 1-band buffer, its values store rho, the distance from a chosen reference axis.

If the buffer is a 3-band buffer, the bands store the following:

Band	Information
Band 0	Stores theta, the polar angle coordinates relative to the chosen reference angle.
Band 1	Stores Y-coordinates, the distances along the chosen reference axis.
Band 2	Stores rho, the distances from the chosen reference axis.

M_SPHERICAL	Specifies that the buffer stores spherical coordinates (theta-phi-rho). In a spherical coordinate system, points are defined relative to a reference axis and a reference angle perpendicular to that axis. Rho is the distance from the origin (the radius), theta (θ) is the elevation angle of the point relative to reference axis, and phi is the azimuthal angle relative to the reference angle.  If the buffer is a 1-band buffer, its values store rho, the distance from a chosen reference axis. If the buffer is a 3-band buffer, the bands store the following: Band: Information Band 0: Stores theta, the elevation angle coordinates relative to the chosen reference axis. Band 1: Stores phi, the azimuth angle coordinates relative to a chosen reference angle. Band 2: Stores rho, the distances from the chosen reference axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the coordinate system type is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPARITY_BASELINE	Sets the stereo baseline value of the stereoscopic camera used to generate the data in the buffer. This is the physical distance between the lenses of the camera. Refer to your camera manual to determine the correct value for this setting, which might differ from the true physical distance between the lenses of your camera. INQ	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	Specifies the stereo baseline value, expressed in meters. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISTANCE_UNIT	Sets the unit to use when the buffer is part of a container and stores natively calibrated distance data. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_INCH	Specifies that the distance data is provided in inches.	c		e	f	g	a		i		j	k	l	m	n	b
M_MILLIMETER	Specifies that the distance data is provided in millimeters.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that the distance data is provided in pixels. This setting is only useful for buffers that have the component type M_COMPONENT_DISPARITY and store a disparity map.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the distance unit is unknown. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_FOCAL_LENGTH	Sets the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer. Refer to your camera manual to determine the correct value for this setting, which might differ from the true focal length of your camera. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer, expressed in pixels. For example, if the buffer has 1000 pixels per millimeter of a single image sensor (after calibration), and the focal length of the lenses in millimeters is 35, the focal length in pixels is 35000. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_INVALID_DATA_FLAG	Sets whether the buffer uses a specific value to indicate invalid data. For 3-band buffers that store coordinates, the invalid data flag should be stored in the Z-axis band. Specify the value that indicates invalid data using M_3D_INVALID_DATA_VALUE. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_FALSE	Specifies that the buffer does not use a special value to indicate invalid data. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the buffer uses a special value to indicate invalid data.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_INVALID_DATA_VALUE	Sets the value used to indicate missing data when M_3D_INVALID_DATA_FLAG is set to M_TRUE. Note that this value is not used or respected by any Aurora Imaging Library function except for MbufConvert3d() when the buffer is a component of the source container. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the value used to indicate missing data. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_X	Sets by how much the X-coordinates stored in the buffer will be offset in the destination point cloud when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the X-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Y	Sets by how much the Y-coordinates stored in the buffer will be offset in the destination point cloud when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Y-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Z	Sets by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Z-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_PRINCIPAL_POINT_X	Sets the X-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies X-position of the principal point, expressed in pixels. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_PRINCIPAL_POINT_Y	Sets the Y-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies Y-position of the principal point, expressed in pixels. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_REPRESENTATION	Sets how 3D data is stored in the buffer; this information is used when the buffer is a range or disparity component of a container. For more information, see MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XYZ	Specifies that the component stores organized and natively calibrated X, Y, and Z-coordinates. This is the default value for 3-band buffers with M_SIZE_Y greater than 1. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedABC_Grid.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XYZ_UNORGANIZED	Specifies that the component stores unorganized and natively calibrated X, Y, and Z-coordinates. This is the default value for 3-band buffers with M_SIZE_Y of 1. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedABC_PointCloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XZ_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates stored in a separate array buffer. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedAC_Linescan.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CALIBRATED_XZ_UNIFORM_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates identified by the row index. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedAC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z	Specifies that the component stores organized and natively calibrated Z-coordinates, without X and Y-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates without X-coordinates; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedC_Linescan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_X_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates, with the X-coordinates identified by column index; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. This corresponds to the GenICam Scan3dOutputMode feature set to RectifiedC_Linescan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_XY	Specifies that the component stores organized and natively calibrated Z-coordinates, with X and Y-coordinates identified by column and row index respectively. The manufacturer of your camera might refer to a range component with this setting as a depth map. This is the default value for 1-band buffers. This corresponds to the GenICam Scan3dOutputMode feature set to RectifiedC.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISPARITY	Specifies that the component stores a disparity map with perspective distortion along the Y-axis. When used with MbufConvert3d(), the Y-coordinate of each point in the resulting point cloud will be generated using the row index of the corresponding pixel and compensating for perspective. This corresponds to the GenICam Scan3dOutputMode feature set to DisparityC. > Typically, this setting should only be used with disparity components that have been generated using area scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_EXTERNAL_Y	Specifies that the component stores a disparity map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. When used with MbufConvert3d(), the Y-coordinate of each point in the resulting point cloud will be taken from the specified buffer. This corresponds to the GenICam Scan3dOutputMode feature set to DisparityC_Linescan. > Typically, this setting should only be used with disparity components that have been generated using line scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_UNIFORM_Y	Specifies that the component stores a disparity map, with Y-values identified by row index. When used with MbufConvert3d(), the Y-coordinates of each point in the resulting point cloud will be generated using the row index of the corresponding pixel. Since the Y-values are uniform, no perspective correction is applied. This partially corresponds to the GenICam Scan3dOutputMode feature set to DisparityC_Linescan, except that the Y-values are not stored in a separate buffer. > Typically, this setting should only be used with disparity components that have been generated using line scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PROJECTED_Z	Specifies that the component stores a projective map. When used with MbufConvert3d(), the X- and Y-coordinates of each point will be generated using the transmitted depth (Z-coordinate values), the camera's intrinsic parameters, and either the column and row index for each corresponding pixel or the A and B coordinate maps provided by the camera. Providing the A and B coordinate maps (M_COMPONENT_COORDINATE_MAP_A_AIL, and M_COMPONENT_COORDINATE_MAP_B_AIL, respectively) will result in a more accurate representation of the camera's lens model (properties of the camera). If the coordinate maps are not provided when used with MbufConvert3d(), the column and row index for each corresponding pixel will be used to generate the X- and Y-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to ProjectedC. > Typically, this setting should only be used with projective maps generated using area scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROJECTED_Z_EXTERNAL_Y	Specifies that the component stores a projective map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. When used with MbufConvert3d(), the Y-coordinate of each point will be taken from the specified buffer. In this case, only the X-coordinates of each point will be generated using the transmitted depth (Z-coordinate values), the camera's intrinsic parameters, and either the column index for each corresponding pixel or the A coordinate map provided by the camera. Providing the A coordinate map (M_COMPONENT_COORDINATE_MAP_A_AIL) will result in a more accurate representation of the camera's lens model (a property of the camera). If the coordinate map is not provided when used with MbufConvert3d(), the column index for each corresponding pixel will be used to generate the X-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to ProjectedC_Linescan. > Typically, this setting should only be used with projective maps generated using line scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNCALIBRATED_Z	Specifies that the component stores organized and uncalibrated Z-coordinates, without X and Y-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to UncalibratedC.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_X	Sets by how much the X-coordinates stored in the buffer will be scaled in the destination point cloud when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies by how much the X-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Y	Sets by how much the Y-coordinates stored in the buffer will be scaled in the destination point cloud when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies how much the Y-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Z	Sets by how much the Z-coordinates stored in the buffer will be scaled in the destination point cloud when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies how much the Z-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SHEAR_X	Sets by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_SHEAR_Z	Sets by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row when the buffer is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Remark

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_IMAGE + M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Type-specific versions of the function when using a C compiler under 64-bit

```
void MbufControlDouble (AIL_ID BufId, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

BufId

See BufId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MbufControlInt64 (AIL_ID BufId, AIL_INT64 ControlType, AIL_INT64 ControlValue)
```

Details

Parameters

BufId

See BufId of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufControlArea

Synopsis

Control a specified area of a buffer.

Syntax

```
void MbufControlArea(
    AIL_ID BufId,           //out
    AIL_INT OffsetX,        //in
    AIL_INT OffsetY,        //in
    AIL_INT SizeX,          //in
    AIL_INT SizeY,          //in
    AIL_INT Band,           //in
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function allows you to control a specified area of a buffer.

Parameters

BufId

Specifies the identifier of the buffer to control.

OffsetX

Specifies the horizontal pixel offset of the area.

OffsetY

Specifies the vertical pixel offset of the area.

SizeX

Specifies the width of the area, in pixels.

SizeY

Specifies the height of the area, in pixels.

Band

Specifies the band of the area. This parameter can be set to one of the following values.

For the band of the area	
Value	Description
M_DEFAULT	Same as M_ALL_BANDS.
M_ALL_BANDS	Specifies all bands (for multi-band buffers).

M_BLUE	Specifies the blue color band (for RGB buffers).
M_GREEN	Specifies the green color band (for RGB buffers).
M_RED	Specifies the red color band (for RGB buffers).
0 <= *Value* < *#bands*	Specifies the index of the band. The relationship between index value and band for RGB, HSL, HSV, and YUV buffers is the following: 0 — Corresponds to the red band for RGB buffers, the hue band for HSL and HSV buffers, or the Y band for YUV buffers. 1 — Corresponds to the green band for RGB buffers, or the saturation band for HSL and HSV buffers, or the U band for YUV buffers. 2 — Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for YUV and HSV buffers.

ControlType

Specifies the type of buffer setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the value required for the setting.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType** and **ControlValue** parameters.

- For the buffer settings

For the buffer settings	
ControlType ControlValue	Description
M_MODIFIED	Signals Aurora Imaging Library that the content in the specified area of the buffer was modified without using Aurora Imaging Library. This control should be used to ensure that Aurora Imaging Library updates its internal information on this area of the buffer. For example, if an area of a buffer that is selected on a display is modified using a pointer to its memory, the display will not be updated automatically. You should use this control type to force a display update of this area. Note that this setting does the same thing as using MbufControl() with M_MODIFIED, but it is faster because it updates a specific area of the buffer, instead of updating the whole buffer. This setting increments the buffer's modification counter. You can inquire about the counter's current value using MbufInquire() with M_MODIFICATION_COUNT.

	M_DEFAULT	Implements the default behavior.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufControlContainer

Synopsis

Control a setting of the specified container, or one of its components.

Syntax

```
void MbufControlContainer(
    AIL_ID ContainerBufId,  //out
    AIL_INT64 Component,    //in
    AIL_INT64 ControlType,  //in
    AIL_DOUBLE ControlValue //in
)
```

Description

This function allows you to control a setting of the specified container, or one of its components.

> Note that presented above is an example prototype of this function. You can call MbufControlContainer() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use MbufControlContainer() and not, for example, MbufControlContainerInt64(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

ContainerBufId

Specifies the identifier of the container to control, or of which to control a component.

Component

Specifies what to control; either the container itself, or a component of the container.

This setting can be used to specify to control the container itself.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to control the container itself																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CONTAINER	Specifies to control a setting of the container.	c		e	f	g	a		i		j	k	l	m	n	b

The following settings can be used to specify a component using a uniquely identifying criterion. An error will be generated if there is no component in the container matching the specified criterion.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying which component to control by a unique criterion																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COMPONENT_BY_ID(AIL_INT ID)	Specifies to control the component, in the container, which has the specified buffer identifier. An error will be generated if the container does not have a component with this buffer identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ID Specifies the Aurora Imaging Library identifier of the component to control.															
M_COMPONENT_BY_INDEX(AIL_INT ComponentIndex)	Specifies to control the component, in the container, at the specified index. An error will be generated if the container does not have a component at this index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ComponentIndex Specifies the index of the component to control.															

The following settings can be used to specify a component using its component type. An error will be generated if there is not exactly one component of this type in the specified container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the component to control by component type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. Coordinates associated with the confidence value 0 are considered invalid data and will not be used by 3D image processing functions. A confidence component is associated with a range or disparity component in the same container when there are no other range, disparity, or confidence components in the container. If there is more than one range, disparity, and/or confidence component in a container (for example, because a single grab into the container transmitted multiple range and confidence components), you will need to either create a child container which contains only the required components using MbufChildContainer(), or free the extra components using MbufFreeComponent().	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the column index of each corresponding pixel and will result in a better representation of the camera's lens model when generating X-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the row index of each corresponding pixel and will result in a better representation of the camera's lens model when generating Y-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 254. You can use custom component types to identify components which store information that does not fit one of the standard component types. In addition, some cameras transmit components with custom component types. Refer to your camera manual to determine what type of information is stored in transmitted components.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. Each pixel of a disparity map indicates the apparent distance (typically measured in pixel units) between where an object appears in the left and right images captured by a stereoscopic camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. Only depth map containers can have a matrix component. A matrix component is associated with a range component in the same container when there is only one range component and no other matrix components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. A mesh component is associated with a range component in the same container when there are no other range or mesh components in that container. A point cloud container which has a mesh component is referred to as a meshed point cloud container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_METADATA	Specifies that the component stores metadata information. Metadata components are used by Aurora Imaging Library internally. Typically, you should ignore metadata components in your application.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. Unlike an intensity component, a multispectral component might include information about non-visible light.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_NORMALS_AIL	Specifies that the component stores normals information for each point in the M_COMPONENT_RANGE component of the container. A normals component is associated with a range component in the same container when there are no other range or normals components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. This can be either a 1-band component that stores a depth map, or a 3-band buffer that stores coordinates of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. Each pixel of a reflectance map indicates how much of the light hitting an object at that location is reflected back. Typically, this is an intensity image of the light spectrum used by a 3D sensor to detect 3D distance/position information. Typically, if the map was generated by a laser profiler, each row indicates the detected intensity of the laser for a single scan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. 3D image processing functions will only operate on points inside the ROI. Only depth map containers can have a region component. A region component is associated with a range component in the same container when there is only one range component and no other region components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. Each pixel of a scatter map indicates how much of the light hitting an object at that location is detected scattering beneath the object's surface.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_UNDEFINED	Specifies that the component stores information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the value needed for the setting.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For specifying settings for a container or a component that is an image buffer
- For specifying settings for any component
- For specifying settings useful with components that store 3D data

The following ControlType and corresponding ControlValue parameter settings are used to control settings of containers and image buffer components.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying settings for a container or a component that is an image buffer																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_INFORMATION_TYPE		Sets the GenDC part type of the component .															
	M_INFO_1D	Specifies to set the GenDC part type to 1D.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_2D	Specifies to set the GenDC part type to 2D.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_2D_H264	Specifies to set the GenDC part type to H.264 data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_2D_JPEG	Specifies to set the GenDC part type to JPEG data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_2D_JPEG2000	Specifies to set the GenDC part type to JPEG 2000 data.	c		e	f	g	a		i		j	k	l	m	n	b

M_INFO_GENICAM_CHUNK	Specifies to set the GenDC part type to GenICam chunk data.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_GENICAM_XML	Specifies to set the GenDC part type to GenICam XML data.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_HOOK	Sets whether to run the user-defined functions hooked to the container or component's modification event upon the event. INFO INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the user-defined functions should not be called. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the user-defined functions should be called.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFIED	Signals Aurora Imaging Library that the buffer content was modified without using Aurora Imaging Library. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Implements the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control all components.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying settings for any component																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_COMPONENT_TYPE	Sets the component type of the component . INFO INQ															
M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 255. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_METADATA	Specifies that the component stores metadata information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_NORMALS_AIL	Specifies that the component stores normals information for each point in the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_UNDEFINED	Specifies that the component contains information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

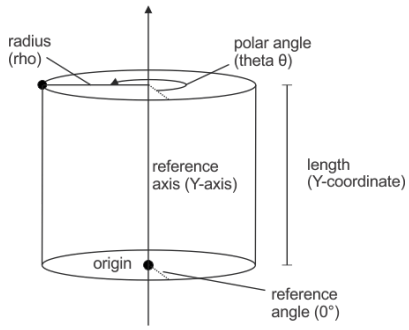
For components with an M_IMAGE attribute and with M_COMPONENT_TYPE set to M_COMPONENT_RANGE or M_COMPONENT_DISPARITY, ControlType and ControlValue can also be set to one of the values below.

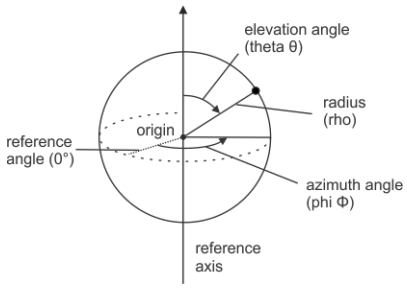
These component settings are useful only when the container is passed as a source to MbufConvert3d(). Additionally, a container cannot be 3D-processable or 3D-displayable unless it has a single range component with all of these settings at their default values (except for M_3D_REPRESENTATION, which must be set to M_CALIBRATED_XYZ or M_CALIBRATED_XYZ_UNORGANIZED).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying settings useful with components that store 3D data																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_3D_ASPECT_RATIO		Sets by how much the focal length in the Y-direction stored in the buffer will be scaled in the destination point cloud to correct for non-square pixels in 3D reconstruction modes that rely on projective geometry. This is useful when the focal length parameters in the camera intrinsic matrix are not equal (in the X- and Y- directions). INQ															
	Value > 0.0	Specifies by how much the focal length the Y-direction will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_COORDINATE_SYSTEM_TYPE	Sets which type of coordinate system to use to interpret the coordinates stored in the bands of the buffer if it is a range component. Note that Aurora Imaging Library does not support any functionality with coordinates stored using a coordinate system type other than M_CARTESIAN. If a buffer stores information defined using another type of coordinate system, you will need to manually convert that information to cartesian coordinates before the buffer can be used with any Aurora Imaging Library processing function. This conversion cannot be done using MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_CARTESIAN	Specifies that the buffer stores right-handed cartesian coordinates. If the buffer is a 1-band buffer, its values will be interpreted as Z-coordinates (equivalent to the pixels of a depth map). If the buffer is a 3-band buffer, the bands store the following: Band: Information Band 0: Stores the X-coordinates. Band 1: Stores the Y-coordinates. Band 2: Stores the Z-coordinates. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CYLINDRICAL	Specifies that the buffer stores cylindrical coordinates (theta-Y-rho). In a cylindrical coordinate system, points are defined relative to a reference axis and reference angle perpendicular to that axis. The Y-coordinate is the distance from the origin along the reference axis, theta (θ) is the polar angle of the point relative to a reference angle, and rho is the distance from that axis along the line described by theta and perpendicular to the reference axis (the radius).  If the buffer is a 1-band buffer, its values store rho, the distance from a chosen reference axis. If the buffer is a 3-band buffer, the bands store the following: Band: Information Band 0: Stores theta, the polar angle coordinates relative to the chosen reference angle. Band 1: Stores Y-coordinates, the distances along the chosen reference axis. Band 2: Stores rho, the distances from the chosen reference axis.															

M_SPHERICAL	Specifies that the buffer stores spherical coordinates (theta-phi-rho). In a spherical coordinate system, points are defined relative to a reference axis and a reference angle perpendicular to that axis. Rho is the distance from the origin (the radius), theta (θ) is the elevation angle of the point relative to reference axis, and phi is the azimuthal angle relative to the reference angle.  If the buffer is a 1-band buffer, its values store rho, the distance from a chosen reference axis. If the buffer is a 3-band buffer, the bands store the following: Band: Information Band 0: Stores theta, the elevation angle coordinates relative to the chosen reference axis. Band 1: Stores phi, the azimuth angle coordinates relative to a chosen reference angle. Band 2: Stores rho, the distances from the chosen reference axis.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the coordinate system type is unknown.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPARITY_BASELINE	Sets the stereo baseline value of the stereoscopic camera used to generate the data in the buffer. This is the physical distance between the lenses of the camera. Refer to your camera manual to determine the correct value for this setting, which might differ from the true physical distance between the lenses of your camera. INQ	c		e	f	g	a		i		j	k	l	m	n	b

Value > 0.0	Specifies the stereo baseline value, expressed in meters. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISTANCE_UNIT	Sets the unit to use when the buffer is part of a container and stores natively calibrated distance data. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_INCH	Specifies that the distance data is provided in inches.	c		e	f	g	a		i		j	k	l	m	n	b
M_MILLIMETER	Specifies that the distance data is provided in millimeters.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that the distance data is provided in pixels. This setting is only useful for buffers that have the component type M_COMPONENT_DISPARITY and store a disparity map.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the distance unit is unknown. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_FOCAL_LENGTH	Sets the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer. Refer to your camera manual to determine the correct value for this setting, which might differ from the true focal length of your camera. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer, expressed in pixels. For example, if the buffer has 1000 pixels per millimeter of a single image sensor (after calibration), and the focal length of the lenses in millimeters is 35, the focal length in pixels is 35000. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_INVALID_DATA_FLAG	Sets whether the buffer uses a specific value to indicate invalid data. For 3-band buffers that store coordinates, the invalid data flag should be stored in the Z-axis band. Specify the value that indicates invalid data using M_3D_INVALID_DATA_VALUE. INQ	c		e	f	g	a		i		j	k	l	m	n	b

M_FALSE	Specifies that the buffer does not use a special value to indicate invalid data. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the buffer uses a special value to indicate invalid data.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_INVALID_DATA_VALUE	Sets the value used to indicate missing data when M_3D_INVALID_DATA_FLAG is set to M_TRUE. Note that this value is not used or respected by any Aurora Imaging Library functions except for MbufConvert3d() when the buffer is a component of the source container. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the value used to indicate missing data. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_X	Sets by how much the X-coordinates stored in the buffer will be offset in the destination point cloud when the container is passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the X-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Y	Sets by how much the Y-coordinates stored in the buffer will be offset in the destination point cloud when the container is passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Y-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Z	Sets by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud when the container is passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Z-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_PRINCIPAL_POINT_X	Sets the X-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies X-position of the principal point, expressed in pixels. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_PRINCIPAL_POINT_Y	Sets the Y-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies Y-position of the principal point, expressed in pixels. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_REPRESENTATION	Sets how 3D data is stored in the buffer; this information is used when the buffer is a range or disparity component of a container. For more information, see MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XYZ	Specifies that the component stores organized and natively calibrated X, Y, and Z-coordinates. This is the default value for 3-band buffers with M_SIZE_Y greater than 1. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedABC_Grid.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XYZ_UNORGANIZED	Specifies that the component stores unorganized and natively calibrated X, Y, and Z-coordinates. This is the default value for 3-band buffers with M_SIZE_Y of 1. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedABC_PointCloud.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XZ_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates stored in a separate array buffer. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedAC_Linescan.	c		e	f	g	a		i		j	k	l	m	n	b

	M_CALIBRATED_XZ_UNIFORM_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates identified by the row index. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedAC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z	Specifies that the component stores organized and natively calibrated Z-coordinates, without X and Y-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedC.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates without X-coordinates; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. This corresponds to the GenICam Scan3dOutputMode feature set to CalibratedC_Linescan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_X_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates, with the X-coordinates identified by column index; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. This corresponds to the GenICam Scan3dOutputMode feature set to RectifiedC_Linescan.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_XY	Specifies that the component stores organized and natively calibrated Z-coordinates, with X and Y-coordinates identified by column and row index respectively. The manufacturer of your camera might refer to a range component with this setting as a depth map. This is the default value for 1-band buffers. This corresponds to the GenICam Scan3dOutputMode feature set to RectifiedC.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISPARITY	Specifies that the component stores a disparity map with perspective distortion along the Y-axis. When used with MbufConvert3d(), the Y-coordinate of each point in the resulting point cloud will be generated using the row index of the corresponding pixel and compensating for perspective. This corresponds to the GenICam Scan3dOutputMode feature set to DisparityC. > Typically, this setting should only be used with disparity components that have been generated using area scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_EXTERNAL_Y	Specifies that the component stores a disparity map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. When used with MbufConvert3d(), the Y-coordinate of each point in the resulting point cloud will be taken from the specified buffer. This corresponds to the GenICam Scan3dOutputMode feature set to DisparityC_Linescan. > Typically, this setting should only be used with disparity components that have been generated using line scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_UNIFORM_Y	Specifies that the component stores a disparity map, with Y-values identified by row index. When used with MbufConvert3d(), the Y-coordinates of each point in the resulting point cloud will be generated using the row index of the corresponding pixel. Since the Y-values are uniform, no perspective correction is applied. This partially corresponds to the GenICam Scan3dOutputMode feature set to DisparityC_Linescan, except that the Y-values are not stored in a separate buffer. > Typically, this setting should only be used with disparity components that have been generated using line scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PROJECTED_Z	Specifies that the component stores a projective map. When used with MbufConvert3d(), the X- and Y-coordinates of each point will be generated using the transmitted depth (Z-coordinate values), the camera's intrinsic parameters, and either the column and row index for each corresponding pixel or the A and B coordinate maps provided by the camera. Providing the A and B coordinate maps (M_COMPONENT_COORDINATE_MAP_A_AIL, and M_COMPONENT_COORDINATE_MAP_B_AIL, respectively) will result in a more accurate representation of the camera's lens model (properties of the camera). If the coordinate maps are not provided when used with MbufConvert3d(), the column and row index for each corresponding pixel will be used to generate the X- and Y-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to ProjectedC. > Typically, this setting should only be used with projective maps generated using area scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROJECTED_Z_EXTERNAL_Y	Specifies that the component stores a projective map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. When used with MbufConvert3d(), the Y-coordinate of each point will be taken from the specified buffer. In this case, only the X-coordinates of each point will be generated using the transmitted depth (Z-coordinate values), the camera's intrinsic parameters, and either the column index for each corresponding pixel or the A coordinate map provided by the camera. Providing the A coordinate map (M_COMPONENT_COORDINATE_MAP_A_AIL) will result in a more accurate representation of the camera's lens model (a property of the camera). If the coordinate map is not provided when used with MbufConvert3d(), the column index for each corresponding pixel will be used to generate the X-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to ProjectedC_Linescan. > Typically, this setting should only be used with projective maps generated using line scan cameras.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNCALIBRATED_Z	Specifies that the component stores organized and uncalibrated Z-coordinates, without X and Y-coordinates. This corresponds to the GenICam Scan3dOutputMode feature set to UncalibratedC.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_X	Sets by how much the X-coordinates stored in the buffer will be scaled in the destination point cloud when the container is passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies by how much the X-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Y	Sets by how much the Y-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies how much the Y-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Z	Sets by how much the Z-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies how much the Z-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SHEAR_X	Sets by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row when it is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_SHEAR_Z	Sets by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row when it is a component of a container passed as a source to MbufConvert3d(). INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler

```
void MbufControlContainerInt64 (AIL_ID ContainerBufId, AIL_INT64 Component, AIL_INT64 ControlType,
AIL_INT64 ControlValue)
```

Details

Parameters

ContainerBufId

See ContainerBufId of the main function for a description.

Component

See Component of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MbufControlContainerDouble (AIL_ID ContainerBufId, AIL_INT64 Component, AIL_INT64 ControlType,
AIL_DOUBLE ControlValue)
```

Details

Parameters

ContainerBufId

See ContainerBufId of the main function for a description.

Component

See Component of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufControlFeature

Synopsis

Controls a feature of a buffer.

Syntax

```
void MbufControlFeature(
    AIL_ID BufId,                   //out
    AIL_INT64 ControlType,          //in
    AIL_CONST_TEXT_PTR FeatureName, //in
    AIL_INT64 UserVarType,          //in
    const void *UserVarPtr          //in
)
```

Description

This function allows you to directly control various GenICam standard feature naming convention (SFNC) features and manufacturer-specific features of buffers. For more information, refer to Using Zebra GenTL Consumer (library).

The features described in this function are primarily available to set existing settings using Aurora Imaging Library code or to set information from which you can build your own interface for the GenTL buffer's description file (XML). Aurora Imaging Library provides two versions of an interface that you can use to interactively inquire the buffer's features. At design-time, you can use Aurora Imaging Intellicam's Feature Browser. At runtime, you can launch Feature Browser, using MbufControl() with M_GC_FEATURE_BROWSER. Both versions of Feature Browser provide a list of available feature names and associated possible values, as well as code snippets with the Aurora Imaging Library functions and Aurora Imaging Library constants associated with the settings selected; you can copy the code snippets to your Aurora Imaging Library application code.

To inquire a buffer-specific feature, use MbufInquireFeature().

Parameters

BufId

Specifies the identifier of the buffer to control. This parameter must be given a valid buffer identifier, previously allocated using MbufAlloc...().

ControlType

Specifies the type of information to control for the specified feature.

See the Parameter associations section for possible values that can be specified.

FeatureName

Specifies the name of the buffer feature to control.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name of the feature																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

AIL_TEXT("*FeatureName*") [1]	Specifies the name of the feature. Note that the feature name is case-sensitive. Refer to the GenTL library's documentation for a list of the features available.			e												b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

UserVarType

Specifies the data type of the address pointed to by the UserVarPtr parameter. If not setting the feature's value (using M_FEATURE_VALUE), set this parameter to **M_DEFAULT**.

Note that, in the case of a string (or a multi-byte register), the size of the array passed to the UserVarPtr parameter must be equal to or larger than the returned information; otherwise an application crash will occur.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the UserVarPtr's data type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TYPE_BOOLEAN	Specifies that UserVarPtr is passed an address of type *AIL_BOOL*.			e												b
M_TYPE_DOUBLE	Specifies that UserVarPtr is passed an address of type *AIL_DOUBLE*.			e												b
M_TYPE_INT64	Specifies that UserVarPtr is passed an address of type *AIL_INT64*.			e												b
M_TYPE_STRING +	Specifies that UserVarPtr is passed an address of type *AIL_TEXT_CHAR*. You must specify a combination value from the following table: • To specify the length of the array			e												b
M_TYPE_UINT8 +	Specifies that UserVarPtr is passed an address of type *AIL_UINT8*. You must specify a combination value from the following table: • To specify the length of the array			e												b

Combination value for M_TYPE_STRING; M_TYPE_UINT8.

You must add the following value to the above-mentioned values to specify the length of the array.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the length of the array

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FEATURE_USER_ARRAY_SIZE(AIL_INT Length)	Specifies the length of the array.			e												b
	Parameters															
	Length Sets the length, in bytes.															

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]
- AIL_BOOL
- AIL_DOUBLE
- AIL_INT
- AIL_INT64
- M_NULL

Specifies the address of the variable in which to write the value of the feature.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType** and **UserVarPtr** parameters.

- For specifying the type of information about the feature to set and the data type returned

To determine the data type of the FeatureName specified, use MbufInquireFeature() with M_FEATURE_VALUE.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of information about the feature to set and the data type returned			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlType		Description															
	UserVarPtr																
M_FEATURE_CHANGE_HOOK		Sets whether to enable an event to occur when the value of the specified feature changes. Once enabled, use MbufHookFunction() with M_FEATURE_CHANGE to hook a specified function to the feature change event. Repeat for each feature that you want to enable a feature change event. Alternatively, to enable an event to occur when the value of any feature changes and at the same time hook a function to any feature change, use MbufHookFunction() with M_FEATURE_CHANGE + M_ALL. INQ															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e												b

	M_DISABLE	Specifies that no event is generated when the value of the specified feature changes. This is the default value.			e												b
	M_ENABLE	Specifies that an event is generated when the value of the specified feature changes.			e												b
M_FEATURE_EXECUTE		Sets that the specified command feature must be executed.			e												b
	UserVarPtr extra info	Data type info: Data type: *Set this parameter to M_NULL*			e												b
M_FEATURE_MAX		Specifies to set the feature to its maximum value.			e												b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *Set this parameter to M_NULL*			e												b
M_FEATURE_MIN		Specifies to set the feature to its minimum value.			e												b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *address of an AIL_INT64*			e												b
M_FEATURE_VALUE		Sets the current value of the feature.			e												b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL* **|** *address of an AIL_DOUBLE* **|** *address of an AIL_INT64* **|** *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e												b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufConvert3d

Synopsis

Converts 3D data to a format that is 3D-processable and/or 3D-displayable.

Syntax

```
void MbufConvert3d(
    AIL_ID SrcContainerOrImageBufId, //in
    AIL_ID DstContainerOrImageBufId, //out
    AIL_ID ExternalYArrayBufId,      //in
    AIL_INT64 DstOptions,            //in
    AIL_INT64 ControlFlag            //in
)
```

Description

This function converts the 3D data in the specified source container or fully-corrected depth map image buffer, and stores it in the specified destination container or image buffer. By default, if the destination is a container, the source data is converted to a point cloud format that is 3D-processable and/or 3D-displayable (depending on whether the destination container has the M_DISP or M_PROC attribute). You can set the DstOptions parameter to M_DEPTH_MAP_CONTAINER to convert the source data to a 3D-processable depth map format. If the destination is an image buffer and the range component of the source container stores a depth map, the source is converted to a fully corrected depth map.

> You can only use an image buffer as a destination if the source is a container with a single range component, and the component has its M_3D_REPRESENTATION set to M_CALIBRATED_Z_UNIFORM_XY.

If the source and destination are both containers, all components of the source container that do not affect whether the container is 3D-processable or 3D-displayable will be copied to the destination.

In some cases, if you have container that stores incomplete or ambiguous 3D data, you can still convert it by specifying the M_COMPENSATE flag. When M_COMPENSATE is specified, Aurora Imaging Library will try to convert the data by making assumptions about the missing or ambiguous information. For example, if the source container has a range component with the M_3D_REPRESENTATION set to M_UNCALIBRATED_Z, it will instead be treated as though it were M_CALIBRATED_Z_UNIFORM_XY. Typically, compensation should only be used for display purposes, because it can result in incorrectly calibrated 3D data in the destination container or image buffer.

If the source container has a range or disparity component with an M_3D_REPRESENTATION setting that specifies an external Y array, you must either set ExternalYArrayBufId to the Aurora Imaging Library identifier of a buffer with an M_ARRAY attribute, or use M_COMPENSATE. Typically, this is only used when the 3D sensor, used to generate the range or disparity component, stores the position of a rotary/linear encoder to identify the position of the conveyor belt when scanning each line. In most cases, these 3D sensors transmit this information as GenICam chunk data, in which case you will need to manually extract the values immediately after the 3D data is grabbed. You can retrieve this data individually for each line using MdigControlFeature() with the feature name "ChunkScanLineSelector" and MdigInquireFeature() with the feature name "ChunkEncoderValue".

> To convert the rotary encoder position values retrieved from the 3D sensor to calibrated Y-coordinates, you will need to set M_3D_SCALE_Y of the range or disparity component to a factor based on the displacement of the conveyor belt for each increment of the encoder. Your 3D sensor might be configured to set this value automatically during acquisition.

Parameters

SrcContainerOrImageBufId

Specifies the identifier of the source container or fully corrected depth map image buffer to convert.

For specifying the source container or fully corrected depth map image buffer	
Value	Description
Buffer identifier	Specifies the Aurora Imaging Library identifier of a fully corrected depth map image buffer. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer and must store a fully-corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). You can mark invalid data in the image buffer by associating the buffer with a region of interest (ROI) using MbufSetRegion(). By default, each pixel outside the ROI will be given a confidence value of 0 in the destination container. If the destination is a 3D-processable depth map container, you can use M_PROPAGATE_REGION to copy the ROI to the M_COMPONENT_REGION_AIL component of the container instead. The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR).
Container identifier	Specifies the Aurora Imaging Library identifier of a container to convert. To be convertible, the source container must have exactly one component with either the component type M_COMPONENT_RANGE or M_COMPONENT_DISPARITY; the container can have other components, but it must not have more than one range or disparity component (if necessary, you can use a child container, created using MbufChildContainer(), to meet this restriction). If the container has a range component, it must have M_3D_COORDINATE_SYSTEM_TYPE set to M_CARTESIAN. The range or disparity component must also meet certain requirements, depending upon its M_3D_REPRESENTATION setting:

3D representation	Requirements
M_CALIBRATED_XYZ	A 3-band range component.
M_CALIBRATED_XYZ_UNORGANIZED	A 3-band range component with an M_SIZE_Y of 1.
M_CALIBRATED_XZ_EXTERNAL_Y	A 3-band range component, with Y values provided by passing an Aurora Imaging Library array buffer to ExternalYArrayBufId. The Y-axis band of the range component will be ignored.
M_CALIBRATED_XZ_UNIFORM_Y	A 3-band range component, with Y values identified by multiplying the row index of each point by the range component's M_3D_SCALE_Y setting. The Y-axis band of the range component will be ignored.
M_CALIBRATED_Z	A 1-band range component. The ControlFlag parameter must be set to M_COMPENSATE.

M_CALIBRATED_Z_EXTERNAL_Y	A 1-band range component, with Y values provided by passing an Aurora Imaging Library array buffer to ExternalYArrayBufId. The ControlFlag parameter must be set to M_COMPENSATE.
M_CALIBRATED_Z_UNIFORM_X_EXTERNAL_Y	A 1-band range component, with X values identified by multiplying the column index of each point by the range component's M_3D_SCALE_X setting and Y values provided by passing an Aurora Imaging Library array buffer to ExternalYArrayBufId.
M_CALIBRATED_Z_UNIFORM_XY	A 1-band range component, with X and Y values identified by multiplying the column and row index of each point by the range component's M_3D_SCALE_X and M_3D_SCALE_Y settings, respectively.
M_UNCALIBRATED_Z	A 1-band range component. The ControlFlag parameter must be set to M_COMPENSATE.
M_DISPARITY	A 1-band disparity component. > Typically, this setting should only be used with disparity components that have been generated using area scan stereoscopic cameras.
M_DISPARITY_EXTERNAL_Y	A 1-band disparity component, with Y values provided by passing an Aurora Imaging Library array buffer to ExternalYArrayBufId. > Typically, this setting should only be used with disparity components that have been generated using line scan stereoscopic cameras.
M_DISPARITY_UNIFORM_Y	A 1-band disparity component, with Y values identified by multiplying the row index of each pixel by the disparity component's M_3D_SCALE_Y setting. > Typically, this setting should only be used with disparity components that have been generated using line scan stereoscopic cameras.
M_PROJECTED_Z	A 1-band range component, with X and Y values identified using the transmitted depth (Z-coordinate values), the camera's intrinsic parameters, and either the column and row index for each corresponding pixel or the A and B coordinate maps provided by the camera. > Typically, this setting should only be used with projective maps generated using area scan cameras.

M_PROJECTED_Z_EXTERNAL_Y	A 1-band range component, with Y values provided by passing an Aurora Imaging Library array buffer to ExternalYArrayBufId, and with X values identified using the transmitted depth (Z-coordinate values), the camera's intrinsic parameters, and the column index for each corresponding pixel or the A coordinate map provided by the camera. > Typically, this setting should only be used with projective maps generated using line scan cameras.

This function uses the 3D settings of the range or disparity component during conversion. For example, all Z-coordinates in the source container's range or disparity component will be offset by the value of M_3D_OFFSET_Z in the destination container or depth map image buffer. In the destination container, the range component will then have the M_3D_OFFSET_Z setting reset to the default value of 0.

If your 3D sensor transmits these 3D settings, they will be set automatically during acquisition. If not, you must modify the 3D settings of components directly using MbufControlContainer() with settings from the table For specifying settings useful with components that store 3D data.

> Note that the M_3D_DISTANCE_UNIT setting of the range or disparity component will be copied to the range component of the destination, but will not be used during conversion.

The source container can have an M_COMPONENT_REGION_AIL component that stores an ROI. By default, each point outside the ROI will be set to the invalid value in the destination image buffer or given a confidence value of 0 in the destination container. You can use M_PROPAGATE_REGION to copy the ROI to an M_RASTER ROI in the destination image buffer or the M_COMPONENT_REGION_AIL component of the destination container instead.

DstContainerOrImageBufId

Specifies the identifier of the destination container or image buffer in which to store the converted data.

For specifying the destination container or image buffer	
Value	Description
Buffer identifier	Specifies the Aurora Imaging Library identifier of a 1-band, 8-bit, 16-bit, or 32-bit unsigned image buffer. In this case, the source must be a container with a range component that stores a depth map. The depth map in the source container will be converted to a fully corrected depth map and stored in the destination image buffer. The image buffer must have been allocated with the same SizeX and SizeY settings as the range component of the source container. This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error. > You can only use an image buffer as a destination if the source is a container with a single range component and the component has its M_3D_REPRESENTATION setting set to M_CALIBRATED_Z_UNIFORM_XY. If ControlFlag is set to M_COMPENSATE, M_3D_REPRESENTATION can also be set to any 3D representation setting that can be interpreted as a depth map.

Container identifier	Specifies the Aurora Imaging Library identifier of a container that is not a child container, and has an M_DISP and/or M_PROC attribute. To make the destination container 3D-processable and/or 3D-displayable, existing components in the destination container with the following component types might be freed, reallocated, or filled with new data: • M_COMPONENT_RANGE. • M_COMPONENT_DISPARITY. • M_COMPONENT_CONFIDENCE. • M_COMPONENT_INTENSITY. • M_COMPONENT_REFLECTANCE. • M_COMPONENT_REGION_AIL. • M_COMPONENT_MESH_AIL. • M_COMPONENT_NORMALS_AIL. > If **MbufConvert3d**() reallocates existing components in the destination container, their Aurora Imaging Library identifiers will change. You can determine the current Aurora Imaging Library identifier of a component with a specific component type, using MbufInquireContainer() with M_COMPONENT_ID.

ExternalYArrayBufId

Specifies the identifier of a 1-band, 1D M_ARRAY buffer containing the Y-axis value for each row of coordinates stored in the range or disparity component of the source container. The array buffer must store one value per row in the range or disparity component of the source container.

This parameter is only used if the source range or disparity component has an M_3D_REPRESENTATION setting that requires an external Y array. Set this parameter to **M_NULL** if not used.

DstOptions

Specifies which operations to apply to the data during conversion.

For specifying operations to apply to the data	
Value	Description
M_DEFAULT	Specifies to convert the data without performing extra operations.
M_DEPTH_MAP_CONTAINER +	Specifies to set the destination container to be a 3D-processable depth map container.
M_NO_REMAP_DEPTH_MAP	Specifies to mark all pixels with the maximum value in the source depth map as invalid during conversion. This can significantly speed up the **MbufConvert3d**() operation if pixels with the maximum value are not important. For example, if the source and destination depth maps are both 8-bit buffers, and the source has valid pixels at the maximum value (255), the pixels will be treated as invalid in the destination. > Note that when the source's depth is greater than that of the destination (for example, a 16-bit source and an 8-bit destination), this setting is ignored and a remap will occur.

M_REMOVE_NON_FINITE	Specifies to mark all non-finite 3D data in the source container as invalid during conversion. Non-finite 3D data is that which has a value of infinity, negative infinity, or NaN in the range or disparity component. This setting is only available when the source is a container. If the destination is a container, all non-finite points are given a confidence value of 0. If the destination is an image buffer, all non-finite points are set to the maximum possible finite value, which marks them as missing or invalid.

Combination value that can be used alone or as combination value for M_DEPTH_MAP_CONTAINER.

You can use the following value on its own, or add it to the above-mentioned value, to specify to propagate the region of interest (ROI) to the destination.

Note that you can only use this value alone when the destination is an image buffer.

For specifying whether to propagate the ROI	
Value	Description
M_PROPAGATE_REGION	Specifies to propagate the ROI of the source into an M_RASTER ROI in the destination depth map image buffer or M_COMPONENT_REGION_AIL component in the destination depth map container. > Note that M_PROPAGATE_REGION is ignored if there is no ROI in the source.

ControlFlag

Specifies whether to generate an error when there is missing or ambiguous information in the source, or to compensate by making assumptions about the missing information.

For specifying the control flag	
Value	Description
M_DEFAULT	Specifies to generate an error if conversion requires information that is missing or ambiguous in the source.
M_COMPENSATE	Specifies to compensate for missing or ambiguous information in the source. The following assumptions are used when compensating: • The coordinates in the source depth map or point cloud are uniformly spaced for any missing axes. For example, a range component with M_3D_REPRESENTATION set to M_CALIBRATED_Z_EXTERNAL_Y will be treated as though it were set to M_CALIBRATED_Z_UNIFORM_X_EXTERNAL_Y. • If the source is a container with no confidence information, all coordinates in the source are considered valid. • If the source is a container, all coordinates in the source are considered natively calibrated. • If M_SIZE_Y of the range component is larger than 1, then the point cloud in the container is considered organized. • Any components in the source container that have the same component type are not copied. • Any confidence, intensity, reflectance, mesh, or normals component in the source container that is not correctly formatted should be ignored.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; ail3d11.lib.
DLL	Requires ail11.dll; ail3d11.dll.

MbufCopy

Synopsis

Copy data from one buffer to another.

Syntax

```
void MbufCopy(
    AIL_ID SrcBufId, //in
    AIL_ID DestBufId //out
)
```

Description

This function copies the specified source buffer data to the specified destination buffer. If the source and destination buffers are of different data types, Aurora Imaging Library converts the data automatically.

If the source buffer depth is greater than that of the destination, the most significant bits are truncated when the data is copied into the destination. If the destination depth is greater than that of the source, the source data is zero or sign-extended (depending on the type of the source) when copied into the destination. If the destination is larger in size than the source, exceeding areas of the buffer are unaffected.

Note, when copying from a non-binary buffer to a binary buffer, all non-zero pixels in the source buffer are represented as ones (1) in the binary buffer. When copying a binary buffer to a buffer of a different depth, each bit is copied into the least-significant bit of a different destination pixel. The remaining bits of the destination pixel are set to 0; to propagate the bit value to all bits, use MimBinarize().

When copying from a floating-point buffer to an integer buffer, the values are truncated.

If the source buffer is a 3-band YUV buffer and the destination buffer is a 1-band buffer, only the Y band (luminance) is copied. If the source buffer is a 3-band RGB buffer and the destination buffer is a 1-band buffer, only the red band is copied.

Note, if the source is a 1-band image buffer associated with a LUT buffer and the destination is a 3-band image buffer, the source data copied to destination is first mapped through the LUT.

Note, if the source and destination are both multi-band buffers, they must have the same number of bands.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcBufId

Specifies the identifier of the source data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DestBufId

Specifies the identifier of the destination data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise,

you will get an error.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufCopyClip

Synopsis

Copy buffer, clipping data outside the destination buffer.

Syntax

```
void MbufCopyClip(
    AIL_ID SrcBufId,  //in
    AIL_ID DestBufId, //out
    AIL_INT DestOffX, //in
    AIL_INT DestOffY  //in
)
```

Description

This function copies the source buffer data to the destination buffer, starting at the specified offset. Data outside of the destination buffer is not copied (it is clipped).

If the source buffer depth is greater than that of the destination, the most significant bits are truncated when the data is copied to the destination. If the destination depth is greater than that of the source, the source data is zero or sign-extended (depending on the type of the source) when copied to the destination.

Note, when copying from a non-binary buffer to a binary buffer, all non-zero pixels in the source buffer are represented as ones (1) in the binary buffer. When copying a binary buffer to a buffer of a different depth, each bit is copied into the least-significant bit of a different destination pixel. The remaining bits of the destination pixel are set to 0; to propagate the bit value to all bits, use MimBinarize().

When copying from a floating-point buffer to an integer buffer, the decimal point values are ignored and the most significant bits are truncated.

Note, if the source and destination are both multi-band buffers, they must have the same number of bands.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcBufId

Specifies the identifier of the source data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DestBufId

Specifies the identifier of the destination data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DestOffX

Specifies the horizontal pixel offset of the destination buffer area at which to start copying data. The offset is relative to the top-left corner of the destination buffer (0, 0). Also, this parameter can be set to a negative value,

and can be specified anywhere outside the destination buffer. Data extending beyond the limits of the destination buffer is not copied (it is clipped).

DestOffY

Specifies the vertical pixel offset of the destination buffer area at which to start copying data. The offset is relative to the top-left corner of the destination buffer (0,0). Also, this parameter can be set to a negative value and can be specified anywhere outside the destination buffer. Data extending beyond the limits of the destination buffer is not copied (it is clipped).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufCopyColor2d

Synopsis

Copy a two-dimensional region of one or all bands of an image buffer to another buffer.

Syntax

```
void MbufCopyColor2d(
    AIL_ID SrcBufId, //in
    AIL_ID DstBufId, //out
    AIL_INT SrcBand, //in
    AIL_INT SrcOffX, //in
    AIL_INT SrcOffY, //in
    AIL_INT DstBand, //in
    AIL_INT DstOffX, //in
    AIL_INT DstOffY, //in
    AIL_INT SizeX,   //in
    AIL_INT SizeY    //in
)
```

Description

This function copies a two-dimensional region of one or all bands of the specified source buffer to the specified band(s) of the destination buffer. It can also be used to retrieve or replace a color band from a color buffer.

If the source is a monochrome buffer and the destination is a multi-band buffer, the unique source buffer band is inserted into the specified band of the destination buffer. If the source is a multi-band buffer and the destination is a monochrome buffer, the specified source buffer band is extracted from the source buffer and written to the destination buffer. If both are multi-band buffers, the specified band(s) is copied from the source to the destination.

If the source buffer depth is greater than that of the destination, the most significant bits are truncated when the data is copied to the destination. If the destination depth is greater than that of the source, the source data is zero or sign-extended (depending on the type of the source) when copied to the destination. Also, the buffers must have the same number of bands if all bands are to be copied.

Note that when copying from a non-binary buffer to a binary destination buffer, all non-zero pixels in the source buffer are represented as ones (1) in the binary destination buffer. When copying a binary buffer to a buffer of a different depth, each bit is copied into the least-significant bit of a different destination pixel. The remaining bits of the destination pixel are set to 0; to propagate the bit value to all bits, use MimBinarize().

Note, if the source is a 1-band image buffer associated with a LUT buffer and the destination is a 3-band image buffer, the source data copied to destination is first mapped through the LUT.

Note that when copying into the U or V band of a YUV destination buffer, the dimensions of the U or V band of the destination buffer must not be smaller than the dimensions of the source buffer.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcBufId

Specifies the identifier of the source data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DstBufId

Specifies the identifier of the destination data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

SrcBand

Specifies the source band from which to copy. SrcBand can be set to one of the following values:

For the source band	
Value	Description
M_ALL_BANDS	Specifies all bands (for multi-band buffers).
M_BLUE	Specifies the blue color band (for RGB buffers).
M_GREEN	Specifies the green color band (for RGB buffers).
M_HUE	Specifies the hue band (for HSL and HSV buffers).
M_LUMINANCE	Specifies the luminance band (for HSL buffers).
M_RED	Specifies the red color band (for RGB buffers).
M_SATURATION	Specifies the saturation band (for HSL and HSV buffers).
M_U	Specifies the U band (for YUV buffers).
M_V	Specifies the V band (for YUV and HSV buffers).
M_Y	Specifies the Y band (for YUV buffers).

0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between index value and band for RGB, HSL, HSV, and YUV buffers is the following: 0 — Corresponds to the red band for RGB buffers, the hue band for HSL and HSV buffers, or the Y band for YUV buffers. 1 — Corresponds to the green band for RGB buffers, or the saturation band for HSL and HSV buffers, or the U band for YUV buffers. 2 — Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for YUV and HSV buffers.

SrcOffX

Specifies the horizontal pixel offset of the region to read relative to the source buffer starting coordinate. The offset must be within the width of the source buffer.

SrcOffY

Specifies the vertical pixel offset of the region to read relative to the source buffer starting coordinate. The offset must be within the height of the source buffer.

DstBand

Specifies the destination band in which to copy. DstBand can be set to the same values as SrcBand.

DstOffX

Specifies the horizontal pixel offset of the region to write relative to the destination buffer starting coordinate. The offset must be within the width of the destination buffer.

DstOffY

Specifies the vertical pixel offset of the region to write relative to the destination buffer starting coordinate. The offset must be within the height of the destination buffer.

SizeX

Specifies the width of the region to be copied, starting from the specified offset (SrcOffX, DstOffX).

SizeY

Specifies the height of the region to be copied, starting from the specified offset (SrcOffY, DstOffY).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufCopyColor

Synopsis

Copy one or all bands of an image buffer.

Syntax

```
void MbufCopyColor(
    AIL_ID SrcBufId,  //in
    AIL_ID DestBufId, //out
    AIL_INT Band      //in
)
```

Description

This function copies one or all bands of the specified source buffer to the specified destination buffer. It can also be used to retrieve or replace a color band from a color image.

If the source is a monochrome buffer and the destination is a multi-band buffer, the source buffer band is copied into the specified band of the destination buffer. If the source is a multi-band buffer and the destination is a monochrome buffer, the specified source buffer band is copied from the source buffer and written to the destination buffer. If both are multi-band buffers, the specified band(s) is copied from the source to the destination.

If the source buffer depth is greater than that of the destination, the most significant bits are truncated when the data is copied to the destination. If the destination depth is greater than that of the source, the source data is zero or sign-extended (depending on the type of the source) when copied to the destination. Also, the buffers must have the same number of bands if all bands are to be copied.

Note, when copying from a non-binary buffer to a binary buffer, all non-zero pixels in the source buffer are represented as ones (1) in the binary buffer. When copying a binary buffer to a buffer of a different depth, each bit is copied into the least-significant bit of a different destination pixel. The remaining bits of the destination pixel are set to 0; to propagate the bit value to all bits, use MimBinarize().

Note, if the source is a 1-band image buffer associated with a LUT buffer and the destination is a 3-band image buffer, the source data copied to destination is first mapped through the LUT.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcBufId

Specifies the identifier of the source data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DestBufId

Specifies the identifier of the destination data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

Band

Specifies the band to copy and/or in which to store the copied data. Set this parameter to one of the following values:

For specifying the band	
Value	Description
M_ALL_BANDS	Specifies all bands (for multi-band buffers).
M_BLUE	Specifies the blue color band (for RGB buffers).
M_GREEN	Specifies the green color band (for RGB buffers).
M_HUE	Specifies the hue band (for HSL and HSV buffers).
M_LUMINANCE	Specifies the luminance band (for HSL buffers).
M_RED	Specifies the red color band (for RGB buffers).
M_SATURATION	Specifies the saturation band (for HSL and HSV buffers).
M_U	Specifies the U band (for YUV buffers).
M_V	Specifies the V band (for YUV and HSV buffers).
M_Y	Specifies the Y band (for YUV buffers).
0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between index value and band for RGB, HSL, HSV, and YUV buffers is the following: 0 — Corresponds to the red band for RGB buffers, the hue band for HSL and HSV buffers, or the Y band for YUV buffers. 1 — Corresponds to the green band for RGB buffers, or the saturation band for HSL and HSV buffers, or the U band for YUV buffers. 2 — Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for YUV and HSV buffers.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.

DLL	Requires ail11.dll.

MbufCopyComponent

Synopsis

Copy a data buffer, or components of a container, to components of a destination container.

Syntax

```
void MbufCopyComponent(
    AIL_ID SrcContainerOrBufId, //in
    AIL_ID DstContainerBufId,   //out
    AIL_INT64 SrcComponent,     //in
    AIL_INT64 Operation,        //in
    AIL_INT64 ControlFlag       //in
)
```

Description

This function copies the specified data buffer, or components of the specified source container, to the specified destination container. If the source is a container, you must specify which components to copy using SrcComponent.

By default, this function allocates one or more buffers and adds them as components of the destination container. The allocated buffers have the same buffer type, size, number of bands, component type, and some settings (such as 3D settings) as the source buffers. All data and other settings are also copied, unless you set the ControlFlag parameter to M_NO_COPY_SOURCE_DATA. Some attributes (such as M_COMPRESS) are not copied by default. You can specify that these attributes are also copied by setting the ControlFlag parameter to M_IDENTICAL.

Optionally, you can specify to replace matching components in the destination container (in some cases, reusing the existing buffers) by specifying the M_REPLACE operation.

When copying components from a container, you can force Aurora Imaging Library to use the destination components as-is and not free or reallocate them. To do so, set the ControlFlag parameter to M_USE_DESTINATION. If there is a mismatch between a component in the source and destination container (for example, the destination component is smaller than the source component), Aurora Imaging Library will attempt to compensate (for example, by only copying some of the data). If there are not the same number of components that meet the specified criteria in the source and destination containers, an error is generated.

Parameters

SrcContainerOrBufId

Specifies the identifier of the source data buffer or container. When copying a buffer to a container, you should first set the buffer's component type using MbufControl() with M_COMPONENT_TYPE.

DstContainerBufId

Specifies the identifier of the destination container.

SrcComponent

Specifies the criterion which will be used to determine which components of the source container to copy. If the source is a data buffer, this parameter must be set to **M_DEFAULT**.

The following settings can be used to uniquely identify a component of the source container.

For specifying which component to copy using a uniquely identifying criterion	
Value	Description
M_COMPONENT_BY_ID(AIL_ID ID)	Specifies the component with the specified Aurora Imaging Library identifier.
	Parameters
	ID Specifies the Aurora Imaging Library identifier of the component.
M_COMPONENT_BY_INDEX(AIL_INT ComponentIndex)	Specifies the component with the specified index in the container.
	Parameters
	ComponentIndex Specifies the index of the component.

The following settings can be used to copy one or more components of the source criteria.

For specifying which component(s) to copy using a generalized criterion	
Value	Description
M_COMPONENT_BY_GROUP_ID(AIL_INT GroupIndex)	Specifies that the component(s) have the specified group ID. The group ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_GROUP_ID.
	Parameters
	GroupIndex Specifies the group ID.
M_COMPONENT_BY_REGION_ID(AIL_INT RegionID)	Specifies that the component(s) have the specified region ID. The region ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_REGION_ID.
	Parameters
	RegionID Specifies the region ID.
M_COMPONENT_BY_SOURCE_ID(AIL_INT SourceID)	Specifies that the component(s) have the specified source ID. The source ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_SOURCE_ID.
	Parameters
	SourceID Specifies the source ID.

M_COMPONENT_ALL	Specifies all components of the container. When used with the M_REPLACE operation, this frees all existing components in the destination container and copies all components of the source container (unless the ControlFlag parameter is set to M_USE_DESTINATION).

The following settings can be used to copy one or more components with the specified component type.

For specifying which component(s) to copy by component type	
Value	Description
M_COMPONENT_CONFIDENCE	Specifies the components that store confidence information for an M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. INFO
M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies the components that store the A coordinate map provided by the camera. INFO
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies the components that store the B coordinate map provided by the camera. INFO
M_COMPONENT_CUSTOM + *n*	Specifies the components that have the specified custom component type , identified by *n*, where n can be a value between 0 and 254. INFO
M_COMPONENT_DISPARITY	Specifies the components that store a disparity map. INFO
M_COMPONENT_INFRARED	Specifies the components that store an intensity image of infrared light.
M_COMPONENT_INTENSITY	Specifies the components that store an intensity image of visible light.
M_COMPONENT_MATRIX_AIL	Specifies the components that store a 4x4 matrix that transforms depth map coordinates. INFO
M_COMPONENT_MESH_AIL	Specifies the components that store mesh information for an M_COMPONENT_RANGE component of the container. INFO
M_COMPONENT_METADATA	Specifies the components that store metadata information. INFO
M_COMPONENT_MULTISPECTRAL	Specifies the components that store an intensity image where each band represents the intensity of a specific wavelength of light. INFO
M_COMPONENT_NORMALS_AIL	Specifies the components that store normals information for each point in the M_COMPONENT_RANGE component of the container. INFO
M_COMPONENT_RANGE	Specifies the components that store 3D distance/position information. INFO
M_COMPONENT_REFLECTANCE	Specifies the components that store a reflectance map. INFO

M_COMPONENT_REGION_AIL	Specifies the components that store a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. INFO
M_COMPONENT_SCATTER	Specifies the components that store a scatter map. INFO
M_COMPONENT_ULTRAVIOLET	Specifies the components that store an intensity image of ultraviolet light.
M_COMPONENT_UNDEFINED	Specifies the components that store information of an unknown type.

Operation

Specifies how to copy the source buffer or component(s) to the destination container.

For specifying the operation to perform	
Value	Description
M_APPEND	Specifies that a new buffer is allocated and added as a component of the destination container for each buffer to be copied.
M_REPLACE	Specifies that existing component(s) in the destination container are replaced with the copied buffer(s). > This operation is not available when SrcComponent is set to a setting from the table For specifying which component to copy using a uniquely identifying criterion. For each buffer to be copied, Aurora Imaging Library determines if an existing component meets the same specified criteria and has the same buffer attributes (such as M_SIZE_X) as the source component; if so, it reuses the existing component. If no matching component is found, a new buffer is allocated with the correct attributes and added to the destination container. Any components in the destination container that meet the same specified criterion, but are not used during the copy operation, are freed. > If the source is a buffer, its M_COMPONENT_TYPE setting is used to determine which component in the destination to replace. An error will be generated if there is more than one component with this component type in the destination container.

ControlFlag

Specifies whether to copy the data, settings, and attributes. This parameter can be set to one of the following:

For specifying how the buffer or container is copied	
Value	Description
M_DEFAULT	Same as M_BASIC_ATTRIBUTE.
M_BASIC_ATTRIBUTE +	Specifies to copy the specified buffer (or component(s) of the specified container) to the destination container, including all data and settings. Optional attributes of buffers are not copied. For example, if the source buffer was allocated with the M_COMPRESS attribute, the destination buffer will be uncompressed.
M_IDENTICAL +	Specifies to copy the specified buffer (or component(s) of the specified container) to the destination container including all data, settings, and attributes (except M_PROC, M_GRAB and M_DISP which are always the same as the attributes of the destination container).

M_USE_DESTINATION	Specifies that all components in the source container that meet the criteria (specified in the SrcComponent parameter) are copied by rank to existing components in the destination container that meet the criteria. For example, if SrcComponent is set to M_COMPONENT_BY_GROUP_ID(), the first component with the specified group ID in the source container is copied to the first component with that group ID in the destination container, the second component with the specified group ID in the source container is copied to the second component with that group ID in the destination container, and so on. If the number of components that meet the specified criteria differs between the source container and destination container, an error is generated. Aurora Imaging Library attempts to compensate for any mismatch in the size and attributes of the source and destination components. For example, if the destination component has the M_COMPRESS attribute but the source component does not, Aurora Imaging Library automatically compresses the data. If Aurora Imaging Library cannot compensate, an error is generated. > This setting is only available for the M_REPLACE operation, with a container as a source.

Combination values for M_BASIC_ATTRIBUTE; M_IDENTICAL.

You can add one or more of the following values to the above-mentioned values to specify how the source is used for the copy operation.

For specifying how the source is used	
Combination value	Description
M_NO_COPY_SOURCE_DATA	Specifies to allocate component(s) in the destination container with the same component type and attributes as the source buffer or component(s), without copying data from the source. In addition, some settings (such as 3D settings, from the table For specifying settings useful with components that store 3D data) are copied.
M_SOURCE_MUST_EXIST	Specifies to generate an error if there is not at least one component in the source container that fits the specified criterion. > This setting is only available when the source is a container.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufCopyCond

Synopsis

Copy conditionally the source buffer to the destination buffer.

Syntax

```
void MbufCopyCond(
    AIL_ID SrcBufId,     //in
    AIL_ID DestBufId,    //out
    AIL_ID CondBufId,    //in
    AIL_INT64 Condition, //in
    AIL_DOUBLE CondValue //in
)
```

Description

This function copies the source buffer data to the destination buffer, modifying only those entries of the destination buffer which have a corresponding entry in the condition buffer that satisfies the specified condition. Other entries are unchanged. If the source and destination buffers are of different data types, Aurora Imaging Library converts the data automatically.

If the source buffer depth is greater than that of the destination, the most-significant bits are truncated when the data is copied to the destination. If the destination depth is greater than that of the source, the source data is zero or sign-extended (depending on the type of the source) when copied to the destination. For example, the data is zero-extended when copying an 8-bit unsigned buffer to a 16-bit unsigned buffer.

> Note that if a 1-band condition buffer is used with a multi-band destination buffer, the one band of the condition buffer will be used for each destination band.

Note, all multi-band buffers passed to this function must have the same number of bands.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcBufId

Specifies the identifier of the source data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DestBufId

Specifies the identifier of the destination data buffer.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

CondBufId

Specifies the identifier of the condition buffer.

Condition

Specifies the condition for which the condition buffer is tested. This parameter can be set to one of the following:

For specifying the condition for which the buffer is tested	
Value	Description
M_DEFAULT	Modify destination buffer entries corresponding to condition buffer entries that are non-zero.
M_EQUAL	Modify destination buffer entries corresponding to condition buffer entries that are equal to CondValue.
M_NOT_EQUAL	Modify destination buffer entries corresponding to condition buffer entries that are not equal to CondValue.

CondValue

Specifies the color value for the specified condition. Even though this value is of type *AIL_DOUBLE*, it is treated as if it had the same type and depth as the condition buffer. If the Condition parameter is set to M_DEFAULT, CondValue is ignored.

For a 3-band condition buffer, this parameter can be set to one of the following color values:

For specifying a color value		
Value	Description	
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies an RGB value when the source buffer, destination buffer, and condition buffer are 8-bit, and the condition buffer is a 3-band buffer. This allows you to compare each band of the condition buffer against a different value.	
		Parameters
		Red Specifies the red band, as a value between 0 and 255.
		Green Specifies the green band, as a value between 0 and 255.
		Blue Specifies the blue band, as a value between 0 and 255.
M_COLOR_BLACK	Specifies the color black.	
M_COLOR_BLUE	Specifies the color blue.	
M_COLOR_BRIGHT_GRAY	Specifies the color bright gray.	
M_COLOR_CYAN	Specifies the color cyan.	

M_COLOR_DARK_BLUE	Specifies the color dark blue.
M_COLOR_DARK_CYAN	Specifies the color dark cyan.
M_COLOR_DARK_GREEN	Specifies the color dark green.
M_COLOR_DARK_MAGENTA	Specifies the color dark magenta.
M_COLOR_DARK_RED	Specifies the color dark red.
M_COLOR_DARK_YELLOW	Specifies the color dark yellow.
M_COLOR_GRAY	Specifies the color gray.
M_COLOR_GREEN	Specifies the color green.
M_COLOR_LIGHT_BLUE	Specifies the color light blue.
M_COLOR_LIGHT_GRAY	Specifies the color light gray.
M_COLOR_LIGHT_GREEN	Specifies the color light green.
M_COLOR_LIGHT_WHITE	Specifies the color light white.
M_COLOR_MAGENTA	Specifies the color magenta.
M_COLOR_RED	Specifies the color red.
M_COLOR_WHITE	Specifies the color white.
M_COLOR_YELLOW	Specifies the color yellow.

For a 1-band or multi-band condition buffer, you can set this parameter to the following grayscale value:

For specifying a grayscale value to be used with all bands	
Value	Description
Value	Specifies a grayscale value. If the condition buffer is binary, this value must be set to 0 or 1. If the condition buffer is multi-band, this value will be used for each band individually.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufCopyMask

Synopsis

Copy buffer with mask.

Syntax

```
void MbufCopyMask(
    AIL_ID SrcBufId,    //in
    AIL_ID DestBufId,   //out
    AIL_INT64 MaskValue //in
)
```

Description

This function copies the specified source buffer data to the specified destination buffer, modifying only the bits of the destination that have a non-zero corresponding bit in the mask.

Note, if the source and destination are both multi-band buffers, they must have the same number of bands.

You can limit this function's results to a region of an image buffer using a region of interest (ROI) set using MbufSetRegion(). The ROI must be defined in raster format (M_RASTER or M_VECTOR_AND_RASTER). An error is generated if the ROI is only in vector format (M_VECTOR). If you specify multiple image buffers with an ROI, results are limited to the portion of the ROIs that intersect.

Parameters

SrcBufId

Specifies the identifier of the source data buffer. Note that the source data buffer cannot be of type floating-point.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

DestBufId

Specifies the identifier of the destination data buffer. Note that the destination data buffer cannot be of type floating-point.

If the source buffer depth is greater than that of the destination, the most significant bits are truncated when the data is copied to the destination. If the destination depth is greater than that of the source, the source data is zero or sign-extended (depending on the type of the source) when copied to the destination.

If you specify an image buffer that has an ROI associated with it, the ROI must be in raster format; otherwise, you will get an error.

MaskValue

Specifies the mask value. Even though this value is of type *AIL_INT64*, it is treated as if it had the same depth as the destination buffer; the most-significant bits that are not required are ignored.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.

DLL	Requires ail11.dll.

MbufCreate2d

Synopsis

Create a two-dimensional data buffer.

Syntax

```
AIL_ID MbufCreate2d(
    AIL_ID SystemId,       //in
    AIL_INT SizeX,         //in
    AIL_INT SizeY,         //in
    AIL_INT Type,          //in
    AIL_INT64 Attribute,   //in
    AIL_INT64 ControlFlag, //in
    AIL_INT Pitch,         //in
    const void *DataPtr,   //in
    AIL_ID *BufIdPtr       //out
)
```

Description

This function creates a two-dimensional data buffer using memory at a specified location, and associates it with a specific Aurora Imaging Library system.

> This function should be used with caution because, when using physical addresses, it provides direct access to any of your computer's memory mapped devices; when using logical addresses, memory protection errors could result.

It is generally better to leave buffer allocation, data loading, and memory control to Aurora Imaging Library (MbufAlloc2d(), MbufGet2d(), MbufPut2d()), since Aurora Imaging Library might require special memory attributes (such as non-paged memory) or alignment to associate the buffer with a particular target system.

You must allocate the appropriate memory before calling **MbufCreate2d**(). MbufInquire() can be used to get the pointer to the data of an Aurora Imaging Library allocated buffer.

[Zebra Radient eV-CL; Zebra Rapixo CL; Zebra Rapixo CoF; Zebra Rapixo CXP]

Note that to create a buffer mapped to on-board memory, it must be shared memory. See your Hardware-specific Notes for more information regarding shared memory.

After creating the two-dimensional data buffer, you should check if the operation was successful, using MappGetError() or by verifying that the two-dimensional data buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the two-dimensional data buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the two-dimensional data buffer's lifetime and you must not manually free it.

> If you create a buffer using **MbufCreate2d**(), freeing the buffer (either manually using MbufFree() or automatically with a smart identifier) only releases the internal structure required for the mapped buffer. You are responsible for freeing the underlying memory used by the buffer.

Parameters

SystemId INQ

Specifies the Aurora Imaging Library system on which to create the buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

SizeX INQ

Specifies the buffer width.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer width																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the buffer width is identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies buffer width. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, width is specified in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SizeY INQ

Specifies the buffer height.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer height																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the buffer height is identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the buffer height. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, height is specified in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Type INQ

Specifies a combination of two values: data type and data depth. Specify the depth of the buffer as one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo,

j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data depth of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the buffer data depth and type are identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
1 +	Specifies a 1-bit (packed binary) buffer. Note that you cannot allocate a 1-bit LUT, kernel, or structuring element buffer.	c		e	f	g	a		i		j	k	l	m	n	b
8 +	Specifies an 8-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
16 +	Specifies a 16-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
32 +	Specifies a 32-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the data depth of the buffer.

You can add one of the following values to the above-mentioned values to set the data type.

Supported data types depend on the specified depth.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type of the buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLOAT	Specifies that the type of data is floating-point. The valid data depth is 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the type of data is signed. The valid data depths are 8, 16 or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the type of data is unsigned. The valid data depths are 1, 8, 16 or 32 bits. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute INQ

Specifies the buffer usage. This allows you to specify the type of buffer, intended purpose, compression type, storage format, data direction, location, internal storage format, memory type, and overscan region of the created buffer; all of which provides additional information for other Aurora Imaging Library functions.

This function should be used with caution because, when using physical addresses, it provides direct access to any of your computer's memory mapped devices; when using logical addresses, memory protection errors could result.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo,

j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer usage																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ARRAY	Specifies a buffer to store array type data. Note that some functions take an M_ARRAY buffer rather than a user-defined array.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data. You must specify a combination value from the following table: • To set the intended purpose of this buffer	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a kernel buffer to store a custom filter for convolution functions. The depth must be 8, 16, or 32-bit integer or floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT	Specifies a buffer to store lookup table data. The valid data depths are 8, 16, or 32-bit.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a buffer to store structuring element data for morphology functions. The data depth must be 32-bit. If signed, the range is -32768 to +32767. If unsigned, the range is 0 to +32767. Note that the data can be set to **M_DONT_CARE** to ignore specific structuring elements.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You must add one or more of the following values to the above-mentioned value to set the intended purpose of this buffer.

Note that this value must not conflict with the attribute of the specified physical memory to which it is mapped. Such a conflict can result in errors that Aurora Imaging Library cannot catch.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_COMPRESS +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an image buffer that can hold compressed data. Note that a buffer with this attribute cannot have the M_SIGNED data type. > If M_COMPRESS is specified, the underlying data must already be compressed with the specified compression type. The Type SizeX and SizeY parameters must also be set to the exact size of the compressed image stored in the underlying data. It is not possible to create a compressed buffer mapped on to only part of a compressed image. When mapping buffers for operations that require a source and destination buffer, and one of the buffers has an M_COMPRESS attribute, the data will be compressed or decompressed depending on the attributes of the destination buffer. If both the source and destination buffers have M_COMPRESS specifiers but different compression types, the data will be re-compressed according to the settings of the destination buffer. > Note that buffers created from preallocated memory with the M_COMPRESS attribute should typically only be passed to Aurora Imaging Library functions as a source. If you pass a compressed buffer created from preallocated memory as a destination for an Aurora Imaging Library function, an error will be generated if the compressed size in memory of the function output is not the same as the size of the allocated memory. This is true even if the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISP	Specifies an image buffer that can be displayed.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies an image buffer in which to grab data.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific														
	The buffer must be allocated in physically contiguous, non-paged memory. Pageable memory grab buffers are not supported for these systems. The ControlFlag parameter should be set to M_PHYSICAL_ADDRESS, or M_AIL_ID if the buffer maps to an existing source buffer with a physical address.	c							i		j	k	l	m		
	Single-band (monochrome) buffers must have a depth of 8 bits or 16 bits to have an M_GRAB attribute.			e	f	g					j	k	l	m	n	b
	Single-band (monochrome) buffers must have a depth of 8 bits to have an M_GRAB attribute.								i							
M_PROC	Specifies an image buffer that can be processed.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

> Note that this value must not conflict with the compression format of the underlying data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type INQ																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data format is 1-band, 8-bit data. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data format is 1-band, 8-bit data.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies the physical nature of the buffer. This parameter can be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the physical nature of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AIL_ID +	Specifies that the new buffer maps to an existing source buffer. DataPtr points to the Aurora Imaging Library identifier of the source buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b
M_HOST_ADDRESS +	Specifies that DataPtr is a logical address that points to the data. Note that when using logical addresses, memory protection errors could result. Note that you can use MbufInquire() with M_HOST_ADDRESS to get the logical address of an Aurora Imaging Library allocated buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b
M_PHYSICAL_ADDRESS +	Specifies that DataPtr is a physical address that points to the data. Note that using physical addresses provides direct access to any of your computer's memory mapped devices. Note that you can use MbufInquire() with M_PHYSICAL_ADDRESS to get the physical address of an Aurora Imaging Library allocated buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the physical nature of the buffer.

You must add one of the following values to the above-mentioned values to set how the pitch is measured.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how the pitch is measured																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PITCH	Specifies that the pitch is in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PITCH_BYTE	Specifies that the pitch is in bytes.	c		e	f	g	a		i		j	k	l	m	n	b

Pitch INQ

Specifies the size of the pitch. The pitch is the number of pixels or bytes (as specified by the ControlFlag parameter) between the start of any two adjacent rows of the buffer. The actual length of a row in the buffer, in physical memory, might be different from the value of the SizeX parameter (for example, due to use of buffer overscan).

> Note that your code should not make assumptions about the actual pitch of the source memory. If the memory was allocated using an MbufAlloc...() function, use MbufInquire() with M_PITCH or M_PITCH_BYTE to establish the required pitch.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the size of the pitch																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that when dealing with a 1-bit buffer, the pitch is a multiple of 4 bytes; otherwise the pitch equals the SizeX parameter.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the pitch in pixels or bytes (as determined by the ControlFlag parameter).	c		e	f	g	a		i		j	k	l	m	n	b

DataPtr

Specifies the Aurora Imaging Library identifier of the buffer, or a pointer to the data array, on which to map the created Aurora Imaging Library buffer.

When the ControlFlag parameter is set to M_AIL_ID, DataPtr specifies the Aurora Imaging Library identifier of the source buffer.

When the ControlFlag parameter is set to M_HOST_ADDRESS, DataPtr specifies a logical address that points to the data.

When the ControlFlag parameter is set to M_PHYSICAL_ADDRESS, DataPtr specifies a physical address that points to the data.

BufIdPtr

Specifies the address of the variable in which to write the buffer identifier or specifies the data type that the function should use to return the buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the created buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the two-dimensional data buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remarks

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.
- *[Aurora Imaging Library Lite]*
  You can allocate an image buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute with Aurora Imaging Library Lite. However, these attributes are not required to work under Aurora Imaging Library Lite.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.

DLL	Requires ail11.dll.

MbufCreateColor

Synopsis

Create a multi-band data buffer.

Syntax

```
AIL_ID MbufCreateColor(
    AIL_ID SystemId,       //in
    AIL_INT SizeBand,      //in
    AIL_INT SizeX,         //in
    AIL_INT SizeY,         //in
    AIL_INT Type,          //in
    AIL_INT64 Attribute,   //in
    AIL_INT64 ControlFlag, //in
    AIL_INT Pitch,         //in
    void **ArrayOfDataPtr, //in
    AIL_ID *BufIdPtr       //out
)
```

Description

This function creates a multi-band data buffer using memory at a specified location, and associates it with a specific Aurora Imaging Library system.

> This function should be used with caution because, when using physical addresses, it provides direct access to any of your computer's memory mapped devices; when using logical addresses, memory protection errors could result.

It is generally better to leave buffer allocation, data loading, and memory control to Aurora Imaging Library (MbufAllocColor(), MbufGetColor(), MbufPutColor()), since Aurora Imaging Library might require special memory attributes (such as non-paged memory) or alignment to associate the buffer with a particular target system.

You must allocate the appropriate memory before calling **MbufCreateColor**(). MbufInquire() can be used to get the pointer to the data of an Aurora Imaging Library allocated buffer.

If creating a buffer over a JPEG stream in one of the supported formats, Aurora Imaging Library can automatically set the size, pitch, and type. To do so, set SizeBand, SizeX, SizeY, Type, and Pitch to M_DEFAULT, and set Attribute to M_COMPRESS without any compression type. Note that in this case, you must provide a host address (M_HOST_ADDRESS).

ControlFlag set to M_HOST_ADDRESS; and M_IMAGE + M_COMPRESS be set without any compression type.

If creating a 3-band buffer, be as specific as possible when setting the buffer attributes (using the Attribute parameter) and make sure to specify the color format of the buffer. The more information known about the associated buffer, the better the results.

[Zebra Radient eV-CL; Zebra Rapixo CL; Zebra Rapixo CoF; Zebra Rapixo CXP]

Note that to create a buffer mapped to on-board memory, it must be shared memory. See your Hardware-specific Notes for more information regarding shared memory.

After creating the data buffer, you should check if the operation was successful, using MappGetError() or by verifying that the data buffer identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the data buffer is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the data buffer's lifetime and you must not manually free it.

> If you create a buffer using **MbufCreateColor**(), freeing the buffer (either manually using MbufFree() or automatically with a smart identifier) only releases the internal structure required for the mapped buffer. You are responsible for freeing the underlying memory used by the buffer.

Parameters

SystemId INQ

Specifies the Aurora Imaging Library system on which to create the buffer. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the Aurora Imaging Library system																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

SizeBand INQ

Specifies the number of (x,y) surfaces (also called bands) that the buffer should have to contain each color or other type of data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of surfaces																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the same number of bands as the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 1024	Specifies the number of bands. There are generally either 1 or 3 bands. When acquiring or processing monochrome images, the buffer requires only 1 band. For RGB color images, it requires 3 color bands. This value can be any non-zero integer value up to 1024 for an image buffer, or 1 or 3 for an array or LUT buffer.	c		e	f	g	a		i		j	k	l	m	n	b

SizeX INQ

Specifies the buffer width.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the buffer width																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the buffer width is identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the buffer width. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, width is specified in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SizeY INQ

Specifies the buffer height.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the buffer height																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the buffer height is identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the buffer height. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, height is specified in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Type INQ

Specifies a combination of two values: data depth and data type, per band. Specify the depth of the buffer per band as one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the depth of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the data depth and type are identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
1 +	Specifies a 1-bit (packed binary) buffer. Note that you cannot create a 1-bit LUT buffer.	c		e	f	g	a		i		j	k	l	m	n	b
8 +	Specifies an 8-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
16 +	Specifies a 16-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
32 +	Specifies a 32-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For the depth of the buffer.

You can add one of the following values to the above-mentioned values to set the data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLOAT	Specifies that the type of data is floating-point. The valid data depth is 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the type of data is signed. The valid data depths are 8, 16, or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the type of data is unsigned. The valid data depths are 1, 8, 16, or 32 bits. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute INQ

Specifies the buffer usage. This allows you to specify the type of buffer, intended purpose, compression type, storage format, data direction, location, internal storage format, memory type, and overscan region of the created buffer; all of which provides additional information for other Aurora Imaging Library functions.

This function should be used with caution because, when using physical addresses, it provides direct access to any of your computer's memory mapped devices; when using logical addresses, memory protection errors could result.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer usage																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ARRAY	Specifies a buffer to store array type data. Note that some functions take an M_ARRAY buffer rather than a user-defined array. Note that this attribute is only supported for 1-band and 3-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data. You must specify a combination value from the following table: • To set the intended purpose of this buffer	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT	Specifies a buffer to store lookup table data. The depth must be 8, 16, or 32-bit integer or floating-point and the internal storage format must be planar (M_PLANAR). Note that this attribute is only supported for 1-band and 3-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE.

You must add one or more of the following values to the above-mentioned value to set the intended purpose of this buffer.

Note that this value must not conflict with the attribute of the specified physical memory to which it is mapped. Such a conflict can result in errors that Aurora Imaging Library cannot catch.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESS +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an image buffer that can hold compressed data. Note that a buffer with this attribute cannot have the M_SIGNED data type. > If M_COMPRESS without any compression type is specified and the underlying data is of type JPEG, SizeBand, Type, SizeX, and SizeY parameters can be set to M_DEFAULT. For all other cases, the underlying data must already be compressed with the specified compression type and color format. The SizeBand, Type SizeX and SizeY parameters must also be set to the exact size of the compressed image stored in the underlying data. It is not possible to create a compressed buffer mapped on to only part of a compressed image. When mapping buffers for operations that require a source and destination buffer, and one of the buffers has an M_COMPRESS attribute, the data will be compressed or decompressed depending on the attributes of the destination buffer. If both the source and destination buffers have M_COMPRESS specifiers but different compression types, the data will be re-compressed according to the settings of the destination buffer. > Note that buffers created from preallocated memory with the M_COMPRESS attribute should typically only be passed to Aurora Imaging Library functions as a source. If you pass a compressed buffer created from preallocated memory as a destination for an Aurora Imaging Library function, an error will be generated if the compressed size in memory of the function output is not the same as the size of the allocated memory. This is true even if the ControlFlag parameter is set to M_AIL_ID. Only 1-band and 3-band buffers can have an M_COMPRESS attribute. You must specify a combination value from the following table: • To set a color buffer format	c		e	f	g	a		i		j	k	l	m	n	b

M_DISP +	Specifies an image buffer that can be displayed. You must specify a combination value from the following table: • To set a color buffer format	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB +	Specifies an image buffer in which to grab data. Only 1-band and 3-band buffers can have an M_GRAB attribute. You must specify a combination value from the following table: • To set a color buffer format	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	The buffer must be allocated in physically contiguous, non-paged memory. Pageable memory grab buffers are not supported for these systems. The ControlFlag parameter should be set to M_PHYSICAL_ADDRESS, or M_AIL_ID if the buffer maps to an existing source buffer with a physical address.	c							i		j	k	l	m		
	Single-band (monochrome) buffers must have a depth of 8 bits or 16 bits to have an M_GRAB attribute. Only some three-band (color) buffers can have an M_GRAB attribute. See the constants for color buffer formats to determine which are supported with an M_GRAB attribute.			e	f	g					j	k	l	m	n	b
	Single-band (monochrome) buffers must have a depth of 8 bits or 16 bits to have an M_GRAB attribute. Only some three-band (color) buffers can have an M_GRAB attribute. See the constants for color buffer formats to determine which are supported with an M_GRAB attribute. Note that color buffers with an M_GRAB attribute are only available for color versions of Zebra Iris GTX (such as 2000C).								i							
M_PROC +	Specifies an image buffer that can be processed. You must specify a combination value from the following table: • To set a color buffer format	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

> Note that this value must not conflict with the compression format of the underlying data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type INQ																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24 + M_PLANAR, M_RGB48 + M_PLANAR, or M_YUV24 + M_PLANAR format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24, M_RGB48, M_YUV9, M_YUV12, M_YUV16 + M_PLANAR, or M_YUV24 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band data in M_RGB24 or M_RGB48 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_RGB24, M_YUV24, M_YUV12, M_YUV9, M_YUV16 + M_PLANAR, or M_YUV16 + M_PACKED format. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_YUV16 + M_PACKED format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE; M_COMPRESS.

You can add one of the following values to the above-mentioned values to set a storage format for image buffers.

When dealing with 3-band buffers, you must specify the appropriate internal storage format of the buffer by adding one of the following values. Note that Aurora Imaging Library needs this information to manipulate the data; for example, you do not want Aurora Imaging Library to interpret a packed data buffer as a planar.

Although there is no right or wrong storage format for Aurora Imaging Library buffers, some operations are optimized for certain formats.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format for image buffers INQ																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). Typically, packed buffers are used for display. Note that it might be slower to process buffers with the M_PACKED attribute. Also, note that you cannot allocate a packed LUT buffer. You must specify a combination value from one of the following tables: • To set a color buffer format • To set a packed color buffer format	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. Typically, planar buffers are used for processing. You must specify a combination value from one of the following tables: • To set a color buffer format • To set a planar color buffer format	c			f		a		i		j	k	l	m		

Combination values for M_PLANAR.

You must add one of the following values to the above-mentioned value to set a planar color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

The specified format can differ from that of the data in memory (unless M_COMPRESS is specified). For example, data in an RGB24 format will be interpreted as YUV data if M_YUV24 is specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB3	Specifies 3-bit color depth (RGB 1:1:1) planar pixels.	c					a		i		j	k	l	m		
M_YUV9	Specifies YUV9 planar standard.	c			f		a		i		j	k	l	m		
M_YUV12	Specifies YUV12 planar standard.	c			f		a		i		j	k	l	m		
M_YUV24	Specifies YUV24 planar standard.	c			f		a		i		j	k	l	m		

Combination values for M_PACKED.

You must add one of the following values to the above-mentioned value to set a packed color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

The specified format can differ from that of the data in memory (unless M_COMPRESS is specified). For example, data in an YUV24 format will be interpreted as RGB data if M_RGB24 is specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth (BGR) packed pixels (BGRBGR).	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.			e		g									n	b
M_BGR32	Specifies 32-bit color depth (BGR) packed pixels (BGRXBGRX). The most-significant byte is a "don't care" byte.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.			e		g									n	b
M_RGB15	Specifies 16-bit color depth packed pixels (XRGB 1:5:5:5). Note that when accessing an M_RGB15 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b

M_RGB16	Specifies 16-bit color depth packed pixels (RGB 5:6:5). Note that when accessing an M_RGB16 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. For more information, see the YUV buffers section of Chapter 23: Data buffers.			e	f	g									n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.			e		g									n	b
M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. For more information, see the YUV buffers section of Chapter 23: Data buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) buffers.			e		g									n	b

Combination values for the values listed in For specifying the intended purpose of the image buffer, For specifying a storage format for image buffers.

You must add one of the following values to the above-mentioned values to set a color buffer format. Alternatively, you can add a value from For specifying a packed color buffer format or from For specifying a planar color buffer format.

Note that the values below cannot be used with grab buffers (M_GRAB), unless otherwise stated.

The specified format can differ from that of the data in memory (unless M_COMPRESS is specified). For example, data in an RGB16 format will be interpreted as YUV data if M_YUV16 is specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed or planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB24	Specifies 24-bit color depth (RGB 8:8:8) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers.								i		j	k	l	m		
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g									n	b

M_RGB48	Specifies 48-bit color depth (RGB 16:16:16). packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.			e	f	g					j	k	l	m	n	b
	This value can be used with planar (M_PLANAR) grab (M_GRAB) buffers.										j	k	l	m		
M_RGB96	Specifies 96-bit color depth (RGB 32:32:32) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16	Specifies YUV16 packed or planar (4:2:2) standard.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) grab (M_GRAB) buffers.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies the physical nature of the buffer. This parameter can be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the physical nature of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AIL_ID +	Specifies that the new buffer maps to an existing source buffer. ArrayOfDataPtr is a pointer to an array that stores a pointer to the Aurora Imaging Library identifier of the source buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b

M_HOST_ADDRESS +	Specifies that ArrayOfDataPtr is a pointer to an array of host addresses that point to the data. The number of pointers in the array depends on the number of bands and whether the data is compressed or packed. Note that when using logical addresses, memory protection errors could result. Note that you can use MbufInquire() with M_HOST_ADDRESS to get the logical address of an Aurora Imaging Library allocated buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b
M_PHYSICAL_ADDRESS +	Specifies that ArrayOfDataPtr is a pointer to an array of physical addresses that point to the data. The number of pointers in the array depends on the number of bands and whether the data is compressed or packed. Note that using physical addresses provides direct access to any of your computer's memory mapped devices. Note that you can use MbufInquire() with M_PHYSICAL_ADDRESS to get the physical address of an Aurora Imaging Library allocated buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For specifying the physical nature of the buffer.

You must add one of the following values to the above-mentioned values to set how the pitch is measured.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how the pitch is measured																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PITCH	Specifies that the pitch is in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PITCH_BYTE	Specifies that the pitch is in bytes.	c		e	f	g	a		i		j	k	l	m	n	b

Pitch INQ

Specifies the size of the pitch. The pitch is the number of pixels or bytes (as specified by the ControlFlag parameter) between the start of any two adjacent rows of the buffer. The actual length of a row in the buffer, in physical memory, might be different from the value of the SizeX parameter (for example, due to use of buffer overscan).

> Note that your code should not make assumptions about the actual pitch of the source memory. If the memory was allocated using an MbufAlloc...() function, use MbufInquire() with M_PITCH or M_PITCH_BYTE to establish the required pitch.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the size of the pitch																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that when dealing with a 1-bit buffer, the pitch is a multiple of 4 bytes; otherwise the pitch equals the SizeX parameter.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the pitch in pixels or bytes (as determined by the ControlFlag parameter).	c		e	f	g	a		i		j	k	l	m	n	b

ArrayOfDataPtr

Specifies one or more data pointers that address the memory to which to map the created Aurora Imaging Library buffer.

When pointing to a single-band buffer, a packed buffer, or a compressed buffer, a single pointer to the packed data must be provided. When pointing to an uncompressed planar buffer, one pointer per band must be provided (except when the ControlFlag parameter is set to M_AIL_ID). Pointers to a 3-band planar buffer must be ordered R-G-B or Y-U-V in the array.

When the ControlFlag parameter is set to M_AIL_ID, ArrayOfDataPtr specifies the address of an array that stores the Aurora Imaging Library identifier of the source buffer.

When the ControlFlag parameter is set to M_HOST_ADDRESS, ArrayOfDataPtr specifies the address of an array that stores pointer(s) to the logical address(es) that points to the data.

When the ControlFlag parameter is set to M_PHYSICAL_ADDRESS, ArrayOfDataPtr specifies the address of an array that stores pointer(s) to the physical address(es) that points to the data.

BufIdPtr

Specifies the address of the variable in which to write the data buffer identifier or specifies the data type that the function should use to return the data buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address for the array buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the created array buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address for the image buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the created image buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address for the LUT buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the created LUT buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the data buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remark

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the

compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufCreateComponent

Synopsis

Create a component in the specified container.

Syntax

```
AIL_ID MbufCreateComponent(
    AIL_ID ContainerBufId,   //in
    AIL_INT SizeBand,        //in
    AIL_INT SizeX,           //in
    AIL_INT SizeY,           //in
    AIL_INT Type,            //in
    AIL_INT64 Attribute,     //in
    AIL_INT64 ControlFlag,   //in
    AIL_INT Pitch,           //in
    void **ArrayOfDataPtr,   //in
    AIL_INT64 ComponentType, //in
    AIL_ID *ComponentIdPtr   //out
)
```

Description

This function creates a data buffer using memory at a specified location, and makes it a component of a specified container. The buffer is allocated on the same system as the container.

> This function should be used with caution because, when using physical addresses, it provides direct access to any of your computer's memory mapped devices; when using logical addresses, memory protection errors could result.

Typically, you should leave buffer allocation, data loading, and memory control to Aurora Imaging Library (MbufAllocComponent(), MbufGetColor(), MbufPutColor()), since Aurora Imaging Library might require special memory attributes (such as non-paged memory) or alignment to associate the buffer component with the same system as the container.

You must allocate the appropriate memory before calling **MbufCreateComponent**() (except when M_PHYSICAL_ADDRESS is specified), and you must free the created buffer when no longer required, either using MbufFreeComponent() or by freeing its container using MbufFree(), before freeing the memory. Freeing a buffer created with **MbufCreateComponent**() will free the internal structure required for a mapped buffer, but it will not free the memory used.

If creating a 3-band buffer, be as specific as possible when setting the buffer attributes (using the Attribute parameter) and make sure to specify the color format of the buffer. The more information known about the associated buffer, the better the results.

[Zebra Radient eV-CL; Zebra Rapixo CL; Zebra Rapixo CoF; Zebra Rapixo CXP]

Note that to create a buffer mapped to on-board memory, it must be shared memory. See your Hardware-specific Notes for more information regarding shared memory.

Parameters

ContainerBufId

Specifies the Aurora Imaging Library identifier of the container in which to create the buffer component. The

buffer will be allocated on the same system as the container.

SizeBand INQ

Specifies the number of (x,y) surfaces (also called bands) that the buffer should have to contain each color or other type of data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the number of surfaces																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the same number of bands as the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 1024	Specifies the number of bands. There are generally either 1 or 3 bands. When acquiring or processing monochrome images, the buffer requires only 1 band. For RGB color images, it requires 3 color bands. This value can be any non-zero integer value up to 1024 for an image buffer, 1 or 3 for an array or LUT buffer, or 1 for a kernel or structuring element buffer.	c		e	f	g	a		i		j	k	l	m	n	b

SizeX INQ

Specifies the buffer width.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the buffer width																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the buffer width is identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the buffer width. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, width is specified in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SizeY INQ

Specifies the buffer height.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the buffer height																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the buffer height is identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the buffer height. The units are determined by the selected buffer attribute. For example, if the buffer has an image buffer attribute, height is specified in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Type INQ

Specifies a combination of two values: data depth and data type, per band. Specify the depth of the buffer per band as one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the depth of the buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the data depth and type are identical to that of the source buffer when the ControlFlag parameter is set to M_AIL_ID.	c		e	f	g	a		i		j	k	l	m	n	b
1 +	Specifies a 1-bit (packed binary) buffer. Note that you cannot create a 1-bit LUT buffer.	c		e	f	g	a		i		j	k	l	m	n	b
8 +	Specifies an 8-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
16 +	Specifies a 16-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b
32 +	Specifies a 32-bit buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For the depth of the buffer.

You can add one of the following values to the above-mentioned values to set the data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FLOAT	Specifies that the type of data is floating-point. The valid data depth is 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the type of data is signed. The valid data depths are 8, 16, or 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the type of data is unsigned. The valid data depths are 1, 8, 16, or 32 bits. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

Attribute

Specifies the buffer usage. This allows you to specify the type of buffer, compression type, storage format, data direction, location, internal storage format, memory type, and overscan region of the created buffer; all of which

provides additional information for other Aurora Imaging Library functions.

Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer usage																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ARRAY	Specifies a buffer to store array type data. Note that some functions take an M_ARRAY buffer rather than a user-defined array. Note that this attribute is only supported for 1-band and 3-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data.	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a kernel buffer to store a custom filter for convolution functions. The depth must be 8, 16, or 32-bit integer or floating-point. Note that this attribute is only supported for 1-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT	Specifies a buffer to store lookup table data. The depth must be 8, 16, or 32-bit integer or floating-point and the internal storage format must be planar (M_PLANAR). Note that this attribute is only supported for 1-band and 3-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT	[For essential Aurora Imaging Library Lite information, see remarks] Specifies a buffer to store structuring element data for morphology functions. The data depth must be 32-bit. If signed, the range is -32768 to +32767. If unsigned, the range is 0 to +32767. Note that the data can be set to **M_DONT_CARE** to ignore specific structuring elements. Note that this attribute is only supported for 1-band buffers.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_IMAGE.

You can add the following value to the above-mentioned value to set the intended purpose of this buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the intended purpose of the image buffer

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESS +	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an image buffer that can hold compressed data. Note that a buffer with this attribute cannot have the M_SIGNED data type. > If M_COMPRESS is specified, the underlying data must already be compressed with the specified compression type and color format. The SizeBand, Type SizeX and SizeY parameters must also be set to the exact size of the compressed image stored in the underlying data. It is not possible to create a compressed buffer mapped on to only part of a compressed image. When mapping buffers for operations that require a source and destination buffer, and one of the buffers has an M_COMPRESS attribute, the data will be compressed or decompressed depending on the attributes of the destination buffer. If both the source and destination buffers have M_COMPRESS specifiers but different compression types, the data will be re-compressed according to the settings of the destination buffer. > Note that buffers created from preallocated memory with the M_COMPRESS attribute should typically only be passed to Aurora Imaging Library functions as a source. If you pass a compressed buffer created from preallocated memory as a destination for an Aurora Imaging Library function, an error will be generated if the compressed size in memory of the function output is not the same as the size of the allocated memory. This is true even if the ControlFlag parameter is set to M_AIL_ID. Only 1-band and 3-band buffers can have an M_COMPRESS attribute. You must specify a combination value from the following table: • To set a color buffer format	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

You can add one of the following values to the above-mentioned value to set the compression type.

The image buffer's data depth dictates which compression type can be selected.

> Note that this value must not conflict with the compression format of the underlying data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24 + M_PLANAR, M_RGB48 + M_PLANAR, or M_YUV24 + M_PLANAR format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG2000 lossy data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band, 8- or 16-bit data in M_RGB24, M_RGB48, M_YUV9, M_YUV12, M_YUV16 + M_PLANAR, or M_YUV24 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data. The supported data formats are: 1-band, 8- or 16-bit data, and 3-band data in M_RGB24 or M_RGB48 format.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossless data in separate fields. The supported data formats are: 1-band, 8- or 16-bit data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_RGB24, M_YUV24, M_YUV12, M_YUV9, M_YUV16 + M_PLANAR, or M_YUV16 + M_PACKED format. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer will be used to hold JPEG lossy data in separate fields. The supported data formats are: 1-band 8-bit data, and 3-band 8-bit data in M_YUV16 + M_PACKED format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_IMAGE; M_COMPRESS.

You can add one of the following values to the above-mentioned values to set a storage format for image buffers.

When dealing with 3-band buffers, you must specify the appropriate internal storage format of the buffer by

adding one of the following values. Note that Aurora Imaging Library needs this information to manipulate the data; for example, you do not want Aurora Imaging Library to interpret a packed data buffer as a planar.

Although there is no right or wrong storage format for Aurora Imaging Library buffers, some operations are optimized for certain formats.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a storage format for image buffers INQ																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). Typically, packed buffers are used for display. Note that it might be slower to process buffers with the M_PACKED attribute. Also, note that you cannot allocate a packed LUT buffer. You must specify a combination value from one of the following tables: • To set a color buffer format • To set a packed color buffer format	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. Typically, planar buffers are used for processing. You must specify a combination value from one of the following tables: • To set a color buffer format • To set a planar color buffer format	c			f		a		i		j	k	l	m		

Combination values for M_PLANAR.

You must add one of the following values to the above-mentioned value to set a planar color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab containers (M_GRAB), unless otherwise stated.

The specified format can differ from that of the data in memory (unless M_COMPRESS is specified). For example, data in an RGB24 format will be interpreted as YUV data if M_YUV24 is specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB3	Specifies 3-bit color depth (RGB 1:1:1) planar pixels.	c					a		i		j	k	l	m		
M_YUV9	Specifies YUV9 planar standard.	c			f		a		i		j	k	l	m		
M_YUV12	Specifies YUV12 planar standard.	c			f		a		i		j	k	l	m		

M_YUV24	Specifies YUV24 planar standard.	c			f		a		i		j	k	l	m		

Combination values for M_PACKED.

You must add one of the following values to the above-mentioned value to set a packed color buffer format. Alternatively, you can add a value from For specifying a packed or planar color buffer format.

Note that the values below cannot be used with grab containers (M_GRAB), unless otherwise stated.

The specified format can differ from that of the data in memory (unless M_COMPRESS is specified). For example, data in an YUV24 format will be interpreted as RGB data if M_RGB24 is specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed color buffer format																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth (BGR) packed pixels (BGRBGR).	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) containers.					g										
M_BGR32	Specifies 32-bit color depth (BGR) packed pixels (BGRXBGRX). The most-significant byte is a "don't care" byte.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) containers.					g										
M_RGB15	Specifies 16-bit color depth packed pixels (XRGB 1:5:5:5). Note that when accessing an M_RGB15 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB16	Specifies 16-bit color depth packed pixels (RGB 5:6:5). Note that when accessing an M_RGB16 + M_PACKED buffer as a 3-band 8-bit buffer, the least significant bits are set to 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. For more information, see the YUV buffers section of Chapter 23: Data buffers.			e	f	g									n	b
	System specific															
	This value can be used with grab (M_GRAB) containers.					g										

M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. For more information, see the YUV buffers section of Chapter 23: Data buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with grab (M_GRAB) containers.					g										

Combination values for the values listed in For specifying the intended purpose of the image buffer, For specifying a storage format for image buffers.

You must add one of the following values to the above-mentioned values to set a color buffer format. Alternatively, you can add a value from For specifying a packed color buffer format or from For specifying a planar color buffer format.

Note that the values below cannot be used with grab containers (M_GRAB), unless otherwise stated.

The specified format can differ from that of the data in memory (unless M_COMPRESS is specified). For example, data in an RGB16 format will be interpreted as YUV data if M_YUV16 is specified.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a packed or planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB24	Specifies 24-bit color depth (RGB 8:8:8) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) buffers in grab (M_GRAB) containers.					g										
M_RGB48	Specifies 48-bit color depth (RGB 16:16:16) planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) buffers in grab (M_GRAB) containers.					g										
M_RGB96	Specifies 96-bit color depth (RGB 32:32:32) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16	Specifies YUV16 packed or planar (4:2:2) standard.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This value can be used with packed (M_PACKED) buffers in grab (M_GRAB) containers.						a									

ControlFlag

Specifies the physical nature of the buffer. This parameter can be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the physical nature of the buffer																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AIL_ID +	Specifies that the new buffer maps to an existing source buffer. ArrayOfDataPtr is a pointer to an array that stores a pointer to the Aurora Imaging Library identifier of the source buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b
M_HOST_ADDRESS +	Specifies that ArrayOfDataPtr is a pointer to an array of host addresses that point to the data. The number of pointers in the array depends on the number of bands and whether the data is compressed or packed. Note that when using logical addresses, memory protection errors could result. Note that you can use MbufInquire() with M_HOST_ADDRESS to get the logical address of an Aurora Imaging Library allocated buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b
M_PHYSICAL_ADDRESS +	Specifies that ArrayOfDataPtr is a pointer to an array of physical addresses that point to the data. The number of pointers in the array depends on the number of bands and whether the data is compressed or packed. Note that using physical addresses provides direct access to any of your computer's memory mapped devices. Note that you can use MbufInquire() with M_PHYSICAL_ADDRESS to get the physical address of an Aurora Imaging Library allocated buffer. You must specify a combination value from the following table: • To set how the pitch is measured	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	This setting must be used if the specified container has the M_GRAB attribute.					g	a									

Combination values for the values listed in For specifying the physical nature of the buffer.

You must add one of the following values to the above-mentioned values to set how the pitch is measured.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying how the pitch is measured																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PITCH	Specifies that the pitch is in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PITCH_BYTE	Specifies that the pitch is in bytes.	c		e	f	g	a		i		j	k	l	m	n	b

Pitch INQ

Specifies the size of the pitch. The pitch is the number of pixels or bytes (as specified by the ControlFlag parameter) between the start of any two adjacent rows of the buffer. The actual length of a row in the buffer, in physical memory, might be different from the value of the SizeX parameter (for example, due to use of buffer overscan).

> Note that your code should not make assumptions about the actual pitch of the source memory. If the memory was allocated using an MbufAlloc...() function, use MbufInquire() with M_PITCH or M_PITCH_BYTE to establish the required pitch.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the size of the pitch																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that when dealing with a 1-bit buffer, the pitch is a multiple of 4 bytes; otherwise the pitch equals the SizeX parameter.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the pitch in pixels or bytes (as determined by the ControlFlag parameter).	c		e	f	g	a		i		j	k	l	m	n	b

ArrayOfDataPtr

Specifies one or more data pointers that address the memory to which to map the created Aurora Imaging Library buffer.

When pointing to a single-band buffer, a packed buffer, or a compressed buffer, a single pointer to the packed data must be provided. When pointing to an uncompressed planar buffer, one pointer per band must be provided (except when the ControlFlag parameter is set to M_AIL_ID). Pointers to a 3-band planar buffer must be ordered R-G-B or Y-U-V in the array.

When the ControlFlag parameter is set to M_AIL_ID, ArrayOfDataPtr specifies the address of an array that stores the Aurora Imaging Library identifier of the source buffer.

When the ControlFlag parameter is set to M_HOST_ADDRESS, ArrayOfDataPtr specifies the address of an array that stores pointer(s) to the logical address(es) that points to the data.

When the ControlFlag parameter is set to M_PHYSICAL_ADDRESS, ArrayOfDataPtr specifies the address of an array that stores pointer(s) to the physical address(es) that points to the data.

ComponentType

Specifies the component type, which identifies what kind of information the buffer component stores.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of component																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default component type. The default component type for image buffers is M_COMPONENT_INTENSITY. The default component type for all other buffers is M_COMPONENT_UNDEFINED.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. Coordinates associated with the confidence value 0 are considered invalid data and will not be used by 3D image processing functions. A confidence component is associated with a range or disparity component in the same container when there are no other range, disparity, or confidence components in the container. If there is more than one range, disparity, and/or confidence component in a container (for example, because a single grab into the container transmitted multiple range and confidence components), you will need to either create a child container which contains only the required components using MbufChildContainer(), or free the extra components using MbufFreeComponent().	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the column index of each corresponding pixel and will result in a better representation of the camera's lens model when generating X-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the row index of each corresponding pixel and will result in a better representation of the camera's lens model when generating Y-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 254. You can use custom component types to identify components which store information that does not fit one of the standard component types. In addition, some cameras transmit components with custom component types. Refer to your camera manual to determine what type of information is stored in transmitted components.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. Each pixel of a disparity map indicates the apparent distance (typically measured in pixel units) between where an object appears in the left and right images captured by a stereoscopic camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. Only depth map containers can have a matrix component. A matrix component is associated with a range component in the same container when there is only one range component and no other matrix components in that container.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. A mesh component is associated with a range component in the same container when there are no other range or mesh components in that container. A point cloud container which has a mesh component is referred to as a meshed point cloud container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_METADATA	Specifies that the component stores metadata information. Metadata components are used by Aurora Imaging Library internally. Typically, you should ignore metadata components in your application.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. Unlike an intensity component, a multispectral component might include information about non-visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies that the component stores normals information for each point in the M_COMPONENT_RANGE component of the container. A normals component is associated with a range component in the same container when there are no other range or normals components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. This can be either a 1-band buffer that stores a depth map, or a 3-band buffer that stores coordinates of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. Each pixel of a reflectance map indicates how much of the light hitting an object at that location is reflected back. Typically, this is an intensity image of the light spectrum used by a 3D sensor to detect 3D distance/position information. Typically, if the map was generated by a laser profiler, each row indicates the detected intensity of the laser for a single scan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. 3D image processing functions will only operate on points inside the ROI. Only depth map containers can have a region component. A region component is associated with a range component in the same container when there is only one range component and no other region components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. Each pixel of a scatter map indicates how much of the light hitting an object at that location is detected scattering beneath the object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_UNDEFINED	Specifies that the component stores information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

ComponentIdPtr

Specifies the address of the variable in which to write the buffer identifier. Since the **MbufCreateComponent**() function also returns the buffer identifier, you can set this parameter to **M_NULL**. If mapping fails, **M_NULL** is written as the identifier.

Return value

The returned value is the buffer identifier. If allocation fails, **M_NULL** is returned.

Remarks

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.
- *[Aurora Imaging Library Lite]*
  You can allocate an image buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute with Aurora Imaging Library Lite. However, these attributes are not required to work under Aurora Imaging Library Lite.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufDiskInquire

Synopsis

Inquire about the data in a file.

Syntax

```
AIL_INT MbufDiskInquire(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 InquireType,       //in
    void *UserVarPtr             //out
)
```

Description

This function inquires about the data in the specified file on disk.

Parameters

FileName

Specifies the file that contains the data.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name or memory stream																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Note, an error occurs if the file does not have a known file format or the file format is not supported. The supported file types include all the formats supported by the MbufExport() and MbufExportSequence() functions. Since a raw data file does not have any information regarding size or type, you can only use **MbufDiskInquire**() to determine the file format of this type of file. This function cannot access a file on the remote computer; the file must be located on the local computer.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

InquireType

Specifies the type of file setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_INT

- AIL_INT64

Specifies the address in which to write the requested information. Since the **MbufDiskInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about general types of file settings
- ForM_ARRAY,M_IMAGE, M_KERNEL,M_LUT and M_STRUCT_ELEMENT
- For M_IMAGE
- For M_KERNEL and M_STRUCT_ELEMENT
- For M_KERNEL

The following inquire types allow you to inquire about general types of file settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about general types of file settings																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_BYTE_ORDER		Inquires the file's byte order.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BIG_ENDIAN	Specifies that the file is in big endian format.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LITTLE_ENDIAN	Specifies that the file is in little endian format.	c		e	f	g	a		i		j	k	l	m	n	b
M_FILE_FORMAT		Inquires the file format. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AIL_NATIVE	Specifies that the data is saved in the Aurora Imaging Library native file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AIL_TIFF	Specifies that the data is saved in an Aurora Imaging Library file format that uses the baseline TIFF 6.0 file format with extran Aurora Imaging Library information included in the comment field. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_AVI_AIL	Specifies an AVI format used to hold images in their Aurora Imaging Library format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AVI_CODEC	Specifies an AVI format used with a codec found on your computer, where the AVI is not an M_AVI_DIB, M_AVI_AIL or M_AVI_MJPG.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AVI_DIB	Specifies an AVI format used to hold non-compressed DIB images . INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_AVI_MJPG	Specifies an AVI format used to hold JPEG compressed sequences. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BMP	Specifies that the data is saved in a BMP file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_GENDC	Specifies that the data is saved in the GenDC file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG2000_LOSSLESS	Specifies that the data is saved in a JPEG2000 lossless file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG2000_LOSSLESS_JP2	Specifies that the data is saved in a JPEG2000 lossless file format using the standard JP2 format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG2000_LOSSY	Specifies that the data is saved in a JPEG2000 lossy file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG2000_LOSSY_JP2	Specifies that the data is saved in a JP2000 lossy file format using the standard JP2 file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSLESS	Specifies that the data is saved in a JPEG lossless file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSLESS_INTERLACED	Specifies that the data is saved in an interlaced JPEG lossless file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSY	Specifies that the data is saved in a JPEG lossy file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSY_INTERLACED	Specifies that the data is saved in an interlaced JPEG lossy file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_JPEG_LOSSY_RGB	Specifies that the data is saved in a JPEG lossy file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PLY_ASCII	Specifies that the data is saved in the PLY file format using ASCII encoding.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PLY_BINARY_LITTLE_ENDIAN	Specifies that the data is saved in the PLY file format using binary encoding in little-endian order.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PNG	Specifies that the data is saved in a PNG file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_RAW	Specifies that the data is saved as raw data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_STL_ASCII	Specifies that the data is saved in the STL file format using ASCII encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_STL_BINARY	Specifies that the data is saved in the STL file format using binary encoding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TIFF	Specifies that the data is saved in a TIFF file format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OBJECT_TYPE		Inquires the type of Aurora Imaging Library object stored in the file. Note that AVI files return M_IMAGE as an object type.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ARRAY	Specifies that the file stores an array buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONTAINER	Specifies that the file stores a container.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IMAGE	Specifies that the file stores an image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_KERNEL	Specifies that the file stores a kernel buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LUT	Specifies that the file stores a LUT buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRUCT_ELEMENT	Specifies that the file stores a struct element buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE	Inquires the file data type and depth. The depth is returned in bits.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_FLOAT	Specifies the data depth and that the data type is floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_SIGNED	Specifies the data depth and that the data type is signed.	c		e	f	g	a		i		j	k	l	m	n	b
depth value + M_UNSIGNED	Specifies the data depth and that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b

You can inquire the following values for M_ARRAY M_IMAGE, M_KERNEL, M_LUT and M_STRUCT_ELEMENT files.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

ForM_ARRAY,M_IMAGE, M_KERNEL,M_LUT and M_STRUCT_ELEMENT																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_RESOLUTION_X	Inquires the X-resolution of the file in pixels per inch (PPI).															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the file has no resolution information.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the X resolution in PPI.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESOLUTION_Y	Inquires the Y-resolution of the file in pixels per inch (PPI). If the file has no resolution information, 0 is returned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that the file has no resolution information.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the Y resolution in PPI.	c		e	f	g	a		i		j	k	l	m	n	b

M_SIGN	Inquires whether the file's data is signed or unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the data is signed.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the data is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BAND +	Inquires the number of bands in the file.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 1024	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BIT	Inquires the file data depth.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 64	Specifies the data depth, in bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X +	Inquires the width of the data in the file.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the data, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y	Inquires the height of the data in the file.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the data, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_SIZE_BAND; M_SIZE_X.

You can add the following value to the above-mentioned values to get the width or the number of bands of the LUT associated with the image.

This option is available only if image data was saved to the file. This option is not available for AVI files.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For obtaining information about the LUT associated with the image																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_LUT	Inquires either the width or the number of bands of the LUT associated with the image in the file.															

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that there is no LUT associated with the image.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies either the width of the LUT associated with the image in the file (for M_SIZE_X) or the number of bands of the LUT associated with the image in the file (for M_SIZE_BAND).	c		e	f	g	a		i		j	k	l	m	n	b

You can inquire the following values for M_IMAGE files (images or AVI sequences).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ASPECT_RATIO		Inquires the aspect ratio of the images in the file.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the aspect ratio.	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATION_PRESENT		Inquires whether the calibration information associated with the buffer was saved, using MbufExport() with M_WITH_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the calibration was not saved.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the calibration was saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPRESSION_TYPE		[For essential Aurora Imaging Library Lite information, see remarks] Inquires the compression type of the images in the file.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that the images are not compressed.	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPRESSION_CCITTFAX3	Specifies that the file holds TIFF CCITT FAX3 data . INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPRESSION_CCITTFAX4	Specifies that the file holds TIFF CCITT FAX4 data . INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPRESSION_CCITTRLE	Specifies that the file holds TIFF CCITT RLE data . INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPRESSION_CCITTRLEW	Specifies that the file holds TIFF CCITT RLEW data . INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPRESSION_LZW	Specifies that the file holds TIFF LZW data .	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPRESSION_PACKBITS	Specifies that the file holds TIFF PackBits data .	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG2000_LOSSLESS	Specifies that the file holds JPEG2000 lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG2000_LOSSY	Specifies that the file holds JPEG2000 lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSLESS	Specifies that the file holds JPEG lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSLESS_INTERLACED	Specifies that the file holds JPEG lossless data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSY	Specifies that the file holds JPEG lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_JPEG_LOSSY_INTERLACED	Specifies that the file holds JPEG lossy data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FRAME_RATE		Inquires the frame rate of an AVI file. This value is only valid for sequences.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the frame rate, in number of images/second.	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT_PRESENT		Inquires the presence of LUT data in the file.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NO	Specifies that LUT data is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_YES	Specifies that LUT data is available.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_IMAGES	Inquires the number of images in an AVI file. This value is only valid for sequences. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of images.	c		e	f	g	a		i		j	k	l	m	n	b

You can inquire the following values for M_KERNEL and M_STRUCT_ELEMENT files.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_KERNEL and M_STRUCT_ELEMENT																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_OFFSET_CENTER_X	Inquires the X-coordinate of the center of the kernel or structuring element.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_CENTER_Y	Inquires the Y-coordinate of the center of the kernel or structuring element.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN	Inquires the overscan type.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that Aurora Imaging Library automatically selects the type of overscan to optimize speed and logic according to the specified operation and the target system.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that no overscan is used, unless processing the border pixels is faster than ignoring them; in the latter case, Aurora Imaging Library automatically selects the overscan to optimize speed according to the specified operation and the target system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FAST	Specifies that Aurora Imaging Library automatically selects the overscan to optimize speed according to the specified neighborhood operation and the target system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MIRROR	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values that mirror the source buffer pixel values. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLACE	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values set to the overscan replacement value (M_OVERSCAN_REPLACE_VALUE).	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLICATE	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values that replicate the border pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRANSPARENT	Specifies a type of overscan that processes the border pixels of a source image using transparent overscan pixel values. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_REPLACE_VALUE		Inquires the replacement value for the overscan pixel values.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLACE_MAX	Specifies that the overscan neighborhood pixel values will be set to the maximum value of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REPLACE_MIN	Specifies that the overscan neighborhood pixel values will be set to the minimum value of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the value of the overscan neighborhood pixels.	c		e	f	g	a		i		j	k	l	m	n	b

You can inquire the following values for M_KERNEL files.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_KERNEL																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ABSOLUTE_VALUE	Inquires the absolute value setting.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to take the absolute value of the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to take the absolute value of the result. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_FACTOR	Inquires the normalization factor.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the normalization factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Inquires the saturation flag.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to saturate results, except when Aurora Imaging Library can take advantage of optimization routines to accelerate the processing. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to saturate results. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information. If the requested information is not available, **M_INVALID** is returned. If an AVI file is specified but its codec is not known, **M_AVI_CODEC_UNSUPPORTED** is returned.

Remarks

- *[Aurora Imaging Library Lite]*
  Note that Aurora Imaging Library Lite does not support the inquiries available for M_KERNEL and M_STRUCT_ELEMENT data buffers.
- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufExport

Synopsis

Export a data buffer or container to a file.

Syntax

```
void MbufExport(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 FileFormat,        //in
    AIL_ID SrcContainerOrBufId   //in
)
```

Description

This function exports a data buffer or container to a file, in the specified output file format.

When exporting an image buffer, all the pixel information in the buffer is exported to the file. To export an image with a LUT (color palette), associate the LUT to the image, using MbufControl(). Upon export, the image is saved with its associated color palette (MIM, TIFF, PNG, and BMP file formats).

When exporting an image buffer in the M_AIL_TIFF file format, you can save the camera calibration information associated with the buffer (if applicable) using M_WITH_CALIBRATION. Additionally, if the image buffer includes any region of interest (ROI) information, set using MbufSetRegion(), the ROI information is also exported to the file (unless you specify M_NO_REGION).

You can also save a buffer in the M_AIL_TIFF file format, using MbufSave().

[Aurora Imaging Library Lite with restriction]

When exporting uncompressed data to a file in an M_JPEG_... or M_JPEG2000_... file format, this function will automatically compress the data, according to the specified file format. The buffer does not need an M_COMPRESS attribute. If you are exporting compressed data to a file in an uncompressed file format, this function will automatically decompress the data.

You can export a buffer or container with all of its data, attributes, and settings (excluding process-dependent settings, such as Aurora Imaging Library identifiers) using the M_AIL_NATIVE file format. You can restore the buffer or container exactly as it was exported using MbufImport().

Parameters

FileName

Specifies the name and path of the file in which to save the data buffer or container. It is recommended that you use the appropriate file extension for the specified format (for example, files in a JPEG format should usually be stored with the jpg file extension). The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten.

This parameter can be set to the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*ftp://user:password@server/pathtofile*") [1]	Export a data buffer to a file on a remote File Transfer Protocol (FTP) server. To access a remote FTP server anonymously, omit the user name and password. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be exported using an FTP file path.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*sftp://user:password@server/pathtofile*") [1]	Export a data buffer to a file on a remote SSH File Transfer Protocol (SFTP) server. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be exported using an SFTP file path.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileFormat INQ

Specifies the file format. This parameter can be set to one of the following values.

The following setting can be used to store either a buffer or a container with an automatically determined file format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the file format automatically																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_USE_EXTENSION	Specifies that the buffer or container is saved in a format automatically determined from the file extension specified in the FileName parameter. An error is generated if the automatically determined file format cannot be used to store the specified buffer or container. If the file extension specifies a jpeg or jpeg 2000 format, the buffer is saved as though it were exported using M_JPEG_LOSSY and M_JPEG2000_LOSSY respectively (except if the file extension is jp2, in which case the buffer is saved as though it were exported using M_JPEG2000_LOSSY_JP2). If the file extension specifies a stl or ply format, the buffer is saved as though it were exported using M_STL_BINARY and M_PLY_BINARY_LITTLE_ENDIAN respectively.	c		e	f	g	a		i		j	k	l	m	n	b

The following file type can be used to store either a buffer or a container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format that can be used with either a buffer or a container																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_AIL_NATIVE	Specifies that the buffer or container is saved in the Aurora Imaging Library native file format. This format stores all contents of the buffer or container, including all attributes and settings (excluding process-dependent settings, such as Aurora Imaging Library identifiers). The recommended file extension when exporting a buffer is mbuf. The recommended file extension when exporting a container is mbufc. > Note that the Aurora Imaging Library native file format is Aurora Imaging Library-specific. You can use the M_AIL_TIFF file format to export buffers to a file that complies with the TIFF standard.	c		e	f	g	a		i		j	k	l	m	n	b

The following file types can be used to store any data buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format that can be used with a buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_AIL_TIFF +	Specifies that the buffer is saved in an Aurora Imaging Library file format that uses the baseline TIFF 6.0 file format with extran Aurora Imaging Library information included in the comment field. The buffer attributes and data type are also saved in the file. This file format supports images that are binary, grayscale, and RGB, as well as palette-color images. The Aurora Imaging Library file format is a single-page, uncompressed TIFF file format. By default, most color image buffers are saved in a packed (chunky) format (in accordance with baseline TIFF 6.0 specifications). Color binary buffers are saved in a 1-bit per pixel format (data is stored in a 3-band, packed binary format). The recommended file extension is mim.	c		e	f	g	a		i		j	k	l	m	n	b
M_RAW	Specifies that the buffer is saved as raw data. The contents are dumped directly (byte stream) into the file and no header is added. If the buffer is multi-band, the buffer is saved in planar format (one band after another).	c		e	f	g	a		i		j	k	l	m	n	b

The following file types can be used to store a buffer in an image format.

Note that the source buffer must have an M_IMAGE attribute when exporting to these formats.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format for use with a buffer that stores image data																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BMP	Specifies that the buffer is saved in a BMP file format. The standard Windows BMP format is used. This file format supports images that are grayscale or RGB. Images in a different format are internally converted before being saved. This file format supports only 8-bit, or 8-bit/band if applicable, buffers. If you are saving a non 8-bit buffer in this file format, only the 8 least-significant bits are saved/band.	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in a JPEG2000 lossless file format. If the buffer is 3-band and does not have an M_COMPRESS + M_JPEG2000_LOSSLESS attribute, the data will be stored in RGB format; otherwise, it will be stored in the same color format as the buffer. This file format supports only 8- and 16-bit buffers. 1-bit buffers are unpacked and saved as 8-bit buffers. 32-bit buffers are saved as 16-bit buffers; only the 16 least-significant bits are saved. In the case of a 32-bit buffer, it is best to shift the buffer by 16 bits to the right before exporting the image to keep the most significant data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSLESS_JP2	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in a JPEG2000 lossless file format using the standard JP2 format. Only the color specification of the image buffer can be specified from the additional JP2 information. If the buffer is 3-band and does not have an M_COMPRESS + M_JPEG2000_LOSSLESS attribute, the data will be stored in RGB format; otherwise, it will be stored in the same color format as the buffer. This file format supports only 8- and 16-bit buffers. 1-bit buffers are unpacked and saved as 8-bit buffers. 32-bit buffers are saved as 16-bit buffers; only the 16 least-significant bits are saved. In the case of a 32-bit buffer, it is best to shift the buffer by 16 bits to the right before exporting the image to keep the most significant data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in a JPEG2000 lossy file format. If the buffer is 3-band and does not have an M_COMPRESS + M_JPEG2000_LOSSY attribute, the data will be stored in YUV16 planar format if the buffer is 8-bit or in RGB format if the buffer is 16-bit; otherwise, it will be stored in the same color format as the buffer. This file format supports only 8- and 16-bit buffers. 1-bit buffers are unpacked and saved as 8-bit buffers. 32-bit buffers are saved as 16-bit buffers; only the 16 least-significant bits are saved. In the case of a 32-bit buffer, it is best to shift the buffer by 16 bits to the right before exporting the image to keep the most significant data.	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG2000_LOSSY_JP2	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in a JP2000 lossy file format using the standard JP2 file format. Only the color specification of the image buffer can be specified from the additional JP2 information. If the buffer is 3-band and does not have an M_COMPRESS + M_JPEG2000_LOSSY attribute, the data will be stored in YUV16 planar format if the buffer is 8-bit or in RGB format if the buffer is 16-bit; otherwise, it will be stored in the same color format as the buffer. This file format supports only 8- and 16-bit buffers. 1-bit buffers are unpacked and saved as 8-bit buffers. 32-bit buffers are saved as 16-bit buffers; only the 16 least-significant bits are saved. In the case of a 32-bit buffer, it is best to shift the buffer by 16 bits to the right before exporting the image to keep the most significant data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in a JPEG lossless file format. If the buffer is 3-band, the data will be stored in RGB format. This file format supports only 8- and 16-bit buffers. 1-bit buffers are unpacked and saved as 8-bit buffers. 32-bit buffers are saved as 16-bit buffers; only the 16 least-significant bits are saved. In the case of a 32-bit buffer, it is best to shift the buffer by 16 bits to the right before exporting the image to keep the most significant data.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in an interlaced JPEG lossless file format. This file format is only applicable to 1-band image buffers. This file format supports only 8- and 16-bit buffers. 1-bit buffers are unpacked and saved as 8-bit buffers. 32-bit buffers are saved as 16-bit buffers; only the 16 least-significant bits are saved. In the case of a 32-bit buffer, it is best to shift the buffer by 16 bits to the right before exporting the image to keep the most significant data.	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in a JPEG lossy file format. If the buffer is 3-band and does not have an M_COMPRESS + M_JPEG_LOSSY attribute, the data will be stored in YUV16 packed format; otherwise, it will be stored in the same color format as the buffer. This file format supports only 8-bit, or 8-bit/band if applicable, buffers. If you are saving a non 8-bit buffer in this file format, only the 8 least-significant bits are saved/band.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in an interlaced JPEG lossy file format. If the buffer is 3-band, the data will be stored in YUV16 packed format. This file format supports only 8-bit, or 8-bit/band if applicable, buffers. If you are saving a non 8-bit buffer in this file format, only the 8 least-significant bits are saved/band.	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_RGB	[For essential Aurora Imaging Library Lite information, see remarks] Specifies that the buffer is saved in a JPEG lossy file format. If the buffer is 3-band, the data will be stored in RGB format. This file format supports only 8-bit, or 8-bit/band if applicable, buffers. If you are saving a non 8-bit buffer in this file format, only the 8 least-significant bits are saved/band.	c		e	f	g	a		i		j	k	l	m	n	b
M_PNG +	Specifies that the buffer is saved in a PNG file format. This file format supports only 1-, 8- and 16-bit buffers. 32-bit buffers are saved as 16-bit buffers; only the 16 least-significant bits are saved. In the case of a 32-bit buffer, it is best to shift the buffer by 16 bits to the right before exporting the image to keep the most significant data.	c		e	f	g	a		i		j	k	l	m	n	b

M_TIFF +	Specifies that the buffer is saved in a TIFF file format. The TIFF file format that is used respects the baseline TIFF 6.0 specification. The TIFF file format that is used supports images that are binary, grayscale, and RGB, as well as palette-color images. Also, this TIFF format supports any data type and depth that Aurora Imaging Library supports for buffers. An image buffer cannot be saved using a multi-page TIFF file format; it is always saved as a single-page TIFF file. By default, most color image buffers are saved in a packed (chunky) format (in accordance with baseline TIFF 6.0 specifications). Color binary buffers are saved in a 1-bit per pixel format (data is stored in a 3-band, packed binary format).	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PNG.

You can add the following value to the above-mentioned value to specify the compression level of a PNG.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression level of a PNG																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PNG_COMPRESSION_LEVEL*n*	Specifies a compression level of *n*, where *n* is a value from 0 to 9. > The highest compression level (that is, 9) will result in the smallest file, but will take longer to save.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_AIL_TIFF.

You can add one of the following values to the above-mentioned value to set whether to save the camera calibration information or region of interest data associated with the buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For saving camera calibration information or ROI data																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NO_REGION	Specifies that any ROI data associated with the buffer is not saved with the buffer contents.	c		e	f	g	a		i		j	k	l	m	n	b
M_WITH_CALIBRATION	Specifies that any camera calibration information associated with the buffer is saved. To restore the associated camera calibration, use McalRestore().	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_AIL_TIFF; M_TIFF.

You can add the following value to the above-mentioned values to set whether to save the color image buffer in

a planar format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For saving a color image in a planar format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PLANAR	Specifies that the color image buffer is saved in a planar format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_TIFF.

You can add the following value to the above-mentioned value to set whether to save the image buffer in big endian format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For saving an image in big endian format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BIG_ENDIAN	Specifies that the image buffer is saved in big endian format.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_TIFF.

You can add one of the following values to the above-mentioned value to set whether to use compression when saving a TIFF image file.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For compressing a TIFF image file																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESSION_CCITTFAX3	Specifies to use CCITT FAX3 compression. > This compression can only be used with monochrome 1-bit buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPRESSION_CCITTFAX4	Specifies to use CCITT FAX4 compression. > This compression can only be used with monochrome 1-bit buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPRESSION_CCITTRLE	Specifies to use CCITT RLE compression. > This compression can only be used with monochrome 1-bit buffers.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPRESSION_CCITTRLEW	Specifies to use CCITT RLEW compression. > This compression can only be used with monochrome 1-bit buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPRESSION_LZW	Specifies to use LZW compression.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPRESSION_NONE	Specifies to use no compression.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPRESSION_PACKBITS	Specifies to use PackBits compression.	c		e	f	g	a		i		j	k	l	m	n	b

The following file type can be used to store any container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format that can be used with a container																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_GENDC	Specifies that the container is saved in the GenDC file format. The recommended file extension is gendc. An error is generated if the source container has one of the following: • A component that is not an M_IMAGE or M_ARRAY buffer. • A component with M_COMPONENT_TYPE set to M_COMPONENT_MESH_AIL or M_COMPONENT_NORMALS_AIL. • A component with M_COMPONENT_GROUP_ID, M_COMPONENT_REGION_ID, or M_COMPONENT_SOURCE_ID set to a value larger than 65535. • A component that is an image buffer compressed in a format other than JPEG or JPEG 2000.	c		e	f	g	a		i		j	k	l	m	n	b

The following file types can be used to store a 3D-processable point cloud container (you can inquire whether a container is 3D-processable using MbufInquireContainer() with M_3D_PROCESSABLE). Any points with a confidence of zero will not be exported. Intensity, reflectance, normals, and mesh components will be included in the exported file.

> Note that these settings are only available when ail3d11.dll is present.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file format for use with a 3D-processable container																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PLY_ASCII +	Specifies that the point cloud container is saved in the PLY file format using ASCII encoding.	c		e	f	g	a		i		j	k	l	m	n	b

M_PLY_ASCII_100_SP7 +	Specifies that the point cloud container is saved in the PLY file format using ASCII encoding with the header following the Aurora Imaging Library 10.0 Service Pack 7 convention.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLY_BINARY_LITTLE_ENDIAN +	Specifies that the point cloud container is saved in the PLY file format using binary encoding in little-endian order.	c		e	f	g	a		i		j	k	l	m	n	b
M_PLY_BINARY_LITTLE_ENDIAN_100_SP7 +	Specifies that the point cloud container is saved in the PLY file format using binary encoding in little-endian order with the header following the Aurora Imaging Library 10.0 Service Pack 7 convention.	c		e	f	g	a		i		j	k	l	m	n	b
M_STL_ASCII	Specifies that the point cloud container is saved in the STL file format using ASCII encoding. > Note that exporting to an STL file requires the presence of a valid mesh component.	c		e	f	g	a		i		j	k	l	m	n	b
M_STL_ASCII_100_SP7	Specifies that the point cloud container is saved in the STL file format using ASCII encoding with the header following the Aurora Imaging Library 10.0 Service Pack 7 convention. > Note that exporting to an STL file requires the presence of a valid mesh component.	c		e	f	g	a		i		j	k	l	m	n	b
M_STL_BINARY	Specifies that the point cloud container is saved in the STL file format using binary encoding. > Note that exporting to an STL file requires the presence of a valid mesh component.	c		e	f	g	a		i		j	k	l	m	n	b

M_STL_BINARY_100_SP7	Specifies that the point cloud container is saved in the STL file format using binary encoding with the header following the Aurora Imaging Library 10.0 Service Pack 7 convention. > Note that exporting to an STL file requires the presence of a valid mesh component.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PLY_ASCII; M_PLY_ASCII_100_SP7; M_PLY_BINARY_LITTLE_ENDIAN; M_PLY_BINARY_LITTLE_ENDIAN_100_SP7.

You can add the following value to the above-mentioned values to set whether to save organization information when exporting a point cloud..

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For saving the organization of a point cloud																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_WITH_ORGANIZATION	Specifies to include point cloud organization information in the exported file. If you later import the file using Aurora Imaging Library (for example, using MbufRestore()), the imported point cloud will retain this organization. > Note that the organization information is stored using Aurora Imaging Library-specific data fields; third-party software will typically ignore the organization information. Invalid points are not exported; they are re-added when you later import the file using Aurora Imaging Library. In this case, the values for invalid points in components other than the confidence component (such as the range and intensity components) will be undefined.	c		e	f	g	a		i		j	k	l	m	n	b

SrcContainerOrBufId

Specifies the identifier of the data buffer or container to save.

Remark

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_IMAGE + M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib;.
DLL	Requires ail11.dll, ail3d11.dll for some formats.

MbufExportSequence

Synopsis

Export a sequence of image buffers to an AVI file.

Syntax

```
void MbufExportSequence(
    AIL_CONST_TEXT_PTR FileName,            //in
    AIL_INT64 FileFormat,                   //in
    const AIL_ID *BufArrayPtrOrSystemIdPtr, //in
    AIL_INT NumOfIds,                       //in
    AIL_DOUBLE FrameRate,                   //in
    AIL_INT64 ControlFlag                   //in
)
```

Description

This function exports a sequence of image buffers to an audio video interleave (AVI) file.

This function creates a new AVI file, or appends to an existing AVI file. If all your images have been acquired, call this function a single time (M_APPEND or M_DEFAULT) to automatically open (or create) the file, write to it, and then close the file. If your images are being acquired over time, call this function to open (or create) the file (M_OPEN or M_OPEN + M_APPEND), call the function again each time you want to write new images to the file (M_WRITE), and call the function once more to close the file (M_CLOSE).

Parameters

FileName

Specifies the name and path of the AVI file.

This parameter can be set to the following value:

For the name and path of the AVI file	
Value	Description
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileFormat INQ

Specifies the format of the file. This parameter can be set to one of the values below.

For specifying the file format	
Value	Description
M_DEFAULT	Specifies that Aurora Imaging Library automatically decides the appropriate format.

M_AVI_AIL	Specifies an AVI format used to hold image buffers in their Aurora Imaging Library format. Since this function saves images in the format in which they are sent, it can be used with any format. Also, since the images are saved "as is", no additional loss is introduced in the images.
	Operating system specific
	[This is only applicable to Windows] To be supported by Windows Media Player, this type of sequence requires a codec. Note that the required codec is provided with Aurora Imaging Library.
M_AVI_DIB	Specifies an AVI format used to hold non-compressed DIB image buffers. If necessary, the image buffers will be converted to a non-compressed DIB format before exporting.
	Operating system specific
	[This is only applicable to Windows] This type of sequence is supported by Windows Media Player.
M_AVI_MJPG	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an AVI format used to hold JPEG compressed sequences. When this format is specified, the image buffers must be in YUV16 packed format. In addition, image buffers must have a width that is a multiple of 16 pixels. For image buffers that have the M_JPEG_LOSSY compression type, the height must be a multiple of 8, and less than or equal to 240 pixels. For image buffers that have the M_JPEG_LOSSY_INTERLACED compression type, the height must be a multiple of 16 pixels, and greater than 240 pixels. If the image buffers are not already in these dimensions and format, Aurora Imaging Library will automatically convert them appropriately.
	Operating system specific
	[This is only applicable to Windows] To be supported by Microsoft Windows Media Player, this type of sequence requires a codec or DirectShow 8.0 (or better).

BufArrayPtrOrSystemIdPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_ID [optionally, in C++: a reference to a constant std::vector<AIL_ID>]

Specifies the image buffers to export or, when opening or closing (M_OPEN, M_OPEN + M_APPEND, or M_CLOSE) a file on a remote computer, an Aurora Imaging Library system allocated on the remote computer.

See the Parameter associations section for possible values that can be specified.

NumOfIds INQ

Specifies the number of Aurora Imaging Library identifiers passed to the BufArrayPtrOrSystemIdPtr parameter (image buffers or system identifiers).

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the BufArrayPtrOrSystemIdPtr parameter.

See the Parameter associations section for possible values that can be specified.

FrameRate

Specifies the frame rate (number of images/sec) of the sequence. The frame rate can be specified with any (or every) call to **MbufExportSequence**(), but it is only checked for validity when closing the AVI file (using M_CLOSE, M_APPEND, or M_DEFAULT). The last call that specifies a valid frame rate will be used.

For specifying the frame rate	
Value	Description
M_DEFAULT	Same as M_NULL.
M_NULL	Specifies to not set or change the frame rate.
Value > 0	Specifies the frame rate, in number of frames/sec.

ControlFlag

Specifies whether to write, overwrite, or append the image buffers to the AVI file.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **BufArrayPtrOrSystemIdPtr**, **NumOfIds**, and **ControlFlag** parameters.

- For specifying how to export the sequence

For specifying how to export the sequence	
ControlFlag	Description
BufArrayPtrOrSystemIdPtr	
NumOfIds	
M_DEFAULT	Opens the AVI file, overwriting it in the process. The file is opened, written into, and then closed. If no file exists, one is created.
BufArrayPtrOrSystemIdPtr	Specifies the address of an array containing the identifiers of the image buffers to export. > Note that the image buffers must be allocated on the same system as the AVI file. The image buffers in this array must be 1- or 3-band buffers and must not have regions of interest (ROI) associated with them. Using an image buffer with an ROI will cause an error.
NumOfIds	Specifies the number of image buffers to export. Note that if the supplied array of image buffers is larger than this number, the remaining buffer identifiers are ignored.
M_APPEND	Appends the image buffers to the file without overwriting it. The file is opened, the specified images are appended, and then the file is closed. If no file exists, one is created.

BufArrayPtrOrSystemIdPtr	Specifies the address of an array containing the identifiers of the image buffers to export. > Note that the image buffers must be allocated on the same system as the AVI file. The image buffers in this array must be 1- or 3-band buffers and must not have regions of interest (ROI) associated with them. Using an image buffer with an ROI will cause an error.
NumOfIds	Specifies the number of image buffers to export. Note that if the supplied array of image buffers is larger than this number, the remaining buffer identifiers are ignored.
M_CLOSE	Closes the AVI file.
BufArrayPtrOrSystemIdPtr	Specifies the computer on which the file is located.
M_NULL	Specifies that the file to be closed is on a local computer. In this case, the file name must not be prefixed with "remote:///".
Remote system identifier	Specifies the address of a variable containing the identifier of a remote system that is on the same remote computer as the AVI file. In this case, the file name must be prefixed with "remote:///".
NumOfIds	Specifies the number of system identifiers passed to the BufArrayPtrOrSystemIdPtr parameter (either a single one, or none).
M_OPEN	Opens the AVI file, overwriting it in the process. Opens the AVI file for writing, and sets the pointer to the beginning of the file. If no file exists, one will be created.
BufArrayPtrOrSystemIdPtr	Specifies the computer on which the file is located.
M_NULL	Specifies that the file to be opened is on a local computer. In this case, the file name must not be prefixed with "remote:///".
Remote system identifier	Specifies the address of a variable containing the identifier of a remote system that is on the same remote computer as the AVI file. In this case, the file name must be prefixed with "remote:///".
NumOfIds	Specifies the number of system identifiers passed to the BufArrayPtrOrSystemIdPtr parameter (either a single one, or none).
M_OPEN + M_APPEND	Opens the AVI file, and sets the pointer to the end of the file without overwriting the existing images. New images are appended to the file. If no file exists, one is created.
BufArrayPtrOrSystemIdPtr	Specifies the computer on which the file is located.
M_NULL	Specifies that the file to be opened is on a local computer. In this case, the file name must not be prefixed with "remote:///".

	Remote system identifier	Specifies the address of a variable containing the identifier of a remote system that is on the same remote computer as the AVI file. In this case, the file name must be prefixed with "remote:///".
	NumOfIds	Specifies the number of system identifiers passed to the BufArrayPtrOrSystemIdPtr parameter (either a single one, or none).
M_WRITE		Writes the specified number of images in the file starting from the current file pointer position. After the write operation, the file pointer is left at the end of the file, ready for the next M_WRITE operation.
	BufArrayPtrOrSystemIdPtr	Specifies the address of an array containing the identifiers of the image buffers to export. > Note that the image buffers must be allocated on the same system as the AVI file. The image buffers in this array must be 1- or 3-band buffers and must not have regions of interest (ROI) associated with them. Using an image buffer with an ROI will cause an error.
	NumOfIds	Specifies the number of image buffers to export. Note that if the supplied array of image buffers is larger than this number, the remaining buffer identifiers are ignored.

Remark

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for M_AVI_MJPG format, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufFree

Synopsis

Free a data buffer or container.

Syntax

```
void MbufFree(
    AIL_ID BufOrContainerBufId //in
)
```

Description

This function deallocates a previously allocated data buffer or container. The memory reserved for the specified buffer or container is released.

> All of a container's components are freed automatically when the container is freed. You can also free components manually using MbufFreeComponent(). **MbufFree**() cannot be used to free buffers that are components.

Before freeing a parent buffer or container, you must free all of its children.

Note that LUT buffers, once associated with another buffer, a digitizer, or a display, should either be disassociated before being freed, or freed after the associated buffer, digitizer, or display is freed.

To disassociate a digitizer or display from a LUT buffer, use MdigControl() with M_LUT_ID set to M_DEFAULT or MdispLut() with M_DEFAULT respectively. To disassociate another buffer from a LUT buffer, use MbufControl() with M_ASSOCIATED_LUT set to M_DEFAULT.

All buffers and containers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

BufOrContainerBufId

Specifies the identifier of the data buffer or container to deallocate.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufFreeComponent

Synopsis

Free one or more buffer components of a specified container.

Syntax

```
void MbufFreeComponent(
    AIL_ID ContainerBufId, //out
    AIL_INT64 Component,   //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function deallocates one or more previously allocated buffer components of the specified container. The memory reserved for the specified component is released.

Child buffers associated with the component must be deallocated, using MbufFree(), prior to deallocating the component.

All buffers allocated on a particular system must be freed before the system can be freed.

Parameters

ContainerBufId

Specifies the Aurora Imaging Library identifier of the container which contains the component(s) to be freed.

Component

Specifies the criterion used to determine which component(s) will be freed from the container. For all settings except M_COMPONENT_ALL or a component type, an error will be generated if no components of the container specified in ContainerBufId meet the criterion.

These settings can be used to free a single component that can be uniquely identified.

For specifying which component to free using a uniquely identifying criterion	
Value	Description
M_COMPONENT_BY_ID(AIL_ID ID)	Specifies the component with the specified Aurora Imaging Library identifier.
	Parameters
	ID Specifies the Aurora Imaging Library identifier of the component.
M_COMPONENT_BY_INDEX(AIL_INT ComponentIndex)	Specifies the component with the specified index in the container.
	Parameters
	ComponentIndex Specifies the index of the component.

These settings can be used to free one or more components that can be identified by a generalized criterion.

For specifying which components to free using a generalized criterion		
Value	Description	
M_COMPONENT_BY_GROUP_ID(AIL_INT GroupIndex)	Specifies that the component(s) have the specified group ID. The group ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_GROUP_ID.	
		Parameters
		GroupIndex Specifies the group ID.
M_COMPONENT_BY_REGION_ID(AIL_INT RegionID)	Specifies that the component(s) have the specified region ID. The region ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_REGION_ID.	
		Parameters
		RegionID Specifies the region ID.
M_COMPONENT_BY_SOURCE_ID(AIL_INT SourceID)	Specifies that the component(s) have the specified source ID. The source ID of a component can be inquired using MbufInquireContainer() with M_COMPONENT_SOURCE_ID.	
		Parameters
		SourceID Specifies the source ID.
M_COMPONENT_ALL	Specifies all components of the container.	

These settings can be used to free one or more components with the specified component type.

For specifying the component to free by component type	
Value	Description
M_COMPONENT_CONFIDENCE	Specifies the components that store confidence information for an M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. INFO
M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies the components that store the A coordinate map provided by the camera. INFO
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies the components that store the B coordinate map provided by the camera. INFO

M_COMPONENT_CUSTOM + *n*	Specifies the components that have a custom component type , identified by *n*, where n can be a value between 0 and 254. INFO
M_COMPONENT_DISPARITY	Specifies the components that store a disparity map. INFO
M_COMPONENT_INFRARED	Specifies the components that store an intensity image of infrared light.
M_COMPONENT_INTENSITY	Specifies the components that store an intensity image of visible light.
M_COMPONENT_MATRIX_AIL	Specifies the components that store a 4x4 matrix that transforms depth map coordinates. INFO
M_COMPONENT_MESH_AIL	Specifies the components that store mesh information for an M_COMPONENT_RANGE component of the container. INFO
M_COMPONENT_METADATA	Specifies the components that store metadata information. INFO
M_COMPONENT_MULTISPECTRAL	Specifies the components that store an intensity image where each band represents the intensity of a specific wavelength of light. INFO
M_COMPONENT_NORMALS_AIL	Specifies the components that store normals information for each point in the M_COMPONENT_RANGE component of the container. INFO
M_COMPONENT_RANGE	Specifies the components that store 3D distance/position information. INFO
M_COMPONENT_REFLECTANCE	Specifies the components that store a reflectance map. INFO
M_COMPONENT_REGION_AIL	Specifies the components that store a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. INFO
M_COMPONENT_SCATTER	Specifies the components that store a scatter map. INFO
M_COMPONENT_ULTRAVIOLET	Specifies the components that store an intensity image of ultraviolet light.
M_COMPONENT_UNDEFINED	Specifies the components that store information of an unknown type.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.

DLL	Requires ail11.dll.

MbufGet1d

Synopsis

Get data from a one-dimensional area of a buffer and place it in a user-supplied array.

Syntax

```
void MbufGet1d(
    AIL_ID SrcBufId,   //in
    AIL_INT OffX,      //in
    AIL_INT SizeX,     //in
    void *UserArrayPtr //out
)
```

Description

This function copies data from a specified one-dimensional area of an Aurora Imaging Library source buffer to a user-supplied array. If the source buffer has multiple bands, this function linearly copies the data from the one-dimensional area of each band. The internal data format of the source buffer need not match the data format of the user-supplied array (planar); an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster. For more information, refer to MbufGetColor2d() with M_PLANAR.

If the specified buffer is a binary image buffer, any extra bits to complete a byte in the array are undefined.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

Parameters

SrcBufId

Specifies the identifier of the source buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

OffX

Specifies the horizontal offset of the required area, relative to the top-left coordinate of the source buffer.

SizeX

Specifies the width of the required area of the source buffer.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:(SizeX) * MbufInquire (M_SIZE_BAND)

Specifies the address of the user array in which to copy the data from the source buffer. Ensure that the user array is large enough to receive the data to be copied from the source buffer.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufGet2d

Synopsis

Get data from a two-dimensional area of a buffer and place it in a user-supplied array.

Syntax

```
void MbufGet2d(
    AIL_ID SrcBufId,   //in
    AIL_INT OffX,      //in
    AIL_INT OffY,      //in
    AIL_INT SizeX,     //in
    AIL_INT SizeY,     //in
    void *UserArrayPtr //out
)
```

Description

This function copies data from a specified two-dimensional area of an Aurora Imaging Library source buffer to a user-supplied array. If the source buffer has multiple bands, this function linearly copies the data from the specified two-dimensional area of each band. The internal data format of the source buffer need not match the data format of the user-supplied array (planar); an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster. For more information, refer to MbufGetColor2d() with M_PLANAR.

If the specified buffer is a binary image buffer, any extra bits to complete a byte in the array are undefined.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

Parameters

SrcBufId

Specifies the identifier of the source buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

OffX

Specifies the horizontal offset of the required area, relative to the top-left coordinate of the source buffer.

OffY

Specifies the vertical offset of the required area, relative to the top-left coordinate of the source buffer.

SizeX

Specifies the width of the required area of the source buffer.

SizeY

Specifies the height of the required area of the source buffer.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of

the same type as the buffer]
Required array size:(SizeX) * MbufInquire (M_SIZE_BAND) * (SizeY)

Specifies the address of the user array in which to copy the data from the source buffer. Ensure that the user array is large enough to receive the data to be copied from the source buffer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufGet

Synopsis

Get data from a buffer and place it in a user-supplied array.

Syntax

```
void MbufGet(
    AIL_ID SrcBufId,   //in
    void *UserArrayPtr //out
)
```

Description

This function copies data from a specified Aurora Imaging Library source buffer to a user-supplied array. If the source buffer has multiple bands, Aurora Imaging Library copies all bands, one after the other. The internal data format of the source buffer need not match the data format of the user-supplied array (planar); an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster. For more information, refer to MbufGetColor() with M_PLANAR.

If the specified buffer is a binary image buffer, any extra bits to complete a byte in the array are undefined.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

Parameters

SrcBufId

Specifies the identifier of the source buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size: MbufInquire (M_SIZE_X) * MbufInquire (M_SIZE_Y) * MbufInquire (M_SIZE_BAND)

Specifies the address of the user array in which to copy the data from the source buffer. Ensure that the user array is large enough to receive the data to be copied from the source buffer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufGetArc

Synopsis

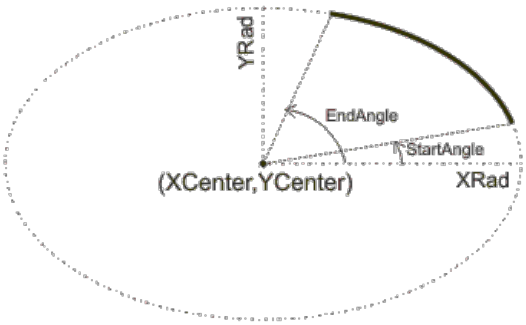
Read the pixels along a specified arc, count the pixels, and store their values, and coordinates, in user-defined arrays.

Syntax

```
AIL_INT MbufGetArc(
    AIL_ID ImageBufId,     //in
    AIL_INT XCenter,       //in
    AIL_INT YCenter,       //in
    AIL_INT XRad,          //in
    AIL_INT YRad,          //in
    AIL_DOUBLE StartAngle, //in
    AIL_DOUBLE EndAngle,   //in
    AIL_INT64 Mode,        //in
    AIL_INT *NbPixelsPtr,  //out
    void *ValueArrayPtr,   //out
    void *PosXArrayPtr,    //out
    void *PosYArrayPtr     //out
)
```

Description

This function reads the series of pixels along an elliptic arc from an image and stores their values, and coordinates, in user-defined arrays. The arc is based on an ellipse centered at XCenter and YCenter, with radii XRad and YRad. The arc is the part of the ellipse between the start angle (StartAngle) and the end angle (EndAngle), both measured with respect to the X-axis. The following image shows how the parameters are used to define an arc:



To determine the exact size of the array needed for a given set of parameter values, call this function twice: the first time to determine the required array size and the second time to fill the arrays. When calling for the first time, specify the same values for ImageBufId, XCenter, YCenter, XRad, YRad, StartAngle, EndAngle, and Mode as you would for the second call to this function. In addition, set NbPixelsPtr to an appropriate address, and set the array parameters to **M_NULL**.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

Parameters

ImageBufId

Specifies the identifier of a single-band (monochrome) source image buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

XCenter

Specifies the X-coordinate of the center of the ellipse from which the arc is defined, in the pixel coordinate system.

YCenter

Specifies the Y-coordinate of the center of the ellipse from which the arc is defined, in the pixel coordinate system.

XRad

Specifies the radius of the ellipse from which the arc is defined, along the X-axis of the pixel coordinate system.

For specifying the radius along the X-axis	
Value	Description
Value >= 0	Specifies the radius along the X-axis.

YRad

Specifies the radius of the ellipse from which the arc is defined, along the Y-axis of the pixel coordinate system.

For specifying the radius along the Y-axis	
Value	Description
Value >= 0	Specifies the radius along the Y-axis.

StartAngle

Specifies the angle at which to start reading the arc, relative to the positive X-axis of the pixel coordinate system.

For specifying the starting angle	
Value	Description
-360.0 <= *Value* <= 360.0	Specifies the angle at which to start reading the arc, in degrees.

EndAngle

Specifies the angle at which to end reading the arc, relative to the positive X-axis of the pixel coordinate system.

For specifying the ending angle	
Value	Description
-360.0 <= *Value* <= 360.0	Specifies the angle at which to end reading the arc, in degrees.

Mode

Reserved for future expansion and must be set to **M_DEFAULT**.

NbPixelsPtr

Specifies the address of the variable in which to write the number of pixels found along the arc. Set this parameter to **M_NULL** if it is not to be evaluated.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_NULL** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

ValueArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (ImageBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:Call **MbufGetArc**() with ValueArrayPtr set to **M_NULL**, PosXArrayPtr set to **M_NULL**, and PosYArrayPtr set to **M_NULL**. The array size is stored in the NbPixelsPtr parameter. Note: The array size is determined by calling the function twice.

Specifies the address of the user array in which to store the pixel values from the image buffer. Ensure that the user array is large enough to receive the data to be stored. When not using this parameter or when determining the required size of this array for subsequent calls to **MbufGetArc**(), set this parameter to **M_NULL**. In this case, nothing is read from the image buffer.

PosXArrayPtr
Accepts the following address:

- Data type: array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
  Required array size:Call **MbufGetArc**() with ValueArrayPtr set to **M_NULL**, PosXArrayPtr set to **M_NULL**, and PosYArrayPtr set to **M_NULL**. The array size is stored in the NbPixelsPtr parameter. Note: The array size is determined by calling the function twice.

Specifies the address of the user array in which to store the X-coordinates of the points along the arc, from the image buffer. Ensure that the user array is large enough to receive the data to be stored. When not using this parameter or when determining the required size of this array for subsequent calls to **MbufGetArc**(), set this value to **M_NULL**. In this case, nothing is read from the image buffer.

PosYArrayPtr
Accepts the following address:

- Data type: array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
  Required array size:Call **MbufGetArc**() with ValueArrayPtr set to **M_NULL**, PosXArrayPtr set to **M_NULL**, and PosYArrayPtr set to **M_NULL**. The array size is stored in the NbPixelsPtr parameter. Note: The array size is determined by calling the function twice.

Specifies the address of the user array in which to store the Y-coordinates of the points along the arc, from the image buffer. Ensure that the user array is large enough to receive the data to be stored. When not using this parameter or when determining the required size of this array for subsequent calls to **MbufGetArc**(), set this value to **M_NULL**. In this case, nothing is read from the image buffer.

Return value

The returned value is the number of pixels found along the arc.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.

DLL	Requires ail11.dll.

MbufGetColor2d

Synopsis

Get data from an area of one or all bands of a buffer and place it in a user-supplied array.

Syntax

```
void MbufGetColor2d(
    AIL_ID SrcBufId,      //in
    AIL_INT64 DataFormat, //in
    AIL_INT Band,         //in
    AIL_INT OffX,         //in
    AIL_INT OffY,         //in
    AIL_INT SizeX,        //in
    AIL_INT SizeY,        //in
    void *UserArrayPtr    //out
)
```

Description

This function copies data from a specific area of one or all bands of a specified Aurora Imaging Library source buffer to a user-supplied array.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

Parameters

SrcBufId

Specifies the identifier of the source buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

DataFormat

Specifies the data format to use to save the data in the user array. The internal data format of the source buffer need not match the specified data format of the user-supplied array; an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster.

See the Parameter associations section for possible values that can be specified.

Band

Specifies the index of the band to copy. This parameter can be set to one of the values below.

For specifying the index of the band to copy	
Value	Description
M_ALL_BANDS	Copies from all bands.
M_BLUE	Copies from the blue band (for RGB buffers).
M_GREEN	Copies from the green band (for RGB buffers).

M_HUE	Copies from the hue band (for HSL and HSV buffers).
M_LUMINANCE	Copies from the luminance band (for HSL buffers).
M_RED	Copies from the red band (for RGB buffers).
M_SATURATION	Copies from the saturation band (for HSL and HSV buffers).
M_V	Copies from the V band (for HSV buffers).
0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between index value and band for RGB, HSL, and HSV buffers is the following: 0 — Corresponds to the red band for RGB buffers, the hue band for HSL and HSV buffers. 1 — Corresponds to the green band for RGB buffers or the saturation band for HSL and HSV buffers. 2 — Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for HSV buffers.

OffX

Specifies the horizontal offset (relative to the top-left source buffer coordinate) of the source buffer area from which to get the data.

OffY

Specifies the vertical offset (relative to the top-left source buffer coordinate) of the source buffer area from which to get the data.

SizeX

Specifies the width of the source buffer area from which to get the data.

SizeY

Specifies the height of the source buffer area from which to get the data.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
- array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]

Specifies the address of the user array in which to copy the data from the source buffer. Ensure that the user array is large enough to receive the data from the specified area and one or all bands of the source buffer.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **DataFormat** parameter and possible values returned to the **UserArrayPtr** parameter.

- For specifying the data format

Note that *SizeX* and *SizeY* denote the source buffer's width and height, respectively.

For specifying the data format	
DataFormat UserArrayPtr - Possible values returned	Description
M_PACKED +	Copies the bands in an interleaved manner (for example, RGB RGB RGB...). This format is only valid when the Band parameter is set to M_ALL_BANDS. You must specify a combination value from the following table: • To set the packed data format
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_PLANAR	Copies the bands one after the other. This format is to be used when copying from all bands of the source buffer.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*
M_SINGLE_BAND	Copies a single band. This format is not valid when the Band parameter is set to M_ALL_BANDS.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*

Combination values for M_PACKED.

You must add one of the following values to the above-mentioned value to set the packed data format.

For more information on these formats, see the RGB buffers section of Chapter 23: Data buffers.

For setting the packed data format	
DataFormat UserArrayPtr - Possible values returned	Description
M_BGR24	Stores the data in a BGR24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*

M_BGR32	Stores the data in a BGR32 packed format. In this format, each pixel is internally stored as four consecutive bytes. Every fourth byte is a "don't care" byte.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_RGB15	Stores the data in an RGB15 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 5-bit green value, a 5-bit red value, and a "don't care" bit (most significant), in little-endian order (RGB 5:5:5).
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_RGB16	Stores the data in an RGB16 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 6-bit green value, and a 5-bit red value (most significant), in little-endian order (RGB 5:6:5).
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_RGB24	Stores the data in an RGB24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufGetColor

Synopsis

Get data from one or all bands of a buffer and place it in a user-supplied array.

Syntax

```
void MbufGetColor(
    AIL_ID SrcBufId,      //in
    AIL_INT64 DataFormat, //in
    AIL_INT Band,         //in
    void *UserArrayPtr    //out
)
```

Description

This function copies data from one or all bands of a specified Aurora Imaging Library source buffer to a user-supplied array.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

Parameters

SrcBufId

Specifies the identifier of the source buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

DataFormat

Specifies the data format to use to save the data in the user array. The internal data format of the source buffer need not match the specified data format of the user-supplied array; an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster.

See the Parameter associations section for possible values that can be specified.

Band

Specifies the index of the band to copy. This parameter can be set to one of the values below.

For specifying the index of the band to copy	
Value	Description
M_ALL_BANDS	Copies from all bands.
M_BLUE	Copies from the blue band (for RGB buffers).
M_GREEN	Copies from the green band (for RGB buffers).
M_HUE	Copies from the hue band (for HSL and HSV buffers).

M_LUMINANCE	Copies from the luminance band (for HSL buffers).
M_RED	Copies from the red band (for RGB buffers).
M_SATURATION	Copies from the saturation band (for HSL and HSV buffers).
M_V	Copies from the V band (for HSV buffers).
0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between index value and band for RGB, HSL, and HSV buffers is the following: 0 — Corresponds to the red band for RGB buffers, the hue band for HSL and HSV buffers. 1 — Corresponds to the green band for RGB buffers or the saturation band for HSL and HSV buffers. 2 — Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for HSV buffers.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
- array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]

Specifies the address of the user array in which to copy data from the source buffer. Ensure that the user array is large enough to receive the data from the specified bands of the source buffer.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **DataFormat** parameter and possible values returned to the **UserArrayPtr** parameter.

- For specifying the data format

Note that *SizeX* and *SizeY* denote the source buffer's width and height, respectively.

For specifying the data format	
DataFormat	Description
UserArrayPtr - Possible values returned	

M_PACKED +	Copies buffer bands in an interleaved manner (for example, RGB RGB RGB...). This format is only valid when the Band parameter is set to M_ALL_BANDS. You must specify a combination value from the following table: • To set the packed data format
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_PLANAR	Copies the bands one after the other. This format is only valid when the Band parameter is set to M_ALL_BANDS.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*
M_SINGLE_BAND	Copies a single band. This format is not valid when the Band parameter is set to M_ALL_BANDS.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*

Combination values for M_PACKED.

You must add one of the following values to the above-mentioned value to set the packed data format.

For more information on these formats, see the RGB buffers section of Chapter 23: Data buffers.

For setting the packed data format	
DataFormat	Description
UserArrayPtr - Possible values returned	
M_BGR24	Stores the data in a BGR24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_BGR32	Stores the data in a BGR32 packed format. In this format, each pixel is internally stored as four consecutive bytes. Every fourth byte is a "don't care" byte.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_RGB15	Stores the data in an RGB15 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 5-bit green value, a 5-bit red value, and a "don't care" bit (most significant), in little-endian order (RGB 5:5:5).
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*

M_RGB16	Stores the data in an RGB16 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 6-bit green value, and a 5-bit red value (most significant), in little-endian order (RGB 5:6:5).
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*
M_RGB24	Stores the data in an RGB24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a std::vector<unsigned-char>]*

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufGetHookInfo

Synopsis

Get information about a hook event.

Syntax

```
AIL_INT MbufGetHookInfo(
    AIL_ID EventId,     //in
    AIL_INT64 InfoType, //in
    void *UserVarPtr    //out
)
```

Description

This function allows you to get information about the event that caused the hook-handler function to be called. **MbufGetHookInfo**() should only be called within the scope of a buffer or container hook-handler function (see MbufHookFunction()).

Parameters

EventId

Specifies the buffer or container event identifier received by the hook-handler function (see MbufHookFunction()).

InfoType

Specifies the event type and the type of information about the event to return.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- a pointer of type void

Specifies the address in which to write the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **InfoType** parameter and possible values returned to the **UserVarPtr** parameter.

- For the event type

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the event type

InfoType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_GC_FEATURE_CHANGE_NAME +	Retrieves the name of the GenICam feature that changed. This information type is only available if **MbufGetHookInfo**() is used from a function hooked to GenTL feature change events using MbufHookFunction() with M_FEATURE_CHANGE.															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e												b
Value	Specifies the feature name.			e												b
M_MODIFIED_BUFFER +	Retrieves information about a modified buffer or container type of event. This event occurs if the hook-handler function was hooked using MbufHookFunction() with M_MODIFIED_BUFFER. You must specify a combination value from the following table: • To set the type of information to return	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an void*	c		e	f	g	a		i		j	k	l	m	n	b
M_OBJECT_ID	Retrieves the identifier of the container that was modified. This event occurs if the hook-handler function was hooked using MbufHookFunction() with M_LAYOUT_MODIFIED.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Container identifier	Specifies the identifier of the container that was modified.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_MODIFIED_BUFFER.

You must add one of the following values to the above-mentioned value to set the type of information to return.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the modified buffer type of event																
InfoType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_BUFFER_ID	Retrieves the Aurora Imaging Library identifier of the modified buffer or container.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_TIME_STAMP	Inquires the time when Aurora Imaging Library finished grabbing into the buffer, expressed in seconds. Typically, this will be in seconds since the Host computer was started. This can be used to see if a frame has been missed since the last time stamp.								i		j	k	l	m		
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*								i		j	k	l	m		
M_MOVED	Returns whether the buffer modification event was triggered by a move operation performed by MbufChildMove(); that is, the buffer is a child buffer and its X/Y offset was changed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NO	Specifies that the buffer modification event was not triggered by a move operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_YES	Specifies that the buffer modification event was triggered by a move operation.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_OFFSET_X	Retrieves the X-offset of the modified region (of the buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_OFFSET_Y	Retrieves the Y-offset of the modified region (of the buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_SIZE_X	Retrieves the width of the modified region (of the buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_SIZE_Y	Retrieves the height of the modified region (of the buffer).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_RESIZED	Returns whether the buffer modification event was triggered by a resize operation performed by MbufChildMove(); that is; the buffer is a child buffer and its XY-size was changed.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_NO	Specifies that the buffer modification event was not triggered by a resize operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_YES	Specifies that the buffer modification event was triggered by a resize operation.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_GC_FEATURE_CHANGE_NAME.

You can add the following value to the above-mentioned value to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																	
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE		Retrieves the length of the string, including the terminating null character ("\0").															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e												b

Return value

The returned value is **M_NULL** if successful. If the operation fails, a non-null (!**M_NULL**) value is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufGetLine

Synopsis

Read the pixels along a specified theoretical line, count the pixels, and store either their values or coordinates in a user-defined array.

Syntax

```
void MbufGetLine(
    AIL_ID ImageBufId,    //in
    AIL_INT StartX,       //in
    AIL_INT StartY,       //in
    AIL_INT EndX,         //in
    AIL_INT EndY,         //in
    AIL_INT64 Mode,       //in
    AIL_INT *NbPixelsPtr, //out
    void *UserArrayPtr    //out
)
```

Description

This function reads the series of pixels along a theoretical line (defined by coordinates) from an image and stores either their values or the X/Y coordinates of each value in a user-defined array. The Bresenham algorithm is used to determine the theoretical line. The line can start and end outside of the buffer, but only pixel values/coordinates within the buffer are store in the user-defined array.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

Parameters

ImageBufId

Specifies the identifier of the source image buffer. This must be a single-band (monochrome) buffer.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

StartX

Specifies the horizontal pixel offset of the starting position of the line, relative to the top-left pixel of the source buffer.

StartY

Specifies the vertical pixel offset of the starting position of the line, relative to the top-left pixel of the source buffer.

EndX

Specifies the horizontal pixel offset of the finishing position of the line, relative to the top-left pixel of the source buffer.

EndY

Specifies the vertical pixel offset of the finishing position of the line, relative to the top-left pixel of the source buffer.

Mode

Specifies the operation mode.

For specifying the operation mode	
Value	Description
M_DEFAULT	Specifies to read or count the pixels along a specified theoretical line and store their values in a user-defined array.
M_POSITION_X	Specifies to read the pixels along a specified theoretical line and store the X-coordinates of the pixels along that line in a user-defined array of integer values.
M_POSITION_Y	Specifies to read the pixels along a specified theoretical line and store the Y-coordinates of the pixels along that line in a user-defined array of integer values.

NbPixelsPtr

Specifies the address of the variable in which to write the number of pixels found along the theoretical line. You can set this parameter to **M_NULL** if you don't want this value to be evaluated.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_NULL** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (ImageBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:Call **MbufGetLine**() with UserArrayPtr set to **M_NULL**. The array size is stored in the NbPixelsPtr parameter. Note: When retrieving the pixels

- Data type: array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
  Required array size:Call **MbufGetLine**() with UserArrayPtr set to **M_NULL**. The array size is stored in the NbPixelsPtr parameter. Note: When retrieving the position.

Specifies the address of the user array in which to store the pixels or the coordinates from the image buffer. Ensure that the user array is large enough to receive the data to be stored. To determine the required size of the array, you can set this parameter to **M_NULL** and pass a non-null address to NbPixelsPtr, which will store the required size of the array. In this case, nothing is read from the image buffer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufGetList

Synopsis

Read the values in the source buffer, at the specified positions, and place their values in a user-defined array.

Syntax

```
void MbufGetList(
    AIL_ID SrcBufId,                //in
    AIL_INT NumPixels,              //in
    const AIL_DOUBLE *PixXArrayPtr, //in
    const AIL_DOUBLE *PixYArrayPtr, //in
    AIL_INT64 InterpolationMode,    //in
    void *UserArrayPtr              //out
)
```

Description

This function reads the values in the source buffer, at the specified positions, and places their values in a user-defined array.

If the source buffer is compressed, Aurora Imaging Library decompresses the data before copying it to the user-supplied array.

When executing the operation on a buffer that has multiple bands, the bands are read sequentially. For example, with a planar RGB buffer, the red band is read first; the red values at the specified positions are placed in the first third of the user-defined array. Then, the green band is read; the green values at the specified positions are placed in the second third of the user-defined array (immediately following the red values). Finally, the blue band is read; the blue values at the specified positions are placed in the last third of the user-defined array (immediately following the green values). If you want to read, for example, 3 pixels, the function will store the red band of each pixel, then the green band, and finally the blue band (RRR, GGG, BBB not RGB, RGB, RGB).

> Note that if the source buffer has multiple bands, its format should match the implicit format of the user array; otherwise an internal conversion has to be done (from the source to a temporary buffer) before executing the operation using the temporary buffer. Likewise, if the source buffer is compressed, an internal decompression has to be done (from the source to a temporary buffer) before executing the operation using the temporary buffer.

> Note that presented above is an example prototype of this function. You can call MbufGetList() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use MbufGetList() and not, for example, MbufGetListInt32(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

SrcBufId

Specifies the identifier of the source buffer. This buffer can be a single-band (monochrome) or multi-band buffer and is not limited to image buffers.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

NumPixels

Specifies the number of values to get.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the PixXArrayPtr or PixYArrayPtr parameter.

PixXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPixels)
- array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]

Specifies the address of an array containing the X-position corresponding to the pixels to get.

PixYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPixels)
- array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]

Specifies the address of an array containing the Y-position corresponding to the pixels to get.

InterpolationMode

Specifies the interpolation mode. The specified mode controls how the function behaves with non-integer positions.

This parameter must be set to one of the values below.

For specifying the interpolation mode	
Value	Description
M_DEFAULT	Same as M_NEAREST_NEIGHBOR with M_OVERSCAN_ENABLE.
M_BICUBIC +	Specifies bicubic interpolation. The value placed in the user-defined array is determined by taking a weighted average of the 16 values (4x4) that surround the specified position. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated according to the depth and data type of the array.
M_BILINEAR +	Specifies bilinear interpolation. The value placed in the user-defined array is determined by taking a weighted average of the 4 values (2x2) that surround the specified position.
M_NEAREST_NEIGHBOR +	Specifies nearest neighbor interpolation. The value placed in the user-defined array is that of the value of the pixel closest to the specified position.

Combination values for the values listed in For specifying the interpolation mode.

You can add one of the following values to the above-mentioned values to determine how the function behaves

with positions that fall outside the source buffer.

For overscan	
Combination value	Description
M_OVERSCAN_CLEAR	Sets the destination position to 0, if the associated point falls outside the source buffer.
M_OVERSCAN_DISABLE	Leaves the destination position as is, if the associated point falls outside the source buffer.
M_OVERSCAN_ENABLE	Uses the values of the source buffer's ancestor buffer, if the associated point falls outside the source buffer. If the source buffer is not a child buffer or if the point falls outside the ancestor buffer, it leaves the destination point as is.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (SrcBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:(NumPixels) * MbufInquire (M_SIZE_BAND)

Specifies the address of a user-defined array in which the values are placed. The data type of this array must match the data type of the source buffer. Its number of entries is determined by the number of values to get and also by the number of bands of the source buffer.

Type-specific versions of the function when using a C compiler

```
void MbufGetListDouble (AIL_ID SrcBufId, AIL_INT NumPixels, const AIL_DOUBLE *PixXArrayPtr, const AIL_DOUBLE *PixYArrayPtr, AIL_INT64 InterpolationMode, void *UserArrayPtr)
```

Details

Parameters

SrcBufId

See SrcBufId of the main function for a description.

NumPixels

See NumPixels of the main function for a description.

PixXArrayPtr

See PixXArrayPtr of the main function for a description.

PixYArrayPtr

See PixYArrayPtr of the main function for a description.

InterpolationMode

See InterpolationMode of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

```
void MbufGetListInt32 (AIL_ID SrcBufId, AIL_INT NumPixels, const AIL_INT32 *PixXArrayPtr, const AIL_INT32 *PixYArrayPtr, AIL_INT64 InterpolationMode, void *UserArrayPtr)
```

Details

Parameters

SrcBufId

See SrcBufId of the main function for a description.

NumPixels

See NumPixels of the main function for a description.

PixXArrayPtr

See PixXArrayPtr of the main function for a description.

PixYArrayPtr

See PixYArrayPtr of the main function for a description.

InterpolationMode

See InterpolationMode of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

```
void MbufGetListInt64 (AIL_ID SrcBufId, AIL_INT NumPixels, const AIL_INT64 *PixXArrayPtr, const AIL_INT64 *PixYArrayPtr, AIL_INT64 InterpolationMode, void *UserArrayPtr)
```

Details

Parameters

SrcBufId

See SrcBufId of the main function for a description.

NumPixels

See NumPixels of the main function for a description.

PixXArrayPtr

See PixXArrayPtr of the main function for a description.

PixYArrayPtr

See PixYArrayPtr of the main function for a description.

InterpolationMode

See InterpolationMode of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufHookFunction

Synopsis

Hook a function to a buffer or container event.

Syntax

```
void MbufHookFunction(
    AIL_ID ContainerOrBufId,                  //in
    AIL_INT HookType,                         //in
    AIL_BUF_HOOK_FUNCTION_PTR HookHandlerPtr, //in
    void *UserDataPtr                         //in-out
)
```

Description

This function allows you to attach or detach a user-defined function to a specified buffer or container event (for example, a modification to the buffer's contents). Once a hook-handler function is defined and hooked to an event, it is automatically called when the event occurs.

You can hook more than one function to an event by making separate calls to **MbufHookFunction**() for each function that you want to hook. Aurora Imaging Library automatically chains and keeps an internal list of all these hooked functions. When a function is hooked, this new function is added to the end of the list. When the event happens, all user-defined functions in the list will be executed in the same order that they were hooked to the event. You can also remove any function from the list; in this case, Aurora Imaging Library preserves the order of the remaining functions in the list.

Parameters

ContainerOrBufId

Specifies the identifier of the buffer or container.

HookType

Specifies the buffer event to which to hook the function. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the buffer event																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_FEATURE_CHANGE +	Hooks the hook-handler function to the event that occurs when the value of a GenICam feature changes. To enable an event to occur when the value of a specific feature changes, use MbufControlFeature() with M_FEATURE_CHANGE_HOOK set to M_ENABLE. Repeat for each feature that you want to enable a feature change event. To enable an event to occur when the value of any feature changes, use M_FEATURE_CHANGE + M_ALL. This setting is only available for hooking to an image buffer. To retrieve the name of the feature that caused the event, use MbufGetHookInfo() with M_GC_FEATURE_CHANGE_NAME.			e												b
M_LAYOUT_MODIFIED +	Calls the hook-handler function when at least 1 component of a container has been added or removed. The hook-handler function is called once at the end of a function regardless of the number of changes.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFIED_BUFFER +	Calls the hook-handler function at the end of any Aurora Imaging Library function that modifies the specified buffer or container. > For a container, this event will occur any time the container is used a destination for an Aurora Imaging Library function (unless none of the container's components is modified), any time a component is added to or removed from the container, as well as any time one of the container's components is modified directly using its Aurora Imaging Library identifier. The hook-handler function will only be called once per call to an Aurora Imaging Library function, unless M_ON_COMPONENT is specified.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_MODIFIED_BUFFER.

You can add the following value to the above-mentioned value to set whether to hook the function to changes in the container's components..

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For hooking the function to changes in components																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ON_COMPONENT	Specifies to hook to changes in each component of the container individually, instead of to the container as a whole. When a component of the container changes, the hook function will be called as though it were hooked to that component. Using MbufGetHookInfo() with M_BUFFER_ID within the hook-handler function will return the buffer identifier of the component.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_FEATURE_CHANGE.

You can add the following value to the above-mentioned value to specify to hook the specified function to all feature change events.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_FEATURE_CHANGE																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL	Specifies to hook the specifies function to all feature change events.			e												b

Combination value for M_FEATURE_CHANGE; M_LAYOUT_MODIFIED; M_MODIFIED_BUFFER.

You can add the following value to the above-mentioned values to set that the function should be unhooked.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the function should be unhooked																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNHOOK	Unhooks the specified function if hooked to the specified event. When you use M_UNHOOK, you must provide the same values for all the parameters as when you originally hooked the function.	c		e	f	g	a		i		j	k	l	m	n	b

HookHandlerPtr

Specifies the address of the function that should be called when the specified event occurs. The hook-handler function must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    AIL_ID EventId,
    void *UserDataPtr
)
```

Parameters

HookType

Type of buffer event that generated the call.

EventId

Event identifier to pass to MbufGetHookInfo() when inquiring about the hooked event.

UserDataPtr

User data pointer that was passed (as UserDataPtr) to **MbufHookFunction**().

Upon successful completion, the hook-handler function should return **M_NULL**. Note MFTYPE and AIL_BUF_HOOK_FUNCTION_PTR are reserved Aurora Imaging Library predefined types for functions and data pointers.

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This

address is passed to the hook-handler function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if not used.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufImport

Synopsis

Import data from a file into a data buffer or container.

Syntax

```
AIL_ID MbufImport(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 FileFormat,        //in
    AIL_INT64 Operation,         //in
    AIL_ID SysId,                //in
    AIL_ID *ContainerOrBufIdPtr  //in-out
)
```

Description

This function imports data from a file into an existing or automatically allocated Aurora Imaging Library data buffer or container. The function assumes that the data in the file is in the specified format.

Note, you can also import data using MbufLoad() or MbufRestore(); however, these functions try to determine the format from the data rather than allowing you to specify the format.

For an M_RESTORE operation of an image, this function tries to allocate the buffer so that it can be used for acquisition (M_GRAB), display (M_DISP), and processing (M_PROC) operations. If the buffer cannot be allocated with the M_GRAB attribute, Aurora Imaging Library allocates one that can be used in all of the above operations except for acquisition (M_GRAB). You can specify that the restored buffer is not allocated with the acquisition or compression attributes (M_GRAB or M_COMPRESS respectively), by combining M_RESTORE with M_NO_GRAB or M_NO_COMPRESS respectively.

For an M_RESTORE operation of a container, this function allocates a container that can be used for display (M_DISP) and processing (M_PROC) operations.

> For an M_RESTORE operation with an M_AIL_TIFF or M_AIL_NATIVE file format that stores a non-image buffer, this function allocates the destination buffer with the same attributes as the original buffer.

If importing a CAD file, the data must be saved in the PLY or STL file format. If the CAD file is in another format, you must convert it to the PLY or STL file format (ASCII encoding or binary encoding) before calling this function. For binary PLY file format, it must be in little-endian format.

If importing an image, all the pixel information in the file is imported and loaded into the buffer. Additionally, if the file was saved in M_AIL_TIFF file format and includes any region of interest (ROI) information, the ROI information is also imported and loaded into the buffer when the FileFormat parameter is set to M_AIL_TIFF. To load the M_AIL_TIFF file without its ROI information, set FileFormat to M_AIL_TIFF + M_NO_REGION. If the file was saved with calibration information that provides a constant pixel size, the calibration information can be loaded and associated with the imported image with M_WITH_CALIBRATION. For more information see the Camera calibration propagation section of Chapter 28: Calibrating your camera setup.

> If the vectorial information of an M_VECTOR_AND_RASTER ROI was input in M_WORLD units, the ROI will be imported as an M_VECTOR ROI.

[Aurora Imaging Library Lite with restriction]

When performing an M_RESTORE operation on a file containing compressed data, the buffer will have an

M_COMPRESS attribute.

[Aurora Imaging Library Lite with restriction]

When importing compressed data, the following applies. When performing an M_LOAD operation and the destination buffer has an M_IMAGE attribute (but not an M_COMPRESS attribute), this function will automatically decompress it. If necessary, the data in the file will be transformed to the format of the buffer. If unsure of the compression type of the data, use M_DEFAULT as the file format rather than M_JPEG_... or M_JPEG2000_...; the data will be read correctly.

[Aurora Imaging Library Lite with restriction]

When importing uncompressed data into a buffer with an M_COMPRESS attribute, this function will automatically compress it, according to the compression settings found in the buffer.

When importing an image file (not stored in the M_AIL_NATIVE file format) that has been saved with an associated LUT (color palette) into a 3-band 8-bit image buffer, the LUT is automatically applied to the data to generate 3-band image data. In this case, a LUT buffer is not created and, therefore, is not associated with the 3-band 8-bit buffer. When importing an image file that has been saved with an associated LUT (color palette) into any other type of image buffer, the LUT is also imported and associated with the resulting image buffer. You can obtain the identifier of the associated LUT, using MbufInquire().

Note that the associated LUT will be automatically selected on the display (MdispLut()) if the image buffer is selected on a display and the default LUT has not been overridden by a former call to MdispLut().

Using MbufDiskInquire(), you can inquire about the dimensions of the data saved in a file (except for raw files, M_GENDC files, and M_AIL_NATIVE files that store a container) without importing it.

After importing or restoring a buffer or container, it is recommended that you check if the operation was successful, using MappGetError(), or by verifying that the returned buffer or container identifier is not M_NULL.

When the restored buffer or container is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the buffer or container's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to import the data. The function handles (internally) the opening and closing of the file.

This parameter can be set to the following value.

For specifying the file name and path	
Value	Description
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file, for example, "*C:\mydirectory\myfile*". To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").

AIL_TEXT("*ftp://user:password@server/pathtofile*") [1]	Import data from a remote File Transfer Protocol (FTP) server into a data buffer or container. To access a remote FTP server anonymously, omit the user name and password. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be imported using an FTP file path.
AIL_TEXT("*http://user:password@server/pathtofile*") [1]	Import data from a remote Hypertext Transfer Protocol (HTTP) server into a data buffer or container. HTTPS is not supported. > Note that importing PLY and STL files from a remote HTTP server is not supported.
AIL_TEXT("*sftp://user:password@server/pathtofile*") [1]	Import data from a remote SSH File Transfer Protocol (SFTP) server into a data buffer or container. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be imported using an SFTP file path.

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileFormat

Specifies the file format. This parameter can be set to one of the following values. Note that except for the M_AIL_NATIVE, M_AIL_TIFF, M_GENDC and M_RAW file formats, data is treated as image data. When performing an M_LOAD operation, the data is internally converted appropriately.

The following file types might require a buffer or a container as a destination. You can determine which object type is required as a destination using MbufDiskInquire() with M_OBJECT_TYPE.

For specifying a file format that might need to be imported to a buffer or container	
Value	Description
M_DEFAULT	Specifies that Aurora Imaging Library automatically determines the file format. If the file format is not supported, its data will be treated as raw data.
M_AIL_NATIVE	Imports a container or buffer saved in the Aurora Imaging Library native file format (typically with the mbuf or mbufc file extension). This format stores the entire content of an Aurora Imaging Library container or Aurora Imaging Library buffer, including settings.

The following file formats must have a buffer as a destination.

For specifying a file format that must be imported to a buffer	
Value	Description

M_AIL_TIFF +	Imports data that is in the M_AIL_TIFF file format (typically with the mim file extension). The baseline TIFF 6.0 specification is used.
M_BMP	Imports image data that is in BMP file format. The standard Windows BMP format is used. If the specified BMP file contains an image whose image format, data type, and/or depth is not supported by Aurora Imaging Library for image buffers, the image will not be imported.
M_JPEG2000_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG2000 lossless image.
M_JPEG2000_LOSSLESS_JP2	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG2000 lossless image, as well as the additional JP2 information.
M_JPEG2000_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG2000 lossy image.
M_JPEG2000_LOSSY_JP2	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG2000 lossy image, as well as the additional JP2 information.
M_JPEG_LOSSLESS	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG lossless image.
M_JPEG_LOSSLESS_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG lossless image stored in two separate fields.
M_JPEG_LOSSY	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG lossy image.
M_JPEG_LOSSY_INTERLACED	[For essential Aurora Imaging Library Lite information, see remarks] Imports a JPEG lossy image stored in two separate fields.
M_JPEG_LOSSY_RGB	[For essential Aurora Imaging Library Lite information, see remarks] Imports a 3-band JPEG lossy image that is in an RGB format.
M_PNG	Imports image data that is in a PNG file format.
M_RAW	Imports raw data.
M_TIFF	Imports image data that is in a TIFF file format. The baseline TIFF 6.0 specification is used. If the specified TIFF file contains an image whose image format, data type, and/or depth is not supported by Aurora Imaging Library for image buffers, the image will not be imported. Multi-page TIFF files can be imported into Aurora Imaging Library buffers but only the first page is saved in the buffer.

Combination values for M_AIL_TIFF.

You can add one or more of the following values to the above-mentioned value to set whether to import the image with additional information.

Note that you can only specify these values if importing an image (for example, you can specify M_NO_REGION + M_WITH_CALIBRATION).

For importing camera calibration or ROI information	
Combination value	Description
M_NO_REGION	Imports the image, excluding any saved ROI information.
M_WITH_CALIBRATION	Imports the image, including its camera calibration information. Note that, this image must have been saved using MbufExport() and it must have a constant pixel size. If the image does not have a constant pixel size, the image will be imported without its camera calibration information; the target buffer will be uncalibrated. If the image is calibrated and it has a constant pixel size, it is associated with a default uniform camera calibration context upon being imported into the buffer (as set using McalUniform()). If the image is imported into an existing buffer that is associated with a camera calibration context, the initial camera calibration context is replaced by that of the imported image.

The following file types must have a container as a destination.

For specifying a file format that must be imported to a container	
Value	Description
M_GENDC	Imports a container saved in the GenDC file format.
M_PLY_ASCII	Imports point cloud data saved in the PLY file format using ASCII encoding. > ail3d<version number>.dll must be present to import this format.
M_PLY_BINARY_LITTLE_ENDIAN	Imports point cloud data saved in the PLY file format using binary encoding in little-endian order. > ail3d<version number>.dll must be present to import this format.
M_STL_ASCII	Imports point cloud data, saved in the STL file format using ASCII encoding. STL files always include mesh data. > ail3d<version number>.dll must be present to import this format.
M_STL_BINARY	Imports point cloud data, saved in the STL file format using binary encoding. STL files always include mesh data. > ail3d<version number>.dll must be present to import this format.

Operation

Specifies the import operation. This parameter can be set to one of the following values.

For specifying the import operation

Value	Description
M_LOAD +	Imports data from the specified file into a previously allocated Aurora Imaging Library data buffer or container. If the destination is a container, all of its existing components will be freed. By default, the contents of the file are loaded, including all data and settings. Optional attributes (such as M_COMPRESS) of data stored in the file are not copied. You can use M_IDENTICAL to specify that the buffer size and optional attributes of the specified file must be loaded from the file. When loading a file to a buffer, the attributes of the destination buffer are used when loading the data (for example, if the destination does not have the M_COMPRESS attribute, the file will be decompressed during the load operation).When loading a file to a container, components in the file are loaded to components allocated without optional attributes (for example, components that store compressed data are loaded to components allocated without the M_COMPRESS attribute).
M_RESTORE +	Imports data from the specified file into an automatically allocated Aurora Imaging Library data buffer or container. Note, you cannot restore (M_RESTORE) a RAW data file (M_RAW) because its dimensions are unknown.

Combination values for M_LOAD.

You can add one of the following values to the above-mentioned value to specify how the contents of the file are loaded into the specified buffer or container..

Note that these settings are only available for loading from an M_AIL_NATIVE or M_GENDC file.

For specifying how the buffer or container is loaded	
Combination value	Description
M_BASIC_ATTRIBUTE	Specifies to load the contents of the file (that stores data suitable for loading into a container) to automatically allocated components that might have different attributes than the components stored in the file. > Note that this setting is only available for a container, not a buffer.
M_IDENTICAL	Specifies to load the contents of the file, including all data, settings, and attributes. This setting is only available for M_AIL_NATIVE and M_GENDC files. An error will be generated if there is a mismatch between the size, number of bands, or attributes (such as M_RGB24) of the destination buffer and the contents of the file.
M_USE_DESTINATION	Specifies to load the contents of the file (that stores data suitable for loading into a container) into the existing components of the destination container. An error will be generated if the file has a different number of components than the destination container, or if at least one of the components in the file cannot be loaded into the component with the same index in the container (for example, because the components of the destination container are of the wrong buffer type to store the imported data). > Note that this setting is only available for a container, not a buffer.

Combination values for M_RESTORE.

You can add one or more of the following values to the above-mentioned value to specify that the buffer is not restored with a certain attribute.

> Note that these values are only available when restoring an image buffer, and not a container.

For specifying that the buffer is not restored with a certain attribute	
Combination value	Description
M_NO_COMPRESS	Specifies that Aurora Imaging Library will not allocate the restored data buffer with an M_COMPRESS attribute.
M_NO_GRAB	Specifies that Aurora Imaging Library will not allocate the restored data buffer with an M_GRAB attribute.

SysId

Specifies the system on which to allocate the buffer or container for an M_RESTORE operation. For an M_LOAD operation, this parameter should be set to **M_NULL**.

For an M_RESTORE operation, this parameter should be set to one of the following values:

For specifying the system on which to allocate the buffer (with M_RESTORE)	
Value	Description
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().

ContainerOrBufIdPtr

Specifies the address of the variable containing the buffer or container identifier, for an M_LOAD operation, or the address of the variable in which to store the new buffer or container identifier (or specifies the data type that the function should use to return the buffer or container identifier), for an M_RESTORE operation.

For an M_LOAD operation, the destination buffer must be large enough in depth and dimensions to hold the data. If the data is deeper than the buffer, the most-significant bits of the data are truncated when loaded into the buffer. If the buffer depth is greater than that of the data, the data is zero or sign-extended (depending on the data type) when loaded into the buffer. If the buffer is larger in size than the data, exceeding areas of the buffer are unaffected.

For an M_RESTORE operation, the destination buffer or container will be allocated with an appropriate size and type to hold the data.

For retrieving the identifier or specifying how to return it	
Value	Description
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the restored buffer or container; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the restored buffer or container; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer or container (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.
Address in which to write the buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored buffer. If restoration fails, **M_NULL** is written as the identifier.
Address in which to write the container identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored container. If restoration fails, **M_NULL** is written as the identifier.
Address of the buffer identifier	Specifies the address of the Aurora Imaging Library identifier of the buffer in which to load the contents of the file.
Address of the container identifier	Specifies the address of the Aurora Imaging Library identifier of the container in which to load the contents of the file.

Return value

The returned value is the buffer or container identifier (for an M_RESTORE operation only) either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If restoration fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remark

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support is reliant upon the presence of a compression/decompression package license. This license is only included by default with the development dongle for the full version of Aurora Imaging Library. In other cases, this license must be purchased separately.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib;.
DLL	Requires ail11.dll, ail3d11.dll for some formats.

MbufImportSequence

Synopsis

Import a sequence of images from an AVI file into separate image buffers.

Syntax

```
void MbufImportSequence(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_INT64 FileFormat,        //in
    AIL_INT64 Operation,         //in
    AIL_ID SystemId,             //in
    AIL_ID *BufArrayPtr,         //in-out
    AIL_INT StartImage,          //in
    AIL_INT NumberOfImages,      //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function imports a sequence of images from an AVI file into separate image buffers. **MbufImportSequence**() can automatically allocate the necessary buffers or you can use previously allocated buffers. In the latter case, the BufArrayPtr parameter should point to an array containing the buffer identifiers. In the former case, **MbufImportSequence**() will write the identifiers of the new buffers into the array pointed to by BufArrayPtr.

Rather than importing all the required images at once, you can also use this function to only open the AVI file for reading. Then, call this function as many times as required to import different sets of images from the file. Once you have finished importing images from the file, you must call this function again to close the file.

Parameters

FileName

Specifies the name and path of the AVI file. This function handles the opening and/or closing of the file depending on the ControlFlag parameter setting.

This parameter can be set to the following value:

For specifying the name and path of the AVI file	
Value	Description
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

FileFormat

Specifies the format of the file. This parameter can be set to one of the values listed below.

For specifying the file format	
Value	Description

M_DEFAULT	Specifies that Aurora Imaging Library automatically determines the file format.
M_AVI_AIL	Specifies an AVI format containing images in their Aurora Imaging Library format.
M_AVI_DIB	Specifies an AVI format containing non-compressed images.
M_AVI_MJPG	[For essential Aurora Imaging Library Lite information, see remarks] Specifies an AVI format containing compressed images.

Operation

Specifies whether to import the specified sequence of images into automatically allocated buffers or previously allocated buffers. If not importing images, this parameter should be set to **M_NULL**.

When importing images, this parameter can be set to one of the following:

For importing images	
Value	Description
M_LOAD	Imports the sequence into previously allocated buffers.
M_RESTORE	Imports the sequence into automatically allocated buffers.

SystemId

Specifies the system on which to allocate the buffers for an M_RESTORE operation. In other cases, this parameter should be set to **M_NULL**.

For an M_RESTORE operation, this parameter should be set to one of the following values:

For specifying the system	
Value	Description
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().

BufArrayPtr
Accepts the address of one of the following:

- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>] (and a maximum array rank for C# of 1)
  Required array size:(NumberOfImages)

Specifies the address of the array containing the buffer identifiers for an M_LOAD operation, or the address of the array in which to store the new buffer identifiers for an M_RESTORE operation. If not importing images, this parameter should be set to **M_NULL**.

If specifying an image buffer, it must be a 1- or 3-band buffer and must not have a region of interest (ROI)

associated with it. Using an image buffer with an ROI will cause an error.

For an M_LOAD operation, the destination buffers should be large enough to hold the imported images.

For an M_RESTORE operation, the destination buffers will be allocated with an appropriate size and type to hold the images. If an M_RESTORE operation fails, zero will be written for the buffer identifiers.

[Aurora Imaging Library Lite with restriction]

When importing compressed images with the M_LOAD operation, if the destination buffers have only an M_IMAGE attribute, the images will automatically be decompressed. When importing uncompressed images with the M_LOAD operation, if the destination buffers have an M_IMAGE + M_COMPRESS attribute, the images will automatically be compressed.

When importing compressed images with the M_RESTORE operation, the M_COMPRESS attribute will automatically be added to the destination buffer (M_IMAGE + M_COMPRESS) and the images will remain compressed in the destination buffer.

StartImage

Specifies the frame number of the image from which to begin importing. If not importing images, this parameter should be set to **M_NULL**.

If importing images, this parameter can be set to one of the following values:

For specifying the frame number of the image from which to begin importing	
Value	Description
M_DEFAULT	Specifies to import the image at the current read position.
Value >= 0	Specifies the frame number of the image from which to begin importing. Note that the frame number of the first image in the sequence is zero.

NumberOfImages

Specifies the number of images, starting at StartImage, to import. If not importing images, this parameter should be set to **M_NULL**.

When importing images, this number cannot be larger than the size of the array pointed to by BufArrayPtr. Note that you can inquire about the number of frames (images) in the AVI file using MbufDiskInquire().

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

ControlFlag

Specifies the function's control flag. This parameter can be set to one of the following:

For specifying the function's control flag	
Value	Description
M_DEFAULT	Opens the AVI file, imports the specified images, and then closes the file.

M_CLOSE	Closes the AVI file, and (re)sets the pointer position to the first image. No images are imported.
M_OPEN	Opens the AVI file for reading, and sets the pointer to the first image. No images are imported.
M_READ	Imports the specified images from the AVI file, starting at the specified StartImage position. After the importing the images, the file pointer is left at the position of the next image, ready for the next M_READ operation.

Remarks

- When a "remote" path is specified (the FileName parameter begins with "remote:///"), the SystemId parameter must point to the remote system when the AVI file is being opened or closed (M_OPEN or M_CLOSE). The buffers that are receiving the images are allocated on this same remote system.
- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for M_AVI_MJPG format and M_IMAGE + M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufInquire

Synopsis

Inquire about an Aurora Imaging Library data buffer setting.

Syntax

```
AIL_INT MbufInquire(
    AIL_ID BufId,          //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function inquires about a specified setting of an Aurora Imaging Library data buffer.

Parameters

BufId

Specifies the identifier of the source data buffer. The buffer must have been previously allocated on the required system using a function of the Buffer module (for example, MbufAllocColor() or MbufImport()).

InquireType

Specifies the type of buffer setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT64
- AIL_UINT64
- same type as the buffer (BufId)

Specifies the address in which to write the requested information. Since the **MbufInquire**() function also returns the requested information, you can set this parameter to **M_NULL**, except when the specified InquireType requires the UserVarPtr parameter to be set to the address of an *AIL_INT64* or *AIL_DOUBLE*. In this case, you must set this parameter to the address of an *AIL_INT64* or *AIL_DOUBLE*, respectively.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring general buffer settings
- For M_KERNEL and M_STRUCT_ELEMENT data buffers
- For M_KERNEL data buffers only

- For M_STRUCT_ELEMENT data buffers only
- For M_IMAGE data buffers only
- For M_IMAGE + M_COMPRESS image buffers
- For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSY, M_JPEG_LOSSY_INTERLACED, or M_JPEG2000_LOSSY)
- For M_IMAGE + M_COMPRESS (with M_JPEG2000_LOSSY or M_JPEG2000_LOSSLESS)
- For M_IMAGE + M_COMPRESS (with M_JPEG2000_LOSSY)
- For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSY, M_JPEG_LOSSY_INTERLACED, M_JPEG_LOSSLESS, or M_JPEG_LOSSLESS_INTERLACED)
- For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSLESS, or M_JPEG_LOSSLESS_INTERLACED)
- For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSY, or M_JPEG_LOSSY_INTERLACED)
- For inquiring settings useful with buffers that are components
- For inquiring settings useful with M_IMAGE buffers that store 3D data and are components

The following inquire types allow you to inquire about general types of buffer settings.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring general buffer settings																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ALLOCATION_OVERSCAN_SIZE		Inquires the size of the overscan region of the image buffer. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the size of the overscan region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANCESTOR_ID		Inquires the Aurora Imaging Library identifier of the ancestor buffer. Only child buffers have an ancestor buffer. The ancestor buffer is the buffer from which the specified buffer (BufId) ultimately originated. It is the root buffer; it does not have a parent buffer (it is not a child buffer of another buffer). Note that the identifier of the specified buffer is returned if it does not have an ancestor buffer. To establish the parent buffer of the specified buffer, use M_PARENT_ID instead.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
	Buffer identifier	Specifies the Aurora Imaging Library identifier of the ancestor buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_ANCESTOR_OFFSET_BAND	Inquires the band offset relative to the ancestor buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the band offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANCESTOR_OFFSET_BIT	Inquires the bit offset relative to the ancestor buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the offset, in bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANCESTOR_OFFSET_X	Inquires the X-offset relative to the ancestor buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the X-offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANCESTOR_OFFSET_Y	Inquires the Y-offset relative to the ancestor buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_BITMAPINFO	[This is only applicable to Windows] Inquires a pointer (LPBITMAPINFO) to the header of the DIB associated with the Aurora Imaging Library buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that there is no DIB associated with the Aurora Imaging Library buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the pointer to the DIB header.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAMERA_CONTAINER_ID	Inquires the GenDC container identifier from the camera. The camera sends the identifier with each transmitted frame.			e	f	g							l	m		b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UINT64*			e	f	g							l	m		b

	0	Specifies that GenDC container identifiers are not supported by the camera.			e	f	g							l	m		b
	Value > 0	Specifies the GenDC container identifier.			e	f	g							l	m		b
M_CAMERA_FRAME_ID		Inquires the frame identifier from the camera. The camera sends the identifier with each transmitted frame.			e	f	g									n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_UINT64*			e	f	g									n	b
	0	Specifies that frame identifiers are not supported by the camera.			e	f	g									n	b
	Value > 0	Specifies the frame identifier.			e	f	g									n	b
M_CAMERA_TIME_STAMP		Inquires the time stamp from the camera expressed in seconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the time stamp, in seconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAMERA_TIME_STAMP_NS		Inquires time stamp from the camera expressed in nanoseconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the time stamp, in nanoseconds.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_TIME_STAMP		Inquires the time stamp given by the camera GenDC stream. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the time stamp, in seconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_TIME_STAMP_IS_PTP		Inquires if M_COMPONENT_TIME_STAMP_NS is relative to the Unix epoch (January 1st 1970 00:00:00).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that M_COMPONENT_TIME_STAMP_NS is not calculated based on the Unix epoch (January 1st 1970 00:00:00).	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that M_COMPONENT_TIME_STAMP_NS is relative to the Unix epoch (January 1st 1970 00:00:00).	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_TIME_STAMP_NS		Inquiries the time stamp given by the camera GenDC stream in nanoseconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the time stamp, in nanoseconds.	c		e	f	g	a		i		j	k	l	m	n	b

M_DATA_FORMAT	Inquires the data storage format of the buffer. This is the color or monochrome storage format of the buffer and includes whether it is in packed or planar format. Note that non-color buffers are stored in planar format and lack a color space. To retrieve the entire list of attributes, use **MbufInquire**() with M_EXTENDED_ATTRIBUTE or M_EXTENDED_FORMAT. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c	d	e	f	g	a	h	i		j	k	l	m	n	b
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
	//Initially, if a buffer was allocated at the same time as its memory Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	//Initially, if a buffer was created from preallocated memory Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c			f		a		i		j	k	l	m		
M_DATA_TYPE	Inquires the buffer data type. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT	Specifies that the buffer uses the float data type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the buffer uses the signed data type.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED	Specifies that the buffer uses the unsigned data type.	c		e	f	g	a		i		j	k	l	m	n	b

M_DC_HANDLE	[This is only applicable to Windows] Inquires the device context handle (HDC) of the buffer. The buffer device context must have been successfully allocated using MbufControl() with M_DC_ALLOC.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that no device context is associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the handle of the device context that is associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIB_HANDLE	[This is only applicable to Windows] Inquires the handle (*HBITMAP*) of the DIB associated with the Aurora Imaging Library buffer. To ensure that the buffer has a DIB handle, the buffer must have been successfully allocated using with M_DIB + M_GDI.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that no DIB handle is associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the handle of the DIB associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTENDED_ATTRIBUTE	Inquires the attributes of the specified buffer. This inquire type only returns attributes that were explicitly set; any attribute left to its default is not returned. To retrieve only the data storage format of the buffer, use **MbufInquire**() with M_DATA_FORMAT. To retrieve the detailed storage format of the specified buffer, use **MbufInquire**() with M_EXTENDED_FORMAT. Note that you cannot set the UserVarPtr parameter to **M_NULL** when InquireType is set to M_EXTENDED_ATTRIBUTE. In addition, a valid *AIL_INT64* pointer must be passed to the function; otherwise, an error will occur. To convert the returned attributes, select the **Benchmarks and Utilities** item in the tree structure of the Aurora Imaging Configurator utility. Then, select the **Buffer format** item. On the **Value type** pane, paste the returned attributes in the text box provided. Then, click on the **Value lookup** button. The results of the translation are presented below the **Value lookup** button. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_ARRAY +	Specifies a buffer to store array type data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data.	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL +	//For 1D or 2D data Specifies a kernel buffer to store a custom filter for convolution functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT +	Specifies a buffer to store lookup table data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT +	//For 1D or 2D data Specifies a buffer to store structuring element data for morphology functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_ATTRIBUTE_NAME +	Inquires the M_EXTENDED_ATTRIBUTE of the buffer, expressed in a human readable format.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_FORMAT		Inquires the detailed storage format of the specified buffer. When you inquire M_EXTENDED_FORMAT, the returned value includes any buffer attributes that you set, any setting that was left to its default, and values that were internally calculated by Aurora Imaging Library. To retrieve only the data storage format of the buffer that you set, use **MbufInquire**() with M_DATA_FORMAT. To retrieve all the attributes of the buffer as set using MbufAlloc...() with Attribute, use **MbufInquire**() with M_EXTENDED_ATTRIBUTE. You can determine generally whether the buffer is in an RGB/BGR, YUV, or monochrome storage format using the buffer's extended format with the **M_IS_FORMAT_RGB_BGR**, **M_IS_FORMAT_YUV**, or **M_IS_FORMAT_MONO** macro respectively. Note that you cannot set the UserVarPtr parameter to **M_NULL** when InquireType is set to M_EXTENDED_FORMAT. In addition, a valid *AIL_INT64* pointer must be passed to the function; otherwise, an error will occur. Some buffer attributes are not returned when using a remote computer because they are not applicable to a remote computer. These attributes include, but are not limited to, M_LINUX_MXIMAGE, M_DIB, M_GDI, M_HOST_MEMORY. To convert the returned attributes, select the **Benchmarks and Utilities** item in the tree structure of the Aurora Imaging Configurator utility. Then, select the **Buffer format** item. On the **Value type** pane, paste the returned attributes in the text box provided. Then, click on the **Value lookup** button. The results of the translation are presented below the **Value lookup** button.	c		e	f	g	a		i		j	k	l	m	n	b
		System specific															
		Note that this inquire type only returns attributes that were explicitly set; any attribute left to its default value is not returned.			e												b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ARRAY +	Specifies a buffer to store array type data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_IMAGE +	Specifies a buffer to store image data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_KERNEL +	//For 1D or 2D data Specifies a kernel buffer to store a custom filter for convolution functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LUT +	Specifies a buffer to store lookup table data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_STRUCT_ELEMENT +	//For 1D or 2D data Specifies a buffer to store structuring element data for morphology functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_FORMAT_NAME +		Inquires the M_EXTENDED_FORMAT of the buffer, expressed in a human readable format.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_TIME_STAMP		Inquires the time stamp of when the grab operation completed, in seconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the time stamp of when the grab operation completed, in seconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_TIME_STAMP_NS		Inquires the time stamp of when the grab operation completed, in nanoseconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0	Specifies the time stamp of when the grab operation completed, in nanoseconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_HOST_ADDRESS		Inquires the Host address of the buffer, if the buffer is visible from the Host address space and is not a planar multi-band buffer. For a planar multi-band buffer, use M_HOST_ADDRESS_BAND + n instead. If available, this address can be used to directly access the data of an Aurora Imaging Library buffer with the Host CPU.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of the same type as BufId*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that the buffer is not visible from the Host address space or the buffer is a planar multi-band buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Host address of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_HOST_ADDRESS_BAND + *n*		Inquires the Host address for the band of the buffer identified by *n*, if the buffer is visible from the Host address space and is a planar multi-band buffer. For any other buffer, use M_HOST_ADDRESS instead. If available, this address can be used to directly access the data of an Aurora Imaging Library buffer with the Host CPU.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of the same type as BufId*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that the buffer is not visible from the Host address space or the buffer is not a planar multi-band buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Host address of the specified band of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX		Inquires the maximum pixel value possible in the buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the expected maximum pixel value within the range of the buffer type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN	Inquires the minimum pixel value possible in the buffer. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the expected minimum pixel value within the range of the buffer type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_COUNT	Inquires the current value of the modification counter of the image buffer. When the image buffer is allocated, the modification counter is initialized to a random non-zero value. The modification counter is incremented each time the image buffer is modified by an Aurora Imaging Library function. If the image buffer is accessed externally, for example, when using MbufCreateColor() or MbufCreate2d(), MbufControl() with M_MODIFIED must be called to indicate that the image buffer's contents have been modified. Calling this function will increment the counter. This feature is useful for optimization. For example, you can avoid repeating certain computations (for example, analysis computations) if you know that the image buffer has not been modified. In this case, inquire the count before the first computation in the sequence of computations, and then inquire it again before repeating the same sequence. If no modifications have been made to the image buffer, you can avoid repeating the sequence unnecessarily.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_HOOK	Inquires the status of the modification hook, which runs a user-defined function upon an event. These user-defined functions are initially hooked to the buffer modification event using MbufHookFunction(). SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the user-defined functions should not be called. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the user-defined functions should be called.	c		e	f	g	a		i		j	k	l	m	n	b
M_NB_CHILD		Inquires the number of child buffers associated with the specified buffer (BufId).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of child buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_OWNER_CONTAINER_ID		Inquires the Aurora Imaging Library identifier of the container in which this buffer is a component.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that the buffer is not a component of a container.	c		e	f	g	a		i		j	k	l	m	n	b
	ContainerID	Specifies the identifier of the container of which this buffer is a component.	c		e	f	g	a		i		j	k	l	m	n	b
M_OWNER_SYSTEM		Inquires the identifier of the system on which the buffer has been allocated. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_OWNER_SYSTEM_TYPE		Inquires the type of system on which the buffer was allocated. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_SYSTEM_CLARITY_UHD_TYPE	Specifies an Aurora Imaging Library Clarity UHD system.	c														
	M_SYSTEM_GENTL_TYPE	Specifies an Aurora Imaging Library GenTL system.			e												
	M_SYSTEM_GEVIQ_TYPE	Specifies an Aurora Imaging Library GevIQ system.				f											
	M_SYSTEM_GIGE_VISION_TYPE	Specifies an Aurora Imaging Library GigE Vision system.					g										
	M_SYSTEM_HOST_TYPE	Specifies the Host.						a									
	M_SYSTEM_IRIS_GTX_TYPE	Specifies an Aurora Imaging Library Iris GTX system.								i							
	M_SYSTEM_RADIENTEVCL_TYPE	Specifies an Aurora Imaging Library Radient eV-CL system.										j					
	M_SYSTEM_RAPIXOCL_TYPE	Specifies an Aurora Imaging Library Rapixo Pro CL system.											k				
	M_SYSTEM_RAPIXOCOF_TYPE	Specifies an Aurora Imaging Library Rapixo CoF system.												l			
	M_SYSTEM_RAPIXOCXP_TYPE	Specifies an Aurora Imaging Library Rapixo CXP system.													m		
	M_SYSTEM_USB3_VISION_TYPE	Specifies an Aurora Imaging Library USB3 Vision system.														n	
	M_SYSTEM_V4L2_TYPE	Specifies an Aurora Imaging Library Video4Linux2 system.															b
M_PARENT_ID		Inquires the Aurora Imaging Library identifier of the parent buffer. Only child buffers have a parent buffer. The parent buffer is the buffer from which the specified buffer (BufId) was defined. The parent buffer can itself have a parent buffer. If the specified buffer has no parent buffer, the identifier of the specified buffer is returned. To establish the ancestor buffer (root buffer) of the specified buffer, use M_ANCESTOR_ID instead.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
	Buffer identifier	Specifies the Aurora Imaging Library identifier of the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARENT_OFFSET_BAND		Inquires the band offset relative to the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the band offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARENT_OFFSET_X		Inquires the X-offset relative to the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_PARENT_OFFSET_Y		Inquires the Y-offset relative to the parent buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_PHYSICAL_ADDRESS		Inquires the physical address of the buffer if it is not a planar multi-band buffer. For a planar multi-band buffer, you can determine its physical address by allocating a child buffer for the required band and then using M_PHYSICAL_ADDRESS to determine its physical address. This type of address is available only for a non-paged buffer mapped to the Host or for a buffer allocated in a frame grabber's on-board memory. This type of address is used mostly for access by DMA capable devices other than the Host CPU. To use this inquire type, the buffer must not have been allocated on a Distributed Aurora Imaging Library remote system. If it is, use M_PHYSICAL_ADDRESS_REMOTE instead.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_NULL	Specifies that the buffer is not visible from the physical address space.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the address of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PHYSICAL_ADDRESS_REMOTE		Inquires the physical address of the buffer on a Distributed Aurora Imaging Library remote system, if it is not a planar multi-band buffer. For a planar multi-band buffer, you can determine its physical address by allocating a child buffer for the required band and then using M_PHYSICAL_ADDRESS_REMOTE to determine its physical address. This type of address is available only for a non-paged buffer mapped to the remote system. This type of address is used mostly for access by DMA capable devices other than the Host CPU. To use this inquire type, the buffer must have been allocated on a Distributed Aurora Imaging Library remote system. If it is not, use M_PHYSICAL_ADDRESS instead.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that the buffer is not visible from the physical address space.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the address of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PITCH		Inquires the number of pixels between the beginnings of any two adjacent lines of the buffer data. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the pitch, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PITCH_BYTE		Inquires the number of bytes between the beginnings of any two adjacent lines of the buffer data. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the pitch, in bytes.	c		e	f	g	a		i		j	k	l	m	n	b

M_SIZE_BAND	Inquires the number of buffer bands. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= 1024	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BIT	Inquires the depth per band. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the depth per band, in bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X	Inquires the width of the buffer. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y	Inquires the height of the buffer. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SURFACE_HANDLE	[This is only applicable to Linux] Inquires the Cairo surface handle of the buffer. The buffer's Cairo surface must have been successfully allocated using MbufControl() with M_SURFACE_ALLOC.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that no Cairo surface is associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the handle of the Cairo surface that is associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_SYSTEM_LOCATION	Inquires whether the buffer is allocated on a local system or a Distributed Aurora Imaging Library remote system.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOCAL	Specifies that the buffer is allocated on a local system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REMOTE	Specifies that the buffer is allocated on a remote system.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE		Inquires the buffer data type and depth. Depth is returned in bits. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FLOAT + *Depth value*	Specifies the data depth and that the data type is floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SIGNED + *Depth value*	Specifies the data depth and that the data type is signed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNSIGNED + *Depth value*	Specifies the data depth and that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b
M_YCBCR_RANGE		Inquires whether pixel values are limited to a signal's YCbCr range. SET	c		e	f	g	a		i		j	k	l	m	n	b
		System specific															
		When grabbing into the specified buffer from a source that transmits in a YCbCr format (for example, an SDI source), the data is not converted to match the encoding that you specify. Instead, this control type is automatically changed to match the encoding of the grabbed data.	c													
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to encode the YUV buffer's pixel values in YCbCr.	c		e	f	g	a		i		j	k	l	m	n	b
	M_YCBCR_HD	Specifies to encode the YUV buffer's pixel values using the high-definition YCbCr standard.	c		e	f	g	a		i		j	k	l	m	n	b
	M_YCBCR_SD	Specifies to encode the YUV buffer's pixel values using the standard-definition YCbCr standard.	c		e	f	g	a		i		j	k	l	m	n	b

	M_YCBCR_UHD	Specifies to encode the YUV buffer's pixel values using the ultra-high-definition YCbCr standard.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT;], [when InquireType=M_EXTENDED_FORMAT: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT].

One of the following values might be combined with the above-mentioned values to determine the location of the buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the location of the buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HOST_MEMORY +	Specifies that the buffer is in Host memory.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF_BOARD	Specifies that the buffer is not in on-board memory. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ON_BOARD +	Specifies that the buffer is in on-board memory. INFO	c							i		j	k	l	m		

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT;], [when InquireType=M_EXTENDED_FORMAT: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT].

One of the following values might be combined with the above-mentioned values to determine whether the buffer was allocated in paged or non-paged memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns whether the buffer was allocated in paged or non-paged memory																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NON_PAGED	Specifies that the buffer is in Aurora Imaging Library reserved, non-pageable memory, if possible. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PAGED	Specifies that the buffer is in pageable memory.	c		e	f	g	a		i		j	k	l	m	n	b

For M_KERNEL and M_STRUCT_ELEMENT data buffers only (see MbufControl() for possible values), you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_KERNEL and M_STRUCT_ELEMENT data buffers																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_OFFSET_CENTER_X	Inquires the X-coordinate of the center of the kernel or structuring element. SET														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < *SizeX*	Specifies the value of the X-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_OFFSET_CENTER_Y	Inquires the Y-coordinate of the center of the kernel or structuring element. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < *SizeY*	Specifies the value of the Y-coordinate.	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN	Inquires the overscan type. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that Aurora Imaging Library automatically selects the type of overscan to optimize speed and logic according to the specified operation and the target system.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no overscan will be used, unless processing the border pixels is faster than ignoring them; in the latter case, Aurora Imaging Library automatically selects the overscan to optimize speed according to the specified operation and the target system.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST	Specifies that Aurora Imaging Library automatically selects the overscan to optimize speed according to the specified neighborhood operation and the target system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIRROR	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values that mirror the source buffer pixel values. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLACE	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values set to the overscan replacement value (M_OVERSCAN_REPLACE_VALUE).	c		e	f	g	a		i		j	k	l	m	n	b

M_REPLICATE	Specifies a type of overscan that processes the border pixels of a source image using overscan pixel values that replicate the border pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TRANSPARENT	Specifies a type of overscan that processes the border pixels of a source image using transparent overscan pixel values. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OVERSCAN_REPLACE_VALUE	Inquires the replacement value for the overscan pixel values. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLACE_MAX	Specifies that the overscan neighborhood pixel values will be set to the maximum value of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_REPLACE_MIN	Specifies that the overscan neighborhood pixel values will be set to the minimum value of the source buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the value of the overscan neighborhood pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Inquires whether results are saturated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to saturate results, except when Aurora Imaging Library can take advantage of optimization routines to accelerate the processing. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to saturate results. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For M_KERNEL data buffers only (see MbufControl() for possible values), you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_KERNEL data buffers only																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ABSOLUTE_VALUE	Inquires whether the absolute value should be taken. SET															

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to take the absolute value of the result.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to take the absolute value of the result. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZATION_FACTOR	Inquires the normalization factor. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the normalization factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For M_STRUCT_ELEMENT data buffers only, you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_STRUCT_ELEMENT data buffers only																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ITER_OVERSCAN	Inquires the type of overscan used to handle overscan operations with iterations for structuring elements. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_GLOBAL	Specifies that for overscan operations with multiple iterations, to use a size equivalent to all iterations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PER_ITER	Specifies that for overscan operations with multiple iterations, each iteration will be according to the structuring element's size and center pixel.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_ELEMENT_VALUES_VALID	Inquires the number of valid structuring element values in the structuring element. This is the number of values that were not set to **M_DONT_CARE**.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the number of valid structuring element values.	c		e	f	g	a		i		j	k	l	m	n	b

For M_IMAGE data buffers only, you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE data buffers only																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_ASSOCIATED_LUT		Inquires the identifier of the LUT buffer associated with the image buffer, if there is one. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies that no LUT buffer is associated with the image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	LUT buffer identifier	Specifies the Aurora Imaging Library identifier of the LUT buffer that is associated with the image buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISPLAY_SIZE_BAND		Inquires the number of bands used for display. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	0	Specifies that no bands are used for display; the buffer is not a 1-band or 3-band buffer and the display bands were not set (MbufControl() with M_DISPLAY_BANDS).	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies that 1 band is used for display.	c		e	f	g	a		i		j	k	l	m	n	b
	3	Specifies that 3 bands are used for display.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFORMATION_TYPE		Inquires the GenDC part type of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_1D	Specifies the GenDC part type is 1D.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_2D	Specifies the GenDC part type is 2D.	c		e	f	g	a		i		j	k	l	m	n	b

	M_INFO_2D_H264	Specifies the GenDC part type is H.264 data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_2D_JPEG	Specifies the GenDC part type is JPEG data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_2D_JPEG2000	Specifies the GenDC part type is JPEG 2000 data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_GENICAM_CHUNK	Specifies the GenDC part type is GenICam chunk data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFO_GENICAM_XML	Specifies the GenDC part type is GenICam XML data.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFORMATION_TYPE_NAME +		Inquires the GenDC part type of the buffer as a string.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_NAME +		Inquires the name of the buffer's pixel format, as defined by the GenICam Pixel Format Naming Convention.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_SIZE_BIT		Inquires the number of bits per pixel, as per the GenICam Pixel Format Naming Convention. Note that this is the total number of bits used to store each pixel across all bands. For example, if the buffer has 3 8-bit bands that store RGB information, this value will be 24.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the number of bit per pixel as per the PFNC.	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_SUPPORT		Inquires whether Aurora Imaging Library supports processing the buffer's given it's pixel format. Note that this setting will only return M_NO if M_PFNC_VALUE was set to an unsupported value during a grab.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_NO	Specifies that image processing is not supported for the buffer's pixel format. An error will be generated when you attempt to use an Aurora Imaging Library processing function with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WITH_COMPENSATION	Specifies that Aurora Imaging Library will process the buffer by copying its data to a temporary buffer with a supported pixel format.	c		e	f	g	a		i		j	k	l	m	n	b
	M_YES	Specifies that the buffer will be processed as normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_VALUE		Inquires the pixel format of the buffer, as defined by the GenICam Pixel Format Naming Convention. This value is either set by the digitizer when grabbing into the buffer, or is determined based on the M_COMPONENT_TYPE of the buffer and the attributes specified when the buffer was allocated. If this value is determined from the attributes of the buffer, and there is no ratified format in the PFNC corresponding to those attributes, the value will be 0.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	PFNCvalue	Specifies the pixel format of the buffer using a value from PFNC (as defined in *pfnc.h*).	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_ACTIVE		Inquires whether the image buffer has an active ROI. The ROI associated with a buffer is active when the buffer has M_REGION_USE set to M_USE.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the image buffer does not have an active ROI.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the image buffer has an active ROI.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGION_LINK	Inquires whether the image buffer is a child buffer with an ROI linked to that of its parent. A child buffer's ROI is linked by default, or using MbufSetRegion() with M_LINK_TO_PARENT. Note, it is possible for the ROI of the child buffer to be linked even if there is no ROI currently defined for the parent buffer. In this case, setting the ROI for the parent buffer will automatically set the ROI for the child buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_LINK_TO_PARENT	Specifies that the image buffer is a child buffer with an ROI linked to that of its parent.	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies that the image buffer is not a child buffer, or that its ROI is not linked to that of its parent.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_TYPE	Inquires whether the image buffer is associated with ROI information, and, if so, in which format the ROI is saved. ROIs are set with MbufSetRegion().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NONE	Specifies that no ROI is defined.	c		e	f	g	a		i		j	k	l	m	n	b
M_RASTER	Specifies that the ROI is in raster format.	c		e	f	g	a		i		j	k	l	m	n	b
M_VECTOR	Specifies that the ROI is in vector format.	c		e	f	g	a		i		j	k	l	m	n	b
M_VECTOR_AND_RASTER	Specifies that the ROI is in raster and vector format.	c		e	f	g	a		i		j	k	l	m	n	b
M_REGION_USE	Inquires whether the ROI associated with the buffer is used with supported image processing operations.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_IGNORE	Specifies that image processing functions will ignore the ROI associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_USE	Specifies that image processing functions will use the ROI associated with the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESOLUTION_X	Inquires the X resolution of the image buffer in pixels per inch (PPI). If M_RESOLUTION_X in MbufControl() is not set, the inquire returns 0. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the X resolution in PPI.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESOLUTION_Y	Inquires the Y resolution of the image buffer in pixels per inch (PPI). If M_RESOLUTION_Y in MbufControl() is not set, the inquire returns 0. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the Y resolution in PPI.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_IMAGE;], [when InquireType=M_EXTENDED_FORMAT: M_IMAGE].

One or more of the following values are combined with the above-mentioned values to determine the intended purpose of the buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the intended purpose of the image buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESS +	Specifies an image buffer that can hold compressed data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISP	Specifies an image buffer that can be displayed.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies an image buffer in which to grab data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC +	Specifies an image buffer that can be processed. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_COMPRESS.

One of the following values might be combined with the above-mentioned value to determine the compression type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the compression type

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_JPEG2000_LOSSLESS	Specifies that the buffer will be used to hold JPEG2000 lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	Specifies that the buffer will be used to hold JPEG2000 lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	Specifies that the buffer will be used to hold JPEG lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	Specifies that the buffer will be used to hold JPEG lossless data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	Specifies that the buffer will be used to hold JPEG lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	Specifies that the buffer will be used to hold JPEG lossy data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PROC.

The following value might be combined with the above-mentioned value to determine whether the buffer was allocated with an overscan region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns whether the buffer was allocated with an overscan region																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOCATION_OVERSCAN	Specifies that the buffer is allocated with an overscan region. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_IMAGE;], [when InquireType=M_EXTENDED_FORMAT: M_IMAGE].

One of the following values might be combined with the above-mentioned values to determine the storage format and location specifier.

You might have set this value, or it could have been automatically selected by Aurora Imaging Library.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the storage format and location specifier																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIB	Specifies that the buffer is a DIB buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GDI	Specifies that the buffer is compatible with GDI. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_LINUX_MXIMAGE	[This is only applicable to Linux] Specifies that the buffer is an X11 Ximage.	c		e	f	g	a				j	k	l	m	n	b

Combination value for [when InquireType=M_EXTENDED_ATTRIBUTE: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT;], [when InquireType=M_EXTENDED_FORMAT: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT].

The following value might be combined with the above-mentioned values to determine whether the buffer is FPGA accessible.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns whether the buffer is FPGA accessible																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FPGA_ACCESSIBLE +	Specifies that the buffer is allocated in a bank of memory that is accessible from the Processing FPGA. INFO										j	k	l	m		

Combination value for M_HOST_MEMORY; M_ON_BOARD; M_FPGA_ACCESSIBLE.

The following value might be combined with the above-mentioned values to determine the memory bank in which the buffer was allocated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the memory bank used																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEMORY_BANK_*n*	Inquires the buffer allocated in the specified memory bank.	c		e	f	g	a		i		j	k	l	m	n	b
	Operating system specific															
	[This is only applicable to Windows] Valid values for *n* are between 0 and 6, inclusive. Note that, to ensure that you are accessing a NUMA-enabled memory bank, you must specify the M_HOST_MEMORY combination value.															
	[This is only applicable to Linux] Note that this value is not available with the M_HOST_MEMORY combination value.															
	System specific															
	Valid values for *n* are between 0 and 1, inclusive. Note that, to ensure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.										j	k				

		In this case, *n* must be set to 0. Note that, to ensure you are accessing an on-board memory bank, you must specify the M_ON_BOARD combination value.												l	m		

Combination value for M_ON_BOARD; M_FPGA_ACCESSIBLE.

The following value might be combined with the above-mentioned values to determine if the buffer is in a specific type of memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location in a specific type of memory																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SHARED	Specifies that the buffer is in shared processing memory. INFO										j	k	l	m		

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_IMAGE;], [when InquireType=M_EXTENDED_FORMAT: M_IMAGE].

One of the following values might be combined with the above-mentioned values to determine the format in which image buffers were stored.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the format in which image buffers were stored																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c			f		a		i		j	k	l	m		

Combination values for [when InquireType=M_DATA_FORMAT: M_PACKED; M_PLANAR;], and for the following values: M_PACKED; M_PLANAR.

One of the following values might be combined with the above-mentioned values to determine the packed or planar color buffer format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the packed or planar color buffer format																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB24	Specifies 24-bit color depth (RGB 8:8:8) packed or planar pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB48	Specifies 48-bit color depth (RGB 16:16:16). INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_RGB96	Specifies 96-bit color depth (RGB 32:32:32) packed or planar pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16	Specifies YUV16 (4:2:2) pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_DATA_FORMAT: M_PACKED;], and for the following values: M_PACKED.

One of the following values might be combined with the above-mentioned values to determine the packed color buffer format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the packed color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth packed pixels (BGRBGR). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BGR32	Specifies 32-bit color depth packed pixels (BGRXBGRX). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB15	Specifies 16-bit color depth packed pixels (XRGB 1:5:5:5). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB16	Specifies 16-bit color depth packed pixels (RGB 5:6:5). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. INFO			e	f	g									n	b
M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_DATA_FORMAT: M_PLANAR;], and for the following values: M_PLANAR.

One of the following values might be combined with the above-mentioned values to determine the planar color buffer format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB3	Specifies 3-bit color depth (RGB 1:1:1) planar pixels.	c					a		i		j	k	l	m		
M_YUV9	Specifies YUV9 planar pixels.	c			f		a		i		j	k	l	m		

M_YUV12	Specifies YUV12 planar pixels.	c			f		a		i		j	k	l	m		
M_YUV24	Specifies YUV24 planar pixels.	c			f		a		i		j	k	l	m		

For M_IMAGE + M_COMPRESS image buffers (see MbufAlloc...() for possible values), you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS image buffers																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_COMPRESSED_DATA_SIZE_BYTE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the size of a compressed buffer, in bytes. Use M_TARGET_SIZE to establish the requested size, in bytes.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the size of a compressed buffer, in bytes.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPRESSION_TYPE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the type of compression. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSLESS	Specifies that the buffer holds JPEG2000 lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	Specifies that the buffer holds JPEG2000 lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS	Specifies that the buffer holds JPEG lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	Specifies that the buffer holds JPEG lossless data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	Specifies that the buffer holds JPEG lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_JPEG_LOSSY_INTERLACED	Specifies that the buffer holds JPEG lossy data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For M_IMAGE + M_COMPRESS image buffers with an M_JPEG_LOSSY, M_JPEG_LOSSY_INTERLACED, or M_JPEG2000_LOSSY compression type, you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSY, M_JPEG_LOSSY_INTERLACED, or M_JPEG2000_LOSSY)																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_Q_FACTOR +		[For essential Aurora Imaging Library Lite information, see remarks] Inquires the quantization factor for both JPEG2000 lossy and JPEG lossy buffers. Note that for 3-band buffers, only the quantization factor associated with the first band is returned. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 99	Specifies the quantization factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Q_FACTOR_CHROMINANCE		[For essential Aurora Imaging Library Lite information, see remarks] Inquires the quantization factor of the U and V bands for both JPEG2000 lossy and JPEG lossy buffers in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 99	Specifies the factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_Q_FACTOR_LUMINANCE		[For essential Aurora Imaging Library Lite information, see remarks] Inquires the quantization factor of the Y band for both JPEG2000 lossy and JPEG lossy buffers in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 99	Specifies the factor. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_QUANTIZATION +	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the quantization table (for a JPEG lossy or JPEG2000 lossy buffer), which is associated with the image buffer. Note that for 3-band buffers, only the identifier of the array buffer associated with the first band is returned. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_QUANTIZATION_CHROMINANCE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the quantization table associated with the U and V bands, for both JPEG2000 lossy and JPEG lossy buffers in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_QUANTIZATION_LUMINANCE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the quantization table associated with the Y band, for both JPEG2000 lossy and JPEG lossy buffers in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b

For M_IMAGE + M_COMPRESS image buffers with an M_JPEG2000_LOSSY or M_JPEG2000_LOSSLESS compression type, you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS (with M_JPEG2000_LOSSY or M_JPEG2000_LOSSLESS)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_DECOMPOSITION_LEVEL +	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the number of iterations the discrete wavelet transform is applied to the image (for single-band images) or on the first band of the image (for 3-band images). SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of iterations (decomposition levels). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_SUBBAND +	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the number of sub-bands rendered from the discrete wavelet transform passed on an image (for 1-band images) or on the first band of a 3-band image.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of sub-bands.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For M_IMAGE + M_COMPRESS (with M_JPEG2000_LOSSY or M_JPEG2000_LOSSLESS); and for the following values: M_Q_FACTOR; M_QUANTIZATION.

You can add one of the following values to the above-mentioned values to set the band about which to inquire.

> Note that when dealing with M_Q_FACTOR and M_QUANTIZATION, the following combination constants are available only for use with JPEG2000 lossy buffers.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_DECOMPOSITION_LEVEL, M_NUMBER_SUBBAND, M_Q_FACTOR (for JPEG2000 lossy buffers), and M_QUANTIZATION (for JPEG2000 lossy buffers)																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BLUE	Inquires about the blue band only (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_GREEN	Inquires about the green band only (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_RED	Inquires about the red band only (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_U	Inquires about the U band only (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_V	Inquires about the V band only (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_Y	Inquires about the Y band only (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b

For M_IMAGE + M_COMPRESS image buffers with an M_JPEG2000_LOSSY compression type, you can set the

InquireType parameter to the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS (with M_JPEG2000_LOSSY)																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_TARGET_SIZE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the requested size of the compressed buffer in bytes. Use M_COMPRESSED_DATA_SIZE_BYTE to establish the real size of the buffer. SET																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies compression will be applied without a target image size.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the size, in bytes. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For M_IMAGE + M_COMPRESS image buffers with an M_JPEG_LOSSY, M_JPEG_LOSSY_INTERLACED, M_JPEG_LOSSLESS, or M_JPEG_LOSSLESS_INTERLACED compression type, you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSY, M_JPEG_LOSSY_INTERLACED, M_JPEG_LOSSLESS, or M_JPEG_LOSSLESS_INTERLACED)																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_HUFFMAN_DC	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the DC Huffman table which is associated with the image buffer. For 3-band buffers, only the identifier of the array buffer associated with the first band is returned. SET																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_HUFFMAN_DC_CHROMINANCE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the DC Huffman table that is associated with the U and V bands of a JPEG lossy buffer in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_DC_LUMINANCE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the DC Huffman table that is associated with the Y band of a JPEG lossy buffer in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b

For M_IMAGE + M_COMPRESS image buffers with an M_JPEG_LOSSLESS or M_JPEG_LOSSLESS_INTERLACED compression type, you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSLESS, or M_JPEG_LOSSLESS_INTERLACED)																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_PREDICTOR	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the type of predictor. This inquire type is supported for JPEG lossless buffers only. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies predictor #0 (no prediction).	c		e	f	g	a		i		j	k	l	m	n	b
1	Specifies predictor #1 (the "pixel-to-the-left" predictor).	c		e	f	g	a		i		j	k	l	m	n	b
2	Specifies predictor #2 (the "pixel-above" predictor).	c		e	f	g	a		i		j	k	l	m	n	b

M_RESTART_INTERVAL	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the number of lines between restart markers (for JPEG lossless buffers) or number of 8x8 blocks of data between restart markers (for JPEG lossy buffers). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies after how many rows (for JPEG lossless buffers) or 8x8 blocks (for JPEG lossy buffers) of data to place restart markers. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For M_IMAGE + M_COMPRESS image buffers with an M_JPEG_LOSSY or M_JPEG_LOSSY_INTERLACED compression type, you can set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For M_IMAGE + M_COMPRESS (with M_JPEG_LOSSY, or M_JPEG_LOSSY_INTERLACED)																
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HUFFMAN_AC	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the AC Huffman table that is associated with the image buffer. For 3-band buffers, only the identifier of the array buffer associated with the first band is returned. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_HUFFMAN_AC_CHROMINANCE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the AC Huffman table that is associated with the U and V bands of a JPEG buffer in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b

M_HUFFMAN_AC_LUMINANCE	[For essential Aurora Imaging Library Lite information, see remarks] Inquires the identifier of the array buffer containing the AC Huffman table that is associated with the Y band of a JPEG buffer in YUV format. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
Array buffer identifier	Specifies the identifier of the array buffer containing the table.	c		e	f	g	a		i		j	k	l	m	n	b

For all buffers, you can set the InquireType parameter to one of the following values. These settings are primarily useful when working with a buffer that is a component of a container, or a buffer that will be copied to a component of a container using MbufCopyComponent().

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring settings useful with buffers that are components																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_COMPONENT_GROUP_ID	Inquires the component group ID assigned to the buffer during acquisition. Refer to your camera manual to learn what component group IDs are assigned to transmitted components.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the component group ID of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INVALID	Inquires whether the information in the buffer was marked as invalid during acquisition.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the information in the buffer has not been marked invalid by the camera. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

M_TRUE	Specifies that the information in the buffer has been marked invalid by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_ID	Inquires the component region ID assigned to the buffer during acquisition. This identifies which region of the image sensor was used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the component region ID of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_OFFSET_X	Inquires the X-offset of the top left corner of the region of the image sensor used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the X-offset of the component region of the buffer, expressed in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_OFFSET_Y	Inquires the Y-offset of the top left corner of the region of the image sensor used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the Y-axis offset of the component region ID of the buffer, expressed in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SOURCE_ID	Inquires the component source ID assigned to the buffer during acquisition. This indicates which of the camera's data sources was used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the component source ID of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_TYPE		Inquires the component type of the buffer, used when the buffer is a component of a container. The component type specifies what kind of information is stored in the component. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 255. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_METADATA	Specifies that the component stores metadata information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_NORMALS_AIL	Specifies that the buffer stores normals information for each point in the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_UNDEFINED	Specifies that the component contains information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_TYPE_NAME +		Inquires the name of the component type of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_EXTENDED_ATTRIBUTE_NAME; M_EXTENDED_FORMAT_NAME; M_INFORMATION_TYPE_NAME; M_PFNC_NAME; M_COMPONENT_TYPE_NAME.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_STRING_SIZE		Retrieves the length of the string, including the terminating null character ("\0").															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

For image buffers with an M_IMAGE attribute and with M_COMPONENT_TYPE set to M_COMPONENT_RANGE or M_COMPONENT_DISPARITY, InquireType can also be set to one of the values below. These settings are primarily useful when working with a buffer that is a component of a container, or a buffer that will be copied as a component of a container using MbufCopyComponent().

These settings are used only when the buffer is the range or disparity component of a container passed as a source to MbufConvert3d(). Additionally, a container cannot be 3D-processable or 3D-displayable unless it has a single range component with all of these settings at their default values (except for M_3D_DISTANCE_UNIT which can be any value and M_3D_REPRESENTATION, which must be set to M_CALIBRATED_XYZ or M_CALIBRATED_XYZ_UNORGANIZED).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring settings useful with M_IMAGE buffers that store 3D data and are components																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_3D_ASPECT_RATIO		Inquires by how much the focal length in the Y-direction stored in the buffer will be scaled in the destination point cloud to correct for non-square pixels in 3D reconstruction modes that rely on projective geometry. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0.0	Specifies by how much the focal length the Y-direction will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_COORDINATE_SYSTEM_TYPE		Inquires which type of coordinate system to use to interpret the coordinates stored in the bands of the buffer if it is a range component. Note that Aurora Imaging Library does not support any functionality with coordinates stored using a coordinate system type other than M_CARTESIAN. If a buffer stores information defined using another type of coordinate system, you will need to manually convert that information to cartesian coordinates before the buffer can be used with any Aurora Imaging Library processing function. This conversion cannot be done using MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CARTESIAN	Specifies that the buffer stores right-handed cartesian coordinates. INFO This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CYLINDRICAL	Specifies that the buffer stores cylindrical coordinates (theta-Y-rho). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_SPHERICAL	Specifies that the buffer stores spherical coordinates (theta-phi-rho). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that the coordinate system type is unknown.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_DISPARITY_BASELINE	Inquires the stereo baseline value of the stereoscopic camera used to generate the data in the buffer. This is the physical distance between the lenses of the camera. Refer to your camera manual to determine the correct value for this setting, which might differ from the true physical distance between the lenses of your camera. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the stereo baseline value, expressed in meters. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISTANCE_UNIT	Inquires the unit to use when the buffer is part of a container and stores natively calibrated distance data. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INCH	Specifies that the distance data is provided in inches.	c		e	f	g	a		i		j	k	l	m	n	b
M_MILLIMETER	Specifies that the distance data is provided in millimeters.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that the distance data is provided in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the distance unit is unknown. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_FOCAL_LENGTH	Inquires the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer. Refer to your camera manual to determine the correct value for this setting, which might differ from the true focal length of your camera. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value > 0.0	Specifies the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer, expressed in pixels. INFO The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_INVALID_DATA_FLAG		Inquires whether the buffer uses a specific value to indicate invalid data. For 3-band buffers that store coordinates, the invalid data flag should be stored in the Z-axis band. Specify the value that indicates invalid data using M_3D_INVALID_DATA_VALUE. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the buffer does not use a special value to indicate invalid data. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the buffer uses a special value to indicate invalid data.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_INVALID_DATA_VALUE		Inquires the value used to indicate missing data when M_3D_INVALID_DATA_FLAG is set to M_TRUE. Note that this value is not used or respected by any Aurora Imaging Library functions except for MbufConvert3d() when the buffer is a component of the source container. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the value used to indicate missing data. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_OFFSET_X	Inquires by how much the X-coordinates stored in the buffer will be offset in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the X-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Y	Inquires by how much the Y-coordinates stored in the buffer will be offset in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Y-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Z	Inquires by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Z-coordinates stored in the buffer will be offset. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_PRINCIPAL_POINT_X	Inquires the X-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies X-position of the principal point, expressed in pixels. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_PRINCIPAL_POINT_Y		Inquires the Y-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies Y-position of the principal point, expressed in pixels. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_REPRESENTATION		Inquires how 3D data is stored in the buffer; this information is used when the buffer is a range or disparity component of a container. For more information, see MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_XYZ	Specifies that the component stores organized and natively calibrated X, Y, and Z-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_XYZ_UNORGANIZED	Specifies that the component stores unorganized and natively calibrated X, Y, and Z-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_XZ_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates stored in a separate array buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_CALIBRATED_XZ_UNIFORM_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates identified by the row index. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z	Specifies that the component stores organized and natively calibrated Z-coordinates, without X and Y-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates without X-coordinates; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_X_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates, with the X-coordinates identified by column index; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_XY	Specifies that the component stores organized and natively calibrated Z-coordinates, with X and Y-coordinates identified by column and row index respectively. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY	Specifies that the component stores a disparity map with perspective distortion along the Y-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_EXTERNAL_Y	Specifies that the component stores a disparity map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_UNIFORM_Y	Specifies that the component stores a disparity map, with Y-values identified by row index. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_PROJECTED_Z	Specifies that the component stores a projective map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROJECTED_Z_EXTERNAL_Y	Specifies that the component stores a projective map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNCALIBRATED_Z	Specifies that the component stores organized and uncalibrated Z-coordinates, without X and Y-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_X	Inquires by how much the X-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies by how much the X-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Y	Inquires by how much the Y-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value != 0.0	Specifies how much the Y-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Z	Inquires by how much the Z-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value != 0.0	Specifies how much the Z-coordinates stored in the buffer will be scaled. The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SHEAR_X		Inquires by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SHEAR_Z		Inquires by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row. The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Remarks

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.
- *[Aurora Imaging Library Lite]*
  While an image buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute are available under Aurora Imaging Library Lite, these attributes are not required for the image buffer to be available to other Aurora Imaging Library Lite functions.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufInquireContainer

Synopsis

Inquire about an Aurora Imaging Library container or component setting.

Syntax

```
AIL_INT MbufInquireContainer(
    AIL_ID ContainerBufId, //in
    AIL_INT64 Component,   //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function inquires about a specified setting of an Aurora Imaging Library container or component.

Parameters

ContainerBufId

Specifies the identifier of the source container. The container must have been previously allocated on the required system using MbufAllocContainer().

Component

Specifies the component to inquire.

This setting can be used to specify to inquire the container itself.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying to inquire the container itself																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CONTAINER	Specifies to inquire a setting of the container.	c		e	f	g	a		i		j	k	l	m	n	b

These settings can be used to inquire the component which meets the specified criterion, or the container itself. In most cases, an error will be generated if there is no component matching this criterion in the container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the component to inquire by a unique identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_BY_ID(AIL_INT ID)	Specifies to control the component, in the container, which has the specified buffer identifier. An error will be generated if the container does not have a component with this buffer identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															

	ID Specifies the Aurora Imaging Library identifier of the component to control.															
M_COMPONENT_BY_INDEX(AIL_INT ComponentIndex)	Specifies to control the component, in the container, at the specified index. An error will be generated if the container does not have a component at this index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	ComponentIndex Specifies the index of the component to control.															

These settings can be used to inquire the component with the specified component type. In most cases, an error will be generated if there is no component, or more than one component, matching this criterion in the container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the component to inquire by component type																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. Coordinates associated with the confidence value 0 are considered invalid data and will not be used by 3D image processing functions. A confidence component is associated with a range or disparity component in the same container when there are no other range, disparity, or confidence components in the container. If there is more than one range, disparity, and/or confidence component in a container (for example, because a single grab into the container transmitted multiple range and confidence components), you will need to either create a child container which contains only the required components using MbufChildContainer(), or free the extra components using MbufFreeComponent().	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the column index of each corresponding pixel and will result in a better representation of the camera's lens model when generating X-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. When using MbufConvert3d() with a projective map, this component can be used instead of the row index of each corresponding pixel and will result in a better representation of the camera's lens model when generating Y-coordinates. A coordinate map component is associated with a range component in the same container when there is only one range component and only one type of each coordinate map component in that container. > Note that this component is only available if it is provided by the camera. For more information refer to your cameras manual.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 254. You can use custom component types to identify components which store information that does not fit one of the standard component types. In addition, some cameras transmit components with custom component types. Refer to your camera manual to determine what type of information is stored in transmitted components.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. Each pixel of a disparity map indicates the apparent distance (typically measured in pixel units) between where an object appears in the left and right images captured by a stereoscopic camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. Only depth map containers can have a matrix component. A matrix component is associated with a range component in the same container when there is only one range component and no other matrix components in that container.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. A mesh component is associated with a range component in the same container when there are no other range or mesh components in that container. A point cloud container which has a mesh component is referred to as a meshed point cloud container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_METADATA	Specifies that the component stores metadata information. Metadata components are used by Aurora Imaging Library internally. Typically, you should ignore metadata components in your application.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. Unlike an intensity component, a multispectral component might include information about non-visible light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_NORMALS_AIL	Specifies that the component stores normals information for each point in the M_COMPONENT_RANGE component of the container. A normals component is associated with a range component in the same container when there are no other range or normals components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. This can be either a 1-band component that stores a depth map, or a 3-band buffer that stores coordinates of 3D points.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. Each pixel of a reflectance map indicates how much of the light hitting an object at that location is reflected back. Typically, this is an intensity image of the light spectrum used by a 3D sensor to detect 3D distance/position information. Typically, if the map was generated by a laser profiler, each row indicates the detected intensity of the laser for a single scan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. 3D image processing functions will only operate on points inside the ROI. Only depth map containers can have a region component. A region component is associated with a range component in the same container when there is only one range component and no other region components in that container.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. Each pixel of a scatter map indicates how much of the light hitting an object at that location is detected scattering beneath the object's surface.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_UNDEFINED	Specifies that the component stores information of an unknown type.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the setting to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT64
- AIL_UINT64

Specifies the address in which to return the value of the inquired setting. Since the **MbufInquireContainer**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a container
- For inquiring about which components are in a container
- For inquiring about settings that apply to both containers and components
- For inquiring about settings that apply only to components
- For inquiring about 3D settings that apply only to range and disparity components that are image buffers

To inquire the container itself, specify M_CONTAINER and one of the following constants.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a container																	
InquireType UserVarPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_3D_CONVERTIBLE	Inquires whether the container can be converted to a 3D-processable and/or natively 3D displayable container using MbufConvert3d(), and whether compensation is required to convert the data. Note that irrespective of the returned value for this inquire type, components with invalid values will result in undefined behavior. This includes non-finite values for valid points (infinity, negative infinity, or NaN), invalid normals, and invalid vertices or edges in mesh triangles. Depending on the type of invalid values, you can use MbufConvert3d() with M_REMOVE_NON_FINITE to remove non-finite values in the range and disparity components. Components with invalid values must be fixed before passing the container to a 3D processing function.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	M_CONVERTIBLE	Specifies that the container stores 3D data that can be converted using MbufConvert3d().	c		e	f	g	a		i		j	k	l	m	n	b
	M_CONVERTIBLE_WITH_COMPENSATION	Specifies that the container stores 3D data that can be converted using MbufConvert3d() with M_COMPENSATE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_CONVERTIBLE	Specifies that the container can be converted us.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPLAYABLE		Inquires whether the container can be used with 3D displays. Any container that is 3D-convertible is also 3D-displayable. A conversion will occur each time the container is modified, unless you convert the container to a format that is natively 3D-displayable. To be natively 3D-displayable, a container must have the attribute M_DISP and must have the following components: • A single range component, which is a 3-band, float data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_RANGE. The component's M_3D_COORDINATE_SYSTEM_TYPE must be set to M_CARTESIAN and M_3D_REPRESENTATION must be set to M_CALIBRATED_XYZ. • A single confidence component, which is a 1-band, 8-bit unsigned data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_CONFIDENCE and with the same dimensions (M_SIZE_X and M_SIZE_Y) as the range component. • Optionally, either a single intensity or single reflectance component, which are 1-band or 3-band, 8- or 16-bit unsigned data image buffers with the M_COMPONENT_TYPE set to M_COMPONENT_INTENSITY or M_COMPONENT_REFLECTANCE. • Optionally, a single mesh component, which is a 1-band, 32-bit unsigned 2D M_ARRAY buffer with the M_COMPONENT_TYPE set to M_COMPONENT_MESH_AIL. • Optionally, a single normals component, which is a 3-band, float data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_NORMALS_AIL. Note that irrespective of the returned value	c		e	f	g	a		i		j	k	l	m	n	b

		for this inquire type, components with invalid values will result in undefined behavior. This includes non-finite values for valid points (infinity, negative infinity, or NaN), invalid normals, and invalid vertices or edges in mesh triangles. Depending on the type of invalid values, you can use MbufConvert3d() with M_REMOVE_NON_FINITE to remove non-finite values in the range and disparity components. Components with invalid values must be fixed before passing the container to a 3D processing function.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPLAYABLE	Specifies that the container can be used with 3D displays.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPLAYABLE_WITH_CONVERSION	Specifies that the container can be used with 3D displays, with automatic conversion.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_DISPLAYABLE	Specifies that the container cannot be used with 3D displays.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_PROCESSABLE	Inquires whether the container can be used for 3D processing. To be 3D-processable, a container must have the attribute M_PROC and must have the following components: • A single range component, which is a 3-band, float data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_RANGE. The component's M_3D_COORDINATE_SYSTEM_TYPE must be set to M_CARTESIAN and M_3D_REPRESENTATION must be set to M_CALIBRATED_XYZ or M_CALIBRATED_XYZ_UNORGANIZED. • A single confidence component, which is a 1-band, 8-bit unsigned data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_CONFIDENCE and with the same dimensions (M_SIZE_X and M_SIZE_Y) as the range component. • Optionally, either a single intensity or single reflectance component, which are 1-band or 3-band, 8- or 16-bit unsigned data image buffers with the M_COMPONENT_TYPE set to M_COMPONENT_INTENSITY or M_COMPONENT_REFLECTANCE. • Optionally, a single mesh component, which is a 1-band, 32-bit unsigned 2D M_ARRAY buffer with the M_COMPONENT_TYPE set to M_COMPONENT_MESH_AIL. • Optionally, a single normals component, which is a 3-band, float data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_NORMALS_AIL. Note that irrespective of the returned value for this inquire type, components with invalid values will result in undefined behavior. This includes non-finite values for valid points (infinity, negative infinity, or NaN), invalid normals, and invalid vertices or edges in mesh triangles. Depending on the type of invalid values, you can use MbufConvert3d() with M_REMOVE_NON_FINITE to remove non-finite values in the range and disparity components. Components with invalid values must be fixed before passing the container to a 3D processing function.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

M_NOT_PROCESSABLE	Specifies that the container cannot be used for 3D processing.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROCESSABLE	Specifies that the container can be used for 3D processing.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_PROCESSABLE_DEPTH_MAP	Inquires whether the container is a 3D-processable depth map container. Note that if a function has an optional functionality that is available for point cloud containers but not depth map image buffers, it will not be available for depth map containers. To store data in a 3D-processable depth map format, a container must have the attribute M_PROC and must have the following components: • A single range component, which is a 1-band, 8-bit, 16-bit, or 32-bit unsigned data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_RANGE. The component's M_3D_COORDINATE_SYSTEM_TYPE must be set to M_CARTESIAN, M_3D_REPRESENTATION must be set to M_CALIBRATED_Z_UNIFORM_XY, M_3D_SCALE_X and M_3D_SCALE_Y must be positive, and M_3D_SHEAR_X and M_3D_SHEAR_Z must be zero. If M_3D_INVALID_DATA_FLAG is set to M_TRUE, M_3D_INVALID_DATA_VALUE must be the maximum value of the buffer. • If M_3D_INVALID_DATA_FLAG is set to M_FALSE, a single confidence component, which is a 1-band, 8-bit unsigned data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_CONFIDENCE and with the same dimensions (M_SIZE_X and M_SIZE_Y) as the range component. • Optionally, a single region component, which is a 1-band, 8-bit unsigned data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_REGION_AIL and with the same dimensions (M_SIZE_X and M_SIZE_Y) as the range component. • Optionally, a single matrix component, which is a 1-band, 32-bit float 4x4 M_ARRAY buffer with the M_COMPONENT_TYPE set to M_COMPONENT_MATRIX_AIL.	c		e	f	g	a		i		j	k	l	m	n	b

		• Optionally, either a single intensity or single reflectance component, which are 1-band or 3-band, 8- or 16-bit unsigned data image buffers with the M_COMPONENT_TYPE set to M_COMPONENT_INTENSITY or M_COMPONENT_REFLECTANCE. • Optionally, a single mesh component, which is a 1-band, 32-bit unsigned 2D M_ARRAY buffer with the M_COMPONENT_TYPE set to M_COMPONENT_MESH_AIL. • Optionally, a single normals component, which is a 3-band, 32-bit float data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_NORMALS_AIL. Note that irrespective of the returned value for this inquire type, components with invalid values will result in undefined behavior. This includes non-finite values for valid points (infinity, negative infinity, or NaN), invalid normals, and invalid vertices or edges in mesh triangles. Depending on the type of invalid values, you can use MbufConvert3d() with M_REMOVE_NON_FINITE to remove non-finite values in the range and disparity components. Components with invalid values must be fixed before passing the container to a 3D processing function.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the container cannot store data in a 3D-processable depth map format.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the container can store data in a 3D-processable depth map format.	c		e	f	g	a		i		j	k	l	m	n	b

		c		e	f	g	a		i		j	k	l	m	n	b
M_3D_PROCESSABLE_MESHED	Inquires whether or not the container can be used for 3D processing that requires the presence of a mesh component. The mesh component must be a 1-band, 32-bit unsigned 2D M_ARRAY buffer with M_COMPONENT_TYPE set to M_COMPONENT_MESH_AIL. To be 3D-processable, a container must have the attribute M_PROC and must have the following components: • A single range component, which is a 3-band, float data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_RANGE. The component's M_3D_COORDINATE_SYSTEM_TYPE must be set to M_CARTESIAN and M_3D_REPRESENTATION must be set to M_CALIBRATED_XYZ or M_CALIBRATED_XYZ_UNORGANIZED. • A single confidence component, which is a 1-band, 8-bit unsigned data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_CONFIDENCE and with the same dimensions (M_SIZE_X and M_SIZE_Y) as the range component. • Optionally, either a single intensity or single reflectance component, which are 1-band or 3-band, 8- or 16-bit unsigned data image buffers with the M_COMPONENT_TYPE set to M_COMPONENT_INTENSITY or M_COMPONENT_REFLECTANCE. • Optionally, a single mesh component, which is a 1-band, 32-bit unsigned 2D M_ARRAY buffer with the M_COMPONENT_TYPE set to M_COMPONENT_MESH_AIL. • Optionally, a single normals component, which is a 3-band, float data image buffer with the M_COMPONENT_TYPE set to M_COMPONENT_NORMALS_AIL. Note that irrespective of the returned value for this inquire type, components with invalid values will result in undefined behavior. This includes non-finite values for valid points (infinity, negative infinity, or NaN), invalid normals, and invalid vertices or edges in mesh triangles. Depending on the type of invalid values, you can use MbufConvert3d() with M_REMOVE_NON_FINITE to remove non-finite values in the range and disparity components. Components with invalid values must be fixed before passing the container to a 3D processing function.															

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the container cannot be used for 3D processing that requires the presence of a correctly formatted mesh component.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the container can be used for 3D processing that requires the presence of a correctly formatted mesh component.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANCESTOR_ID	Inquires the Aurora Imaging Library identifier of the ancestor container. Only child containers have an ancestor container. The ancestor container is the container from which the specified container ultimately originated. It is the root container; it does not have a parent container (it is not a child container of another container). If the specified container is not a child container, the identifier of the container itself is returned as the ancestor container. To establish the parent container of the specified container, use M_PARENT_ID instead.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_GROUP_ID_LIST +	Inquires the list of unique group IDs of components in the container.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_ID_LIST +	Inquires the list of unique region IDs of components in the container.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SOURCE_ID_LIST +	Inquires the list of unique source IDs of components in the container.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_TYPE_LIST +	Inquires the list of unique component types of components in the container.	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_LAYOUT_MODIFICATION_COUNT	Inquires the current value of the modification counter of the specified container. When the container is allocated, the layout modification counter is initialized to a default value. The layout modification counter is incremented each time a component is added or removed from the container. This feature is useful for optimization. For example, you can avoid repeating certain computations (for example, analysis computations) if you know that the image buffer has not been modified. In this case, inquire the count before the first computation in the sequence of computations, and then inquire it again before repeating the same sequence. If no modifications have been made to the image buffer, you can avoid repeating the sequence unnecessarily. Note that Component must be set to M_CONTAINER.			e	f	g						l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e	f	g						l	m	n	b
Value	Specifies the current value of the modification counter.			e	f	g						l	m	n	b
M_OWNER_SYSTEM	Inquires the identifier of the system on which the container has been allocated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
M_OWNER_SYSTEM_TYPE	Inquires the type of system on which the container was allocated.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_SYSTEM_CLARITY_UHD_TYPE	Specifies an Aurora Imaging Library Clarity UHD system. INFO	c														

	M_SYSTEM_GENTL_TYPE	Specifies an Aurora Imaging Library GenTL system. INFO			e												
	M_SYSTEM_GEVIQ_TYPE	Specifies an Aurora Imaging Library GevIQ system. INFO				f											
	M_SYSTEM_GIGE_VISION_TYPE	Specifies an Aurora Imaging Library GigE Vision system. INFO					g										
	M_SYSTEM_HOST_TYPE	Specifies the Host. INFO						a									
	M_SYSTEM_IRIS_GTX_TYPE	Specifies an Aurora Imaging Library Iris GTX system. INFO								i							
	M_SYSTEM_RADIENTEVCL_TYPE	Specifies an Aurora Imaging Library Radient eV-CL system. INFO										j					
	M_SYSTEM_RAPIXOCL_TYPE	Specifies an Aurora Imaging Library Rapixo Pro CL system. INFO											k				
	M_SYSTEM_RAPIXOCOF_TYPE	Specifies an Aurora Imaging Library Rapixo CoF system. INFO												l			
	M_SYSTEM_RAPIXOCXP_TYPE	Specifies an Aurora Imaging Library Rapixo CXP system. INFO													m		
	M_SYSTEM_USB3_VISION_TYPE	Specifies an Aurora Imaging Library USB3 Vision system. INFO														n	
	M_SYSTEM_V4L2_TYPE	Specifies an Aurora Imaging Library Video4Linux2 system. INFO															b
M_PARENT_ID		Inquires the Aurora Imaging Library identifier of the parent container. Only child container have a parent container. The parent container is the container from which the specified container (ContainerBufId) was defined. The container can itself have a parent container. If the specified container has no parent container, the identifier of the specified container is returned. To establish the ultimate ancestor container of the specified container, use M_ANCESTOR_ID instead. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b

M_SYSTEM_LOCATION	Inquires whether the specified container is allocated on a system on the local computer or the remote computer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_LOCAL	Specifies that the container is allocated on a local system.	c		e	f	g	a		i		j	k	l	m	n	b
M_REMOTE	Specifies that the container is allocated on a remote system.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire which components are in the container, specify one of the following constants:

> These settings will not return an error if there is more or less than one component that matches the criterion specified by Component.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about which components are in a container																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_COMPONENT_COUNT	Inquires how many components are in the container. If Component is set to M_CONTAINER, the returned value will be the number of all components in the container. Otherwise, the returned value will be the number of components in the container which meet the specified criteria.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_LIST +	Inquires the Aurora Imaging Library identifiers of components in the container. If Component is set to M_CONTAINER, the returned array will contain the Aurora Imaging Library identifiers of all components in the container. Otherwise, the returned array will contain the Aurora Imaging Library identifiers of all components in the container which meet the specified criteria.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_COMPONENT_GROUP_ID_LIST; M_COMPONENT_REGION_ID_LIST; M_COMPONENT_SOURCE_ID_LIST; M_COMPONENT_TYPE_LIST; M_COMPONENT_LIST.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

To inquire a container or component, specify one of the following constants.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about settings that apply to both containers and components																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CAMERA_CONTAINER_ID	Inquires the GenDC container identifier from the camera. The camera sends the identifier with each transmitted frame.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UINT64*			e	f	g							l	m		b
0	Specifies that GenDC container identifiers are not supported by the camera.			e	f	g							l	m		b
Value > 0	Specifies the GenDC container identifier.			e	f	g							l	m		b
M_CAMERA_FRAME_ID	Inquires the frame identifier from the camera. The camera sends the identifier with each transmitted frame.			e	f	g									n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UINT64*			e	f	g									n	b
0	Specifies that frame identifiers are not supported by the camera.			e	f	g									n	b
Value > 0	Specifies the frame identifier.			e	f	g									n	b
M_CAMERA_TIME_STAMP	Inquires the time stamp from the camera expressed in seconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the time stamp, in seconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_CAMERA_TIME_STAMP_NS		Inquires time stamp from the camera expressed in nanoseconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the time stamp, in nanoseconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_ATTRIBUTE		Inquires the attributes of the container or component. To retrieve only the data storage format of the buffer, use **MbufInquireContainer**() with M_DATA_FORMAT. To retrieve the detailed storage format of the specified buffer, use **MbufInquireContainer**() with M_EXTENDED_FORMAT. Note that you cannot set the UserVarPtr parameter to **M_NULL** when InquireType is set to M_EXTENDED_ATTRIBUTE. In addition, a valid *AIL_INT64* pointer must be passed to the function; otherwise, an error will occur. To convert the returned attributes, select the **Benchmarks and Utilities** item in the tree structure of the Aurora Imaging Configurator utility. Then, select the **Buffer format** item. On the **Value type** pane, paste the returned attributes in the text box provided. Then, click on the **Value lookup** button. The results of the translation are presented below the **Value lookup** button. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ARRAY +	Specifies a buffer to store array type data. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_CONTAINER	Specifies a container.	c		e	f	g	a		i		j	k	l	m	n	b
	M_IMAGE +	Specifies a buffer to store image data.	c		e	f	g	a		i		j	k	l	m	n	b
	M_KERNEL +	//For 1D or 2D data Specifies a kernel buffer to store a custom filter for convolution functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_LUT +	Specifies a buffer to store lookup table data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_STRUCT_ELEMENT +	//For 1D or 2D data Specifies a buffer to store structuring element data for morphology functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_ATTRIBUTE_NAME +		Inquires the human-readable extended attributes of the container or component.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_TIME_STAMP		Inquires the time stamp of when the grab operation completed, in seconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0.0	Specifies the time stamp of when the grab operation completed, in seconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB_TIME_STAMP_NS		Inquires the time stamp of when the grab operation completed, in nanoseconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the time stamp of when the grab operation completed, in nanoseconds.	c		e	f	g	a		i		j	k	l	m	n	b

M_MODIFICATION_COUNT	Inquires the current value of the modification counter of the image buffer. When the image buffer is allocated, the modification counter is initialized to a random non-zero value. The modification counter is incremented each time the image buffer is modified by an Aurora Imaging Library function. If the image buffer is accessed externally, for example, when using MbufCreateColor() or MbufCreate2d(), MbufControl() with M_MODIFIED must be called to indicate that the image buffer's contents have been modified. Calling this function will increment the counter. This feature is useful for optimization. For example, you can avoid repeating certain computations (for example, analysis computations) if you know that the image buffer has not been modified. In this case, inquire the count before the first computation in the sequence of computations, and then inquire it again before repeating the same sequence. If no modifications have been made to the image buffer, you can avoid repeating the sequence unnecessarily.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODIFICATION_HOOK	Inquires the status of the modification hook, which runs a user-defined function upon an event. These user-defined functions are initially hooked to the buffer modification event using MbufHookFunction(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the user-defined functions should not be called. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the user-defined functions should be called. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire a component, specify one of the following constants.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about settings that apply only to components

InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_COMPONENT_GROUP_ID	Inquires the component group ID assigned to the buffer during acquisition. INFO															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the component group ID of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_ID	Inquires the Aurora Imaging Library identifier of the component.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that no component in the container matches the specified criterion.	c		e	f	g	a		i		j	k	l	m	n	b
M_MULTIPLE_IDS	Specifies that more than one component in the container matches the specified criterion.	c		e	f	g	a		i		j	k	l	m	n	b
Component identifier	Specifies the Aurora Imaging Library identifier of the component.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_INVALID	Inquires whether the information in the buffer was marked as invalid by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the information in the buffer has not been marked invalid by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the information in the buffer has been marked invalid by the camera.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_REGION_ID	Inquires the component region ID assigned to the buffer during acquisition. This identifies which region of the image sensor was used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the component region ID of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_OFFSET_X	Inquires the X-offset of the top left corner of the region of the image sensor used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the X-offset of the component region of the buffer, expressed in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_REGION_OFFSET_Y	Inquires the Y-offset of the top left corner of the region of the image sensor used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the Y-axis offset of the component region ID of the buffer, expressed in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_SOURCE_ID	Inquires the component source ID assigned to the buffer during acquisition. This indicates which of the camera's data sources was used to generate the data in the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the component source ID of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_TIME_STAMP	Inquires the time stamp given by the camera GenDC stream. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the time stamp, in seconds.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_TIME_STAMP_IS_PTP	Inquires if M_COMPONENT_TIME_STAMP_NS is relative to the Unix epoch (January 1st 1970 00:00:00).	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that M_COMPONENT_TIME_STAMP_NS is not calculated based on the Unix epoch (January 1st 1970 00:00:00).	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that M_COMPONENT_TIME_STAMP_NS is relative to the Unix epoch (January 1st 1970 00:00:00).	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_TIME_STAMP_NS	Inquiries the time stamp given by the camera GenDC stream in nanoseconds. Due to clock differences, this time stamp should not be compared to the other available time stamps.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
0	Specifies that this time stamp is not supported by the camera.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the time stamp, in nanoseconds.	c		e	f	g	a		i		j	k	l	m	n	b

M_COMPONENT_TYPE		Inquires the component type of the buffer, used when the buffer is a component of a container. The component type specifies what kind of information is stored in the component. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_CONFIDENCE	Specifies that the component stores confidence information for the M_COMPONENT_RANGE or M_COMPONENT_DISPARITY component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_COORDINATE_MAP_A_AIL	Specifies that the component stores the A coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_COORDINATE_MAP_B_AIL	Specifies that the component stores the B coordinate map provided by the camera. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_CUSTOM + *n*	Specifies that the component has a custom component type, identified by *n*, where n can be a value between 0 and 255. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_DISPARITY	Specifies that the component stores a disparity map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_INFRARED	Specifies that the component stores an intensity image of infrared light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_INTENSITY	Specifies that the component stores an intensity image of visible light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MATRIX_AIL	Specifies that the component stores a 4x4 matrix that transforms depth map coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MESH_AIL	Specifies that the component stores mesh information for the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_COMPONENT_METADATA	Specifies that the component stores metadata information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_MULTISPECTRAL	Specifies that the component stores an intensity image where each band represents the intensity of a specific wavelength of light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_NORMALS_AIL	Specifies that the buffer stores normals information for each point in the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_RANGE	Specifies that the component stores 3D distance/position information. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_REFLECTANCE	Specifies that the component stores a reflectance map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_REGION_AIL	Specifies that the component stores a region of interest (ROI) for the M_COMPONENT_RANGE component of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_SCATTER	Specifies that the component stores a scatter map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_ULTRAVIOLET	Specifies that the component stores an intensity image of ultraviolet light. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPONENT_UNDEFINED	Specifies that the component contains information of an unknown type. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPONENT_TYPE_NAME +		Inquires the component type of the buffer in a human-readable format.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b

M_DATA_FORMAT	Inquires the data storage format of the buffer. This is the color or monochrome storage format of the buffer and includes whether it is in packed or planar format. Note that non-color buffers are stored in planar format and lack a color space. To retrieve the entire list of attributes, use MbufInquire() with M_EXTENDED_ATTRIBUTE or M_EXTENDED_FORMAT. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c	d	e	f	g	a	h	i		j	k	l	m	n	b
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	This value can have multiple meanings, as indicated below.	c		e	f	g	a		i		j	k	l	m	n	b
	//Initially, if a buffer was allocated at the same time as its memory Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	//Initially, if a buffer was created from preallocated memory Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c			f		a		i		j	k	l	m		
M_DATA_TYPE	Inquires the buffer data type. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT	Specifies that the buffer uses the float data type.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED	Specifies that the buffer uses the signed data type.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNSIGNED	Specifies that the buffer uses the unsigned data type.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_FORMAT	Inquires the detailed storage format of the specified buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_ARRAY	Specifies a buffer to store array type data. INFO INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_IMAGE +	Specifies a buffer to store image data. INFO INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_KERNEL +	Specifies a kernel buffer to store a custom filter for convolution functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LUT	Specifies a buffer to store lookup table data. INFO INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRUCT_ELEMENT +	Specifies a buffer to store structuring element data for morphology functions. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTENDED_FORMAT_NAME +	Inquires the M_EXTENDED_FORMAT of the buffer, expressed in a human readable format.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFORMATION_TYPE	Inquires the GenDC part type of the buffer.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_1D	Specifies the GenDC part type is 1D. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_2D	Specifies the GenDC part type is 2D. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_2D_H264	Specifies the GenDC part type is H.264 data. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_INFO_2D_JPEG	Specifies the GenDC part type is JPEG data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_2D_JPEG2000	Specifies the GenDC part type is JPEG 2000 data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_GENICAM_CHUNK	Specifies the GenDC part type is GenICam chunk data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INFO_GENICAM_XML	Specifies the GenDC part type is GenICam XML data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_INFORMATION_TYPE_NAME +	Inquires the GenDC part type of the buffer as a string.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_NAME +	Inquires the name of the buffer's pixel format, as defined by the GenICam Pixel Format Naming Convention.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_SIZE_BIT	Inquires the number of bits per pixel, as per the GenICam Pixel Format Naming Convention. INFO	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the number of bit per pixel as per the PFNC.	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_SUPPORT	Inquires whether Aurora Imaging Library supports processing the buffer's given it's pixel format. INFO	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NO	Specifies that image processing is not supported for the buffer's pixel format. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_WITH_COMPENSATION	Specifies that Aurora Imaging Library will process the buffer by copying its data to a temporary buffer with a supported pixel format.	c		e	f	g	a		i		j	k	l	m	n	b
	M_YES	Specifies that the buffer will be processed as normal.	c		e	f	g	a		i		j	k	l	m	n	b
M_PFNC_VALUE		Inquires the pixel format of the buffer, as defined by the GenICam Pixel Format Naming Convention. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	PFNCvalue	Specifies the pixel format of the buffer using a value from PFNC (as defined in *pfnc.h*). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PITCH		Inquires the number of pixels between the beginnings of any two adjacent lines of the buffer data. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the pitch, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PITCH_BYTE		Inquires the number of bytes between the beginnings of any two adjacent lines of the buffer data. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the pitch, in bytes.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BAND		Inquires the number of buffer bands. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	1 <= *Value* <= 1024	Specifies the number of bands.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_BIT		Inquires the depth per band. SET SET	c		e	f	g	a		i		j	k	l	m	n	b

UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the depth per band, in bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_X	Inquires the width of the buffer. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIZE_Y	Inquires the height of the buffer. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE	Inquires the buffer data type and depth. Depth is returned in bits. SET SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FLOAT + *Depth value*	Specifies the data depth and that the data type is floating-point.	c		e	f	g	a		i		j	k	l	m	n	b
M_SIGNED + *Depth value*	Specifies the data depth and that the data type is signed.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNSIGNED + *Depth value*	Specifies the data depth and that the data type is unsigned.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_EXTENDED_ATTRIBUTE_NAME; M_COMPONENT_TYPE_NAME; M_EXTENDED_FORMAT_NAME; M_INFORMATION_TYPE_NAME; M_PFNC_NAME.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_STRING_SIZE		Retrieves the length of the string, including the terminating null character ("\0").														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT;], [when InquireType=M_EXTENDED_FORMAT: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT].

One of the following values might be combined with the above-mentioned values to determine the location of the buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the location of the buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_HOST_MEMORY +	Specifies that the buffer is in Host memory. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_OFF_BOARD	Specifies that the buffer is not in on-board memory. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ON_BOARD +	Specifies that the buffer is in on-board memory. INFO	c							i		j	k	l	m		

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT;], [when InquireType=M_EXTENDED_FORMAT: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT].

One of the following values might be combined with the above-mentioned values to determine whether the buffer was allocated in paged or non-paged memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns whether the buffer was allocated in paged or non-paged memory																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NON_PAGED	Specifies that the buffer is in Aurora Imaging Library reserved, non-pageable memory, if possible. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PAGED	Specifies that the buffer is in pageable memory. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_IMAGE;], [when InquireType=M_EXTENDED_FORMAT: M_IMAGE].

One or more of the following values are combined with the above-mentioned values to determine the intended purpose of the buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the intended purpose of the image buffer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_COMPRESS +	Specifies an image buffer that can hold compressed data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DISP	Specifies an image buffer that can be displayed. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GRAB	Specifies an image buffer in which to grab data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC +	Specifies an image buffer that can be processed. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_IMAGE;], [when InquireType=M_EXTENDED_FORMAT: M_IMAGE].

One of the following values might be combined with the above-mentioned values to determine the storage format and location specifier.

You might have set this value, or it could have been automatically selected by Aurora Imaging Library.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the storage format and location specifier																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DIB	Specifies that the buffer is a DIB buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_GDI	Specifies that the buffer is compatible with GDI. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LINUX_MXIMAGE	[This is only applicable to Linux] Specifies that the buffer is an X11 Ximage. INFO	c		e	f	g	a				j	k	l	m	n	b

Combination values for M_COMPRESS.

One of the following values might be combined with the above-mentioned value to determine the compression type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the compression type																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_JPEG2000_LOSSLESS	Specifies that the buffer will be used to hold JPEG2000 lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG2000_LOSSY	Specifies that the buffer will be used to hold JPEG2000 lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_JPEG_LOSSLESS	Specifies that the buffer will be used to hold JPEG lossless data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSLESS_INTERLACED	Specifies that the buffer will be used to hold JPEG lossless data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY	Specifies that the buffer will be used to hold JPEG lossy data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_JPEG_LOSSY_INTERLACED	Specifies that the buffer will be used to hold JPEG lossy data in separate fields. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PROC.

The following value might be combined with the above-mentioned value to determine whether the buffer was allocated with an overscan region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns whether the buffer was allocated with an overscan region																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALLOCATION_OVERSCAN	Specifies that the buffer is allocated with an overscan region. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when InquireType=M_EXTENDED_ATTRIBUTE: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT;], [when InquireType=M_EXTENDED_FORMAT: M_ARRAY; M_IMAGE; M_KERNEL; M_LUT; M_STRUCT_ELEMENT].

The following value might be combined with the above-mentioned values to determine whether the buffer is FPGA accessible.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns whether the buffer is FPGA accessible																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FPGA_ACCESSIBLE +	Specifies that the buffer is allocated in a bank of memory that is accessible from the Processing FPGA. INFO										j	k	l	m		

Combination values for [when InquireType=M_EXTENDED_ATTRIBUTE: M_IMAGE;], [when InquireType=M_EXTENDED_FORMAT: M_IMAGE].

One of the following values might be combined with the above-mentioned values to determine the format in which image buffers were stored.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the format in which image buffers were stored

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PACKED +	Specifies that the buffer's bands are stored in packed format; that is, each pixel's bands are stored together (RGB RGB RGB...). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PLANAR +	Specifies that the buffer's bands are stored in planar format; that is, each pixel's bands are stored in separate planes. INFO	c			f		a		i		j	k	l	m		

Combination values for [when InquireType=M_DATA_FORMAT: M_PACKED; M_PLANAR;], and for the following values: M_PACKED; M_PLANAR.

One of the following values might be combined with the above-mentioned values to determine the packed or planar color buffer format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the packed or planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB24	Specifies 24-bit color depth (RGB 8:8:8) packed or planar pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB48	Specifies 48-bit color depth (RGB 16:16:16). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB96	Specifies 96-bit color depth (RGB 32:32:32) packed or planar pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16	Specifies YUV16 (4:2:2) pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_DATA_FORMAT: M_PACKED;], and for the following values: M_PACKED.

One of the following values might be combined with the above-mentioned values to determine the packed color buffer format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the packed color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BGR24	Specifies 24-bit color depth packed pixels (BGRBGR). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_BGR32	Specifies 32-bit color depth packed pixels (BGRXBGRX). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_RGB15	Specifies 16-bit color depth packed pixels (XRGB 1:5:5:5). INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_RGB16	Specifies 16-bit color depth packed pixels (RGB 5:6:5). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_YUV16_UYVY	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the UYVY order. INFO			e	f	g									n	b
M_YUV16_YUYV	Specifies YUV16 packed (4:2:2) pixels, whereby the bands of each pixel are stored in the YUYV order. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for [when InquireType=M_DATA_FORMAT: M_PLANAR;], and for the following values: M_PLANAR.

One of the following values might be combined with the above-mentioned values to determine the planar color buffer format.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the planar color buffer format																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB3	Specifies 3-bit color depth (RGB 1:1:1) planar pixels. INFO	c					a		i		j	k	l	m		
M_YUV9	Specifies YUV9 planar pixels. INFO	c			f		a		i		j	k	l	m		
M_YUV12	Specifies YUV12 planar pixels. INFO	c			f		a		i		j	k	l	m		
M_YUV24	Specifies YUV24 planar pixels. INFO	c			f		a		i		j	k	l	m		

Combination value for M_HOST_MEMORY; M_ON_BOARD; M_FPGA_ACCESSIBLE.

The following value might be combined with the above-mentioned values to determine the memory bank in which the buffer was allocated.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns the memory bank used																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MEMORY_BANK_*n*	Inquires the buffer allocated in the specified memory bank. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_ON_BOARD; M_FPGA_ACCESSIBLE.

The following value might be combined with the above-mentioned values to set a location in a specific type of memory.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a location in a specific type of memory

Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SHARED	Specifies that the buffer is in shared processing memory. INFO										j	k	l	m		

For components with an M_IMAGE attribute and with M_COMPONENT_TYPE set to M_COMPONENT_RANGE or M_COMPONENT_DISPARITY, InquireType can also be set to one of the values below.

These settings are used only when the component is the range or disparity component of a container that is used as a source with MbufConvert3d(). Additionally, a container cannot be 3D-processable or 3D-displayable unless it has a single range component with all of these settings at their default values (except for M_3D_DISTANCE_UNIT which can be any value and M_3D_REPRESENTATION, which must be set to M_CALIBRATED_XYZ or M_CALIBRATED_XYZ_UNORGANIZED).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about 3D settings that apply only to range and disparity components that are image buffers																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_3D_ASPECT_RATIO	Inquires by how much the focal length in the Y-direction stored in the buffer will be scaled in the destination point cloud to correct for non-square pixels in 3D reconstruction modes that rely on projective geometry. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies by how much the focal length the Y-direction will be scaled. INFO The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_COORDINATE_SYSTEM_TYPE	Inquires which type of coordinate system to use to interpret the coordinates stored in the bands of the buffer if it is a range component. Note that Aurora Imaging Library does not support any functionality with coordinates stored using a coordinate system type other than M_CARTESIAN. If a buffer stores information defined using another type of coordinate system, you will need to manually convert that information to cartesian coordinates before the buffer can be used with any Aurora Imaging Library processing function. This conversion cannot be done using MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CARTESIAN	Specifies that the buffer stores right-handed cartesian coordinates. INFO This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_CYLINDRICAL	Specifies that the buffer stores cylindrical coordinates (theta-Y-rho). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SPHERICAL	Specifies that the buffer stores spherical coordinates (theta-phi-rho). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_UNKNOWN	Specifies that the coordinate system type is unknown. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISPARITY_BASELINE	Inquires the stereo baseline value of the stereoscopic camera used to generate the data in the buffer. This is the physical distance between the lenses of the camera. Refer to your camera manual to determine the correct value for this setting, which might differ from the true physical distance between the lenses of your camera. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the stereo baseline value, expressed in meters. INFO The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_DISTANCE_UNIT		Inquires the unit to use when the buffer is part of a container and stores natively calibrated distance data. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_INCH	Specifies that the distance data is provided in inches. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MILLIMETER	Specifies that the distance data is provided in millimeters. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that the distance data is provided in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_UNKNOWN	Specifies that the distance unit is unknown. INFO This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_FOCAL_LENGTH		Inquires the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer. Refer to your camera manual to determine the correct value for this setting, which might differ from the true focal length of your camera. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the focal length of the lenses of the stereoscopic camera used to generate the data in the buffer, expressed in pixels. INFO The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_INVALID_DATA_FLAG	Inquires whether the buffer uses a specific value to indicate invalid data. For 3-band buffers that store coordinates, the invalid data flag should be stored in the Z-axis band. Specify the value that indicates invalid data using M_3D_INVALID_DATA_VALUE. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the buffer does not use a special value to indicate invalid data. INFO This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the buffer uses a special value to indicate invalid data. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_INVALID_DATA_VALUE	Inquires the value used to indicate missing data when M_3D_INVALID_DATA_FLAG is set to M_TRUE. Note that this value is not used or respected by any Aurora Imaging Library functions except for MbufConvert3d() when the buffer is a component of the source container. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the value used to indicate missing data. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_X	Inquires by how much the X-coordinates stored in the buffer will be offset in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies by how much the X-coordinates stored in the buffer will be offset. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Y	Inquires by how much the Y-coordinates stored in the buffer will be offset in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Y-coordinates stored in the buffer will be offset. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_OFFSET_Z	Inquires by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Z-coordinates stored in the buffer will be offset. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_PRINCIPAL_POINT_X	Inquires the X-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies X-position of the principal point, expressed in pixels. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_PRINCIPAL_POINT_Y	Inquires the Y-position of the principal point of the buffer. This is the point in the buffer which the optical axis of the camera intersects. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies Y-position of the principal point, expressed in pixels. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_REPRESENTATION	Inquires how 3D data is stored in the buffer; this information is used when the buffer is a range or disparity component of a container. For more information, see MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XYZ	Specifies that the component stores organized and natively calibrated X, Y, and Z-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XYZ_UNORGANIZED	Specifies that the component stores unorganized and natively calibrated X, Y, and Z-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XZ_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates stored in a separate array buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_XZ_UNIFORM_Y	Specifies that the component stores organized and natively calibrated X and Z-coordinates, with Y-coordinates identified by the row index. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_CALIBRATED_Z	Specifies that the component stores organized and natively calibrated Z-coordinates, without X and Y-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_CALIBRATED_Z_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates without X-coordinates; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_X_EXTERNAL_Y	Specifies that the component stores organized and natively calibrated Z-coordinates, with the X-coordinates identified by column index; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_CALIBRATED_Z_UNIFORM_XY	Specifies that the component stores organized and natively calibrated Z-coordinates, with X and Y-coordinates identified by column and row index respectively. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY	Specifies that the component stores a disparity map with perspective distortion along the Y-axis. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_EXTERNAL_Y	Specifies that the component stores a disparity map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISPARITY_UNIFORM_Y	Specifies that the component stores a disparity map, with Y-values identified by row index. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROJECTED_Z	Specifies that the component stores a projective map. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROJECTED_Z_EXTERNAL_Y	Specifies that the component stores a projective map; Y-coordinates are stored in a separate buffer that has an M_ARRAY attribute and is not part of the container. INFO	c		e	f	g	a		i		j	k	l	m	n	b

	M_UNCALIBRATED_Z	Specifies that the component stores organized and uncalibrated Z-coordinates, without X and Y-coordinates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_X		Inquires by how much the X-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value != 0.0	Specifies by how much the X-coordinates stored in the buffer will be scaled. INFO The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Y		Inquires by how much the Y-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value != 0.0	Specifies how much the Y-coordinates stored in the buffer will be scaled. INFO The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SCALE_Z		Inquires by how much the Z-coordinates stored in the buffer will be scaled in the destination point cloud when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value != 0.0	Specifies how much the Z-coordinates stored in the buffer will be scaled. INFO The default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b

M_3D_SHEAR_X	Inquires by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the X-coordinates stored in the buffer will be offset in the destination point cloud from the X-coordinates in the previous row. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
M_3D_SHEAR_Z	Inquires by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row when it is a component of a container passed as a source to MbufConvert3d(). SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies by how much the Z-coordinates stored in the buffer will be offset in the destination point cloud from the Z-coordinates in the previous row. INFO The default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information. If the requested information cannot be retrieved, **M_ERROR** is returned.

Remarks

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

- *[Aurora Imaging Library Lite]*
  While an image buffer with an M_KERNEL or an M_STRUCT_ELEMENT attribute are available under Aurora Imaging Library Lite, these attributes are not required for the image buffer to be available to other Aurora Imaging Library Lite functions.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufInquireFeature

Synopsis

Inquires a feature of the buffer.

Syntax

```
void MbufInquireFeature(
    AIL_ID BufId,                   //in
    AIL_INT64 InquireType,          //in
    AIL_CONST_TEXT_PTR FeatureName, //in
    AIL_INT64 UserVarType,          //in
    void *UserVarPtr                //out
)
```

Description

This function allows you to directly inquire various GenICam GenTL standard feature naming convention (SFNC) features and buffer-specific features. When used with a GenICam GenTL SFNC-compliant camera, it allows you to directly inquire various buffer-specific features specified with the GenTL buffer description file (XML). Note that for the purpose of this function, a feature can also be of type category or command. Although category and command features lack a feature value, all other attributes of the feature can be inquired. For a list of standard category feature names, see the GenICam GenTL Standard Feature Naming Convention or your GenTL-compliant camera's documentation.

The features described in this function are primarily available to check existing settings using Aurora Imaging Library code or to retrieve information from which you can build your own interface for the GenTL buffer's description file (XML). Aurora Imaging Library provides two versions of an interface that you can use to interactively inquire the camera's features. At design-time, you can use Aurora Imaging Intellicam's Feature Browser. At runtime, you can launch Feature Browser, using MbufControl() with M_GC_FEATURE_BROWSER. Both versions of Feature Browser provide a list of available feature names and associated possible values, as well as code snippets with the Aurora Imaging Library functions and Aurora Imaging Library constants associated with the settings selected; you can copy the code snippets to your Aurora Imaging Library application code.

Occasionally, the returned information is a string whose length can change between when you inquires its size and when you inquire its value (for example, the temperature can change from 99 to 100 between calls). In this case, when you inquire the information's length to allocate an array of an appropriate size, you might allocate an array that is too small to retrieve the data and experience an application crash. To avoid an application crash, specify the length of the array passed to the UserVarPtr parameter using the M_FEATURE_USER_ARRAY_SIZE() macro; the data will be truncated to fit in the array and an error will be generated.

To control a buffer-specific feature, use MbufControlFeature().

Parameters

BufId

Specifies the identifier of the buffer to inquire. This parameter must be given a valid buffer identifier, previously allocated using MbufAlloc...().

InquireType

Specifies the type of information to inquire about the feature.

See the Parameter associations section for possible values that can be specified.

FeatureName

Specifies the name of the buffer feature to inquire. Note that this parameter expects a case-sensitive string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the name of the feature																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*FeatureName*") 1	Specifies the name of the feature. Note that the feature name is case-sensitive. Refer to your GenTL library's documentation for a list of the features available.			e												b
AIL_TEXT("*Root*") 1	Specifies to inquire the highest-level feature of the XML structure; this feature is of type category and typically, its subfeatures are also of type category. Note that this value is only available when using M_SUBFEATURE_....			e												b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

UserVarType

Specifies the data type of the address pointed to by the UserVarPtr parameter. If not inquiring the feature's value (using M_FEATURE_VALUE), set this parameter to **M_DEFAULT**.

Note that, in the case of a string (or a multi-byte register), the size of the array passed to the UserVarPtr parameter must be equal to or larger than the returned information; otherwise an application crash will occur.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the UserVarPtr's data type																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_TYPE_BOOLEAN	Specifies that UserVarPtr is passed an address of type *AIL_BOOL*.			e												b
M_TYPE_DOUBLE	Specifies that UserVarPtr is passed an address of type *AIL_DOUBLE*.			e												b
M_TYPE_INT64	Specifies that UserVarPtr is passed an address of type *AIL_INT64*.			e												b
M_TYPE_STRING +	Specifies that UserVarPtr is passed an address of type *AIL_TEXT_CHAR*. You must specify a combination value from the following table: • To specify the length of the array			e												b

M_TYPE_UINT8 +	Specifies that UserVarPtr is passed an address of type *AIL_UINT8*. You must specify a combination value from the following table: • To specify the length of the array			e												b

Combination value for M_TYPE_STRING; M_TYPE_UINT8.

You must add the following value to the above-mentioned values to specify the length of the array.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the length of the array																	
Combination value	Description		(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FEATURE_USER_ARRAY_SIZE(AIL_INT Length)	Specifies the length of the array.				e												b
		Parameters															
		Length Sets the length, in bytes.															

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_BOOL [optionally, in C++: a reference to a std::vector<AIL_BOOL>]
- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]
- AIL_BOOL
- AIL_DOUBLE
- AIL_INT
- AIL_INT64

Specifies the address of the variable in which to return the requested information.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For specifying the type of information to inquire about the feature and the data type returned
- For inquiring about the enumeration entry of a feature's value
- For inquiring about the subfeatures of a feature

The following inquire types are used to specify the type of information about the feature to inquire and the data type returned.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo,

For specifying the type of information to inquire about the feature and the data type returned																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_FEATURE_ACCESS_MODE	Inquires whether the specified feature is implemented, available, readable, and/or writable. Note that an error is generated if the feature does not exist in the camera's device description file (XML). To learn if the specified feature exists in the file and is implemented on your camera, use M_FEATURE_PRESENT.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e												b
Bit-encoded access mode value	Specifies a bit-encoded value that details the access mode. To establish the access mode from the encoded value, use the following macros: • To learn whether the specified feature is available, use the **M_FEATURE_IS_AVAILABLE** macro. Note that if a feature is not available, use **M_FEATURE_IS_IMPLEMENTED** to learn whether the specified feature is only temporarily unavailable or actually not implemented on your camera. A feature that is temporarily unavailable typically relies on the setting of another feature (for example, when a trigger feature is set to off, its associated trigger polarity feature would be unavailable). • To learn whether the specified feature is implemented, use the **M_FEATURE_IS_IMPLEMENTED** macro. For example, a Bayer color feature would not be implemented on a monochrome camera because the hardware cannot support it. • To learn whether the specified feature can be read, use the **M_FEATURE_IS_READABLE** macro. • To learn whether the specified feature can be written to, use the **M_FEATURE_IS_WRITABLE** macro.			e												b

M_FEATURE_CACHING_MODE	Inquires whether a copy of the specified feature's value is stored in Host memory for faster access and retrieval. If so, the feature is known as a cachable feature. A cachable feature is typically only retrieved from the camera whenever the cache is invalidated (such as, before the first read and before the first read to occur after a write operation). A cachable feature is typically a value that only changes based on user-interaction (setting a control value). For example, a temperature feature is not cachable. To return a true/false value for this inquire, use the **M_FEATURE_IS_CACHABLE** macro on the returned result.			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e											b
M_FEATURE_CACHING_MODE_NONE	Specifies that the feature is not cachable.			e											b
M_FEATURE_CACHING_MODE_WRITE_AROUND	Specifies that feature's values are written only to the camera, and the cache is updated the next time the feature's value is inquired.			e											b
M_FEATURE_CACHING_MODE_WRITE_THROUGH	Specifies that when information is written to the camera, the cache is updated simultaneously.			e											b
M_FEATURE_CHANGE_HOOK	Inquires whether an event occurs when the value of the specified feature changes. If enabled, MbufHookFunction() with M_FEATURE_CHANGE hooks a specified function to the feature change event. SET			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e											b
M_DISABLE	Specifies that no event is generated when the value of the specified feature changes. This is the default value.			e											b
M_ENABLE	Specifies that an event is generated when the value of the specified feature changes.			e											b
M_FEATURE_DEPRECATED	Inquires whether the specified feature is deprecated and should no longer be used.			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e											b

	M_FALSE	Specifies that the feature is not depreciated.			e											b
	M_TRUE	Specifies that the feature is depreciated.			e											b
M_FEATURE_DESCRIPTION +		Inquires the description of the specified feature.			e											b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
	Value	Specifies the description of the feature.			e											b
M_FEATURE_DISPLAY_NAME +		Inquires the specified feature's display name. Note that this can differ slightly from the internally-used feature name (M_FEATURE_NAME).			e											b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
	Value	Specifies the display name of the feature.			e											b
M_FEATURE_EXECUTE_COMPLETED		Inquires whether the specified executable camera feature (command) has finished executing on your camera. Note that to enable command execution-completion polling, set MsysControl() with M_GC_FEATURE_EXECUTE_POLLING_MODE to M_AUTOMATIC.			e											b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL*			e											b
	M_FALSE	Specifies that the executable camera feature has not finished executing.			e											b
	M_TRUE	Specifies that the executable camera feature has finished executing.			e											b
M_FEATURE_INCREMENT		Inquires the value by which the feature's value increments. For example, if a feature's possible values are 2, 4, 6, ..., then the feature's value increments by 2.			e											b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e											b
	Value	Specifies the feature's incremental value.			e											b

M_FEATURE_MAX	Inquires the maximum possible value for the feature.			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *address of an AIL_INT64*			e											b
Value	Specifies the maximum possible value for the feature.			e											b
M_FEATURE_MIN	Inquires the minimum possible value for the feature.			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *address of an AIL_INT64*			e											b
Value	Specifies the maximum possible value for the feature.			e											b
M_FEATURE_NAME +	Inquires the specified internally-used feature name. Note that this can differ slightly from the displayed name of the feature (M_FEATURE_DISPLAY_NAME).			e											b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
Value	Specifies the feature's name.			e											b
M_FEATURE_POLLING_INTERVAL	Inquires the interval between inquiring an executable feature's completion status, when automatically polling (using MsysControl() with M_GC_FEATURE_EXECUTE_POLLING_MODE).			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e											b
Value >= -1	Specifies the polling interval, in msecs.			e											b
M_FEATURE_PRESENT	Inquires whether the specified feature is present in the camera's device description file. Note that, if the feature is listed in the camera's device description file (XML), use M_FEATURE_ACCESS_MODE and the **M_FEATURE_IS_IMPLEMENTED** macro to determine if the feature is supported. If the feature is not present in the file, M_FEATURE_PRESENT will return M_NO, whereas M_FEATURE_ACCESS_MODE will generate an error.			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL*			e											b

	M_NO	Specifies that the specified feature is not present.			e											b
	M_YES	Specifies that the specified feature is present.			e											b
M_FEATURE_REPRESENTATION		Inquires the display format for the feature's value. The display format dictates the size of the field and its form (check box, slider, text-box, etc.) used in a user interface for the GenTL buffer's description file (XML), such as Feature Browser (accessible through Aurora Imaging Intellicam or MbufControl() with M_GC_FEATURE_BROWSER. Note that this inquire type can only be used with a feature that has a data type of *AIL_INT64* or *AIL_DOUBLE*.			e											b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e											b
	M_FEATURE_REPRESENTATION_BOOLEAN	Specifies that the feature should be displayed as a checkbox.			e											b
	M_FEATURE_REPRESENTATION_HEX_NUMBER	Specifies that the feature should be displayed as a hexadecimal value.			e											b
	M_FEATURE_REPRESENTATION_IPV4_ADDRESS	Specifies that the feature should be displayed as an IP address in dotted decimal notation.			e											b
	M_FEATURE_REPRESENTATION_LINEAR	Specifies that the feature should be displayed as a slider with the appropriate range shown.			e											b
	M_FEATURE_REPRESENTATION_LOGARITHMIC	Specifies that the feature should be displayed as a slider with the appropriate logarithmic range shown.			e											b
	M_FEATURE_REPRESENTATION_MAC_ADDRESS	Specifies that the feature should be displayed as a hexadecimal MAC address.			e											b
	M_FEATURE_REPRESENTATION_PURE_NUMBER	Specifies that the feature should be displayed as edit text-box with a decimal display.			e											b

M_FEATURE_SELECTOR_COUNT	Inquires the number of selectors that are ancestors of the feature. Selectors take an enumerator or integer to index which instance of a subfeature should be accessed, when multiple instances of a subfeature exist. For example, the GenICam standard `TriggerSelector` feature is used to select which type of trigger to configure (for example, `FrameStart` and `AcquisitionStart` are both enumerators that can be passed to `TriggerSelector`). All subfeatures of the selector (such as its source) are referenced from the trigger selector (`TriggerSource[AcquisitionStart]`). This inquire type only returns a value greater than 0 if the feature is a subfeature of a selector. It returns 1 if the feature is a subfeature of a selector. It returns 2 if its selector is also referenced by another selector. The M_FEATURE_SELECTOR_COUNT for `TriggerMode`, in this example, would be 1. To inquires the name of the selector associated with the specified feature, use M_FEATURE_SELECTOR_NAME + n.			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e											b
Value	Specifies the number of ancestor selectors of the feature.			e											b
M_FEATURE_SELECTOR_NAME + *n* +	Inquires the name of the ancestor selector of the specified subfeature, where *n* is the number of generations. So, if inquiring the feature selector name of the 1st selector of `TriggerSource`, the returned name would be `TriggerSelector`. To inquires the number of selectors that are associated with the specified feature, use M_FEATURE_SELECTOR_COUNT.			e											b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
Value	Specifies the name associated with the selector index.			e											b
M_FEATURE_SIZE	Inquires the size of the feature's value.			e											b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e											b

	Value	Specifies the feature's size, in bytes.			e										b
M_FEATURE_STREAMABLE		Inquires whether the specified feature is streamable. A streamable feature can be stored in the local camera's description file (an XML file stored on your computer). Streamable features are typically not persistent values (such as pixel format and image size). In most cases, a streamable feature is capable of being saved and loaded from file at a later date, possibly on a different camera.			e										b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e										b
	M_FALSE	Specifies that the feature is not streamable.			e										b
	M_TRUE	Specifies that the feature is streamable.			e										b
M_FEATURE_TOOLTIP +		Inquires the tooltip of the specified feature.			e										b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e										b
	Value	Specifies the feature's tooltip.			e										b
M_FEATURE_TYPE		Inquires the specified feature's type.			e										b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e										b
	M_TYPE_BOOLEAN	Specifies that the feature's value is a boolean.			e										b
	M_TYPE_CATEGORY	Specifies that the feature is a category feature, and its value cannot be inquired.			e										b
	M_TYPE_COMMAND	Specifies that the feature is a command and its value cannot be inquired.			e										b
	M_TYPE_DOUBLE	Specifies that the feature's value is a floating-point.			e										b
	M_TYPE_ENUMERATION	Specifies that the feature's value is an enumeration.			e										b
	M_TYPE_INT64	Specifies that the feature's value is a 64-bit integer.			e										b

	M_TYPE_REGISTER	Specifies that the feature's value is mapped to a multi-byte register.			e											b
	M_TYPE_STRING	Specifies that the feature's value is a string.			e											b
M_FEATURE_VALUE +		Inquires the current value of the feature.			e											b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_BOOL* **|** *address of an AIL_DOUBLE* **|** *address of an AIL_INT64* **|** *array of type AIL_UINT8 [optionally, in C++: a reference to a std::vector<AIL_UINT8>]* **|** *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
	Value	Specifies the feature's value.			e											b
M_FEATURE_VALUE_ARRAY		Inquires the array of values of the specified feature. The array of values will be inquired without needing to be indexed by a selector. This is internally handled by Aurora Imaging Library and simplifies implementation.			e											b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_BOOL [optionally, in C++: a reference to a std::vector<AIL_BOOL>]* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*			e											b
M_FEATURE_VALUE_ARRAY_SIZE		Inquires the number of elements in the specified feature's array of values. For use with M_FEATURE_VALUE_ARRAY.			e											b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e											b
	Value >= 0	Specifies the number of elements in the feature array.			e											b

M_FEATURE_VISIBILITY	Inquires the visibility level of the specified feature. The visibility level indicates the level of expertise required to view or select the feature. When creating a custom user interface for the camera's device description file (XML), the visibility level of the specified feature can be used to potentially hide the specified feature. The current level of visibility should be set within your custom user interface, and the enumeration entry for the visibility level of the specified feature must be compared against it. If the specified enumeration entry has a level of visibility higher than the current level of visibility, it is hidden.			e												b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e												b
M_FEATURE_VISIBILITY_BEGINNER	Specifies that the feature is suggested for beginners.			e												b
M_FEATURE_VISIBILITY_EXPERT	Specifies that the feature is suggested for experts.			e												b
M_FEATURE_VISIBILITY_GURU	Specifies that the feature is suggested for advanced experts.			e												b
M_FEATURE_VISIBILITY_INVISIBLE	Specifies that the feature should not be shown in the user interface for the camera's device description file (XML). Note that an invisible feature can still be set or inquired using MbufControlFeature() and **MbufInquireFeature**(), respectively.			e												b

When the specified feature is of type enumeration, you can use one of the following to inquire about the specified enumeration entry in the supported enumeration list of the feature. When the information pertains to a specific enumeration entry (for example, M_FEATURE_ENUM_ENTRY_ACCESS_MODE + n), the value is described in relation to its position in the enumerated list, where *n* is the index into the enumerated list.

> Note that these values can only be used when the feature type to inquire is an enumeration.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the enumeration entry of a feature's value																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_FEATURE_ENUM_ENTRY_ACCESS_MODE + *n*		Inquires whether the specified enumeration entry is implemented, available, readable, and/or writable, where *n* is the index into the enumerated list. Note that an error is generated if the enumeration entry does not exist in the camera's device description file (XML). To learn if the specified enumeration entry exists in the file and is implemented on your camera, use the **M_FEATURE_IS_IMPLEMENTED** macro.													
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e										b
	Bit-encoded access mode value	Specifies a bit-encoded value that details the access mode. To establish the access mode from the encoded value, use the following macros: • To learn whether the specified enumeration entry is available, use the **M_FEATURE_IS_AVAILABLE** macro. Note that if an enumeration entry is not available, use **M_FEATURE_IS_IMPLEMENTED** to learn whether the specified enumeration entry is only temporarily unavailable or actually not implemented on your camera. An enumeration entry that is temporarily unavailable typically relies on the setting of another feature or enumeration entry. • To learn whether the specified enumeration entry is implemented, use the **M_FEATURE_IS_IMPLEMENTED** macro. • To learn whether the specified enumeration entry can be read, use the **M_FEATURE_IS_READABLE** macro.			e										b
M_FEATURE_ENUM_ENTRY_COUNT		Inquires the total number of enumeration entries in the supported enumerated list of the feature. Note that this inquire type can only be used when the feature to inquire is of type enumeration.			e										b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e										b
	Value	Specifies the total number of enumeration entries.			e										b

M_FEATURE_ENUM_ENTRY_DESCRIPTION + *n* +	Inquires the description of the specified enumeration entry, where *n* is the index into the enumerated list. Note that this inquire type can only be used when the feature type to inquire is an enumeration.			e											b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
Value	Specifies the enumeration entry's description.			e											b
M_FEATURE_ENUM_ENTRY_DISPLAY_NAME + *n* +	Inquires the display name of the specified enumeration entry, where *n* is the index into the enumerated list. Note that this can differ slightly from the internally-used name of the entry (M_FEATURE_ENUM_ENTRY_NAME + n).			e											b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
Value	Specifies the enumeration entry's display name.			e											b
M_FEATURE_ENUM_ENTRY_NAME + *n* +	Inquires the internally-used name of the specified enumeration entry of the feature, where *n* is the index into the enumerated list. Note that this can differ slightly from the display name of the entry (M_FEATURE_ENUM_ENTRY_DISPLAY_NAME + n).			e											b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
Value	Specifies the enumeration entry's name.			e											b
M_FEATURE_ENUM_ENTRY_TOOLTIP + *n* +	Inquires the tool-tip for the specified enumeration entry, where *n* is the index into the enumerated list.			e											b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
Value	Specifies the enumeration entry's tooltip.			e											b
M_FEATURE_ENUM_ENTRY_VALUE + *n*	Inquires the enumeration integer value of the specified enumeration entry, where *n* is the index into the enumerated list.			e											b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e													b
	Value	Specifies the enumeration entry's integer value.			e													b
M_FEATURE_ENUM_ENTRY_VISIBILITY + *n*		Inquires the visibility level of the specified enumeration entry, where *n* is the index into the enumerated list. The visibility level indicates the level of expertise required to view or select the enumeration entry. When creating a custom user interface for the camera's device description file (XML), the visibility level of the specified enumeration entry can be used to potentially hide the specified enumeration entry. The current level of visibility should be set within your custom user interface, and the visibility level of the specified enumeration entry must be compared against it. If the specified enumeration entry has a level of visibility higher than the current level of visibility, it is hidden.			e												b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e													b
	M_FEATURE_VISIBILITY_BEGINNER	Specifies that the enumeration entry is suggested for beginners.			e													b
	M_FEATURE_VISIBILITY_EXPERT	Specifies that the enumeration entry is suggested for experts.			e													b
	M_FEATURE_VISIBILITY_GURU	Specifies that the enumeration entry is suggested for advanced experts.			e													b
	M_FEATURE_VISIBILITY_INVISIBLE	Specifies that the enumeration entry should not be shown in the user interface for the camera's device description file (XML). Note that an invisible feature can still be set or inquired using MbufControlFeature() and **MbufInquireFeature**(), respectively.			e													b

When the specified feature is of type category or "Root", you can use one of the following to inquire about its subfeatures. This is useful for enumerating the features of your camera.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the subfeatures of a feature																		
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																	

M_SUBFEATURE_COUNT		Inquires the number of subfeatures or subcategories that the specified category feature (node) has.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*			e											b
	Value	Specifies the number of subfeatures.			e											b
M_SUBFEATURE_NAME + *n* +		Inquires the name of the specified subfeature, where *n* is the index of subfeature.			e											b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*			e											b
	Value	Specifies the name of the subfeature.			e											b
M_SUBFEATURE_TYPE + *n*		Inquires the type of the specified subfeature, where *n* is the index of subfeature.			e											b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*			e											b
	M_TYPE_BOOLEAN	Specifies that the feature's value is a boolean.			e											b
	M_TYPE_CATEGORY	Specifies that the feature is a category.			e											b
	M_TYPE_COMMAND	Specifies that the feature is a command to be executed.			e											b
	M_TYPE_DOUBLE	Specifies that the feature's value is a floating-point.			e											b
	M_TYPE_ENUMERATION	Specifies that the feature's value is an enumeration.			e											b
	M_TYPE_INT64	Specifies that the feature's value is a 64-bit integer.			e											b
	M_TYPE_REGISTER	Specifies that the feature is mapped to a multi-byte register.			e											b
	M_TYPE_STRING	Specifies that the feature's value is a string.			e											b

Combination value for M_FEATURE_DESCRIPTION; M_FEATURE_DISPLAY_NAME; M_FEATURE_NAME; M_FEATURE_SELECTOR_NAME + n; M_FEATURE_TOOLTIP; M_FEATURE_VALUE; M_FEATURE_ENUM_ENTRY_DESCRIPTION + n; M_FEATURE_ENUM_ENTRY_DISPLAY_NAME + n; M_FEATURE_ENUM_ENTRY_NAME + n; M_FEATURE_ENUM_ENTRY_TOOLTIP + n; M_SUBFEATURE_NAME + n.

You can add the following value to the above-mentioned values to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

To inquire the size of a string			(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InquireType combination value		Description															
	UserVarPtr - Possible values returned																
M_STRING_SIZE		Retrieves the length of the string, including the terminating null character ("\0"). Note that, when used with M_FEATURE_VALUE, you can only use this combination value if the feature is of type string.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*			e												b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufLink

Synopsis

Link one buffer or container to another.

Syntax

```
void MbufLink(
    AIL_ID SrcContainerOrBufId,     //in
    AIL_ID TargetContainerOrBufId, //out
    AIL_INT64 LinkOperation,        //in
    AIL_INT ControlFlag             //in
)
```

Description

This function links two buffers or containers so that subsequent modifications in the source are copied to the target until the buffers/containers are unlinked. For instance, when using the Distributed Aurora Imaging Library monitoring configuration, you can link a publishing application's image buffer to the monitoring application's image buffer. Any modifications to the published buffer will be copied to the local buffer.

Parameters

SrcContainerOrBufId

Specifies the identifier of the source buffer or container. This buffer or container remains independent and can be modified as per the application's standard mechanisms.

TargetContainerOrBufId

Specifies the identifier of the target buffer or container. The contents of this buffer or container will have the same modifications applied to it as are applied to SrcContainerOrBufId.

LinkOperation

Specifies the link operation to perform.

To specify the link operation	
Value	Description
M_LINK	Specifies to link the two specified buffers.
M_UNLINK	Specifies to unlink the two specified buffers.

ControlFlag

Specifies whether to convert 3D information copied from the source container. If unused, this parameter must be set to **M_DEFAULT**.

For specifying to compensate for missing information in the source container	
Value	Description

M_COMPENSATE	Specifies to automatically convert the information copied from the source container to the destination container so that the destination container is 3D-processable (if the destination container has the attribute M_PROC) and/or 3D-displayable (if the destination container has the attribute M_DISP). This is equivalent to using MbufConvert3d() with M_COMPENSATE. > An error will be generated any time the source container is modified and its contents cannot be copied because it cannot be converted. > Note that this setting is only available when ail3d<version number>.dll is present.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib;.
DLL	Requires ail11.dll, ail3d11.dll for 3D compensation.

MbufLoad

Synopsis

Load data from a file into a data buffer or container.

Syntax

```
void MbufLoad(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContainerOrBufId      //out
)
```

Description

This function loads data from a file into a previously allocated data buffer or container. The function detects the file format from the data.

You can perform the same operation as **MbufLoad**() using MbufImport(), which uses the specified file format to open the file instead of trying to determine the format from the data.

When loading an image file, all the pixel information in the file is loaded in the buffer. Additionally, if the file was saved in M_AIL_TIFF file format and includes any region of interest (ROI) information, the ROI information is also loaded in the buffer. Note that if the vectorial information of an M_VECTOR_AND_RASTER ROI was input in M_WORLD units, the ROI will be loaded as an M_VECTOR ROI.

When loading an image file that has been saved with an associated LUT (color palette) into a 3-band 8-bit image buffer, the LUT is automatically applied to the data to generate 3-band image data. In this case, a LUT buffer is not created and, therefore, is not associated with the 3-band 8-bit buffer. When loading an image file that has been saved with an associated LUT (color palette) into any other type of image buffer, the LUT is also imported and associated with the resulting image buffer. You can obtain the identifier of the associated LUT, using MbufInquire(). Note that the associated LUT will be automatically selected on the display (MdispLut()) if the image buffer is selected on a display and the default LUT has not been overidden by a former call to MdispLut().

When loading an image file into a container, the destination's container components are removed and a new component is allocated containing the image data from the specified source file.

When loading a container file, the destination's container components are removed and one or more components are allocated containing the component data from the specified source container file.

Using MbufDiskInquire(), you can inquire about the dimensions of the data saved in a file (except for raw files, M_GENDC files, and M_AIL_NATIVE files that store a container) without loading it.

[Aurora Imaging Library Lite with restriction]

When loading compressed data and the destination buffer has an M_IMAGE attribute (but not an M_COMPRESS attribute), this function will automatically decompress it. If necessary, the data in the file will be transformed to the format of the buffer. When loading uncompressed data into a buffer with an M_COMPRESS attribute, this function will automatically compress it, according to the compression settings found in the buffer.

Parameters

FileName

Specifies the name and path of the file from which to load the data. The function handles (internally) the opening and closing of the file.

This parameter can be set to the following value:

For specifying the file name and path	
Value	Description
AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file, for example, "*C:\mydirectory\myfile*". Typically, data buffers have a MIM file extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").
AIL_TEXT("*ftp://user:password@server/pathtofile*") [1]	Load data from a remote File Transfer Protocol (FTP) server into a data buffer. To access a remote FTP server anonymously, omit the user name and password. > FTP-SSL (FTPS) is not supported.
AIL_TEXT("*http://user:password@server/pathtofile*") [1]	Load data from a remote HTTP server into a data buffer. Import data from a remote HTTP server into a data buffer. HTTPS is not supported.
AIL_TEXT("*sftp://user:password@server/pathtofile*") [1]	Load data from a remote SSH File Transfer Protocol (SFTP) server into a data buffer. > FTP-SSL (FTPS) is not supported.

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContainerOrBufId

Specifies the identifier of the destination buffer or container. For the data buffer, if the data is deeper than the buffer, the most-significant bits of the data are truncated when loaded into the buffer. If the buffer depth is greater than that of the data, the data is zero or sign-extended (depending on the data type) when loaded into the buffer. If the buffer is larger in size than the data, exceeding areas of the buffer are unaffected. If the buffer is smaller in size than the data, the data will be cropped. For containers, the component's size will always reflect what is in the file.

Remark

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_IMAGE + M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufPut1d

Synopsis

Put data from a user-supplied array into a one-dimensional area of a buffer.

Syntax

```
void MbufPut1d(
    AIL_ID DestBufId,        //out
    AIL_INT OffX,            //in
    AIL_INT SizeX,           //in
    const void *UserArrayPtr //in
)
```

Description

This function copies data from a user-supplied array to a specific one-dimensional area of the specified Aurora Imaging Library destination buffer. If the destination buffer has multiple bands, Aurora Imaging Library copies the data from the user-supplied array to the destination buffer sequentially. The internal data format of the user-supplied array need not match the data format of the Aurora Imaging Library destination buffer (planar); an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster. For more information, refer to MbufPutColor2d() with M_PLANAR.

Parameters

DestBufId

Specifies the identifier of the destination buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

OffX

Specifies the horizontal offset of the destination buffer area in which to put data, relative to the destination buffer's top-left pixel.

SizeX

Specifies the width of the destination buffer area in which to put the data.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:(SizeX) * MbufInquire (M_SIZE_BAND)

Specifies the address of the user array from which to put data into the destination buffer. Ensure that user array is large enough to contain the data to be copied into the destination buffer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufPut2d

Synopsis

Put data from a user-supplied array into a two-dimensional area of a buffer.

Syntax

```
void MbufPut2d(
    AIL_ID DestBufId,        //out
    AIL_INT OffX,            //in
    AIL_INT OffY,            //in
    AIL_INT SizeX,           //in
    AIL_INT SizeY,           //in
    const void *UserArrayPtr //in
)
```

Description

This function copies data from a user-supplied array to a two-dimensional area of the specified Aurora Imaging Library destination buffer. If the user-supplied array has multiple bands, Aurora Imaging Library copies the data from the user-supplied array to the destination buffer sequentially. The internal data format of the user-supplied array need not match the data format of the Aurora Imaging Library destination buffer (planar); an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster. For more information, refer to MbufPutColor2d() with M_PLANAR.

Parameters

DestBufId

Specifies the identifier of the destination buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

OffX

Specifies the horizontal offset of the destination buffer area in which to put the data, relative to the destination buffer's top-left pixel.

OffY

Specifies the vertical offset of the destination buffer area in which to put the data, relative to the destination buffer's top-left pixel.

SizeX

Specifies the width of the destination buffer area in which to put the data.

SizeY

Specifies the height of the destination buffer area in which to put the data.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:(SizeX) * MbufInquire (M_SIZE_BAND) * (SizeY)

Specifies the address of the user array from which to copy data into the destination buffer. Ensure that user array is large enough to contain the data to be copied to the destination buffer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufPut

Synopsis

Put data from a user-supplied array into a data buffer.

Syntax

```
void MbufPut(
    AIL_ID DestBufId,        //out
    const void *UserArrayPtr //in
)
```

Description

This function copies data from a user-supplied array to a specified Aurora Imaging Library destination buffer. If the destination buffer has multiple bands, Aurora Imaging Library copies the data from the user-supplied array to the destination buffer sequentially. The internal data format of the user-supplied array need not match the data format of the Aurora Imaging Library destination buffer (planar); an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster. For more information, refer to MbufPutColor() with M_PLANAR.

Parameters

DestBufId

Specifies the identifier of the destination buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size: MbufInquire (M_SIZE_X) * MbufInquire (M_SIZE_Y) * MbufInquire (M_SIZE_BAND)

Specifies the address of the user array from which to put data into the destination buffer. Ensure that user array is large enough to contain the data to be copied to the destination buffer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufPutColor2d

Synopsis

Put data from a user-supplied array into an area of one or all bands of a data buffer.

Syntax

```
void MbufPutColor2d(
    AIL_ID DestBufId,        //out
    AIL_INT64 DataFormat,    //in
    AIL_INT Band,            //in
    AIL_INT OffX,            //in
    AIL_INT OffY,            //in
    AIL_INT SizeX,           //in
    AIL_INT SizeY,           //in
    const void *UserArrayPtr //in
)
```

Description

This function copies data from a user-supplied array to a specified area in one or all bands of a specified Aurora Imaging Library destination buffer.

Parameters

DestBufId

Specifies the identifier of the destination buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

DataFormat

Specifies the data format of the user-supplied array. The internal data format of the destination buffer need not match the specified data format of the user-supplied array; an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster.

See the Parameter associations section for possible values that can be specified.

Band

Specifies the index of the band in which to copy. This parameter can be set to one of the values below.

For specifying the index of the band	
Value	Description
M_ALL_BANDS	Copies to all bands.
M_BLUE	Copies to the blue band (for RGB buffers).
M_GREEN	Copies to the green band (for RGB buffers).
M_HUE	Copies to the hue band (for HSL, and HSV buffers).

M_LUMINANCE	Copies to the luminance band (for HSL buffers).
M_RED	Copies to the red band (for RGB buffers).
M_SATURATION	Copies to the saturation band (for HSL and HSV buffers).
M_V	Specifies the V band (for HSV buffers).
0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between index value and band for RGB, HSL, and HSV buffers is the following: 0 — Corresponds to the red band for RGB buffers, or the hue band for HSL and HSV buffers. 1 — Corresponds to the green band for RGB buffers, or the saturation band for HSL and HSV buffers. 2 — Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for HSV buffers.

OffX

Specifies the horizontal offset of the destination buffer area in which to put the data, relative to the destination buffer's top-left coordinate.

OffY

Specifies the vertical offset of the destination buffer area in which to put the data, relative to the destination buffer's top-left coordinate.

SizeX

Specifies the width of the destination buffer area in which to put the data.

SizeY

Specifies the height of the destination buffer area in which to put the data.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
- array of type unsigned-char [optionally, in C++: a reference to a constant std::vector<unsigned-char>]

Specifies the address of the user array from which to copy data into the destination buffer. Ensure that user array is large enough to contain the data to be copied to the destination buffer.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **DataFormat** and **UserArrayPtr** parameters.

- For specifying the data format

Note that *SizeX* and *SizeY* denote the destination buffer's width and height, respectively.

For specifying the data format	
DataFormat / UserArrayPtr	Description
M_PACKED +	Copies the bands in an interleaved manner (for example, RGB RGB RGB...). This format is only valid when the Band parameter is set to M_ALL_BANDS. You must specify a combination value from the following table: • To set the packed data format
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a constant std::vector<unsigned-char>]*
M_PLANAR	Copies the bands one after the other. This format is to be used when copying to all bands (M_ALL_BANDS) of the destination buffer.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*
M_SINGLE_BAND	Copies to a single band. This format is not valid when the Band parameter is set to M_ALL_BANDS.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*

Combination values for M_PACKED.

You must add one of the following values to the above-mentioned value to set the packed data format.

For more information on these formats, see the RGB buffers section of Chapter 23: Data buffers.

For setting the packed data format	
DataFormat / UserArrayPtr	Description
M_BGR24	Stores the data in a BGR24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a constant std::vector<unsigned-char>]*
M_BGR32	Stores the data in a BGR32 packed format. In this format, each pixel is internally stored as four consecutive bytes. Every fourth byte is a "don't care" byte.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a constant std::vector<unsigned-char>]*

M_RGB15	Stores the data in a RGB15 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 5-bit green value, a 5-bit red value, and a "don't care" bit (most significant), in little-endian order (RGB 5:5:5).
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a constant std::vector<unsigned-char>]*
M_RGB16	Stores the data in a RGB16 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 6-bit green value, and a 5-bit red value (most significant), in little-endian order (RGB 5:6:5).
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a constant std::vector<unsigned-char>]*
M_RGB24	Stores the data in a RGB24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
UserArrayPtr extra info	Data type info: Data type: *array of type unsigned-char [optionally, in C++: a reference to a constant std::vector<unsigned-char>]*

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufPutColor

Synopsis

Put data from a user-supplied array into one or all bands of a data buffer.

Syntax

```
void MbufPutColor(
    AIL_ID DestBufId,        //out
    AIL_INT64 DataFormat,    //in
    AIL_INT Band,            //in
    const void *UserArrayPtr //in
)
```

Description

This function copies data from a user-supplied array to one or all bands of a specified Aurora Imaging Library destination buffer.

Parameters

DestBufId

Specifies the identifier of the destination buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

DataFormat

Specifies the data format of the user-supplied array. The internal data format of the user-supplied array need not match the data format of the Aurora Imaging Library destination buffer (planar); an internal conversion will be performed if necessary. Note, however, if the formats do match, the operation will be much faster.

See the Parameter associations section for possible values that can be specified.

Band

Specifies in which band to put the data. Set this parameter to one of the following values:

For specifying the band	
Value	Description
M_ALL_BANDS	Specifies all bands (for multi-band buffers).
M_BLUE	Specifies the blue color band (for RGB buffers).
M_GREEN	Specifies the green color band (for RGB buffers).
M_HUE	Specifies the hue band (for HSL and HSV buffers).
M_LUMINANCE	Specifies the luminance band (for HSL buffers).

M_RED	Specifies the red color band (for RGB buffers).
M_SATURATION	Specifies the saturation band (for HSL and HSV and HSV buffers).
M_U	Specifies the U band (for YUV buffers).
M_V	Specifies the V band (for YUV and HSV buffers).
M_Y	Specifies the Y band (for YUV buffers).
0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between index value and band for RGB, HSL, HSV, and YUV buffers is the following: 0 — Corresponds to the red band for RGB buffers, the hue band for HSL and HSV buffers, or the Y band for YUV buffers. 1 — Corresponds to the green band for RGB buffers, or the saturation band for HSL and HSV buffers, or the U band for YUV buffers. 2 — Corresponds to the blue band for RGB buffers, or the luminance band for HSL buffers, or the V band for YUV and HSV buffers.

UserArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]
- array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]

Specifies the address of the user array from which to copy data into the destination buffer. Ensure that user array is large enough to contain the data to be copied to the destination buffer.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **DataFormat** and **UserArrayPtr** parameters.

- For specifying the data format

Note that *SizeX* and *SizeY* denote the destination buffer's width and height, respectively.

For specifying the data format	
DataFormat	Description
UserArrayPtr	

M_PACKED +	Copies the bands in an interleaved manner (for example, RGB RGB RGB...). This format is only valid when the Band parameter is set to M_ALL_BANDS. You must specify a combination value from the following table: • To set the packed data format
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]*
M_PLANAR	Copies the bands one after the other. This format is to be used when copying to all bands (M_ALL_BANDS) of the destination buffer.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*
M_SINGLE_BAND	Copies to a single band. This format is not valid when the Band parameter is set to M_ALL_BANDS.
UserArrayPtr extra info	Data type info: Data type: *array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]*

Combination values for M_PACKED.

You must add one of the following values to the above-mentioned value to set the packed data format.

For more information on these formats, see the RGB buffers section of Chapter 23: Data buffers.

For setting the packed data format	
DataFormat UserArrayPtr	Description
M_BGR24	Stores the data in a BGR24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]*
M_BGR32	Stores the data in a BGR32 packed format. In this format, each pixel is internally stored as four consecutive bytes. Every fourth byte is a "don't care" byte.
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]*
M_RGB15	Stores the data in a RGB15 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 5-bit green value, a 5-bit red value, and a "don't care" bit (most significant), in little-endian order (RGB 5:5:5).
UserArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]*
M_RGB16	Stores the data in a RGB16 packed format. In this format, each pixel is internally stored as a 16-bit word with a 5-bit blue value (least significant), a 6-bit green value, and a 5-bit red value (most significant), in little-endian order (RGB 5:6:5).

	UserArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]*
M_RGB24		Stores the data in a RGB24 packed format. In this format, each pixel is internally stored as three consecutive bytes.
	UserArrayPtr extra info	Data type info: Data type: *array of type AIL_UINT8 [optionally, in C++: a reference to a constant std::vector<AIL_UINT8>]*

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufPutLine

Synopsis

Write a specified series of values along a specified theoretical line in an image.

Syntax

```
void MbufPutLine(
    AIL_ID ImageBufId,       //out
    AIL_INT StartX,          //in
    AIL_INT StartY,          //in
    AIL_INT EndX,            //in
    AIL_INT EndY,            //in
    AIL_INT64 Mode,          //in
    AIL_INT *NbPixelsPtr,    //out
    const void *UserArrayPtr //in
)
```

Description

This function reads pixel values from a user-defined array and writes them to the series of pixels, in the specified image, along the theoretical line defined by specified coordinates. The Bresenham algorithm is used to determine the theoretical line. The line can start and end outside of the buffer. The first pixel value in the array will be written to the first valid pixel in the buffer. If the user-array contains more entries than there are valid pixels on the theoretical line in the buffer, the excess entries will be ignored. If the user-array is smaller than the number of valid pixels on the theoretical line, an error will occur.

Parameters

ImageBufId

Specifies the identifier of the destination image buffer. This must be a single-band (monochrome) buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

StartX

Specifies the horizontal pixel offset of the starting position of the line, relative to the top-left pixel of the destination buffer.

StartY

Specifies the vertical pixel offset of the starting position of the line, relative to the top-left pixel of the destination buffer.

EndX

Specifies the horizontal pixel offset of the finishing position on the line, relative to the top-left pixel of the destination buffer.

EndY

Specifies the vertical pixel offset of the finishing position on the line, relative to the top-left pixel of the destination buffer.

Mode

Specifies the operation mode. Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

NbPixelsPtr

Specifies the address of the variable in which to write the number of pixels found along the theoretical line. You can set this parameter to **M_NULL** if you don't want this value to be evaluated.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_NULL** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (ImageBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:Call **MbufPutLine**() with UserArrayPtr set to **M_NULL**. The array size is stored in the NbPixelsPtr parameter. Note: The array size is determined by calling the function twice.

Specifies the address of the user array containing the pixels to write to the image buffer. Ensure that the user array is at least as large as the value returned by NbPixelsPtr. To determine the number of pixel values required, you can set this parameter to **M_NULL** and pass a non-null address to NbPixelsPtr, which will store the size of the required array. In this case, nothing is written to the image buffer.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufPutList

Synopsis

Overwrite the values in the destination buffer at the specified positions, with values from a user-defined array.

Syntax

```
void MbufPutList(
    AIL_ID DestBufId,               //out
    AIL_INT NumPixels,              //in
    const AIL_DOUBLE *PixXArrayPtr, //in
    const AIL_DOUBLE *PixYArrayPtr, //in
    AIL_INT64 OverscanMode,         //in
    const void *UserArrayPtr        //in
)
```

Description

This function overwrites the values in the destination buffer at the specified positions, with values taken from a user-defined array. If non-integer positions are given, the values are written to the closest integer positions.

When executing the operation on a buffer that has multiple bands, the bands are processed sequentially. For example, with a planar RGB buffer, the red band is processed first; the red values at the specified positions are replaced by those in the first third of the user-defined array. Then, the green band is processed; the green values at the specified positions are replaced by those in the second third of the user-defined array (immediately following the red values). Finally, the blue band is processed; the blue values at the specified positions are replaced by those in the last third of the user-defined array (immediately following the green values). If you want to overwrite, for example, 3 pixels, the function will process the red band of each pixel, then green band, and finally the blue band (RRR, GGG, BBB not RGB, RGB, RGB).

> Note that if the destination buffer has multiple bands, its format should match the implicit format of the user array; otherwise, the operation is executed using a temporary buffer and then an internal conversion has to be done (from the temporary buffer to the destination). Likewise, if the destination buffer is compressed, the operation is executed using a temporary buffer and then an internal compression has to be done (from the temporary buffer to the destination).

> Note that presented above is an example prototype of this function. You can call MbufPutList() with the same prototype as any of the type-specific functions below, without using the function's type-specific name; that is, use MbufPutList() and not, for example, MbufPutListInt32(). The data is automatically cast into the appropriate type. This is true except when using a C compiler. When using a C compiler, you must always call this function with its type-specific name (you cannot use the standard name of this function).

Parameters

DestBufId

Specifies the identifier of the destination buffer. This buffer can be a single-band (monochrome) or multi-band buffer and is not limited to image buffers.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

NumPixels

Specifies the number of values to put into the destination buffer.

> When using a standard vector (std::vector) overload function in C++, you can pass **M_DEFAULT** to this parameter; the size will be determined automatically based on the number of items in the vector passed to the parameter.

PixXArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPixels)
- array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]

Specifies the address of an array containing the X-coordinate of the positions in the buffer to overwrite.

PixYArrayPtr
Accepts the address of one of the following:

- array of type AIL_DOUBLE [optionally, in C++: a reference to a constant std::vector<AIL_DOUBLE>] (and a maximum array rank for C# of 1)
  Required array size:(NumPixels)
- array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]

Specifies the address of an array containing the Y-coordinate of the positions in the buffer to overwrite.

OverscanMode

Specifies the overscan mode. This specified mode controls how the function behaves with positions that fall outside the destination buffer.

This parameter must be set to one of the values below.

For specifying the overscan mode	
Value	Description
M_DEFAULT	Same as M_OVERSCAN_ENABLE.
M_OVERSCAN_DISABLE	Specifies that if a position falls outside the destination buffer, the corresponding entry in the user-defined array is not used.
M_OVERSCAN_ENABLE	Specifies that if a position falls outside the destination buffer, the function will try to modify the destination's ancestor (if applicable), if the given position is within its limits. Otherwise, the corresponding entry in the user-defined array is not used.

UserArrayPtr
Accepts the following address:

- Data type: array of the same type as the buffer (DestBufId) [optionally, in C++: a reference to a std::vector of the same type as the buffer]
  Required array size:(NumPixels) * MbufInquire (M_SIZE_BAND)

Specifies the address of a user-defined array which contains values to write into the destination buffer. The data type of this array must match the data type of the destination buffer. Its number of entries is determined by the number of values to put and also by the number of bands of the destination buffer.

Type-specific versions of the function when using a C compiler

```
void MbufPutListDouble (AIL_ID DestBufId, AIL_INT NumPixels, const AIL_DOUBLE *PixXArrayPtr, const
AIL_DOUBLE *PixYArrayPtr, AIL_INT64 OverscanMode, const void *UserArrayPtr)
```

Details

Parameters

DestBufId

See DestBufId of the main function for a description.

NumPixels

See NumPixels of the main function for a description.

PixXArrayPtr

See PixXArrayPtr of the main function for a description.

PixYArrayPtr

See PixYArrayPtr of the main function for a description.

OverscanMode

See OverscanMode of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

```
void MbufPutListInt32 (AIL_ID DestBufId, AIL_INT NumPixels, const AIL_INT32 *PixXArrayPtr, const AIL_INT32 *PixYArrayPtr, AIL_INT64 OverscanMode, const void *UserArrayPtr)
```

Details

Parameters

DestBufId

See DestBufId of the main function for a description.

NumPixels

See NumPixels of the main function for a description.

PixXArrayPtr

See PixXArrayPtr of the main function for a description.

PixYArrayPtr

See PixYArrayPtr of the main function for a description.

OverscanMode

See OverscanMode of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

```
void MbufPutListInt64 (AIL_ID DestBufId, AIL_INT NumPixels, const AIL_INT64 *PixXArrayPtr, const AIL_INT64 *PixYArrayPtr, AIL_INT64 OverscanMode, const void *UserArrayPtr)
```

Details

Parameters

DestBufId

See DestBufId of the main function for a description.

NumPixels

See NumPixels of the main function for a description.

PixXArrayPtr

See PixXArrayPtr of the main function for a description.

PixYArrayPtr

See PixYArrayPtr of the main function for a description.

OverscanMode

See OverscanMode of the main function for a description.

UserArrayPtr

See UserArrayPtr of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufRestore

Synopsis

Restore data from disk into an automatically allocated data buffer or container.

Syntax

```
AIL_ID MbufRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SystemId,             //in
    AIL_ID *ContainerOrBufIdPtr  //out
)
```

Description

This function restores the data that was previously saved to a file and loads it into an automatically allocated buffer or container. It tries to detect the file format from the data.

This function will allocate the destination buffer or container with the same attributes as the original buffer or container, with the exception of an M_IMAGE buffer.

In the case of an M_IMAGE buffer, the **MbufRestore**() function tries to allocate the buffer so that it can be used for acquisition (M_GRAB), display (M_DISP), and processing (M_PROC) operations. If the buffer cannot be allocated with the M_GRAB attribute, Aurora Imaging Library allocates one that can be used in all of the above operations except for acquisition (M_GRAB).

When restoring a container, this function allocates a container that can be used for display (M_DISP) and processing (M_PROC) operations.

If restoring an image, all the pixel information in the file is restored and loaded into the buffer. Additionally, if the file was saved in M_AIL_TIFF file format and includes any region of interest (ROI) information, the ROI information is also restored and loaded into the buffer.

> If the vectorial information of an M_VECTOR_AND_RASTER ROI was input in M_WORLD units, the ROI will be restored as an M_VECTOR ROI.

[Aurora Imaging Library Lite with restriction]

When restoring compressed data, the buffer will have an M_COMPRESS attribute.

When restoring an image file that was saved with an associated LUT (color palette), the LUT is also restored and associated with the restored image buffer. You can obtain the identifier of the associated LUT, using MbufInquire().

Note, you can perform the same operation as **MbufRestore**() using MbufImport(), which uses the specified file format to restore the data instead of trying to determine the format from the data.

After restoring the buffer or container, you should check if the operation was successful, using MappGetError() or by verifying that the buffer or container identifier returned is not **M_NULL** (or *nullptr* if M_UNIQUE_ID was specified).

When the restored buffer or container is no longer required, release it using MbufFree() unless M_UNIQUE_ID was specified during restoration; if M_UNIQUE_ID was specified, the smart identifier manages the buffer or container's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the data buffer or container. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

For specifying the file name and path	
Value	Description
M_INTERACTIVE	Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example "*C:\mydirectory\myfile*"). Typically, data buffer files can contain any of the file extensions listed in MbufImport(). To specify the file on the remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").
AIL_TEXT("*ftp://user:password@server/pathtofile*") 1	Restore data from a remote File Transfer Protocol (FTP) server into an automatically allocated data buffer. To access a remote FTP server anonymously, omit the user name and password. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be restored using an FTP file path.
AIL_TEXT("*http://user:password@server/pathtofile*") 1	Restore data from a remote HTTP server into an automatically allocated data buffer. HTTPS is not supported.
AIL_TEXT("*sftp://user:password@server/pathtofile*") 1	Restore data from a remote SSH File Transfer Protocol (SFTP) server into an automatically allocated data buffer. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be restored using an SFTP file path.

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SystemId

Specifies the system on which to allocate the buffer or container.

This parameter should be set to one of the following values:

For specifying the system	
Value	Description

M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().

ContainerOrBufIdPtr

Specifies the address of the variable in which to write the buffer or container identifier or specifies the data type that the function should use to return the buffer or container identifier.

For retrieving the identifier or specifying how to return it	
Value	Description
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer or container ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated buffer or container ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_BUF_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the buffer or container (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.
Address in which to write the buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored buffer. If restoration fails, **M_NULL** is written as the identifier.
Address in which to write the container identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored container. If restoration fails, **M_NULL** is written as the identifier.

Return value

The returned value is the buffer or container identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_BUF_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Remark

- *[Aurora Imaging Library Lite]*
  Note that during development and at runtime, compression support, particularly for an M_IMAGE + M_COMPRESS buffer type, requires the presence of an Aurora Imaging Library license that grants access to the compression/decompression package. This access is only granted by default with the development license dongle for the full version of Aurora Imaging Library. In other cases, you must purchase access to this package separately.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib;.
DLL	Requires ail11.dll, ail3d11.dll for some formats.

MbufSave

Synopsis

Save a data buffer or container in a file, using an Aurora Imaging Library output file format.

Syntax

```
void MbufSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContainerOrBufId      //in
)
```

Description

This function saves a previously allocated data buffer or container to a file, in an Aurora Imaging Library file format.

If a container is specified, it is saved in the Aurora Imaging Library native file format. This format stores all data and settings of the container and its components.

If a buffer is specified, it is saved in the Aurora Imaging Library TIFF file format. This is a baseline TIFF 6.0 file format with extra information included in the comment field. The extra information includes the buffer attributes and data type.

The Aurora Imaging Library TIFF file format supports images that are binary, grayscale, and RGB, as well as palette-color images. Images in a different format are internally converted before being saved. In addition, by default, most color image buffers are saved in a packed (chunky) format (in accordance with baseline TIFF 6.0 specifications). Color binary buffers are saved in a 1-bit per pixel format (data is stored in a 3-band, packed binary format).

When saving an image buffer (M_IMAGE), all the pixel information is stored in the file. In addition, if the image buffer includes any region of interest (ROI) information, set using MbufSetRegion(), the ROI information is also stored in the file.

When saving an image buffer (M_IMAGE) that has an associated LUT buffer (color palette), the content of the LUT is also saved with the image.

Note, you can perform the same operation as **MbufSave**(), using MbufExport() with its FileFormat parameter set to M_AIL_TIFF or M_AIL_NATIVE.

Parameters

FileName

Specifies the name and path of the file in which to save the file. It is recommended that you use the MIM file extension for buffers, and the MBUFC file extension for containers. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten.

This parameter can be set to one of the following values:

For specifying the file name and path	
Value	Description
M_INTERACTIVE	Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.

AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, data buffer files can contain any of the file extensions listed in MbufImport. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").
AIL_TEXT("*ftp://user:password@server/pathtofile*") [1]	Save a data buffer to a file on a remote File Transfer Protocol (FTP) server. To access a remote FTP server anonymously, omit the user name and password. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be saved using an FTP file path.
AIL_TEXT("*sftp://user:password@server/pathtofile*") [1]	Save a data buffer to a file on a remote SSH File Transfer Protocol (SFTP) server. > FTP-SSL (FTPS) is not supported. > Note that GenDC, STL, and PLY files cannot be saved using an SFTP file path.

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContainerOrBufId

Specifies the identifier of the data buffer or container to save. This image buffer can have an ROI set using MbufSetRegion().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib;.
DLL	Requires ail11.dll.

MbufStream

Synopsis

Load, restore, or save a buffer or container from/to a file or memory stream.

Syntax

```
void MbufStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SystemId,               //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContainerOrBufIdPtr,   //in-out
    AIL_INT *SizeByteVarPtr        //in-out
)
```

Description

This function can load, restore, or save a buffer or container from/to a file or memory stream.

To inquire the number of bytes necessary to save a buffer or container, you should first call this function (**MbufStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file.

Typically the stream is saved in the Aurora Imaging Library native file format. Optionally, you can save a buffer to the PNG format or a container to a GenDC-compatible format by setting the ControlFlag parameter to M_PNG or M_GENDC, respectively. You must specify the same format when loading or restoring a stream saved in a PNG or GenDC-compatible format. When loading or restoring a PNG from memory, you must specify the size using the SizeByteVarPtr parameter.

Optionally, when saving a container, you can specify to losslessly compress the data in the stream by setting the ControlFlag parameter to M_COMPRESS. This is not available when saving a container in a format that is GenDC-compatible.

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

For an M_RESTORE operation of an image, this function tries to allocate the buffer so that it can be used for acquisition (M_GRAB), display (M_DISP), and processing (M_PROC) operations. If the buffer cannot be allocated with the M_GRAB attribute, Aurora Imaging Library allocates one that can be used in all of the above operations except for acquisition (M_GRAB). You can specify that the restored buffer is not allocated with the acquisition or compression attributes (M_GRAB or M_COMPRESS respectively), by combining M_RESTORE with M_NO_GRAB or M_NO_COMPRESS respectively.

For an M_RESTORE operation of a container, this function allocates a container that can be used for display (M_DISP) and processing (M_PROC) operations.

Using **MbufStream**(), you can choose to save a backwards-compatible version of the buffer or container, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions,

you can also load or restore buffers or containers saved using any Aurora Imaging Library version with which this function was available. Settings that do not exist in the lower version will be filled with default values when the buffer or container is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SystemId

Specifies the system on which to restore the buffer or container.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the buffer or container. This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the type of stream																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream. The file is saved in the Aurora Imaging Library native file format, unless ControlFlag is set to M_GENDC or M_PNG; in which case, the file is saved in the specified file format. The GenDC file format is only available when saving/loading a container. The PNG file format is only available when saving/loading buffers.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream. The memory stream is saved in the Aurora Imaging Library native file format, unless ControlFlag is set to M_GENDC or M_PNG; in which case, the memory stream is saved in the specified file format. The GenDC file format is only available when saving/loading a container. The PNG file format is only available when saving/loading a buffer. > When loading or restoring a PNG from memory, you must specify the size using the SizeByteVarPtr parameter.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the buffer or container.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies the control flag for the operation. Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

ContainerOrBufIdPtr

Specifies the address of the variable in which to write, or from which to read, the buffer or container identifier.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write, or from which to read, the size of the buffer or container, in bytes.

If the size is not required, you can set this parameter to **M_NULL**.

> The size of the buffer or container varies depending on the Aurora Imaging Library version that you specify.

See the Parameter associations section for possible values that can be returned/specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SystemId**, **Operation**, **Version**, **ControlFlag**, **ContainerOrBufIdPtr**, and **SizeByteVarPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																		
Operation		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	MemPtrOrFileName - Possible values passed or returned																	
	SystemId																	
	Version																	
	ControlFlag																	
	ContainerOrBufIdPtr - Possible values passed or returned																	
	SizeByteVarPtr - Possible values passed or returned																	

M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a buffer or container to a memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SystemId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the buffer or container must be compatible when saved. The size of a buffer or container might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContainerOrBufIdPtr	Specifies the address of the variable containing the identifier of the buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
ControlFlag	Specifies how the data is stored in the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENDC	Specifies that the data is saved using a GenDC-compatible format. > This setting is only available for a container and for not a buffer.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PNG	Specifies that the data is saved using the PNG file format. > This setting is only available for a buffer and not a container.	c		e	f	g	a		i		j	k	l	m	n	b
SizeByteVarPtr		Specifies the address of the variable in which to store the size of the buffer or container, in bytes.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD		Loads the content of a specified file or memory stream into a previously allocated buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName		Specifies the file or memory stream from which to load the buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), or URL when the StreamType parameter is set to M_FILE. Files that store a buffer or container in the Aurora Imaging Library native format typically have the mbuf or mbufc extension respectively. Files that store a format in the GenDC format typically have the gendc extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SystemId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContainerOrBufIdPtr	Specifies the address of the variable containing the identifier of the buffer or container. All previous settings are overwritten with those of the loaded buffer or container, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the buffer or container is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
ControlFlag	Specifies how to load the data from the file or memory stream into the buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior. When loading a container, this is equivalent to M_BASIC_ATTRIBUTE.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENDC +	Specifies that the stream is in a GenDC-compatible format. > This setting is only available for a container and not a buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PNG	Specifies to load the contents of a file or memory stream that is in the PNG file format. > This setting is only available for a buffer and not a container. > When using this setting with M_MEMORY, you must specify the size using the SizeByteVarPtr parameter.	c		e	f	g	a		i		j	k	l	m	n	b
SizeByteVarPtr	Specifies the address of the variable that contains the size of the buffer, in bytes. This is required when restoring the buffer from a PNG file format. Otherwise, this should be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that this parameter is unused.	c		e	f	g	a		i		j	k	l	m	n	b
SizeByteVarPtr	The address of the variable that contains the size of the buffer, in bytes.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE +	Restores a buffer or container from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b

MemPtrOrFileName	Specifies the file or memory stream from which to load the buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), or URL when the StreamType parameter is set to M_FILE. Files that store a buffer or container in the Aurora Imaging Library native format typically have the mbuf or mbufc extension respectively. Files that store a format in the GenDC format typically have the gendc extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SystemId	Specifies the system on which to allocate the buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContainerOrBufIdPtr	Specifies the address of the variable in which to write the identifier of the buffer or container. If the operation is not successful, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag	Specifies how the data is stored in the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENDC	Specifies that the data is saved using a GenDC-compatible format. > This setting is only available for a container and not a buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_PNG	Specifies that the data is saved using the PNG file format. > This setting is only available for a buffer and not a container. > When using this setting with M_MEMORY, you must specify the size using the SizeByteVarPtr parameter.	c		e	f	g	a		i		j	k	l	m	n	b
SizeByteVarPtr	Specifies the address of the variable that contains the size of the buffer, in bytes, when restoring the buffer from a PNG file format. Otherwise, this should be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that this parameter is unused.	c		e	f	g	a		i		j	k	l	m	n	b
SizeByteVarPtr	Specifies the address of the variable that contains the size of the buffer or container, in bytes.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE	Saves a buffer or container to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream from which to load the buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

		AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), or URL when the StreamType parameter is set to M_FILE. File streams that store a buffer or container in the Aurora Imaging Library native format typically have the mbuf or mbufc extension respectively. File streams that store a format in the GenDC format typically have the gendc extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SystemId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		Specifies the Aurora Imaging Library version with which the buffer or container must be compatible when saved.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContainerOrBufIdPtr		Specifies the address of the variable containing the identifier of the buffer or container.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlFlag		Specifies how to save the data in the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b
		M_COMPRESS	Specifies to losslessly compress the data in the stream. > This setting is only available for a container and not a buffer.	c		e	f	g	a		i		j	k	l	m	n	b
		M_GENDC	Specifies to save the data using a GenDC-compatible format. When saving using a GenDC-compatible format, some attributes and settings of the container and its components might be removed. > This setting is only available for a container and not a buffer. An error is generated if the source container has one of the following: • A component that is not an M_IMAGE or M_ARRAY buffer. • A component with M_COMPONENT_TYPE set to M_COMPONENT_MESH_AIL or M_COMPONENT_NORMALS_AIL. • A component with M_COMPONENT_GROUP_ID, M_COMPONENT_REGION_ID, or M_COMPONENT_SOURCE_ID set to a value larger than 65535. • A component that is an image buffer compressed in a format other than JPEG or JPEG 2000.	c		e	f	g	a		i		j	k	l	m	n	b
		M_PNG +	Specifies to save the data using the PNG file format. > This setting is only available for a buffer and not a container.	c		e	f	g	a		i		j	k	l	m	n	b

	SizeByteVarPtr	Specifies the address of the variable in which to place the size of the buffer or container, in bytes. This should be set to M_NULL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NULL	Specifies that this parameter is unused.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Combination values that can be used alone or as combination values for [when Operation=M_LOAD: M_GENDC].

You can use one of the following values on its own, or add it to the above-mentioned value, to specify what to load from a file or stream.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what to load from the contents of a file or memory																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_BASIC_ATTRIBUTE	Specifies to load the contents of the file or memory stream (that stores data suitable for loading into a container) to automatically allocated components that might have different attributes than the original stored components. > This setting is only available for a container and not a buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_IDENTICAL	Specifies to load all of the contents of the file or memory stream to the specified buffer (or to the specified container). When loading a container, this will includes all data, settings, and attributes (except M_PROC, M_GRAB, and M_DISP which are always the same as the attributes of the destination container). When loading a buffer, an error will be generated if there is a mismatch between the size, number of bands, or attributes (such as M_RGB24) of the destination buffer and the contents of the file.	c		e	f	g	a		i		j	k	l	m	n	b
M_USE_DESTINATION	Specifies to load the contents of the file or memory stream (that stores data suitable for loading into a container) into the existing components of the destination container. An error will be generated if the file or memory stream has a different number of components than the destination container. > This setting is only available for a container and not a buffer.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_RESTORE.

You can add one or more of the following values to the above-mentioned value to specify that the buffer is not

restored with a certain attribute.

> Note that these values are only available when restoring an image buffer, and not a container.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying that the buffer is not restored with a certain attribute																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NO_COMPRESS	Specifies that Aurora Imaging Library will not allocate the restored data buffer with an M_COMPRESS attribute.	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_GRAB	Specifies that Aurora Imaging Library will not allocate the restored data buffer with an M_GRAB attribute.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when Operation=M_SAVE: M_PNG].

You can add the following value to the above-mentioned value to specify the compression level of a PNG.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the compression level of a PNG																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_PNG_COMPRESSION_LEVEL*n*	Specifies a compression level of *n*, where *n* is a value from 0 to 9. > The highest compression level (that is, 9) will result in the smallest file, but will take longer to save.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib;.
DLL	Requires ail11.dll.

MbufSetRegion

Synopsis

Set the region of interest (ROI) of an image buffer.

Syntax

```
void MbufSetRegion(
    AIL_ID ImageBufId,           //out
    AIL_ID ImageOrGraphicListId, //in
    AIL_INT64 Label,             //in
    AIL_INT64 Operation,         //in
    AIL_DOUBLE Param             //in
)
```

Description

This function sets the region of interest (ROI) of an image buffer; if supported, Aurora Imaging Library functions will only operate on this region of the buffer. **MbufSetRegion**() can also be used to modify, delete, copy, extract, or link the ROI.

The ROI of an image buffer can be of any shape and can be composed of several non-contiguous areas. When dealing with processing operations that respect the ROI, all pixels outside the ROI are ignored; to disable their use of the ROI, use MbufControl() with M_REGION_USE.

You can define an ROI from an image mask, a 2D graphics list, or the valid pixels in a depth map. When defined from an image mask, the region of interest consists of the pixels corresponding to the non-zero pixels in the image mask. When defined from the graphics contained in a 2D graphics list, only areas corresponding to the graphics are processed. If the graphics defining the ROI are not filled, the ROI will consist of only the graphics' outline (MgraControlList() with M_LINE_THICKNESS). However, if the combination constant M_FILL_REGION is specified when calling **MbufSetRegion**(), the ROI is created such that non-filled graphics are considered filled. This is useful if you want to display the 2D graphics list defining the region of interest, or modify it interactively, without obstructing the target image. You can also specify the combination constant M_USE_LINE_THICKNESS_1 when calling **MbufSetRegion**() to ignore the M_LINE_THICKNESS setting of the graphics in the 2D graphics list and use the default value of one pixel. This is useful when interactively creating a region of interest from graphics in a 2D graphics list that have line thickness greater than one pixel for visual clarity.

> The image mask must be the same size as the image buffer.

For a fully-corrected depth map image buffer, you can also define an ROI from the valid pixels of the image (using M_RASTERIZE_DEPTH_MAP_VALID_PIXELS). In this case, any pixel in the image that is set to the maximum possible value (for example, 255 for an 8-bit buffer) is considered invalid and is excluded from the ROI.

Note that, unlike a child buffer, an ROI does not have its own Aurora Imaging Library identifier and cannot have results returned with respect to it.

When defining the ROI with an image mask, this raster (bitmap) information is copied and associated with the image buffer, creating an M_RASTER ROI. When defining the ROI with a 2D graphics list, this vectorial information and/or its corresponding raster (bitmap) version is copied and associated with the image buffer, creating an M_VECTOR ROI, an M_RASTER ROI, or an M_VECTOR_AND_RASTER ROI, depending on how you set the Operation parameter. Some functions (for example, MblobCalculate()) expect an M_RASTER ROI, while other functions (for example, McodeRead()) expect an M_VECTOR ROI; creating an M_VECTOR_AND_RASTER ROI

allows more Aurora Imaging Library modules to use the ROI in the source image.

When an ROI is defined, the information from the specified 2D graphics list or image mask is copied and the copy is associated with the source image buffer. Any subsequent changes to the original 2D graphics list or image mask will not affect the ROI. To update the ROI using a new or modified 2D graphics list or image mask, call **MbufSetRegion**() again with the new information. The old ROI information is replaced with the new ROI information.

An M_VECTOR ROI or an M_VECTOR_AND_RASTER ROI can be associated with the same camera calibration information as the image. This ensures that whenever the relative coordinate system moves (using McalRelativeOrigin(), McalSetCoordinateSystem(), or McalFixture()), the ROI moves accordingly. To associate an ROI with camera calibration information, you must define the ROI with a 2D graphics list. All graphics in the 2D graphics list that were created in world units (using MgraControl() with M_INPUT_UNITS set to M_WORLD) will be associated with the same camera calibration information as the image; all other graphics in the 2D graphics list, as well as any generated raster information, will not be calibrated. Note that defining an M_VECTOR_AND_RASTER ROI with a 2D graphics list containing graphics created in world units will generate an error if the specified image buffer is not already calibrated.

If the camera calibration information associated with the image buffer changes, the raster information of an M_VECTOR_AND_RASTER ROI will be discarded, causing the ROI to become an M_VECTOR ROI. Calling **MbufSetRegion**() with the Operation parameter set to M_RASTERIZE and the ImageOrGraphicListId parameter set to M_NULL transforms the ROI to an M_VECTOR_AND_RASTER ROI, such that the raster data is synchronized with the vector information.

Not all functions support an image buffer with an ROI. If you pass an image buffer with an ROI to a processing or analysis function that does not use the ROI of an image buffer, an error will be generated. The functions' descriptions state whether they support the use of an ROI.

> If a child buffer is allocated using a parent buffer with an ROI, the child buffer's ROI is automatically linked with that of the parent buffer (the same as using this function with M_LINK_TO_PARENT and the Aurora Imaging Library identifier of the child buffer). You can also set an ROI in the child buffer directly. Note that the ROI of the child buffer is only accessible using the child buffer's Aurora Imaging Library identifier.

You cannot set an ROI in a YUV image buffer unless its format is YUV24 Planar, and it is not a child buffer.

Parameters

ImageBufId

Specifies the identifier of the image buffer for which to set the ROI, or from which to copy or extract the ROI.

See the Parameter associations section for possible values that can be returned.

ImageOrGraphicListId

Specifies the identifier of the image buffer or 2D graphics list that contains the data to use to set the ROI, or to which to copy or extract the ROI information. If not used, set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be specified.

Label

Reserved for future expansion and must be set to **M_DEFAULT**.

Operation

Specifies the type of operation to perform.

See the Parameter associations section for possible values that can be specified.

Param

Reserved for future expansion and must be set to **M_DEFAULT**.

Parameter associations

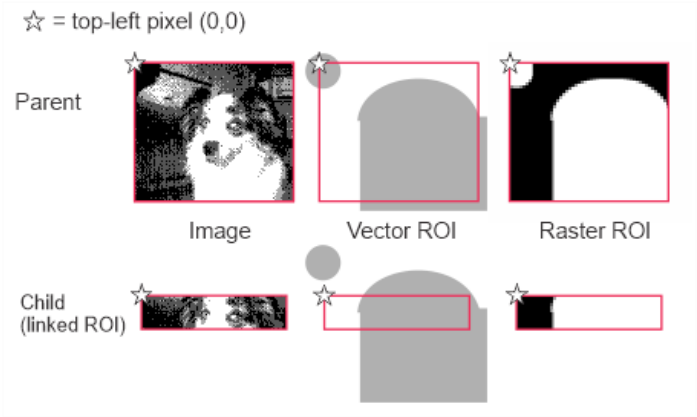
The table below lists possible values for the **ImageOrGraphicListId** and **Operation** parameters and possible values returned to the **ImageBufId** parameter.

- For specifying the operation to perform for the ROI(s)

The following Operation and corresponding ImageBufId and ImageOrGraphicListId parameter settings are used to perform operations on ROIs related to the specified buffer(s).

For specifying the operation to perform for the ROI(s)	
Operation ImageBufId - Possible values returned ImageOrGraphicListId	Description
M_DEFAULT	Same as M_RASTERIZE.
M_COPY	Specifies to copy the ROI of the source image buffer and set the copy as the ROI of the destination image buffer. If the source buffer has an ROI only in an M_RASTER format, and the destination buffer is smaller than the source buffer, the copied ROI is cropped to the size of the destination buffer. If the destination buffer is larger than the source buffer, the copied ROI retains the size of the source buffer's ROI. This is true even if the source buffer is a child buffer that has an ROI that is linked to the ROI of a larger parent buffer (by default, or using **MbufSetRegion**() with M_LINK_TO_PARENT); only the part of the linked ROI that is within the area of the source buffer is copied. If the source buffer has an ROI in an M_VECTOR format, the entire ROI is copied, including parts of the ROI that are outside the source image. If the ROI is in an M_VECTOR_AND_RASTER format, the entire vector portion of the ROI is copied, including parts of the ROI that are outside the source image. The raster portion of the ROI is recalculated based on the region that is visible in the destination image (if the source image and destination image are not the same size).
ImageBufId	Returns the following.
Image buffer identifier from which to copy the ROI	Specifies the identifier of the source image buffer from which to copy the ROI.
ImageOrGraphicListId	Specifies the following.

Image buffer identifier for which to define the ROI	Specifies the identifier of the destination image buffer to which to copy the ROI. If the image buffer is already associated with an ROI, this association is removed and the image buffer is associated with the new ROI. If the image buffer is a child buffer with an ROI linked to that of its parent, the ROIs are unlinked.
M_DELETE	Specifies to remove ROI information from the image buffer.
ImageBufId	Returns the following.
Image buffer identifier for which to remove the ROI	Specifies the Aurora Imaging Library identifier of the image buffer from which to remove ROI information.
M_EXTRACT	Specifies to extract the 2D graphics list or image mask used to define the ROI of the source image buffer, and store it in the destination image buffer or 2D graphics list. When you pass a 2D graphics list as a destination, the graphics used to define the source ROI are added to the destination 2D graphics list. Existing graphics in the destination 2D graphics list are not removed. If the source buffer has an ROI in an M_RASTER format, ImageOrGraphicListId must be set to the Aurora Imaging Library identifier of an image buffer. If the source buffer has an ROI in an M_VECTOR format, ImageOrGraphicListId must be set to the Aurora Imaging Library identifier of a 2D graphics list. If the source buffer has an M_VECTOR_AND_RASTER ROI, ImageOrGraphicListId can be set to the Aurora Imaging Library identifier of either a 2D graphics list or an image buffer. If the destination image buffer is larger than the source image, the existing image mask is copied; the image mask is not recalculated based on the size of the destination image.
ImageBufId	Returns the following.
Image buffer identifier from which to extract the ROI	Specifies the Aurora Imaging Library identifier of the source image buffer from which to extract the 2D graphics list or image mask used to define the ROI.
ImageOrGraphicListId	Specifies one of the following.
2D graphics list identifier in which to store the extracted 2D graphics	Specifies the identifier of the destination 2D graphics list in which to store the 2D graphics of the extracted 2D graphics list. The 2D graphics list must have been previously allocated using MgraAlloc().
Image buffer identifier in which to store the extracted image mask	Specifies the identifier of the destination image buffer in which to store the extracted image mask.

M_LINK_TO_PARENT	Specifies to link the ROI of the image buffer to the ROI of its parent buffer. The child buffer's offset and size are used to determine what portion of the parent's ROI to use for the child. The ROI information for the parent and child buffer occupy the same memory space. Therefore, modifying the ROI of the parent buffer affects the ROI of the child buffer. Modifying the ROI of the child buffer does not affect the ROI of the parent buffer; instead it unlinks the ROIs. The ROI of a child buffer is linked to the ROI of its parent buffer by default. This image shows the relationship between the linked ROIs of a parent and child buffer.  You can unlink the ROI by setting an ROI for the child buffer using **MbufSetRegion**() with any other type of operation. To retain the child buffer's current ROI when unlinking, set the Operation parameter to M_COPY, and set both the ImageBufId and ImageOrGraphicListId parameters to the Aurora Imaging Library identifier of the child buffer. If the specified buffer is not a child buffer, an error is generated.
ImageBufId	Returns the following.
Child image buffer identifier of which to link the ROI	Specifies the Aurora Imaging Library identifier of the child image buffer, of which to link the ROI with that of its parent buffer.
M_NO_RASTERIZE +	Specifies to create an M_VECTOR ROI, defined by the graphics of a 2D graphics list. The content of the 2D graphics list is copied, and this vectorial information is associated with the specified image buffer. Only the areas defined by the graphics of the 2D graphics list are processed by Aurora Imaging Library functions. If a shape is not filled, only the pixels corresponding to its contour will be processed by Aurora Imaging Library functions unless the combination constant M_FILL_REGION is specified.
ImageBufId	Returns the following.
Image buffer identifier for which to set the ROI	Specifies the Aurora Imaging Library identifier of the image buffer for which to set the ROI. If the image buffer is already associated with an ROI, this association is removed and the image buffer is associated with the new ROI. If the image buffer is a child buffer with an ROI linked to that of its parent, the ROIs are unlinked.

	ImageOrGraphicListId	Specifies the following.
	2D graphics list identifier to use to define the ROI	Specifies the identifier of the 2D graphics list to use to define the ROI. The 2D graphics list must have been previously allocated and filled with 2D graphics using the graphics functions (Mgra...()).
M_RASTERIZE +		Specifies to create an M_RASTER ROI defined by an image mask, an M_VECTOR_AND_RASTER ROI defined by a 2D graphics list, or to change an M_VECTOR ROI to an M_VECTOR_AND_RASTER ROI. Aurora Imaging Library functions will only operate on source image pixels corresponding to either non-zero value pixels of the image mask or areas defined by the graphics of the 2D graphics list, depending on whether the Aurora Imaging Library function requires raster or vectorial information. When creating an ROI defined by an image mask, the content of the image mask is copied and this raster (bitmap) information is stored as the raster portion of the ROI associated with the specified image buffer. When creating an ROI defined by a 2D graphics list, or changing an M_VECTOR ROI to an M_VECTOR_AND_RASTER ROI, an image mask is generated from the 2D graphics list or existing vectorial information. This raster (bitmap) information and the vectorial information (from the existing ROI or specified 2D graphics list) are stored as the raster portion and vector portion of the ROI associated with the image buffer. When generating the mask, pixels corresponding to areas defined by the vectorial information are set to a non-zero value, and all other image mask pixels are set to a zero value. If a shape is not filled, only the pixels corresponding to its contour will be set to a non-zero value, unless the combination constant M_FILL_REGION is specified. M_FILL_REGION is not available when creating an ROI from an image mask. This is the default value.
	ImageBufId	Returns the following.
	Image buffer identifier for which to set the ROI	Specifies the Aurora Imaging Library identifier of the image buffer for which to set the ROI. If the image buffer is already associated with an ROI, this association is removed and the image buffer is associated with the new ROI (except when ImageOrGraphicListId is set to M_NULL. If the image buffer is a child buffer with an ROI linked to that of its parent, the ROIs are unlinked.
	ImageOrGraphicListId	Specifies one of the following.
	M_NULL	Specifies to rasterize the existing M_VECTOR ROI of the image buffer. This changes the M_VECTOR ROI to an M_VECTOR_AND_RASTER ROI. Note that this operation has no effect on an M_RASTER or M_VECTOR_AND_RASTER ROI.
	2D graphics list identifier to use to define the ROI	Specifies the identifier of the 2D graphics list to use to define the ROI. The 2D graphics list must have been previously allocated and filled with 2D graphics using the graphics functions (Mgra...()).

Image mask buffer identifier to use to define the ROI	Specifies the identifier of the image buffer to use as an image mask to define the ROI. The image mask must be the same size as the image buffer, and must be a 1-band image buffer.
M_RASTERIZE_AND_DISCARD_LIST +	Specifies to create an M_RASTER ROI defined by a 2D graphics list. An image mask is generated from the specified 2D graphics list, and stored as the raster ROI associated with the image buffer. When generating the mask, pixels corresponding to areas defined by the 2D graphics list are set to a non-zero value, and all other mask pixels are set to a zero value. If a shape is not filled, only the pixels corresponding to its contour will be set to a non-zero value, unless the combination constant M_FILL_REGION is specified. This operation does not store the vectorial information from the 2D graphics list in the ROI. Aurora Imaging Library functions will only operate on source image pixels corresponding to non-zero value pixels of the image mask.
ImageBufId	Returns the following.
Image buffer identifier for which to set the ROI	Specifies the Aurora Imaging Library identifier of the image buffer for which to set the ROI. If the image buffer is already associated with an ROI, this association is removed and the image buffer is associated with the new ROI. If the image buffer is a child buffer with an ROI linked to that of its parent, the ROIs are unlinked.
ImageOrGraphicListId	Specifies the following.
2D graphics list identifier to use to define the ROI	Specifies the identifier of the 2D graphics list to use to define the ROI. The 2D graphics list must have been previously allocated and filled with 2D graphics using the graphics functions (Mgra...()).
M_RASTERIZE_DEPTH_MAP_VALID_PIXELS	Specifies to create an M_RASTER ROI for a fully-corrected depth map image buffer; in this case, the ROI is defined by the valid pixels of the depth map. Any pixel in the image that is set to the maximum possible value (for example, 255 for an 8-bit buffer) is considered invalid and is excluded from the ROI. You can either set and define the ROI using the depth map alone, or you could use the depth map and an image mask buffer. If you choose to use both the depth map and the mask buffer, the ROI will be defined by the pixels that are both valid in the depth map and non-zero in the image mask buffer. Note that if the mask buffer is a depth map, the ROI will be defined by the pixels that are valid in both depth maps (not equal to the invalid value). Note that, if some pixels of the depth map are later changed to be valid or invalid (for example during a grab), the ROI is not dynamically updated; you must call this function again to update the ROI. You can also create an M_RASTER ROI defined by an image buffer or 2D graphics list for a fully-corrected depth map image buffer. To do so, use M_RASTERIZE.
ImageBufId	Returns the following.

	Fully-corrected depth map image buffer identifier for which to set the ROI	Specifies the Aurora Imaging Library identifier of the image buffer for which to set the ROI. The image buffer must be a 1-band, 8-bit, 16-bit, or 32-bit unsigned buffer and must store a fully-corrected depth map (that is, if you call McalInquire() with M_DEPTH_MAP, the function returns M_TRUE). If the image buffer is already associated with an ROI, this association is removed and the image buffer is associated with the new ROI. If the image buffer is a child buffer with an ROI linked to that of its parent, the ROIs are unlinked.
ImageOrGraphicListId		Specifies one of the following.
	M_NULL	Specifies that the depth map will be the only buffer used to define the ROI.
	Image mask buffer identifier to use to define the ROI	Specifies the identifier of the image buffer to use as an image mask to define the ROI. The ROI will be defined by the pixels that are both valid in the depth map and non-zero in the image mask buffer (or not equal to the invalid value if this image buffer stores a fully-corrected depth map). The image mask must be the same size as the depth map image buffer, and must be a 1-band image buffer.

Combination values for M_NO_RASTERIZE; M_RASTERIZE; M_RASTERIZE_AND_DISCARD_LIST.

You can add one or more of the following values to the above-mentioned values to set whether to interpret the graphics in the 2D graphics list as being filled or as having a line thickness of 1 pixel.

For interpreting the graphics in the 2D graphics list	
Combination value	Description
M_FILL_REGION	Specifies that the graphics in the 2D graphics list are interpreted as if they are filled when defining the ROI. If one or more graphics of the 2D graphics list are already filled (for example, a rectangle created with MgraRectFill()), M_FILL_REGION has no effect on the already filled graphics.
M_USE_LINE_THICKNESS_1	Specifies that the graphics in the 2D graphics list are interpreted as if their M_LINE_THICKNESS setting is set to 1 pixel when defining the ROI.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.

MbufTransfer

Synopsis

Copy a two-dimensional region of one or all bands from the source buffer into a two-dimensional region of one or all bands in the destination buffer, using a specified transfer function and transfer type.

Syntax

```
void MbufTransfer(
    AIL_ID SrcBufId,            //in
    AIL_ID DestBufId,           //out
    AIL_INT SrcOffX,            //in
    AIL_INT SrcOffY,            //in
    AIL_INT SrcSizeX,           //in
    AIL_INT SrcSizeY,           //in
    AIL_INT SrcBand,            //in
    AIL_INT DestOffX,           //in
    AIL_INT DestOffY,           //in
    AIL_INT DestSizeX,          //in
    AIL_INT DestSizeY,          //in
    AIL_INT DestBand,           //in
    AIL_INT64 TransferFunction, //in
    AIL_INT64 TransferType,     //in
    AIL_INT64 OperationFlag,    //in
    void *ExtraParameterPtr     //in
)
```

Description

This function copies a two-dimensional region of one or all bands of the source buffer to a two-dimensional region of one or all bands of the destination buffer, using a specified transfer function and transfer type.

If the size of the regions in the source and destination buffers differ, the size of the largest region is reduced so that the size of the source and destination regions become equal.

Parameters

SrcBufId

Specifies the identifier of the source buffer. When the TransferFunction parameter is set to M_CLEAR, you must set this parameter to **M_NULL**; when the TransferFunction parameter is set to M_BYTE_SWAP, you can set this parameter to **M_NULL** to swap bytes within the same buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

[All except Zebra Concord PoE with ToE, Zebra Indio, Zebra Rapixo CoF, and Zebra Rapixo CXP]

Floating-point buffers are only supported when the TransferFunction parameter is set to M_COPY.

DestBufId

Specifies the identifier of the destination buffer.

If the specified buffer is an image buffer with an associated region of interest (ROI), an error will occur.

[All except Zebra Concord PoE with ToE and Zebra Indio]

Floating-point buffers are only supported when the TransferFunction parameter is set to M_COPY.

SrcOffX

Specifies the horizontal offset of the source region, relative to the top-left coordinate of the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the horizontal offset of the source region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the horizontal offset of the source region is ignored when the TransferFunction parameter is set to M_CLEAR.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the horizontal offset of the source region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SrcOffY

Specifies the vertical offset of the source region, relative to the top-left coordinate of the source buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the vertical offset of the source region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0 pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the vertical offset of the source region is ignored when the TransferFunction parameter is set to M_CLEAR.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the vertical offset of the source region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SrcSizeX

Specifies the width of the source region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the width of the source region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default width of the source region. The default is equal to the difference between the width of the source buffer and the horizontal offset of the source region.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the width of the source region is ignored when the TransferFunction parameter is set to M_CLEAR.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the width of the source region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SrcSizeY

Specifies the height of the source region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the height of the source region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default height of the source region. The default is equal to the difference between the height of the source buffer and the vertical offset of the source region.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the height of the source region is ignored when the TransferFunction parameter is set to M_CLEAR.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the source region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

SrcBand

Specifies the band of the region from which to copy. This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the band of the region from which to copy																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL_BANDS.	c		e	f	g	a		i		j	k	l	m	n	b
M_NULL	Specifies that the band of the source region is ignored when the TransferFunction parameter is set to M_CLEAR.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies all bands (for multi-band buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_BLUE	Specifies the blue band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_GREEN	Specifies the green band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_HUE	Specifies the hue band (for HSL buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_LUMINANCE	Specifies the luminance band (for HSL buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_RED	Specifies the red band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b

M_SATURATION	Specifies the saturation band (for HSL buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_U	Specifies the U band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_V	Specifies the V band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_Y	Specifies the Y band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < *#bands*	Specifies the index of the band to copy. The relationship between the index value and band for RGB and HSL buffers is the following: 0 – Corresponds to the red band for RGB buffers or the hue band for HSL buffers. 1 – Corresponds to the green band for RGB buffers or the saturation band for HSL buffers. 2 – Corresponds to the blue band for RGB buffers or the luminance band for HSL buffers.	c		e	f	g	a		i		j	k	l	m	n	b

DestOffX

Specifies the horizontal offset of the destination region, relative to the top-left coordinate of the destination buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the horizontal offset of the destination region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default horizontal offset of the destination region. The default is equal to the horizontal offset of the source region.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the horizontal offset of the destination region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

DestOffY

Specifies the vertical offset of the destination region, relative to the top-left coordinate of the destination buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the vertical offset of the destination region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default vertical offset of the destination region. The default is equal to the vertical offset of the source region.	c		e	f	g	a		i		j	k	l	m	n	b

Value	Specifies the value of the vertical offset of the destination region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

DestSizeX

Specifies the width of the destination region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the width of the destination region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default width of the destination region. The default is equal to the difference between the destination buffer width and the horizontal offset of the destination region.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the width of the destination region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

DestSizeY

Specifies the height of the destination region.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the height of the destination region																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default height of the destination region. The default is equal to the difference between the destination buffer height and the vertical offset of the destination region.	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the height of the destination region, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

DestBand

Specifies the band of the destination region in which to copy.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the band of the region in which to copy																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL_BANDS.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL_BANDS	Specifies all bands (for multi-band buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_BLUE	Specifies the blue band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_GREEN	Specifies the green band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b

M_HUE	Specifies the hue band (for HSL buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_LUMINANCE	Specifies the luminance band (for HSL buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_RED	Specifies the red band (for RGB buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_SATURATION	Specifies the saturation band (for HSL buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_U	Specifies the U band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_V	Specifies the V band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
M_Y	Specifies the Y band (for YUV buffers).	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < *#bands*	Specifies the index of the band in which to copy. The relationship between the index value and band for RGB and HSL buffers is the following: 0 – Corresponds to the red band for RGB buffers or the hue band for HSL buffers. 1 – Corresponds to the green band for RGB buffers or the saturation band for HSL buffers. 2 – Corresponds to the blue band for RGB buffers or the luminance band for HSL buffers.	c		e	f	g	a		i		j	k	l	m	n	b

TransferFunction

Specifies the function to use in the transfer. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the function to use in the transfer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_BYTE_SWAP	Specifies a transfer function that extracts the bytes in each pixel of the source region, swaps them, and copies the new pixels into the destination region. Note that M_BYTE_SWAP is only supported when the source and destination buffers are both either 1-band 16-bit buffers or 1-band 32-bit buffers and have the same internal storage specifiers. When using 16-bit buffers, the first byte is swapped with the last byte. When using 32-bit buffers, the first byte is swapped with the last byte and the two middle bytes are swapped. You can swap bytes within the same buffer. To do so, set the SrcBufId parameter to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_CLEAR	Specifies a transfer function that clears the destination region. Set all source buffer parameters to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
M_COMPOSITION	Specifies a transfer function that copies all pixels, except for pixels in the source region that are equal to the composition color specified by the OperationFlag parameter.	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	Note that M_COMPOSITION is only supported with 1-band 8-bit buffers, 1-band 16-bit buffers, M_RGB16 buffers, M_BGR24 buffers, and M_BGR32 buffers.	c		e	f	g	a		i		j	k	l	m	n	b
	The source and destination buffers must have the same number of bands, the same data-depth per band, and the same storage specifiers, unless the source is a 1-band, 8-bit buffer. In which case, the destination buffer can be a 3-band 8-bit buffer and the source region band is copied into each of the bands of the destination region. If both regions are multi-band, each source band is copied to the corresponding band of the destination.			e	f	g			i		j	k	l	m	n	b

M_COPY +	Specifies a transfer function that copies the data from the source region into the destination region. If the source and destination buffers are of different data types, Aurora Imaging Library converts the data automatically. When copying from a floating-point buffer to an integer buffer, the values are truncated. If the source is a 3-band region of a YUV buffer and the destination is a 1-band region, only the Y band (luminance) is copied. If the source is a 3-band region of an RGB buffer and the destination is a 1-band region, only the red band is copied. If both regions are multi-band, each source band is copied to the corresponding band of the destination. Note that when copying from a non-binary buffer to a binary destination buffer, all non-zero pixels in the source buffer are represented as ones (1) in the binary destination buffer. When copying a binary buffer to a buffer of a different depth, each bit is copied into the least-significant bit of a different destination pixel using the specified transfer function. The remaining bits of the destination pixel are set to 0; to propagate the bit value to all bits, use MimBinarize().	c		e	f	g	a		i		j	k	l	m	n	b
	System specific															
	In addition, if the source is a 1-band image buffer associated with a LUT buffer and the destination is 3-band, the source data is first mapped through the LUT before being copied to the destination.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_COPY.

You can add the following value to the above-mentioned value to set the scale of the M_COPY transfer function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the scale of M_COPY transfer function																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SCALE	Specifies that the data of the source region must be scaled to fit in the destination region.	c		e	f	g	a		i		j	k	l	m	n	b

TransferType

Specifies the type of transfer. If M_DEFAULT is not selected, then any combination of the transfer modes listed below can be specified for the TransferType parameter. From this combination of transfer modes, Aurora Imaging Library selects the most efficient mode to transfer data. If the most efficient transfer mode fails, then Aurora Imaging Library automatically selects the next most efficient transfer mode from the combination of transfer modes. If all selected transfer modes fail, an error is returned. The efficiency of the transfer mode depends on the internal format of the source and destination buffers, as well as the hardware available in your computer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of transfer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Transfers data using the most efficient transfer mode among the transfer modes listed.	c		e	f	g	a		i		j	k	l	m	n	b
M_AIL_MODE +	Transfers data using the standard Aurora Imaging Library mode. This transfer mode is always possible.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIB_MODE +	[This is only applicable to Windows] Transfers data using the DIB interface. Both the source and destination buffers must have the M_DIB attribute. This transfer mode is only possible when the TransferFunction parameter is set to M_COPY.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For specifying the type of transfer.

You can add the following value to the above-mentioned values to set whether to transfer the data synchronously.

By default, the **MbufTransfer**() function chooses the value that is the most efficient for data transfer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying synchronous data transfer																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_SYNCHRONOUS	Specifies that the transfer of data is synchronous.	c		e	f	g	a		i		j	k	l	m	n	b

OperationFlag

Specifies the color value that is passed to the transfer function. If the TransferFunction parameter is set to M_COPY or M_BYTE_SWAP, this parameter must be set to **M_NULL**.

If the TransferFunction parameter is set to M_CLEAR or M_COMPOSITION, you can set this parameter to one of the following values for a 3-band image buffer:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a color value																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_RGB888(AIL_INT Red, AIL_INT Green, AIL_INT Blue)	Specifies an RGB value. Specify an RGB value when the source buffer is a 3-band buffer.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	Red Specifies the red band, as a value between 0 and 255.															

		Green Specifies the green band, as a value between 0 and 255.													
		Blue Specifies the blue band, as a value between 0 and 255.													
M_COLOR_BLACK		Specifies the color black.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BLUE		Specifies the color blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_BRIGHT_GRAY		Specifies the color bright gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_CYAN		Specifies the color cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_BLUE		Specifies the color dark blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_CYAN		Specifies the color dark cyan.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_GREEN		Specifies the color dark green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_MAGENTA		Specifies the color dark magenta.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_RED		Specifies the color dark red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_DARK_YELLOW		Specifies the color dark yellow.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GRAY		Specifies the color gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_GREEN		Specifies the color green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_BLUE		Specifies the color light blue.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GRAY		Specifies the color light gray.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_GREEN		Specifies the color light green.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_LIGHT_WHITE		Specifies the color light white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_MAGENTA		Specifies the color magenta.	c		e	f	g	a		i		j	k	l	m	n	b

M_COLOR_RED	Specifies the color red.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_WHITE	Specifies the color white.	c		e	f	g	a		i		j	k	l	m	n	b
M_COLOR_YELLOW	Specifies the color yellow.	c		e	f	g	a		i		j	k	l	m	n	b

For a 1-band or multi-band image buffer, you can set this parameter to the following grayscale value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a grayscale value to be used with all bands																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
Value	Specifies a grayscale value used to clear the 1-band or multi-band buffer. This value is cast to the buffer type and depth. If the destination buffer is multi-band, this value will be replicated in each band.	c		e	f	g	a		i		j	k	l	m	n	b

ExtraParameterPtr

Specifies a pointer to the Aurora Imaging Library identifier of the overlay buffer, if applicable. Set this parameter to **M_NULL** in all other cases.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib.
DLL	Requires ail11.dll.